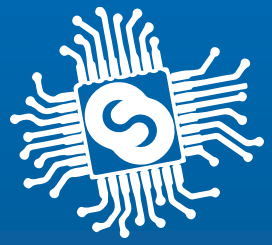
Cursul #1



PM

Intro & De ce

Disclaimer

- *Slide-urile acestea reprezintă un rezumat.*
- *Slide-urile nu acoperă toate explicațiile, simulările, demonstrațiile oferite în timpul cursului.*
- *Slide-urile nu limitează în vreun fel materia cerută la examen.*
- *Pentru varianta completă, sunteți așteptați la curs.*

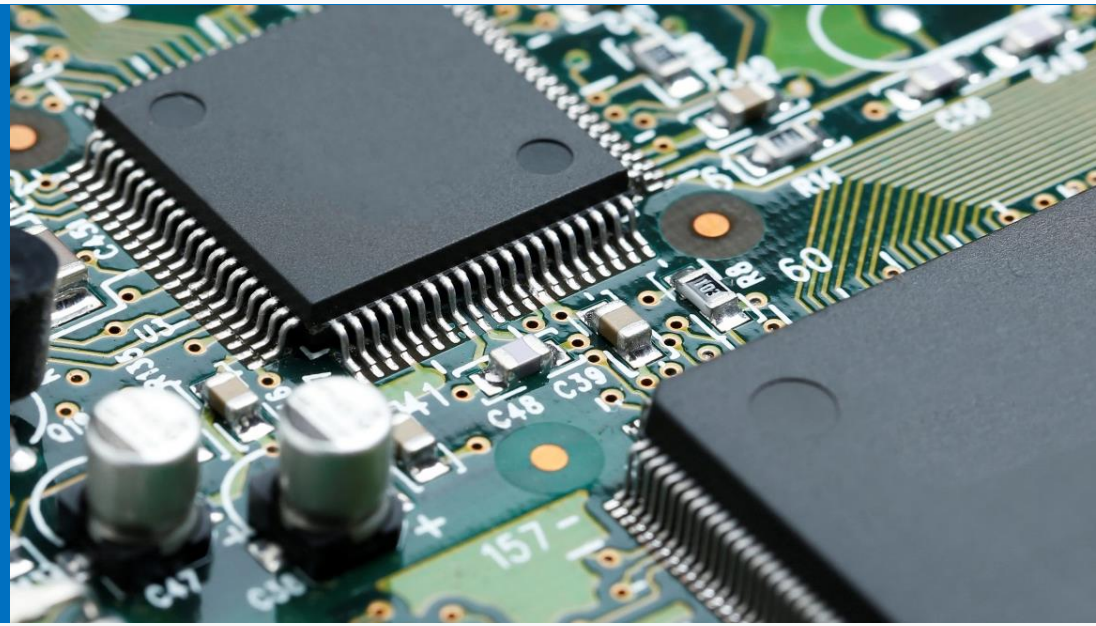
© Elemente din curs preluate din:
- *notițe curs PM, Dan Tudose*
- *notițe curs PM, Andrei Voinescu*
- *scoala de vara EAP 2016, Marius Nicolae*
- *notite curs EEA/ED, Razvan Tataroiu & Daniel Rosner*

Cuprins

- Intro
- Structură punctaj
- Introducere / despre sisteme
- Despre curs / cuprins
- Reguli
- Next time

Intro

Intro - PM



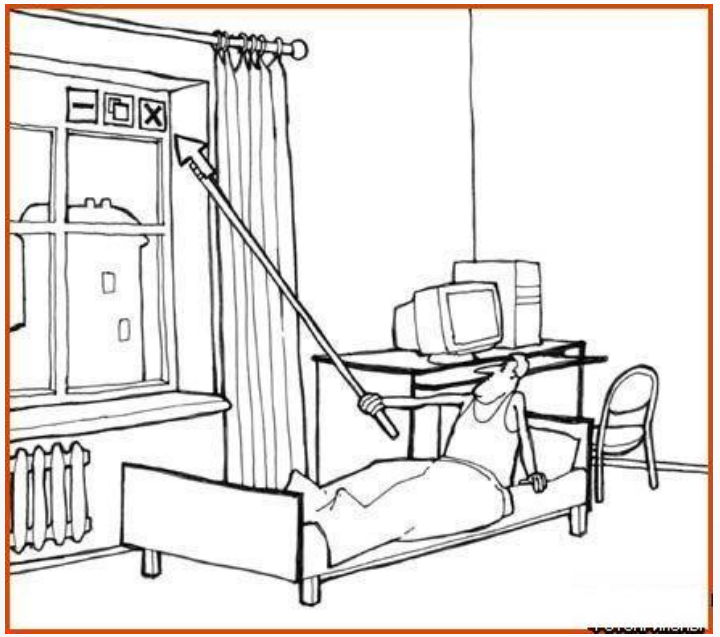
PM

Inginerie

- scientific understanding of the natural world

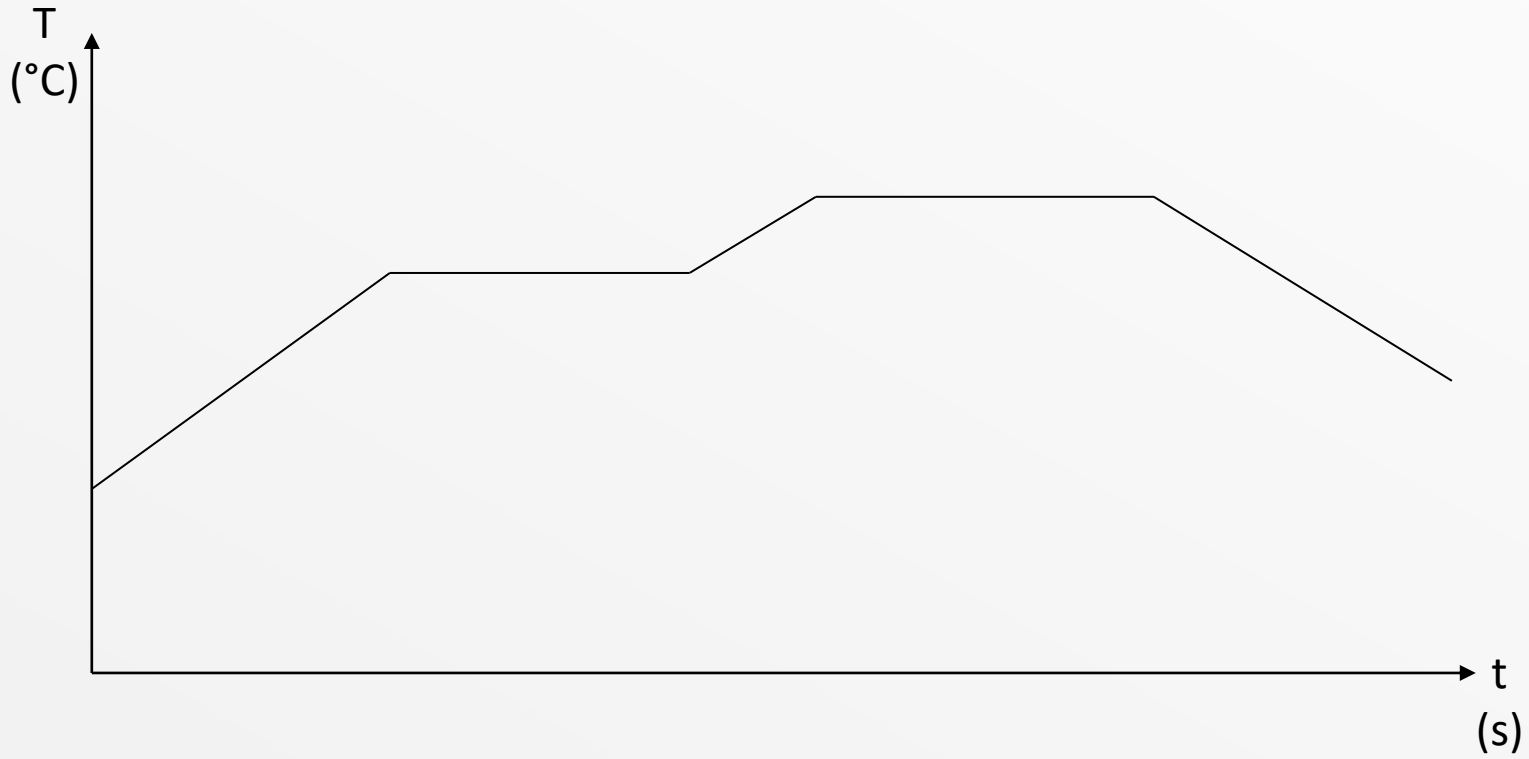
>> used to invent, design, and build things

>> to solve problems and achieve practical goals

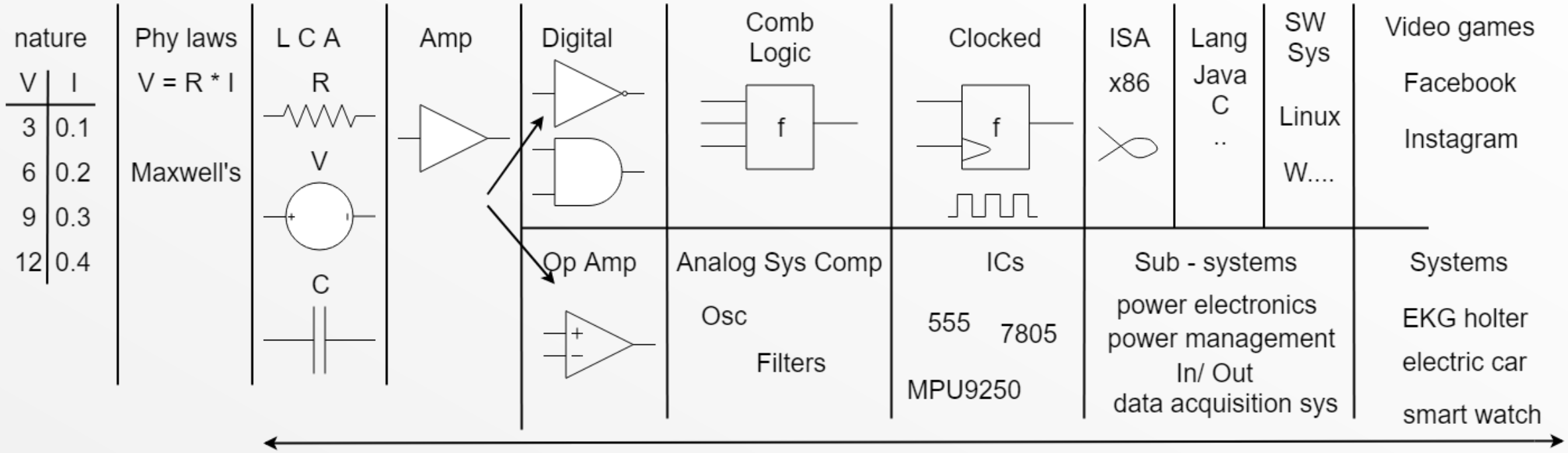


© https://study.com/academy/lesson/what-is-engineering-definition-types.html

Inginer

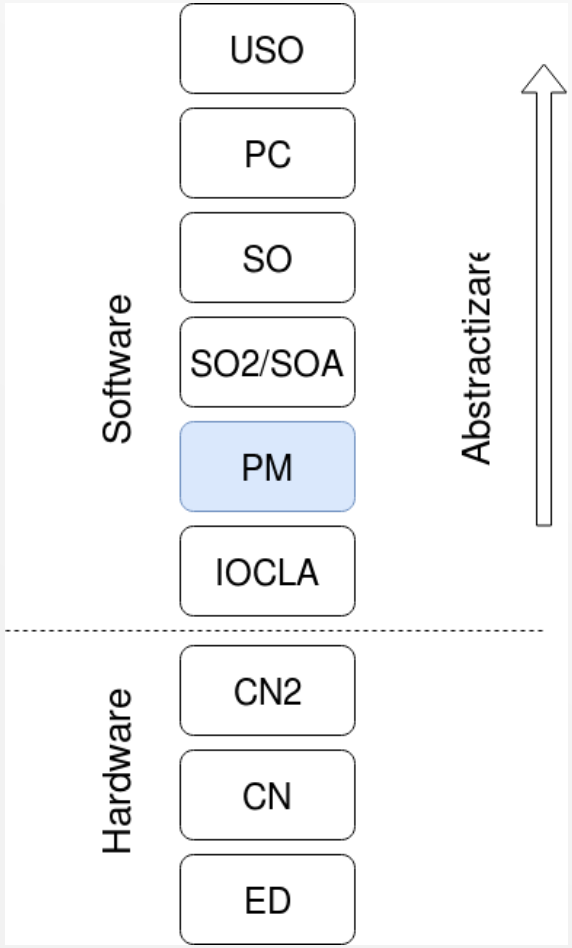


Abstractizare



© Anant Agarwal and Jeffrey Lang, course materials for 6.002 Circuits and Electronics, Spring 2007. MIT

PM @ ACS



- **ED, CN, CN2** - bazele sistemelor de calcul
- **USO** - utilizator al unei abstractzări la nivel de shell
- **PC** se bazează pe abstracțiile de sistem de operare și pe libc
- **SO** se bazează pe ceea ce oferă sistemul de operare
- **SO2** interfațează hardware-ul, dar se concentrează pe a oferi funcționalități nivelelor de deasupra
- **PM** interfațează hardware-ul

POO, PP, PA

PM @ ACS



Perspectiva programatorului asupra felului în care funcționează un sistem de calcul

Cum ajunge un program scris într-un limbaj de nivel înalt să fie executat pe mașina de calcul?

Perspectiva Architectului de Sistem: Cum să proiectez un sistem de calcul care să satisfacă cerințele de design.

Ce se intâmplă la mijloc?



Cum putem să proiectăm un procesor complex pornind de la porți logice și conexiuni?

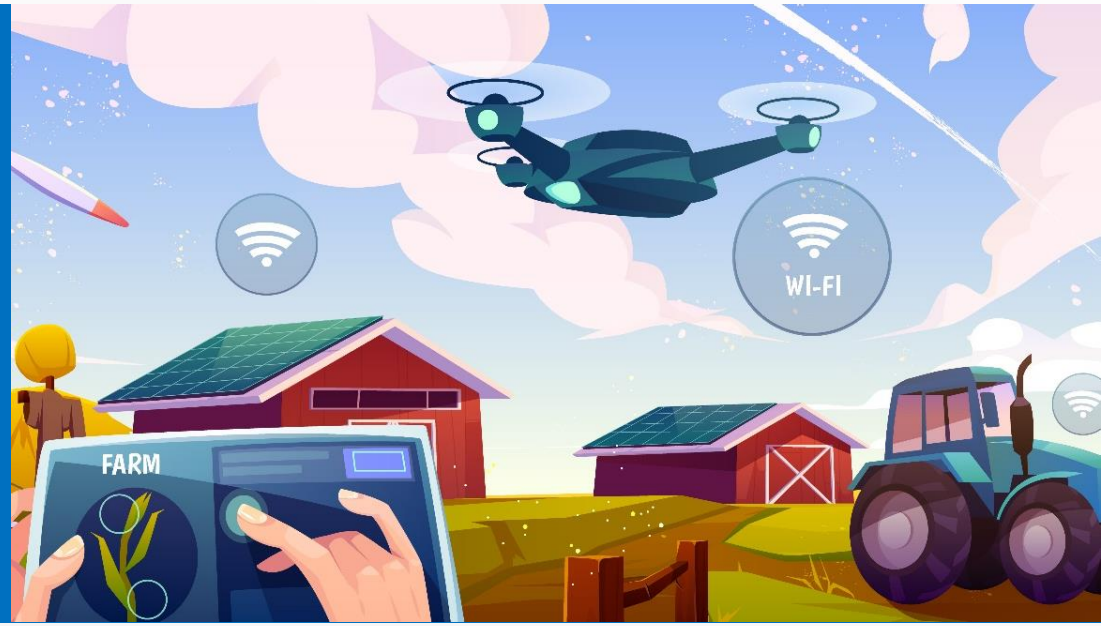
Perspectiva inginerului HW asupra felului în care merge un calculator

CN1 & 2

De ce PM?

- Sisteme de calcul cu microprocesoare > peste tot
- Întrebări pentru un inginer:
  - Ce se află într-un sistem de calcul?
  - Cum interacționează componentele?
  - Cum proiectez un sistem ce interacționează cu mediul fizic?
  - Cum aleg cea mai bună variantă de hardware pentru un sistem embedded ?
- “Data based decisions” – based on IoT infrastructure require:
  - Actual physical sensors
  - Lots of IoT custom hardware

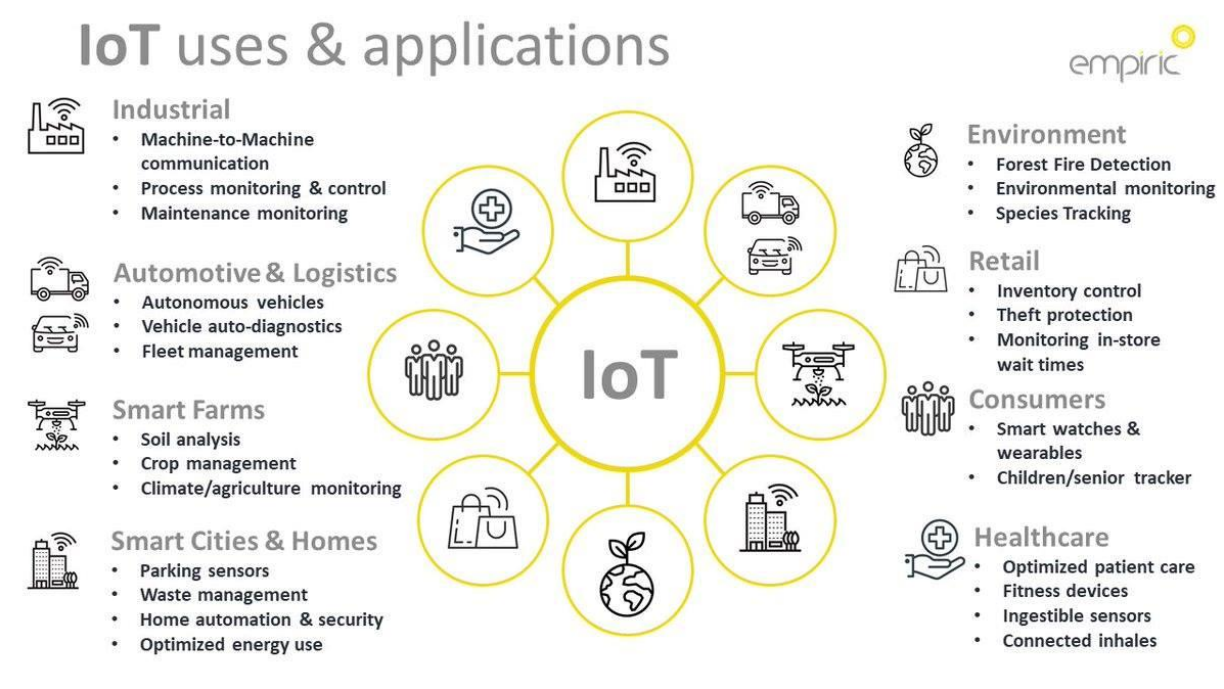
IoT

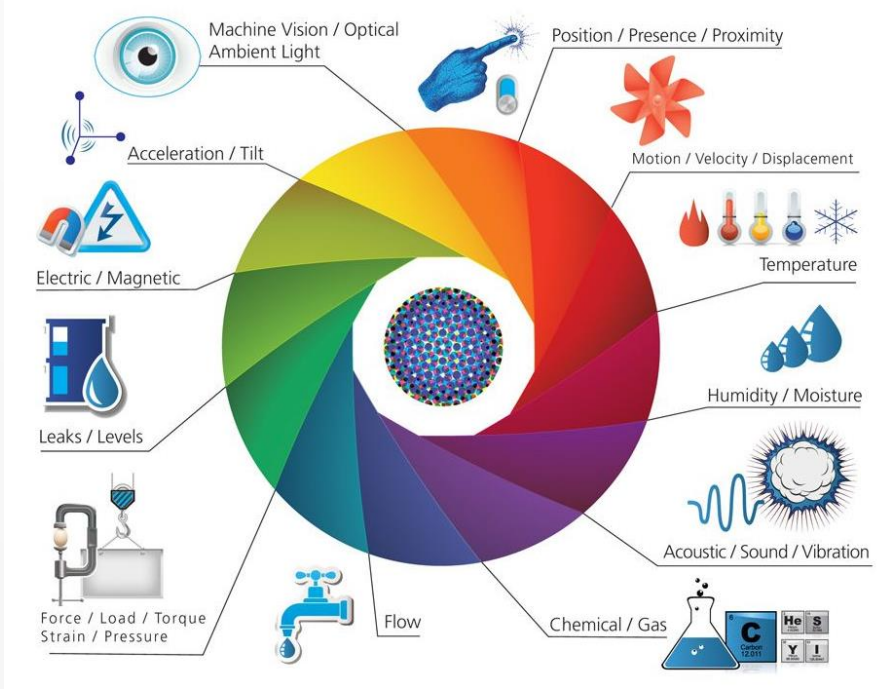


PM

© https://www.freepik.com/vectorpouch

IoT





© https://www.facebook.com/empiricsolutions/photos/the-global-iot-application-platform-and-services-revenue-is-forecast-to-amount-t/2947857835263186/

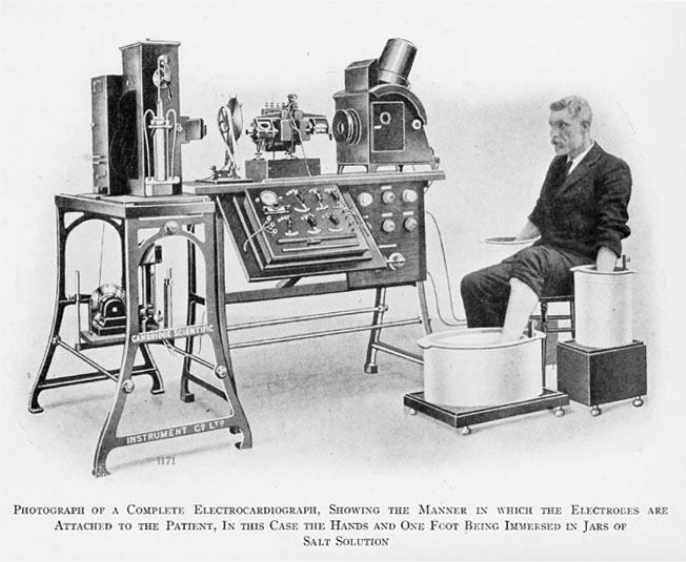
© https://www.postscapes.com/what-exactly-is-the-internet-of-things-infographic/

Unde suntem

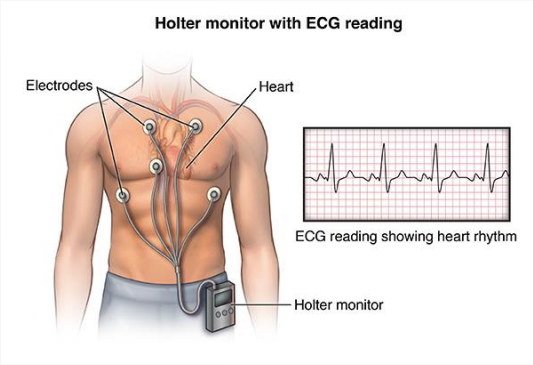




Unde suntem



PHOTOGRAPH OF A COMPLETE ELECTROCARDIOGRAPH, SHOWING THE MANNER IN WHICH THE ELECTRODES ARE ATTACHED TO THE PATIENT, IN THIS CASE THE HANDS AND ONE FOOT BEING IMMEESED IN JARS OF SALT SOLUTION





© https://en.wikipedia.org/wiki/File:Willem_Einthoven_ECG.jpg

https://www.samsung.com/us/apps/samsung-health-monitor/

Embedded systems (sisteme incorporate)

- În general, au o funcție dedicată;
- Constrângeri uzuale:
  - Cerințe de timp real
  - Timp fix de raspuns:
    - control (ex: eșantionare la timp constant)
    - safety (raspuns în timp limitat la detecție)
  - Resurse limitate (putere de procesare / memorie)
  - Cerințe de robustețe (aka up-time >>)

Exemple



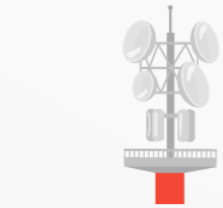
PM

© https://www.freepik.com/vectorpouch

Calculatoare și
Tehnologia Informației

Ingineria Sistemelor -
Automatică și Informatică Aplicată

Discutie



Data transmission



Automatic Control



Automatic Control



Signal Processing

Huma Computer
Interaction

Computer Graphics

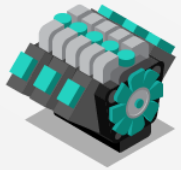






Embedded
Programming

Automatic Control





Automatic Control

Automatic Control

Embedded
Programming

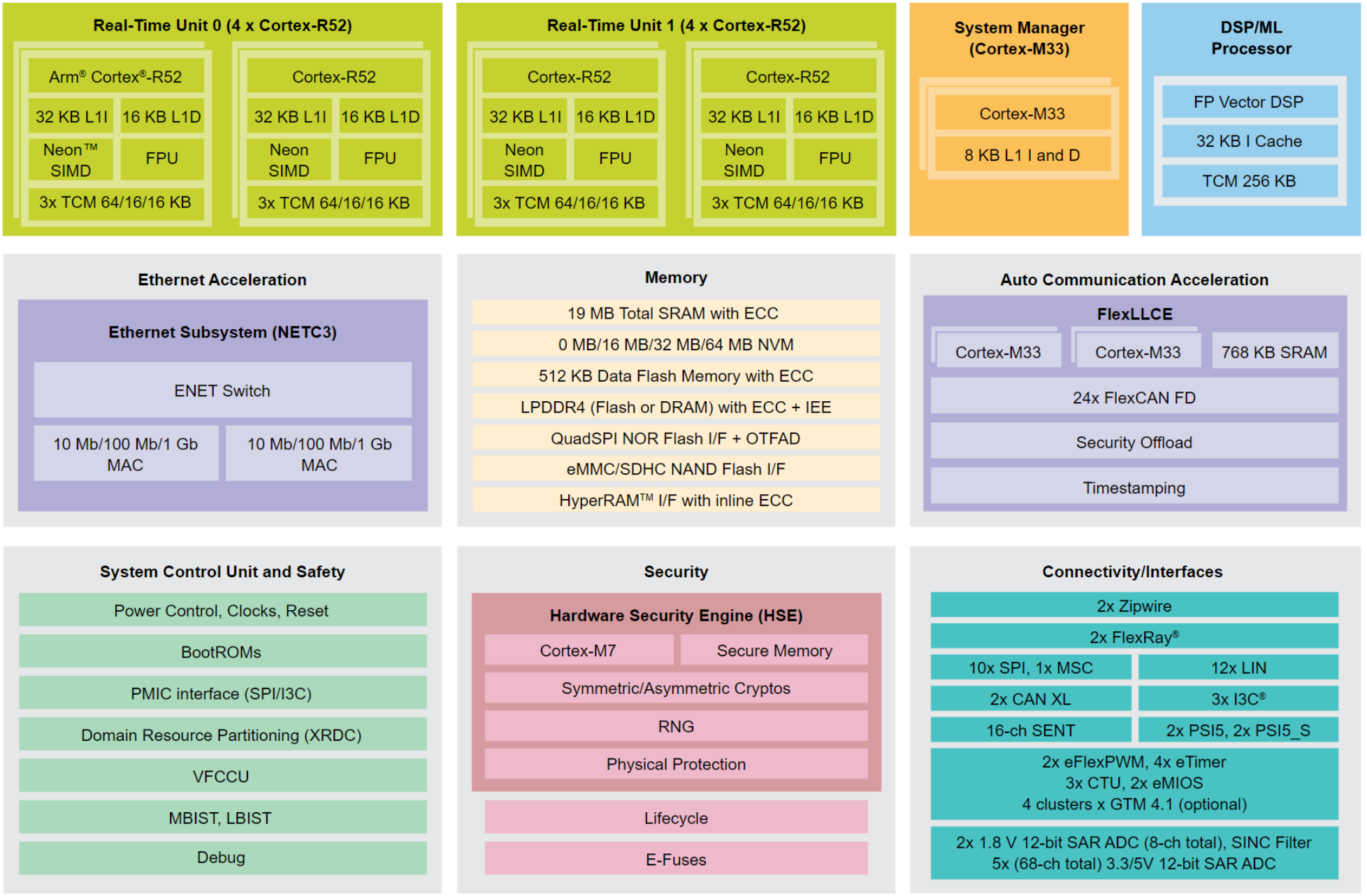




Communication
Protocols

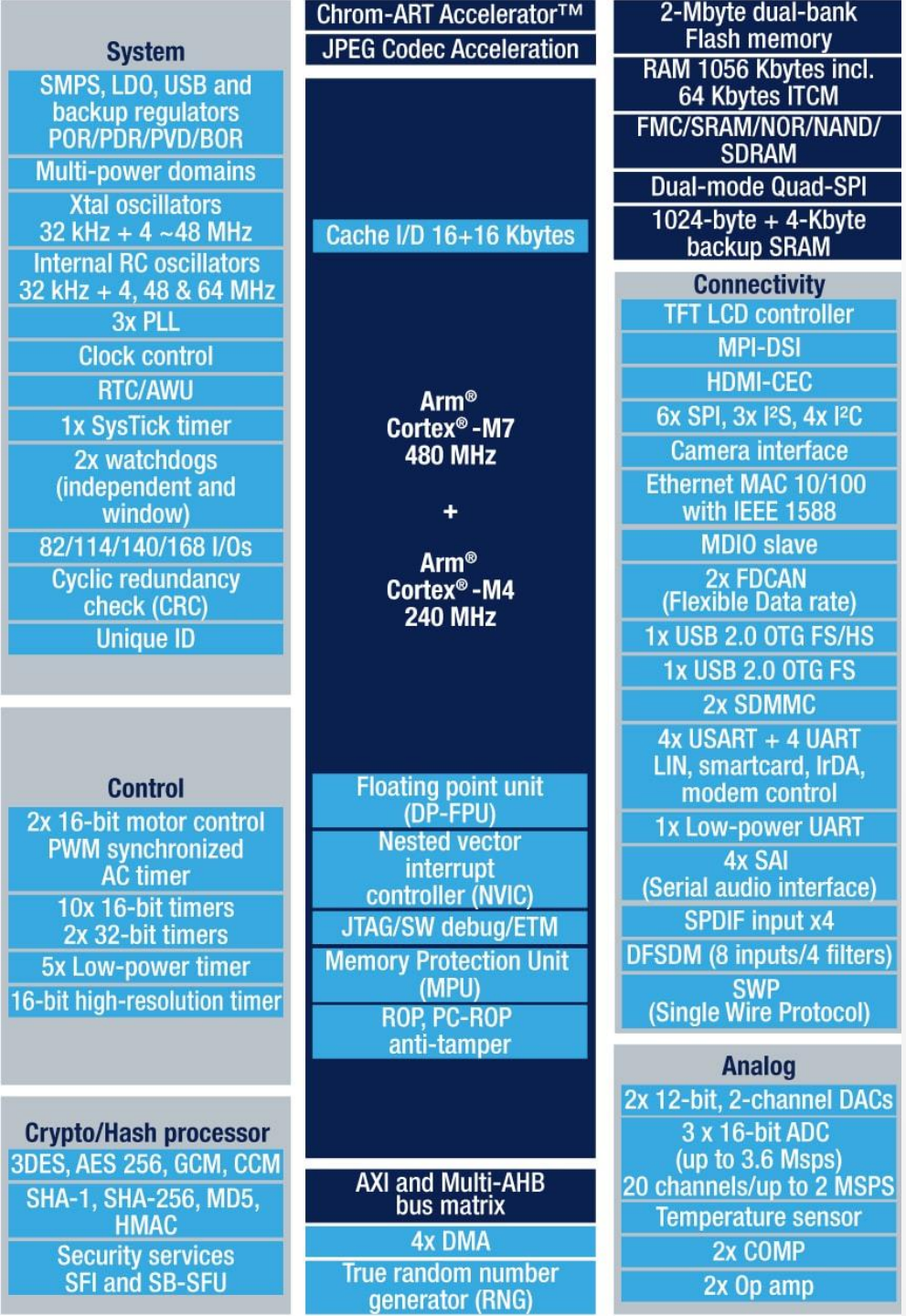
Cloud Integration

Discutie



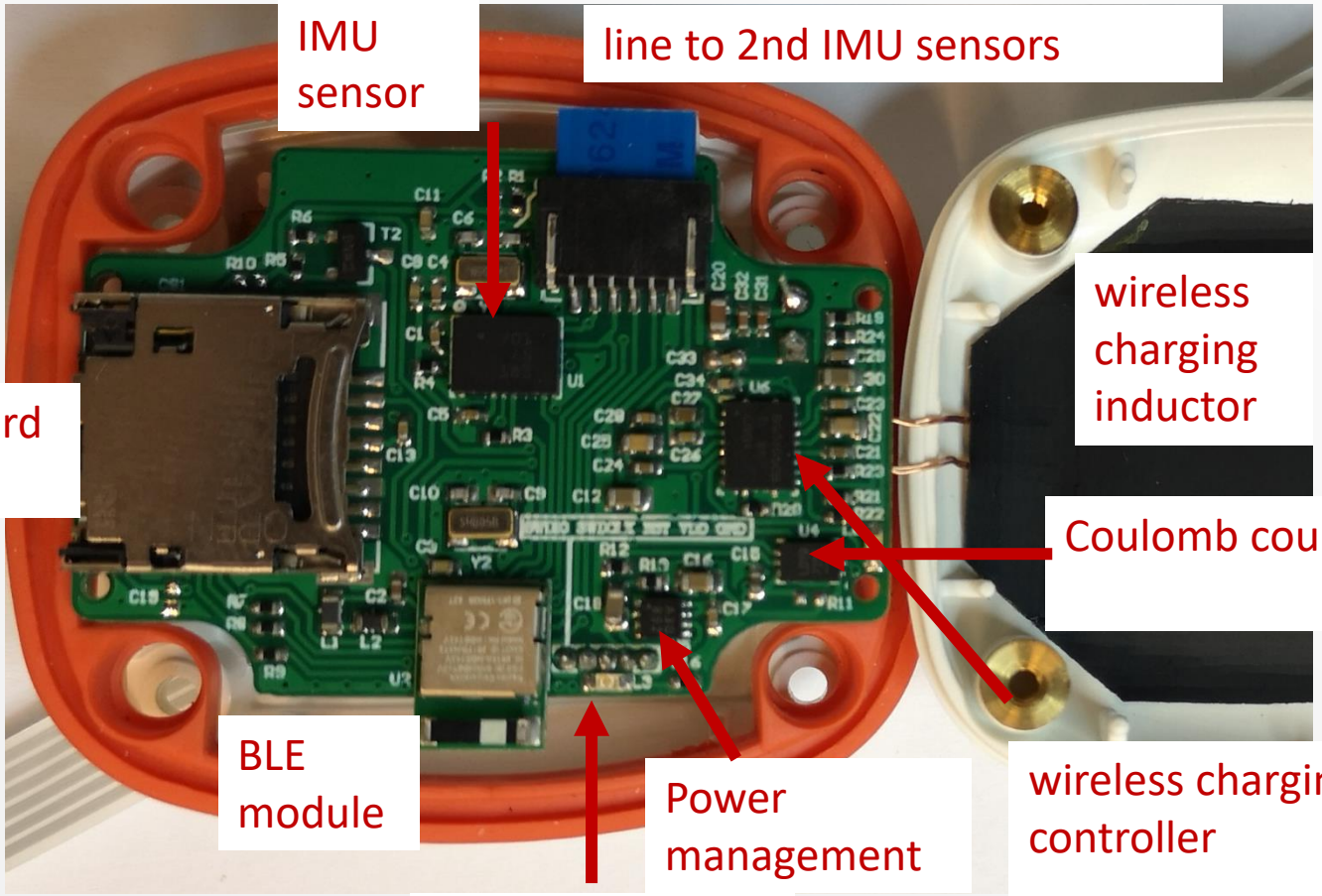
© https://www.nxp.com/products/processors-and-microcontrollers/s32-automotive-platform/s32z-and-s32e-real-time-processors/s32e2-safe-and-secure-high-performance-real-time-processors-with-actuation-support:S32E2

Discutie

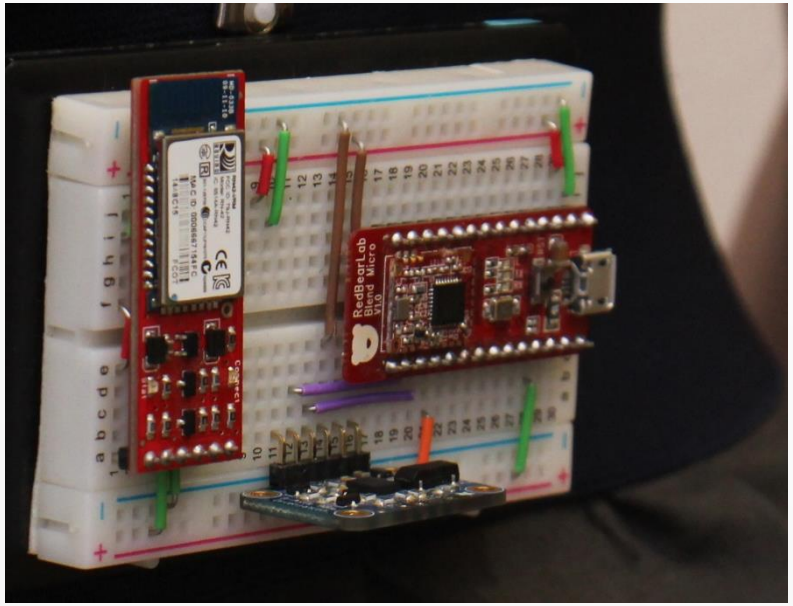


https://www.st.com/en/microcontrollers-microprocessors/stm32h757zi.html

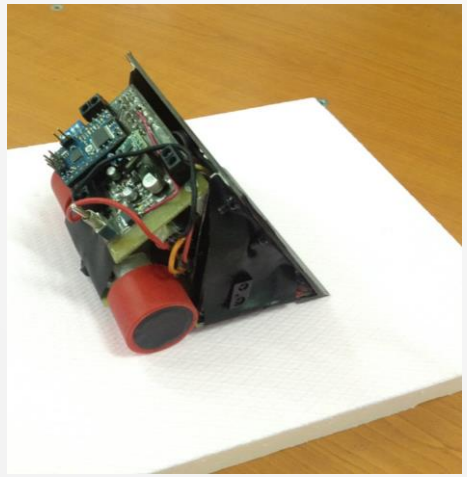
Exemple - ENTy

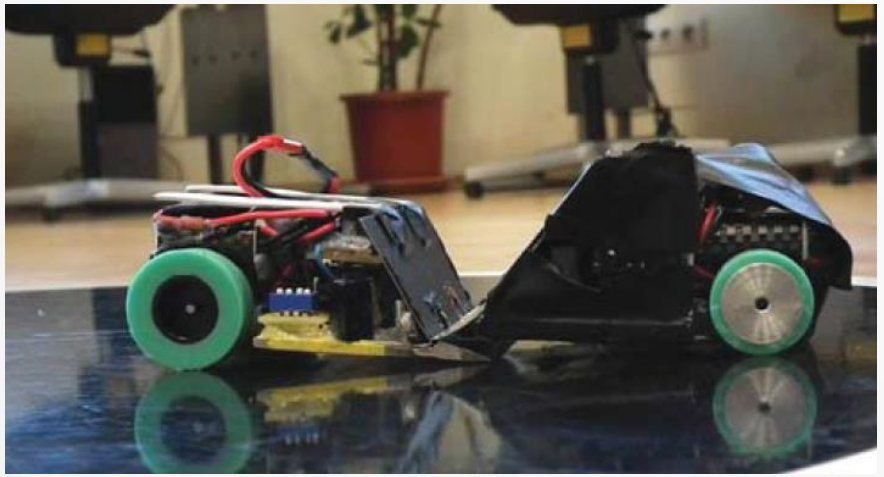


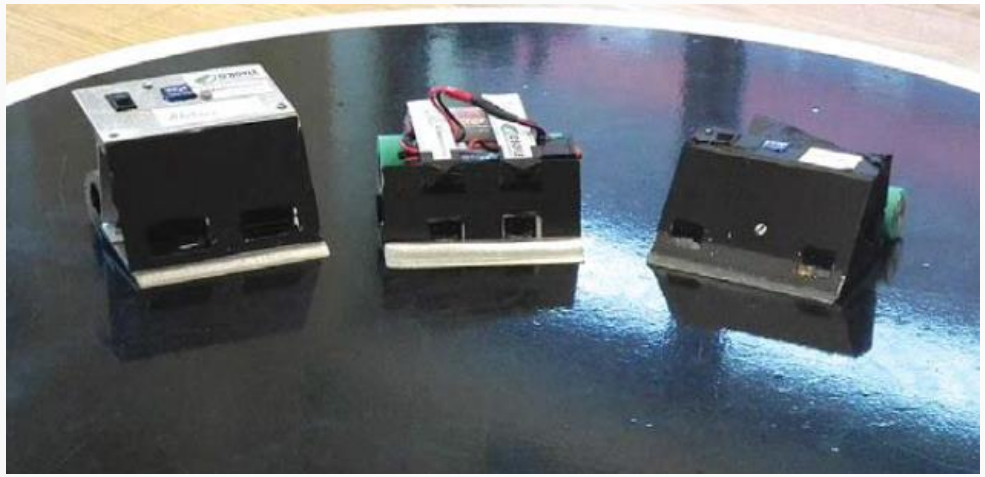




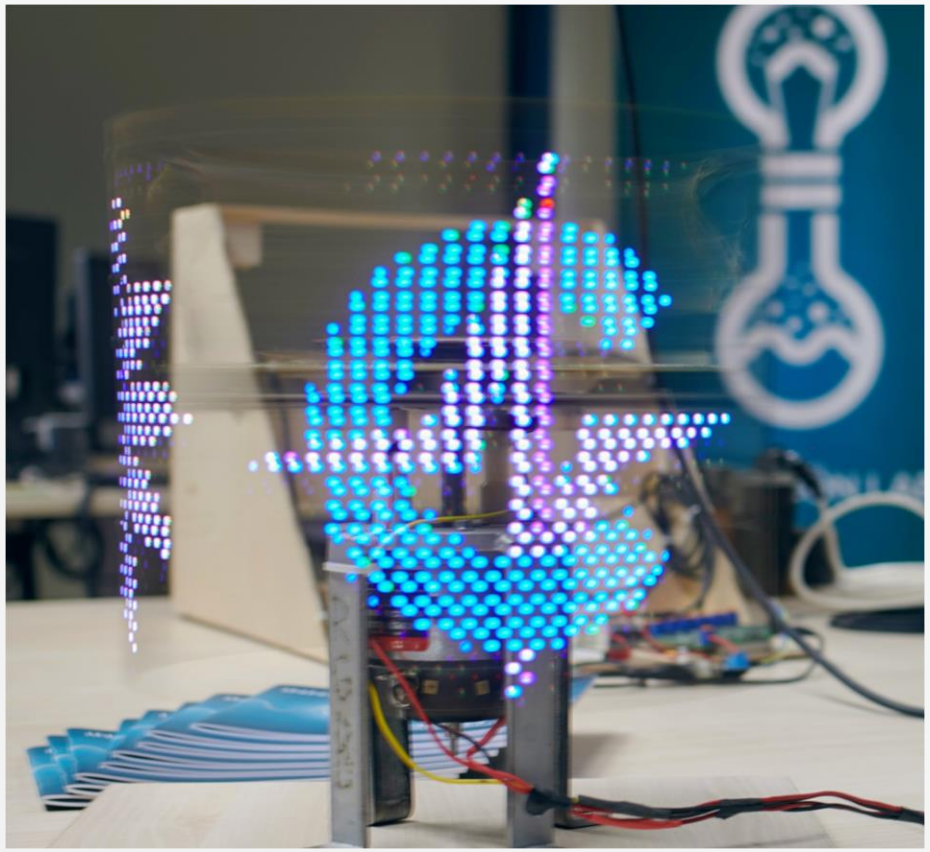
Exemple – mini-sumo

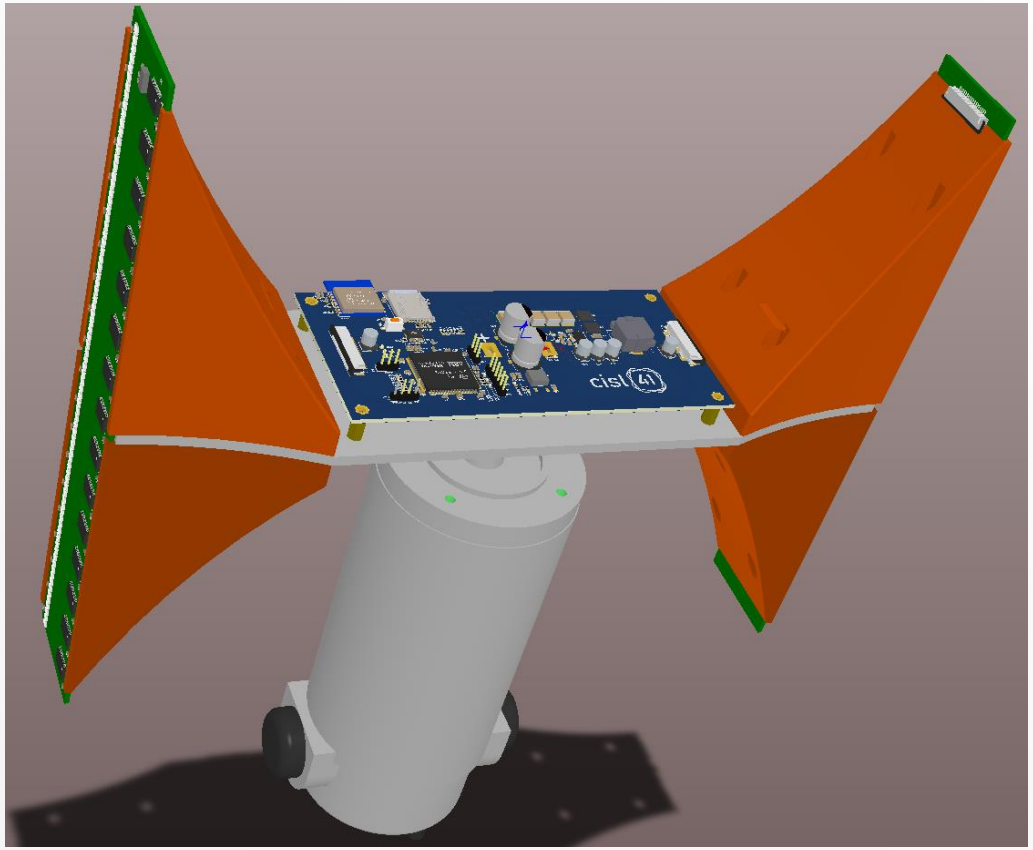






Exemple – POV





Exemple companii - angajator (în RO)



Echipa de PM



PM

© https://www.freepik.com/vectorpouch

Asistenții











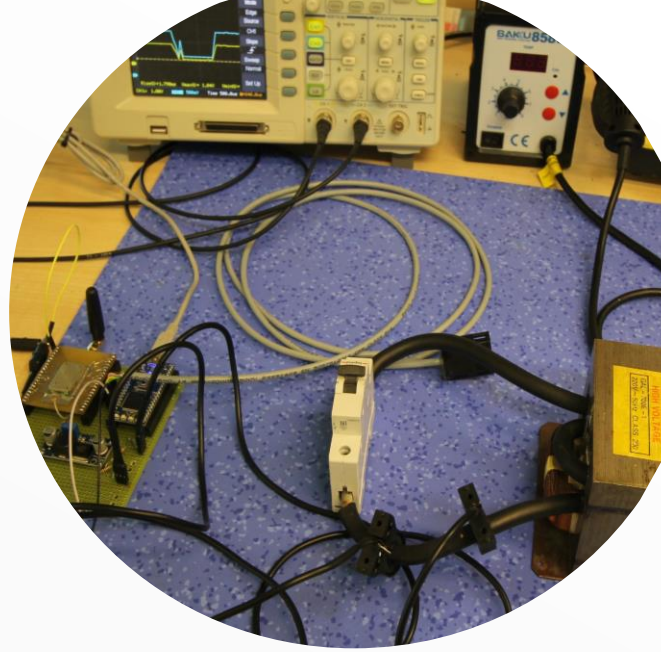


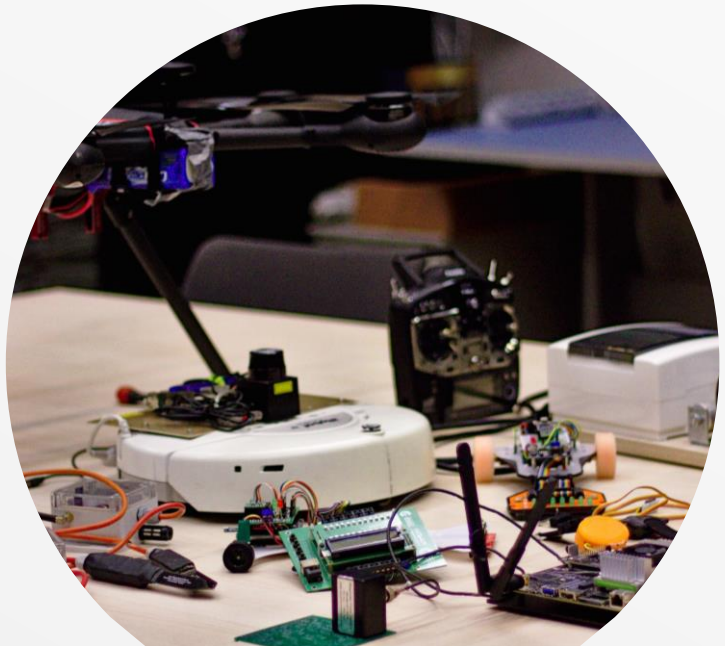


About me

- Cursuri
  - (II) Electronică Analogică
  - (III) Proiectarea cu Microprocesoare
  - (MI/MII) Sisteme Fizico-Cibernetice
- VZ & PoliFest
- Innovation Labs
- Concursuuuuri (tech)
- Tech
  - MedTech
  - Energy
- C310
- daniel.rosner@upb.ro









Structură



PM

© https://www.freepik.com/vectorpouch

Curs

- Prezentarea diferitelor periferice, metode de comunicare
- Metode de programare pentru microcontroller-ul de laborator
- Alte variante hardware
- Proiectarea unei plăci de bază, bazată pe 8086
- ... și bineînțeles o serie de surprize plăcute

Laborator

- 7 săptămâni – laborator:
  - Aplicații introductive
  - USART, LCD
  - Întreruperi, Timere
  - Timere, PWM
  - SPI. FAT Filesystem. Player Audio
  - ADC. I2C. LCD grafic
- 7 săptămâni – proiect:
  - work work work

Structură punctaj



PM

© https://www.freepik.com/vectorpouch

Structură punctaj

- 1 punct activitate laborator
- 1 punct lucrare laborator (colocviu final)
- 3 puncte proiect
- 3 puncte examen final
- 2 puncte activitate curs (teste/ activitate)
- + 0.75 bonus top 30 de proiecte din an (top 7%)
- + 0.75 bonus top 10 proiecte din an

Reguli promovare

- 50% parcurs;
- 50% examen;
- 50% prezențe laborator;
- 7 prezente la curs pentru a intra in examen;

- Obs curs:
  - nu tinem pe 17 Aprilie, 1 Mai si 29 Mai (liber + ult saptamana pt preg proiect);
  - Ultimile 4 cursuri practice sunt Q&A din proiect – fara prezenta;
  - => 14 cursuri cu prezenta
  - Punctajul maxim pe activitatea la curs se poate obtine din 11 cursuri;

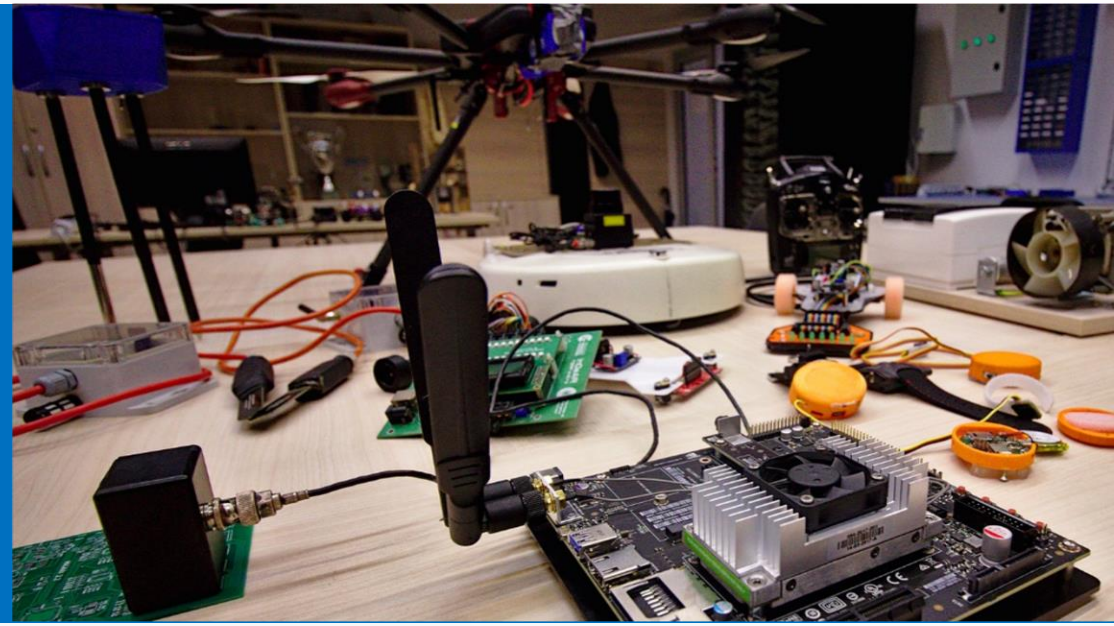
Extra

- Bonus pentru rezultate la concursuri & activități
  - până la 1 punct pentru rezulate in top la concursuri de profil tehnic
  - până la 0,5 puncte bonus pentru implicare în activități studențesti de voluntariat
  - *> mail în presesiune cu Subject: [Bonus_PM] Nume_Prenume_32xCC*

Echivalări

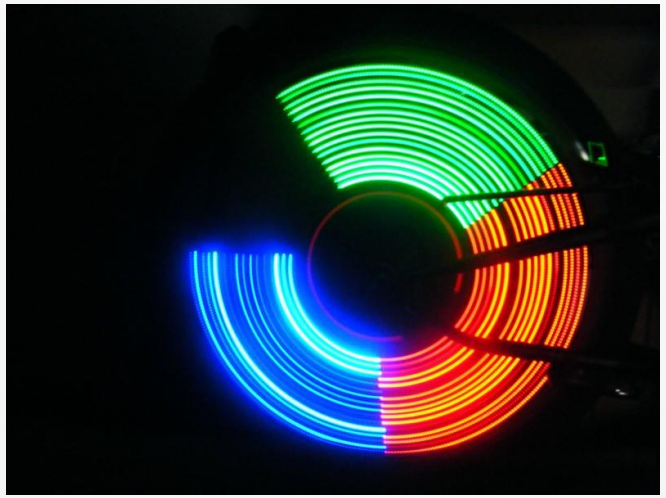
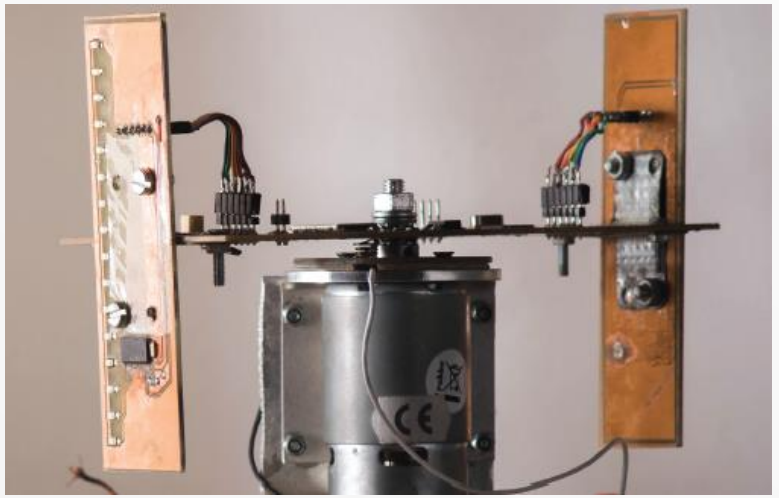
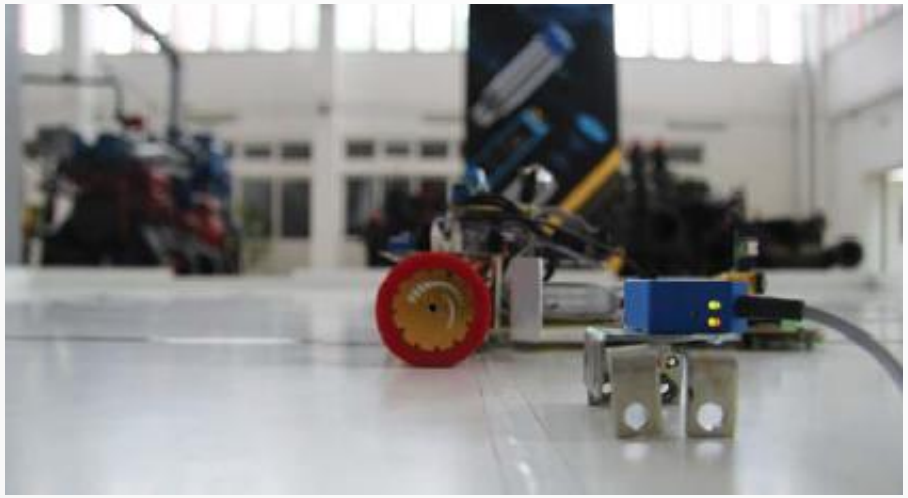
- Pana la 3 puncte pentru rezultate la concursuri tehnice. Ex:
  - ACM (top 50% la world finals);
  - Innovation Labs (SemiFinale);
  - Suceava Hard and Soft (top 50% echipe);

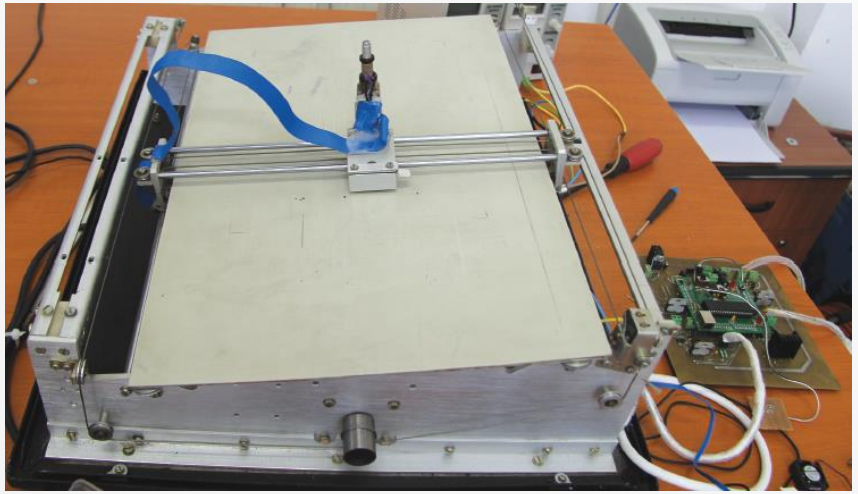
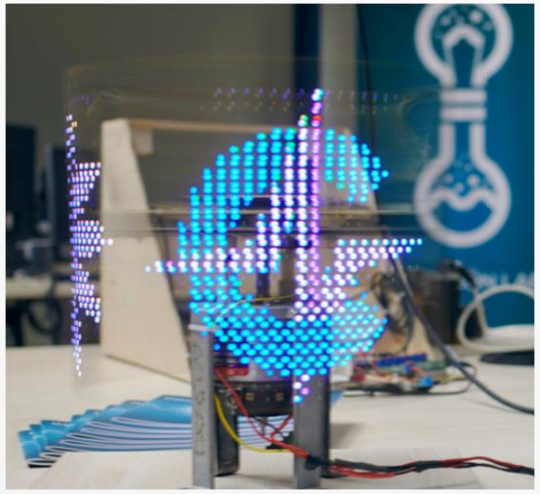
Despre proiect



PM

Ce & de ce

Structură punctaj proiect

- **3p** din punctajul final

24-Apr-2023	28-Apr-2023	Proiect	Confirmare tema proiect + cine vrea sa invete sa lipeasca
1-May-2023	5-May-2023	Proiect	Milestone - documentatie
8-May-2023	12-May-2023	Proiect	Laborator help lipit
15-May-2023	19-May-2023	Proiect	Milestone - Hard
22-May-2023	26-May-2023	Proiect	Milestone - Soft, all done
29-May-2023	2-Jun-2023	Prezentare Proiecte	PM Fair

Pentru orice milestone intarziat – maxim 70% din punctaj

	"proiecte hardware heavy"	"proiecte software heavy"	proiect "echilibrat"
Milestone - documentatie	0.4	0.4	0.4
Milestone - Hard	0.9	0.7	0.8
Milestone - Soft, all done	0.7	0.9	0.8
PM Fair	1	1	1

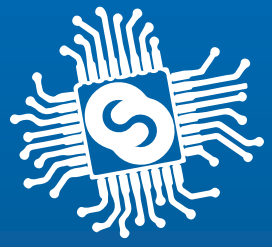
Alegerea temei [S10]

- Tema trebuie să fie aprobată de către asistentul de laborator
- Nu se aprobă teme banale; ex:
  - Ceas digital, termometru digital;
  - Ca reguli de referință:
    - Nu poate să fie mai simplu decât un laborator de PM ☺
    - Nu poate să fie bazat pe un tutorial youtube de 15 minute ☺
- ToDo pentru S10
  - Temă
  - Schemă bloc

La cursul următor

- Istoria arhitecturilor de calcul
- Aplicații microprocesor
- Recapitulare electronica
- Documentația hardware
- GPIO
- Arduino
- Hello World

Cursul #2&3



PM

Disclaimer

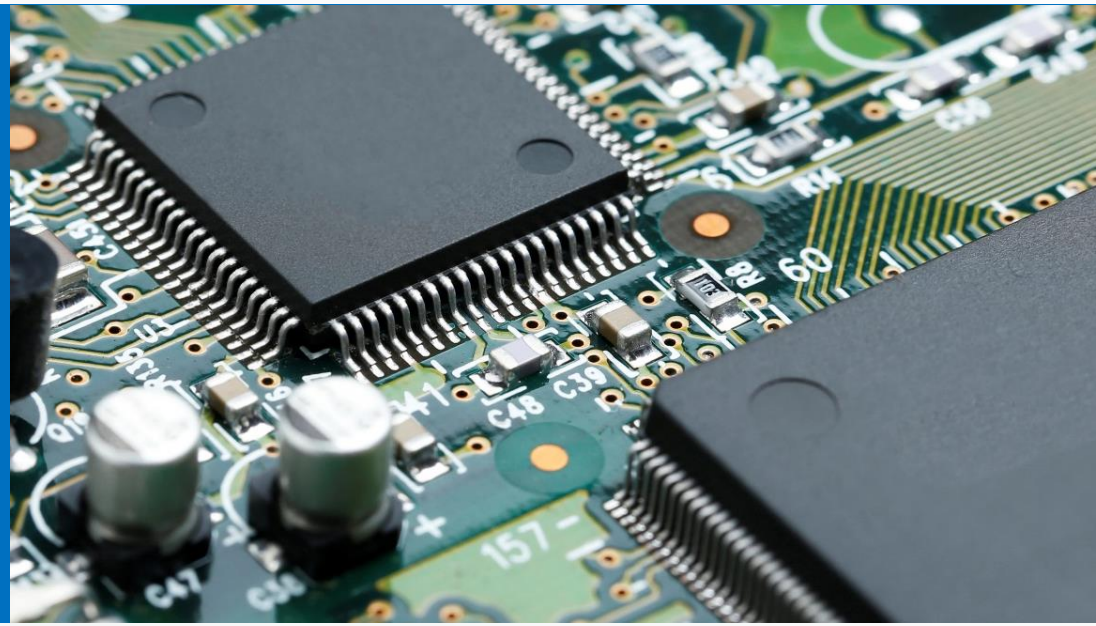
- *Slide-urile acestea reprezintă un rezumat.*
- *Slide-urile nu acoperă toate explicațiile, simulările, demonstrațiile oferite în timpul cursului.*
- *Slide-urile nu limitează în vreun fel materia cerută la examen.*
- *Pentru varianta completă, sunteți așteptați la curs.*

© Elemente din curs preluate din:
- *notițe curs PM 2021, Dan Tudose*
- *notițe curs PM 2021, Andrei Voinescu*
- *scoala de vara eap 2016, Marius Nicolae*
- *notite curs EEA/ED –Razvan Tataroiu, Daniel Rosner*

Cuprins

- Microprocesoare vs microcontrolere
- Scurta istorie a microprocesoarelor
- Scurta istorie a microcontrolerelor
- Familia AVR
- ATMega 328P
- Logica digitala
- GPIO
- Software programming
- Next time

Microprocesoare vs Microcontrolere



PM

Ce este un microprocesor?

- Un dispozitiv electronic pe o singură plachetă de siliciu care execută instrucțiuni
- Întrebări cheie:
  - De unde încarcă instrucțiuni?
  - De unde preia intrările?
  - Unde stochează ieșirile?

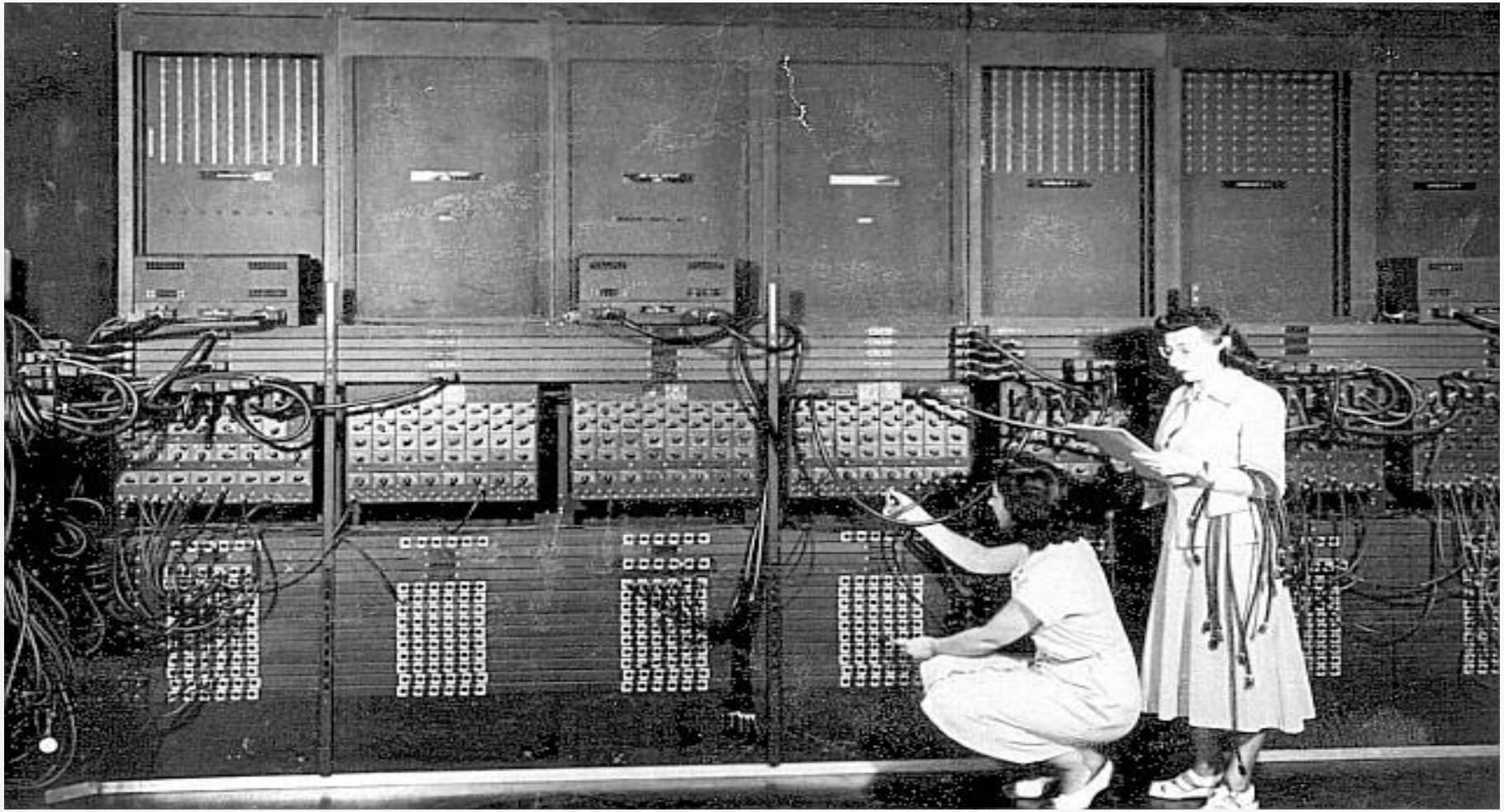
Arhitecturi principale

- Din punctul de vedere al accesului la memorie, sunt 2 arhitecturi:
  - von Neumann, unde memoria conține atât instrucțiuni cât și date.

    PC-urile actuale sunt toate von Neumann
  - Harvard, unde accesul la memorie se face pe magistrale separate, una pentru date, una pentru instrucțiuni.

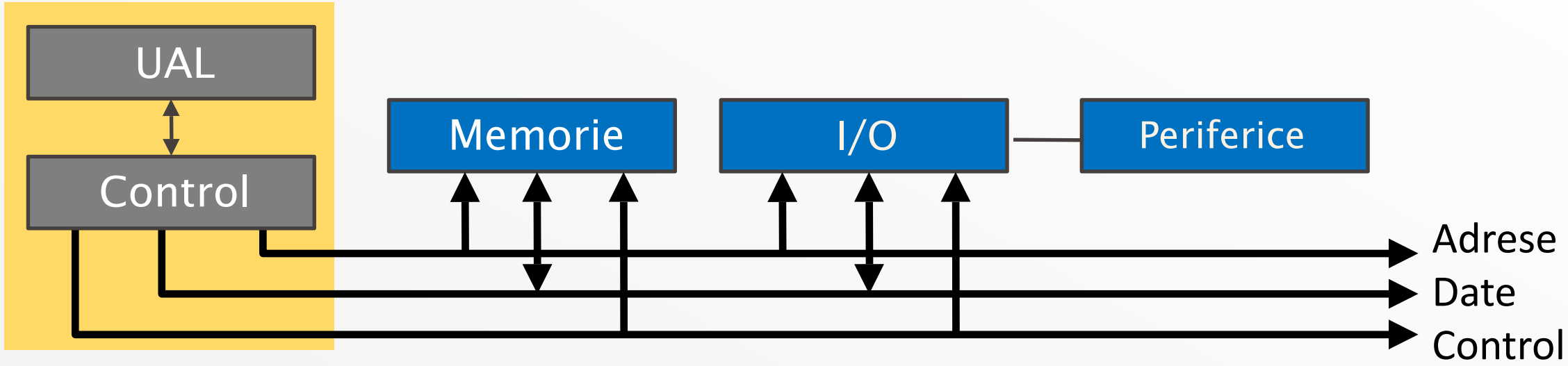
    Microcontrollere AVR de la laborator sunt Harvard

Viața înainte de von Neumann



Microprocesoare

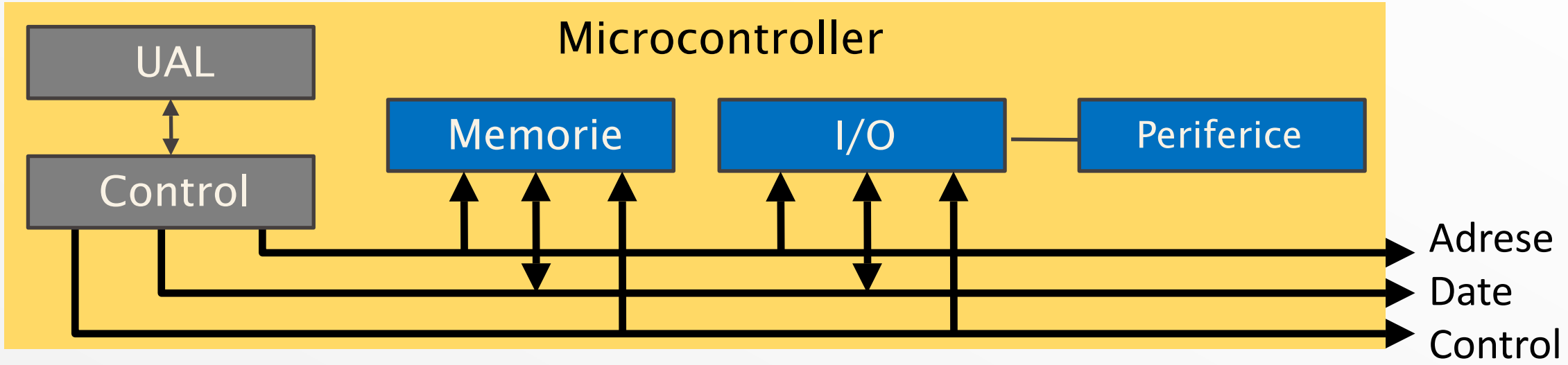
Microprocesor



High performance, general purpose, pentru PC & workstations
Instruction decode and control, arithmetic/logic operations, registers, timing, external control
Cost tipic: $75 - $500

Cerere anuală: zeci de milioane

//versus - Microcontroler



Nivel mult mai mare de integrare pentru controlul embedded

Funcții de microprocesor, plus memorie și periferice on-chip (ex. porturi, timere)

"Swiss army knife" al tehnologiei

Cost tipic: $0.1- $25

Cerere anuală: miliarde!

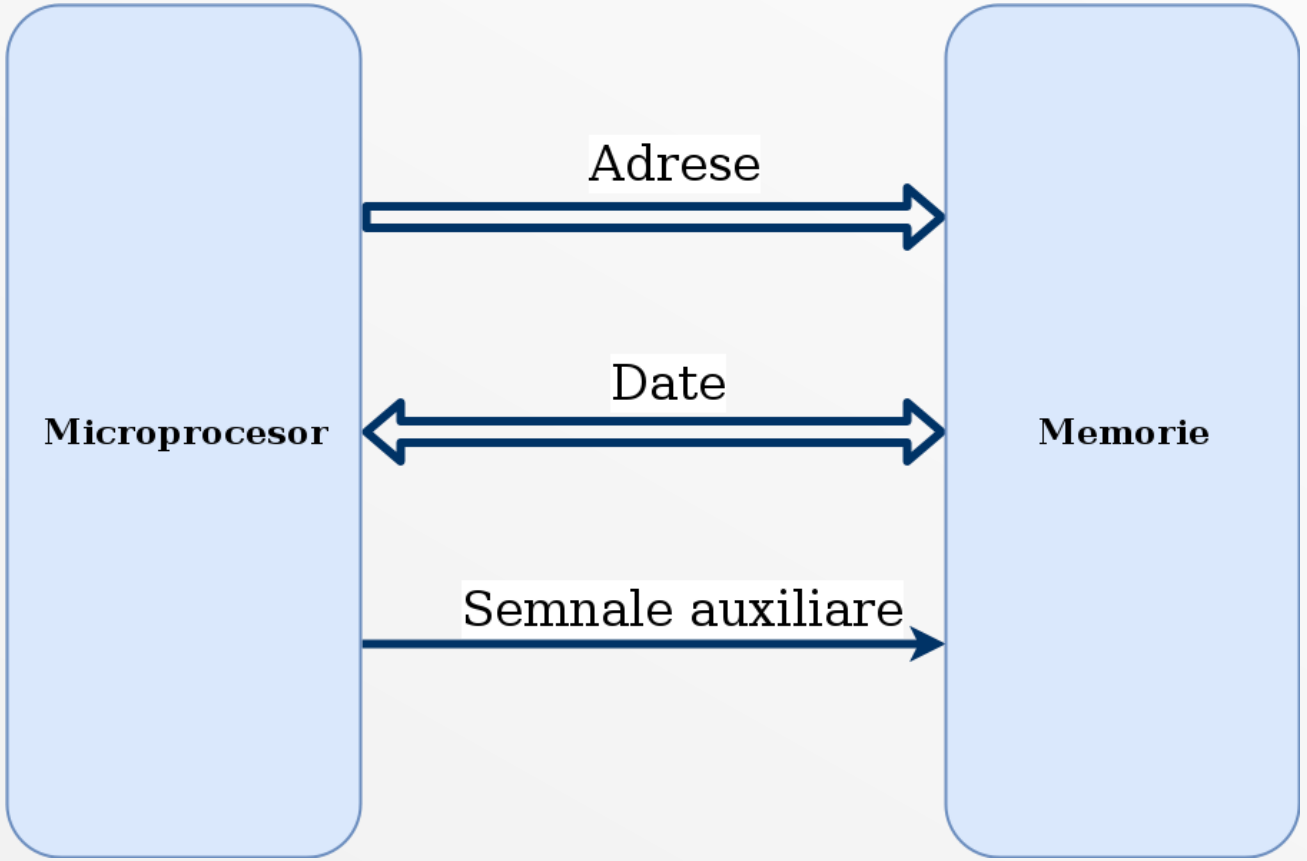
Necesitatea unei motherboard

- Un microprocesor are nevoie de:
  - Memorie
  - Periferice I/O
  - Circuit de ceas
  - Circuit de RESET



Ce este o magistrală?

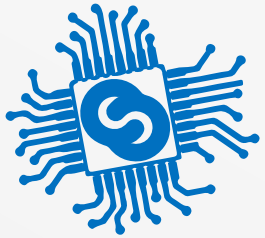
- Magistrala asigură transferul de date bidirecțional între procesor și o locație din memorie
- Accesul procesorului la memorie are loc doar prin magistrală



Cum se face I/O pe un microprocesor?

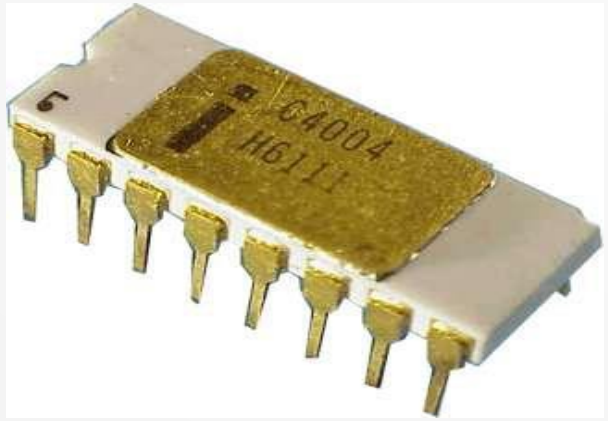
- Sunt două metode de a accesa periferice I/O:
- Memory-mapped I/O:
  - Perifericul I/O are o adresă, registrele lui sunt accesibile la acea adresă
- Port-mapped I/O:
  - Microprocesorul are instrucțiuni speciale de acces al perifericelor I/O, care se traduc la exteriorul chip-ului prin alte semnale auxiliare decăt cele pentru citire/scriere din/ în memorie

Scurtă istorie & exemple - microprocesoare



PM

INTEL 4004



- **1971**
- First Intel uP
- Word width: 4-bit
- 2300 transistors
- Clock: 108KHz/500/740
- 46 instructions
- Registers: 16 x 4-bit
- Stack: 12 x 4-bit
- Address space
  - 1Kb of program, 4Kb of data

INTEL 4040



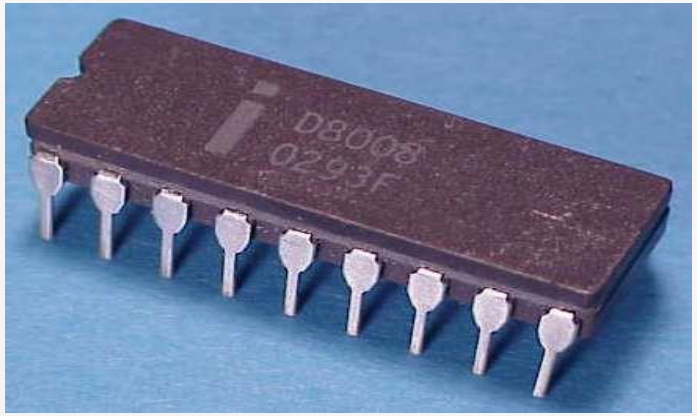
- **1974**
- 60K instructions/sec.
- 3000 transistors

Support chips:

- 4201 – Clock Generator 500 to 740 kHz using 4 to 5.185MHz crystals
- 4269 - Programmable keyboard/display
- 3216 and 3226 4-bit parallel bus drivers

Obs:
Fairchild Semiconductors >>
Busicom >> <<

INTEL 8008



- **1972**
- First 8-bit
- Clock: 800KHz
- 3500 transistors
- 48 instructions
- Registers: 6 x 8-bit
- Stack: 17 x 7-bit
- Address space: 16KB

INTEL 8080



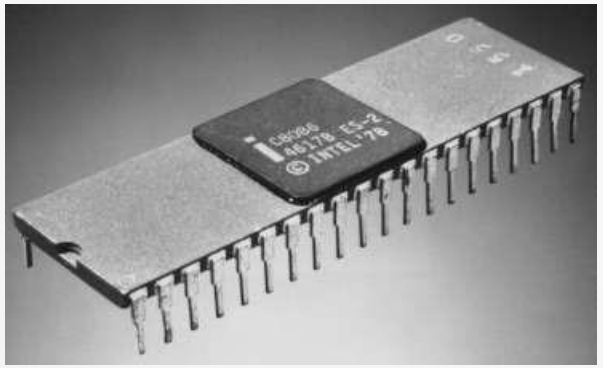
- **1974**.
- 2 MHz.
- 6,000 transistors
- 10 times faster than 8008.
- Address space: 64KB
- Registers: 6 x 8-bit
- Distinct Address & Data Buses

INTEL 8085



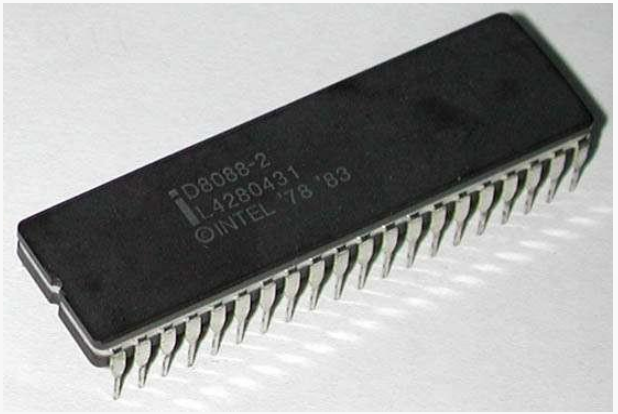
- **1976**
- 3MHz
- 8-bit data bus 16-bit address bus
- 6,500 transistors.
- 246 instructions
- > 100 million copies sold

INTEL 8086



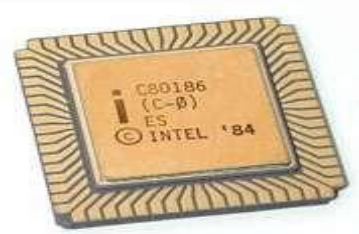
- **1978**
- First 16-bit μP
- 4.77 MHz, 8 MHz and 10 MHz
- 16-bit data bus and 20-bit address bus
- 29,000 transistors
- 2.5 MIPS
- Access - 1 MB of memory
- Multiply and Divide instructions

INTEL 8088



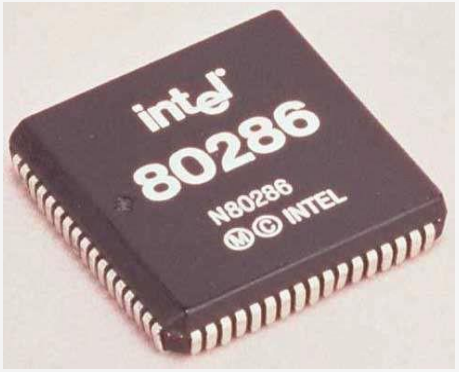
- **1979**.
- cheaper 8088
- **> primul PC IBM**

INTEL 80186



- **1982**
- 6MHz

INTEL 80286



- **1982**
- 8MHz
- 16-bit data bus and 24-bit address bus
- 16 MB of addressable memory
- 4 MISP

INTEL 80386



- **1986 - 2007**
- First **32-bit** µP
- 32-bit data bus & 32-bit address bus
- **4 GB of addressable memory**
- 2,750,000 transistors
- Clock speed varied from 16 MHz to 33 MHz depending upon the various versions.

INTEL 80486



- **1989**
- 1.2 million transistors
- 8 KB of cache
- 16 MHz to 100 MHz

Pentium



- **1993 –**
- Original name 80586
- 66Mhz
- 32-bit data and address bus
- 4GB of memory
- Cache memory
  - 8KB data
  - 8KB Instructions

Intel core i3/5/7/9

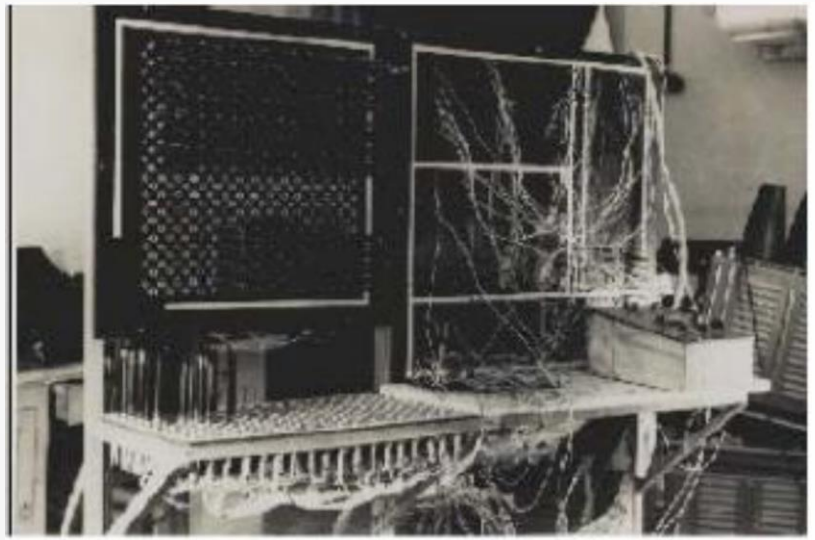


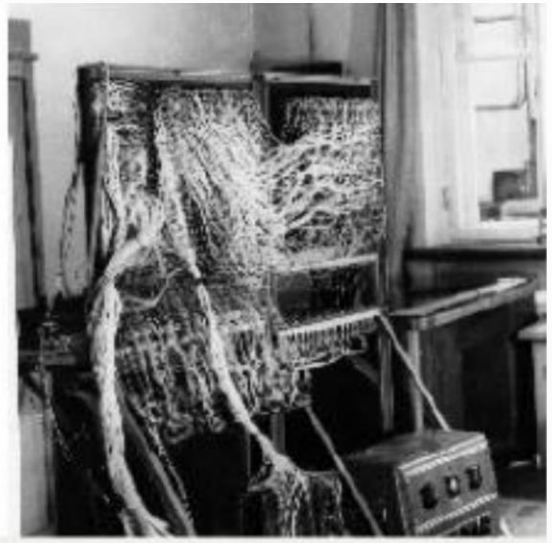
- **2006 –**
- 4 cores (i7, i5) 2 cores(i3), 10-18 (i9)
- 781 – 7 billion transistors
- 64 KB of L1 cache per core, 256 KB of L2 cache and 8 MB of L3 cache

Calculatoare proiectate de membrii Departamentului Calculatoare @ UPB

Electrointegrator pentru soluționarea unor probleme de câmp.

Adrian Petrescu (1963)





MAC-1 Calculator Analogic cu 30 de amplificatoare operaționale.

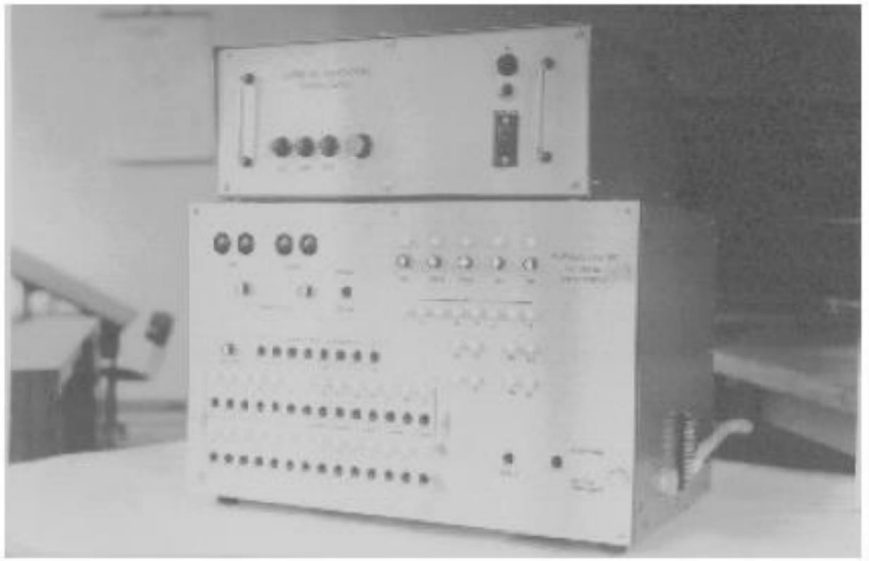
Adrian Petrescu, Petre Dimo, Ivan Sipos (1965)



Calculatoare proiectate de membrii Departamentului Calculatoare @ UPB

Microcalculatorul MC-1

Adrian Petrescu, Nicolae Țăpuș, Trandafir Moisa (1972)



Microcalculatorul FELIX MC-8

Adrian Petrescu, Nicolae Țăpuș, Trandafir Moisa (1975)



Calculatoare proiectate de membrii Departamentului Calculatoare @ UPB

Microcalculatorul FELIX M-18

Adrian Petrescu, Nicolae Țăpuș, Trandafir Moisa (1978)



Microcalculatorul aMIC

Adrian Petrescu, Francisc Iacob (1984)



Calculatoare proiectate de membrii Departamentului Calculatoare @ UPB

Microcalculatorul FELIX HC-85

Adrian Petrescu, Francisc Iacob (1985)

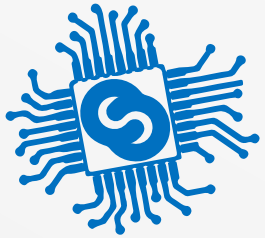


Microcalculatorul FELIX-PC

Adrian Petrescu, Nicolae Țăpuș, Trandafir Moisa, Irina Athanasiu (1985)

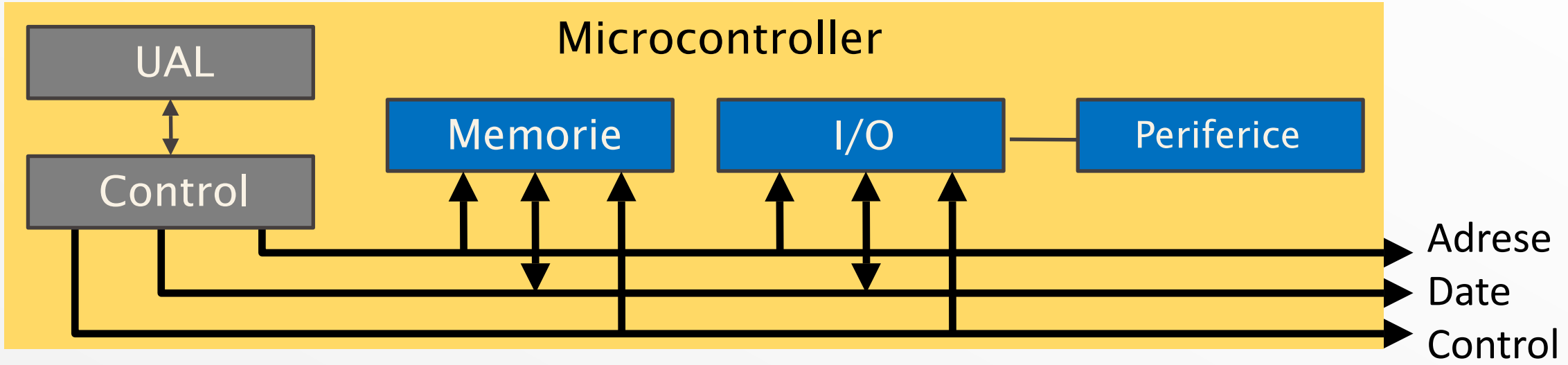


Scurtă istorie & exemple - microcontrolere



PM

Microcontroler



Nivel mult mai mare de integrare pentru controlul embedded

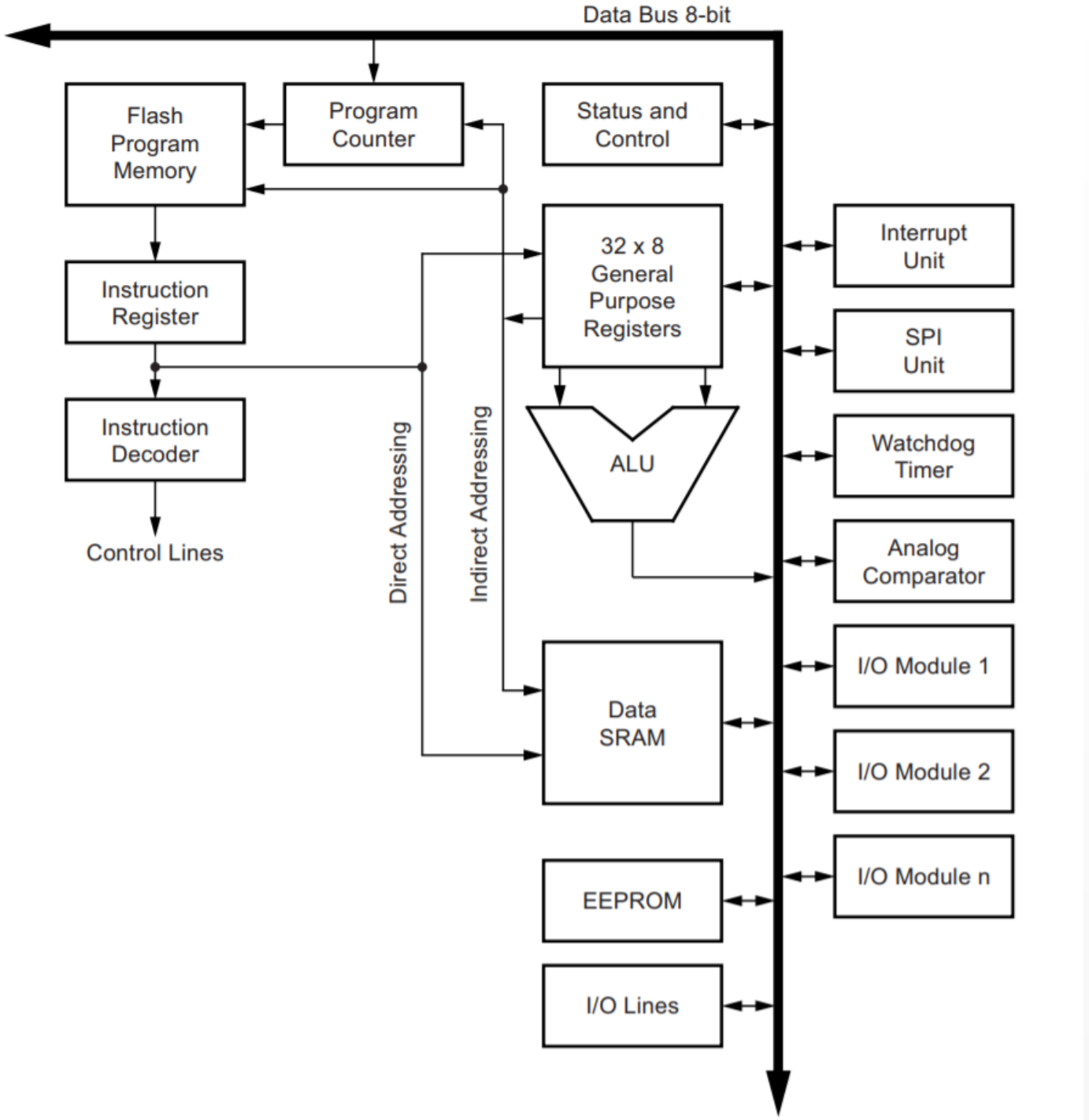
Funcții de microprocesor, plus memorie și periferice on-chip (ex. porturi, timere)

"Swiss army knife" al tehnologiei

Cost tipic: $0.1- $25

Cerere anuală: miliarde!

Schemă bloc de referință



Microcontroler

- microcontroller (MCU) – un mini calculator pe un singur cip de siliciu ce integrează:
  - procesorul
  - Memoria de date
  - Memoria de program
  - periferice



- *În contrast cu un microprocesor ce are nevoie de alte cipuri externe pentru memorie, control, periferice*

INTEL P8051



- **1980**
- 8-bit
- 12 MHz

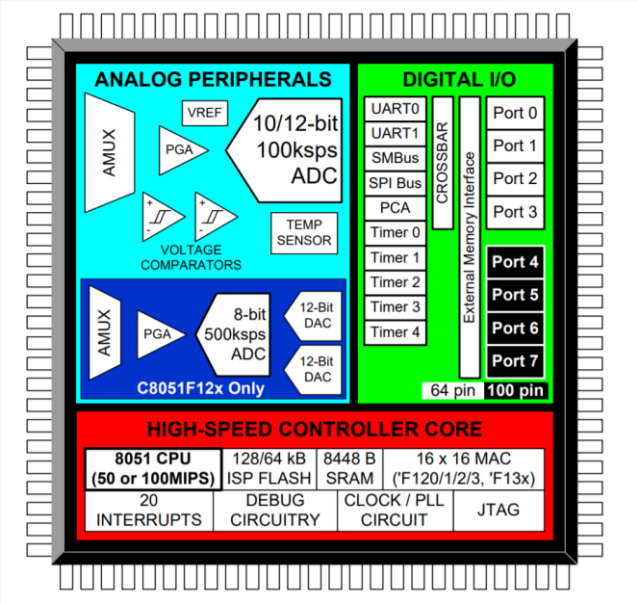
versiuni curente (2022)

AT89C5131A-L



High-performance flash version of the 80C51 single-chip 8-bit microcontroller with full speed USB functions.

C8051F131-GQ



MCU 8051 100 MHz 128 kB MAC 8-bit MCU

https://ro.mouser.com/datasheet/2/368/C8051F12x-13x-1397919.pdf

MSP430 (TI)



- Dezvoltat în 1993
- Arhitectură von-Neumann pe 16 biți
- Consum redus de energie
- Două procesoare compatibile
  - MSP430 CPU
  - MSP430 CPUx (MSP430X)
- În continuare foarte folosite de industrie si hobby-iști

Familia PIC (Microchip)

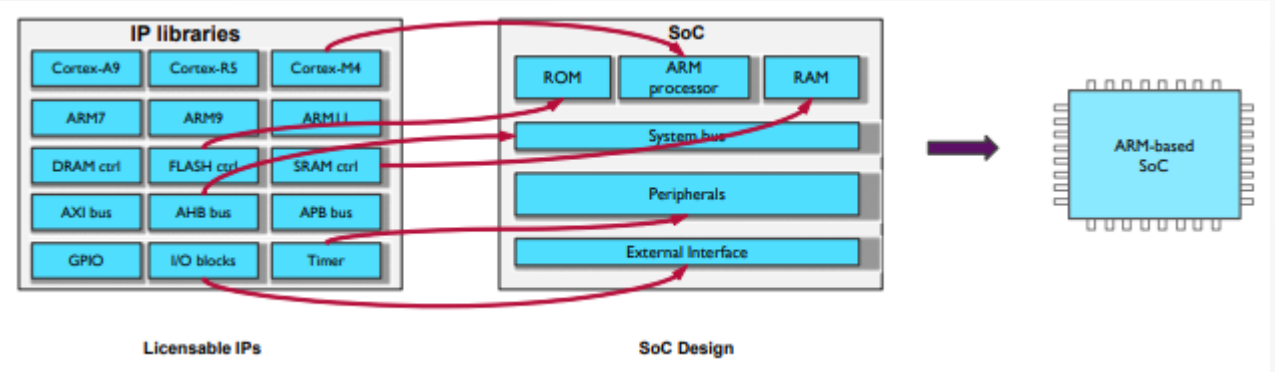


- Dezvoltat în 1976 de General Instrument
- Arhitectură Harvard pe 8 biți
- Acum este realizata de Microchip Technology
- Inițial Peripheral Interface Controller, în prezent Programable Intelligent Computer
- Nu există nicio distincție între spațiul de memorie și spațiul de registre
- PIC16C5x, PIC16C64, PIC16C71 și PIC16C84

ARM Cortex-M



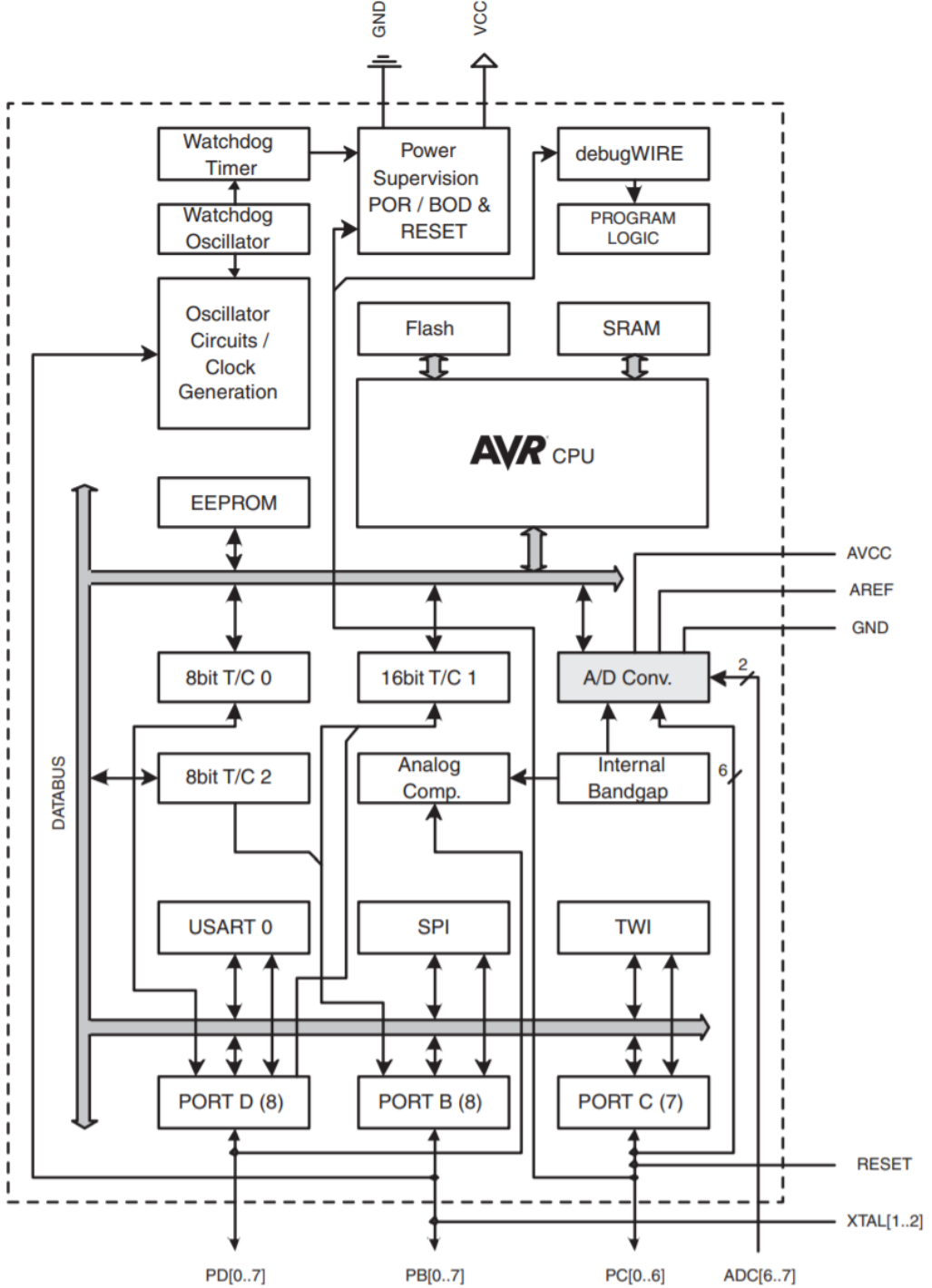
- Dezvoltat începând cu 2004
- Arhitectură ARM pe 32 de biți, licențiate de Arm Holdings
- Permite integrarea facilă a elementelor de siliciu ARM si 3rd party
- Dimensiunea instrucțiunii pentru M0 de 16 biți
- Cortex-M0, Cortex-M0+, Cortex-M1, Cortex-M3, Cortex-M4, Cortex-M7, Cortex-M23, Cortex-M33, Cortex-M35P, Cortex-M55



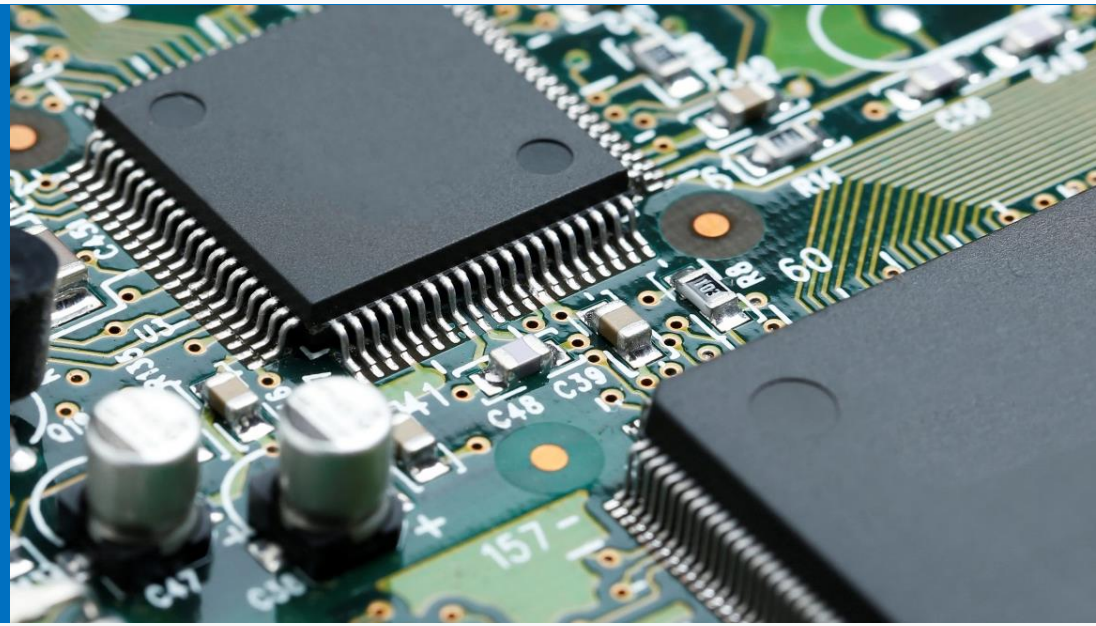
Schemă bloc ATMega328P



https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf



AVR



PM

Specificități AVR

- Arhitectură Harvard:
  - Există o memorie de program (32kB)
  - Există o memorie de date (2kB)
- Intrările și ieșirile sunt port-mapped
  - Există instrucțiuni speciale (in / out) de acces la memoria I/O
  - Compilatorul se ocupă singur de diferențierea între scriere în memorie și scriere către I/O

Flow-ul unui program pe AVR

- Încărcarea programului

  Cu ajutorul unui programator extern sau a unui bootloader, binarul de AVR trebuie să ajungă în memoria de program

- Lansarea în execuție

  La resetarea microcontroller-ului, se pornește execuția începând cu o adresa prestabilita din etapa de proiectare

- Fire de execuție

  Unul singur, dat de registrul PC (Program Counter)

- Interacțiune cu exteriorul

  Doar prin instrucțiuni in/out (port-mapped I/O)

Mediul de programare

- În C, foarte similar cu programarea pe PC
- Fără consolă ☺
- GCC este configurat pentru un sistem pe 8 biti (int & pointerii sunt pe 16 biti)
- Avem macrodefiniții pentru toate adresele relevante de I/O,

compilatorul știe automat să folosească instrucțiunile dedicate pentru scrierile și citirile de la aceste adrese

```
#define PORTC *(volatile uint8_t *) (0x020)
```

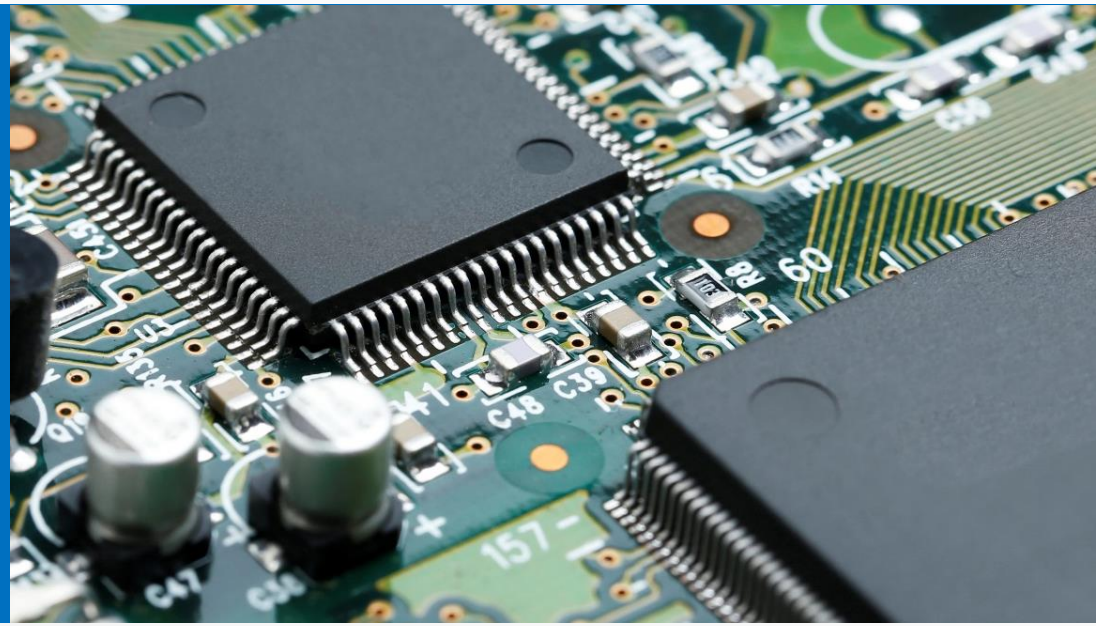
Exemplu cod pe 328P

```
#include <avr/ io.h>                    //header cu registrele I/O
#define F_CPU 16000000UL          //frecventa de lucru
#include <util / delay.h>
int main ( )
{
        DDRC = (1 << PC4 );      //PIN PC4 setat ca out
        while (1)
        {
        PORTC ^= (1 << PC4 );   //PIN PC4 setat alternativ 0/1
        _delay_ms (250);
        }
}
```

Examplu limbaj de asamblare pe 328P

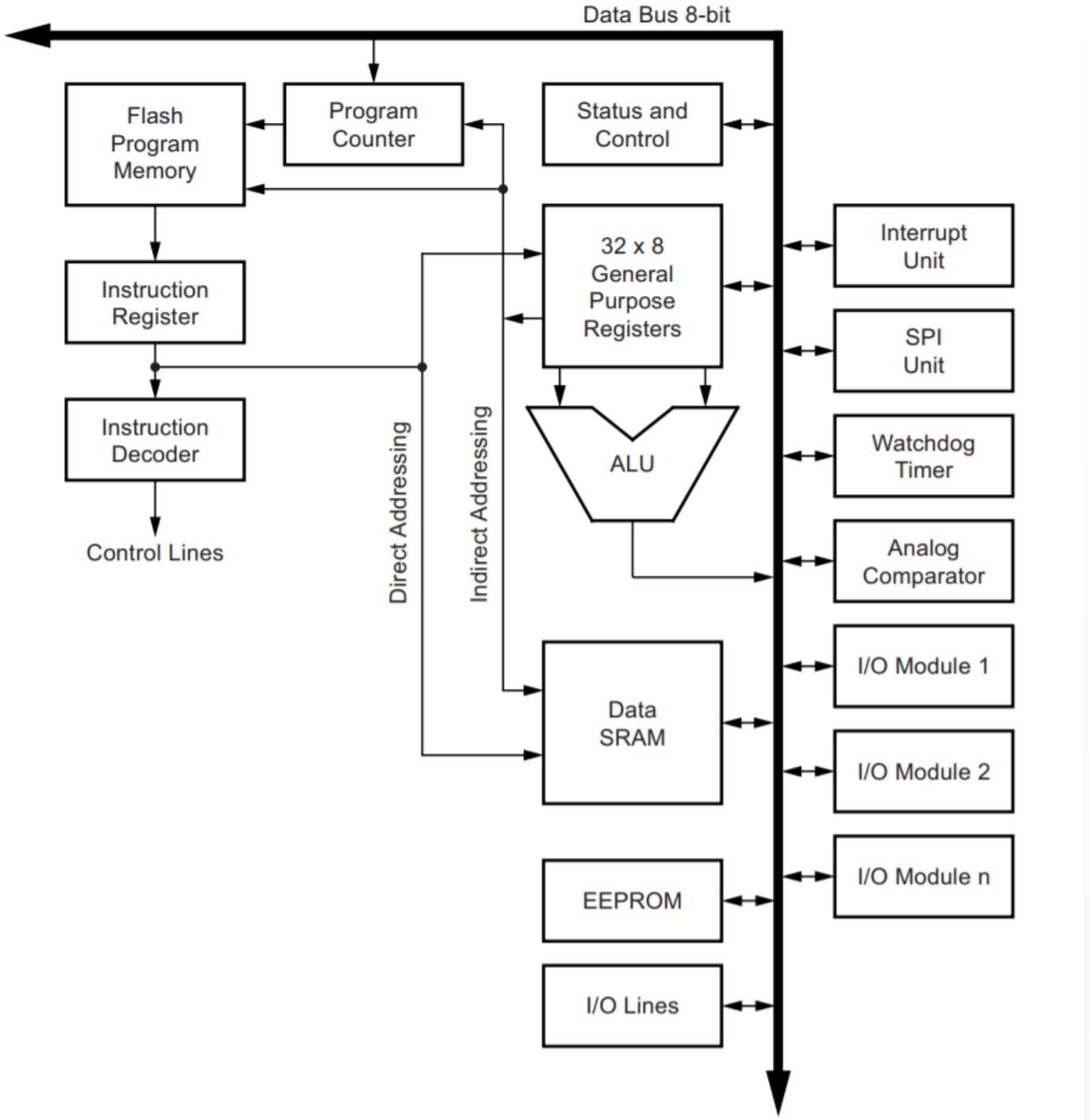
```
00000000 <__vectors>:
   0:   0c 94 3e 00     jmp 0x7c            ; 0x7c <__ctors_end>
                                            ; care face jump catre main dupa initializa
00000094 <main>:
  94:   80 e1           ldi r24, 0x10       ; (1 << 4)
  96:   87 b9           out 0x07, r24       ; scriere in DDRC
  98:   90 e1           ldi r25, 0x10       ; (1 << 4)
  9a:   88 b1           in  r24, 0x08       ; citire din PORTC
  9c:   89 27           eor r24, r25        ; r24 = r24 XOR r25
  9e:   88 b9           out 0x08, r24       ; scriere in PORTC
  a0:   2f ef           ldi r18, 0xFF       ; 255
  a2:   34 e3           ldi r19, 0x34       ; 52
  a4:   8c e0           ldi r24, 0x0C       ; 12
  a6:   21 50           subi    r18, 0x01   ; 1 ciclu
  a8:   30 40           sbci    r19, 0x00   ; 1 ciclu
  aa:   80 40           sbci    r24, 0x00   ; 1 ciclu
  ac:   e1 f7           brne    .-8         ; 0xa6 <main+0x12>
                                            ; 1 ciclu daca nu face conditia, 2 altfel
                                            ; practic se numara de la 0x0C34FF la 0
                                            ; 0x0C34FF * 5 cicli - 1 de iesire
  ae:   00 c0           rjmp    .+0         ; 0xb0 <main+0x1c>
  b0:   00 00           nop                 ; nop + rjmp = 3 cicli
  b2:   f3 cf           rjmp    .-26        ; 0x9a <main+0x6>
```

Atmega 328

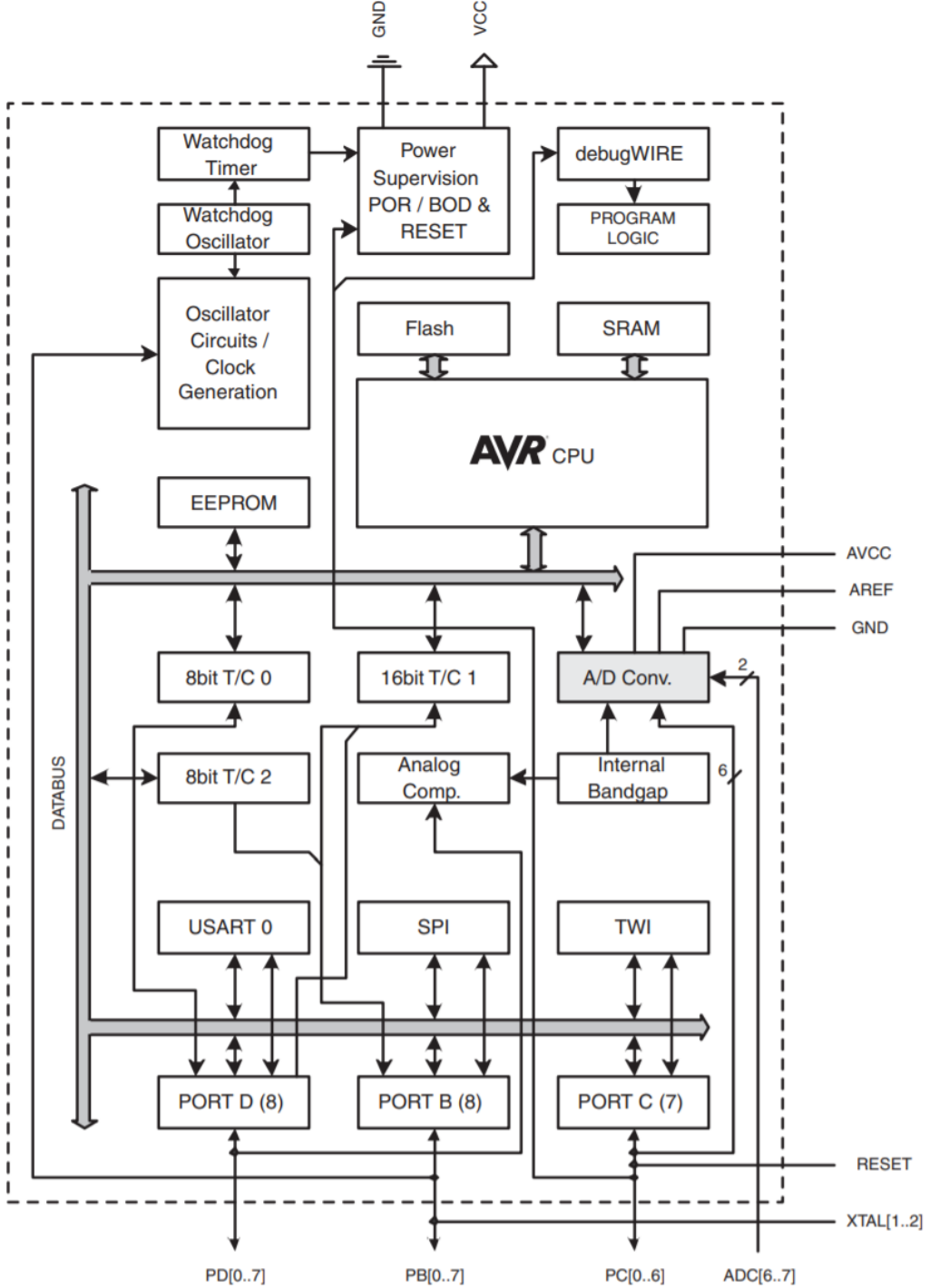


PM

Schemă bloc ATMega328P

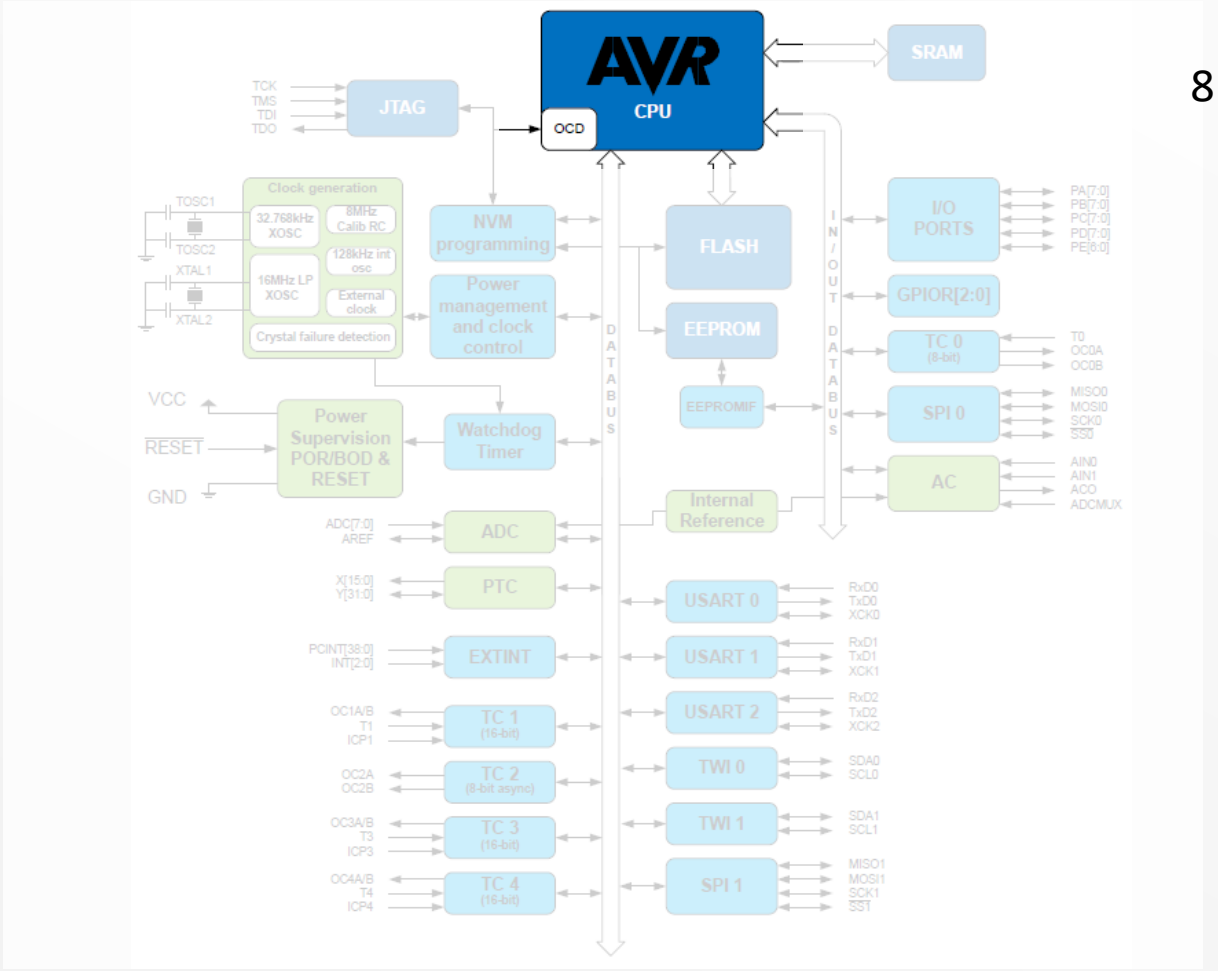


Schemă bloc ATMega328P



https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

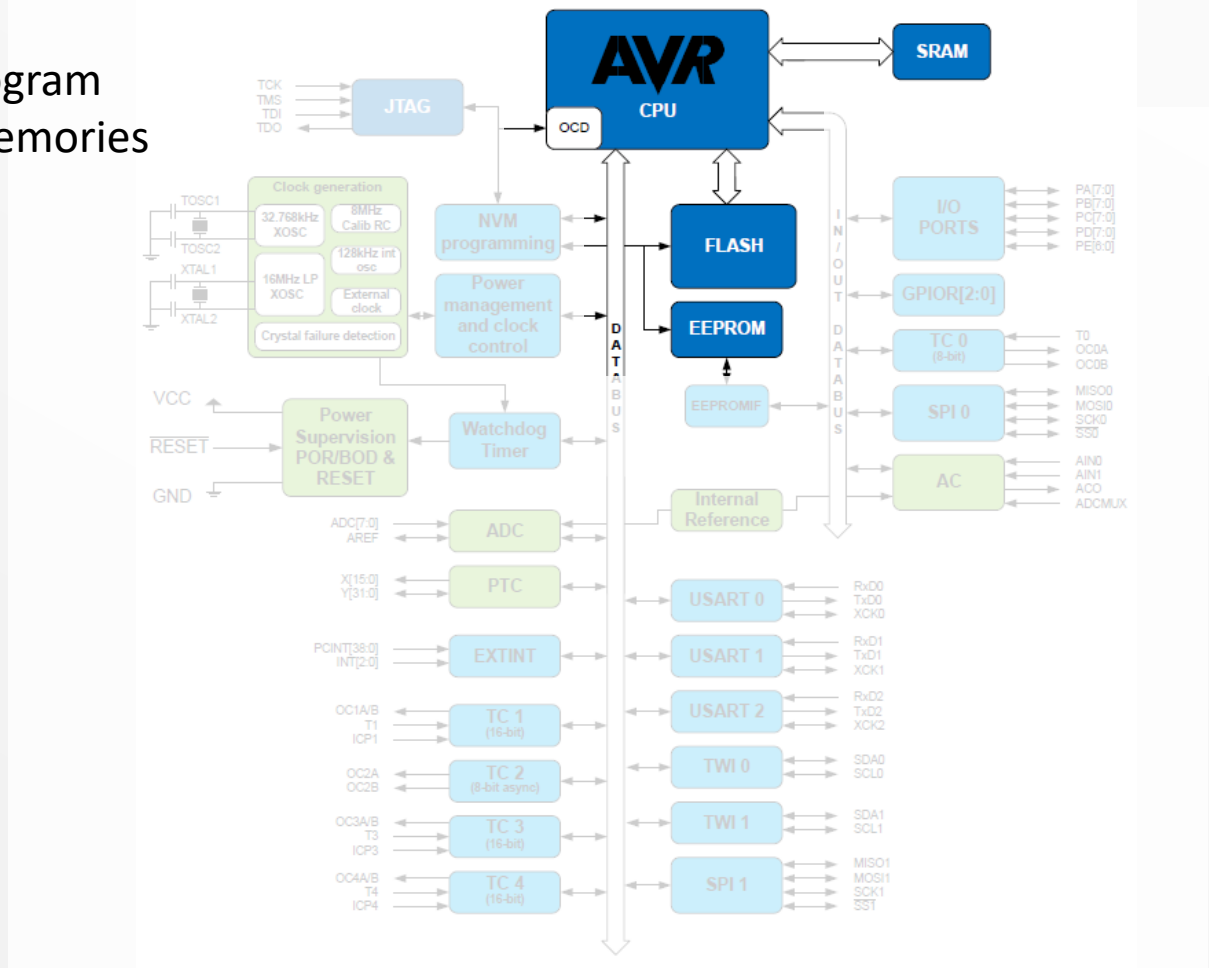
Arhitectura lui ATMega328P



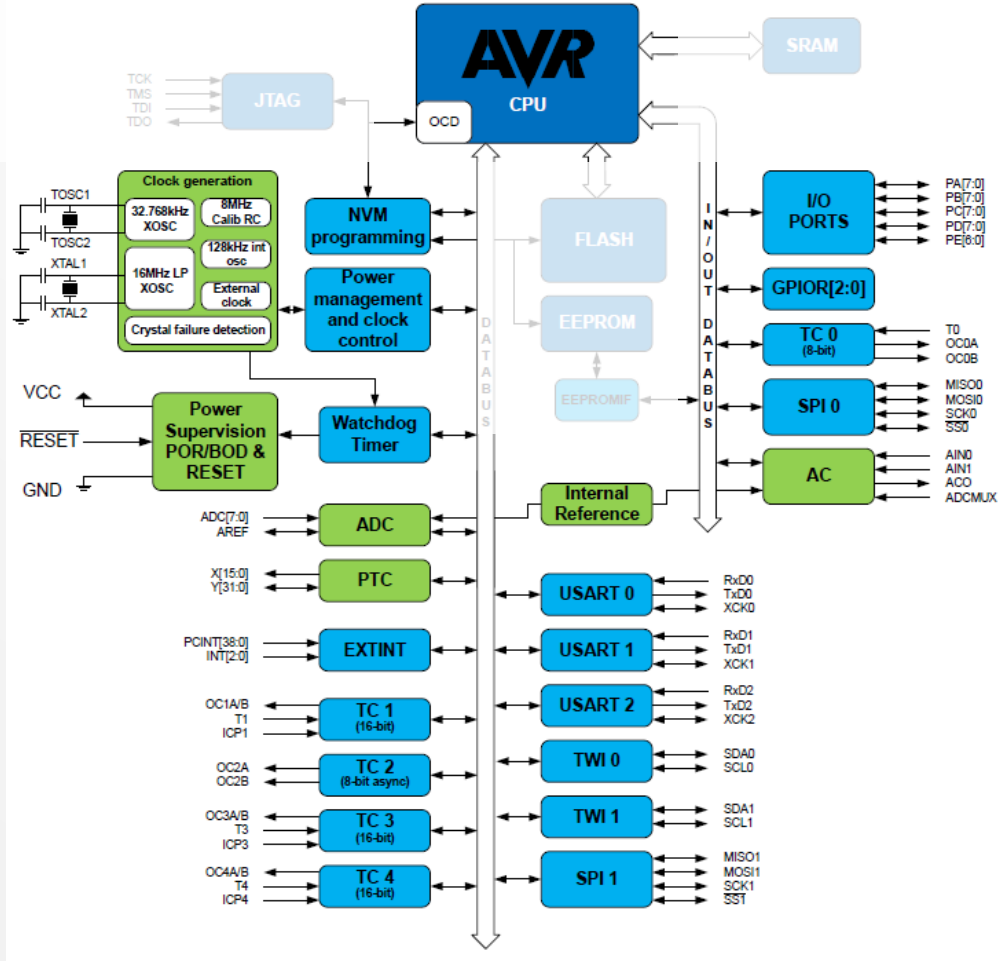
8 bit CPU core

Arhitectura lui ATMega328P

On-chip program
and data memories

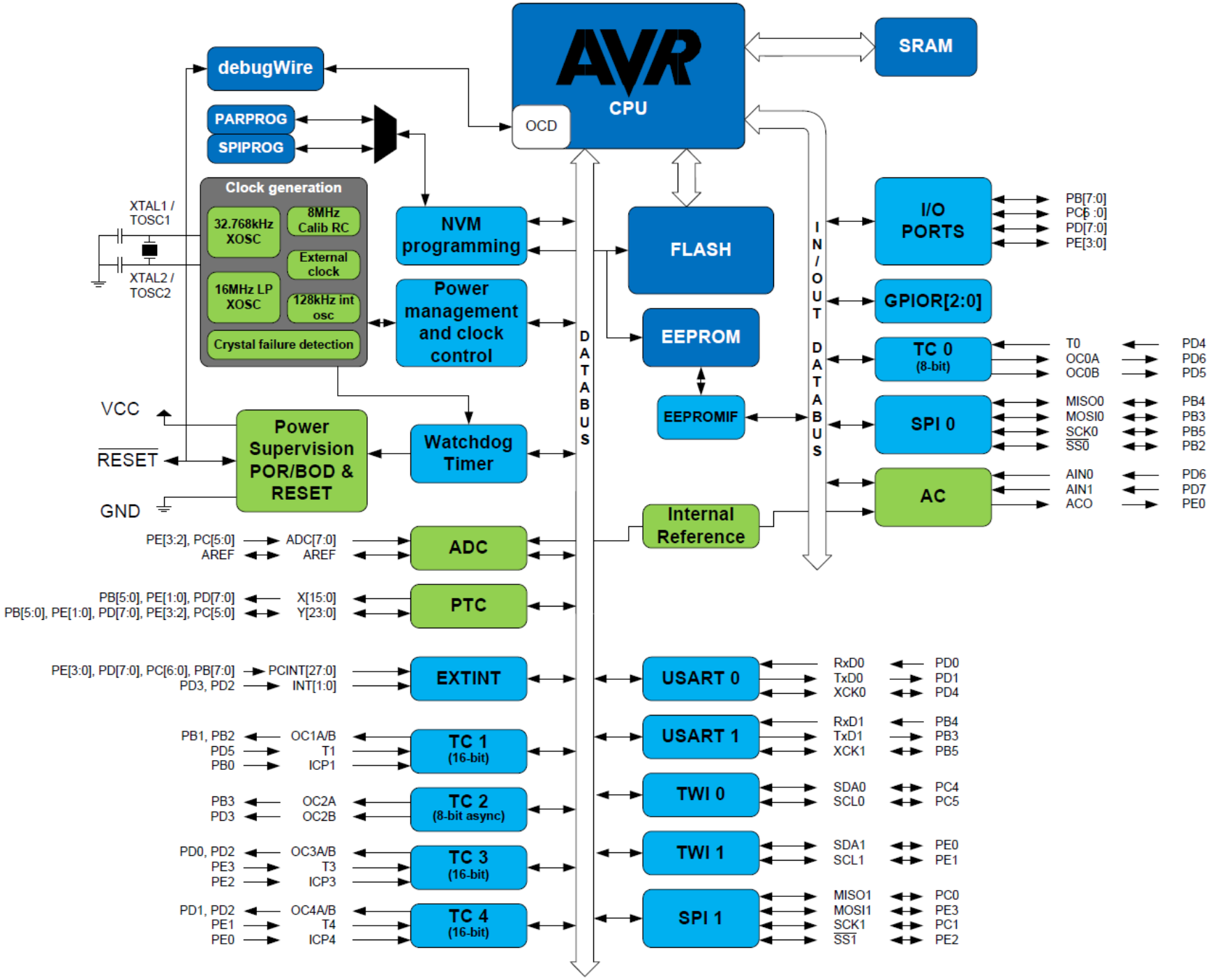


Arhitectura lui ATMega328P



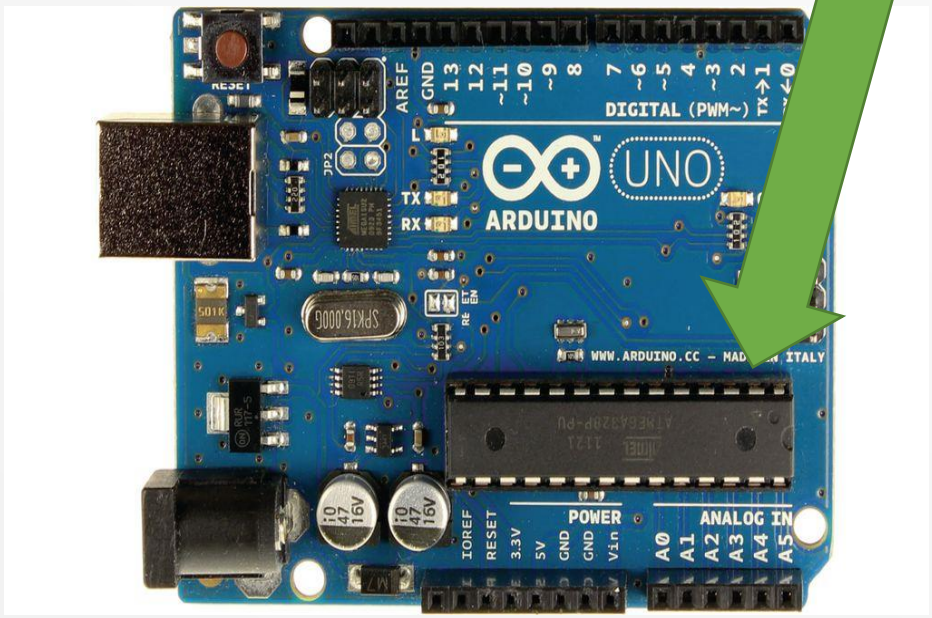
More than plenty peripherals

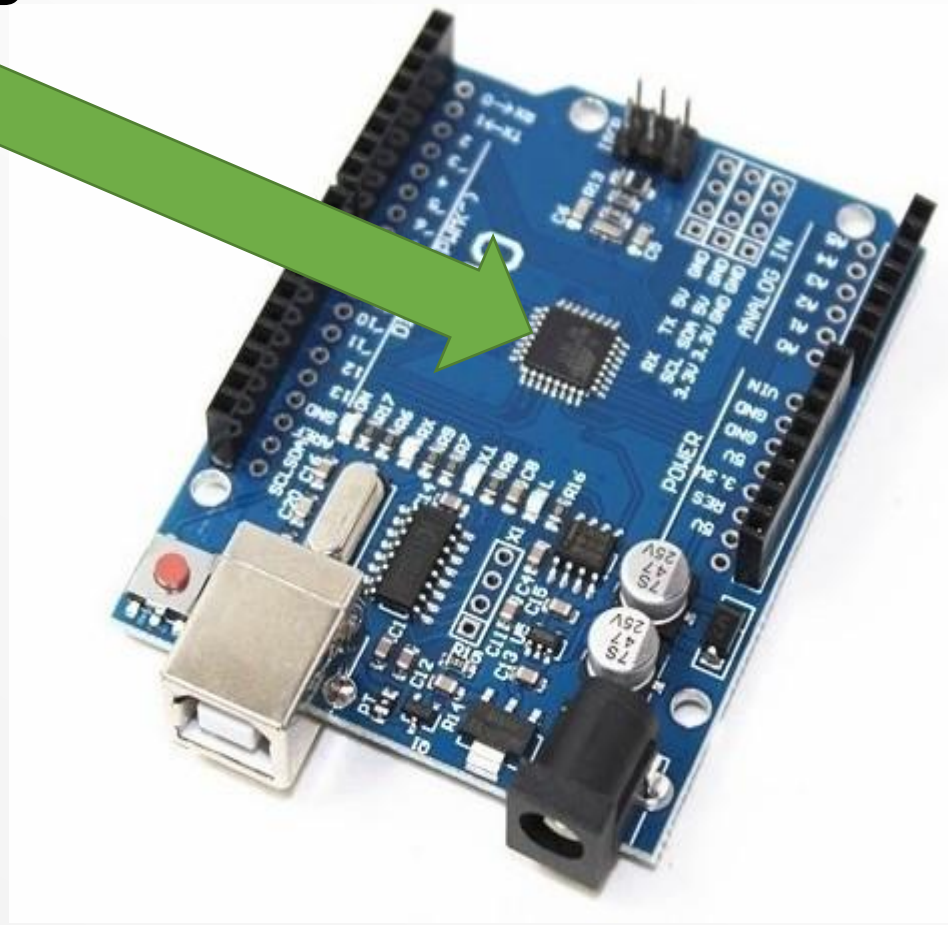
ATMega328P


AVR
CPU
OCD
SRAM
debugWire
PARPROG
SPIPROG
Clock generation
32.768kHz XOSC
8MHz Calib RC
External clock
16MHz LP XOSC
128kHz int osc
Crystal failure detection
XTAL1 / TOSC1
XTAL2 / TOSC2
NVM programming
Power management and clock control
FLASH
EEPROM
EEPROMIF
I/O PORTS
PB[7:0]
PC[6:0]
PD[7:0]
PE[3:0]
GPIOR[2:0]
TC 0 (8-bit)
T0
OC0A
OC0B
PD4
PD6
PD5
SPI 0
MISO0
MOSI0
SCK0
SS0
PB4
PB3
PB5
PB2
AC
AIN0
AIN1
ACO
PD6
PD7
PE0
Internal Reference
DATABUS
I/O DATABUS
VCC
RESET
GND
Power Supervision POR/BOD & RESET
Watchdog Timer
ADC
PE[3:2], PC[5:0]
AREF
ADC[7:0]
AREF
PTC
PB[5:0], PE[1:0], PD[7:0]
PB[5:0], PE[1:0], PD[7:0], PE[3:2], PC[5:0]
X[15:0]
Y[23:0]
EXTINT
PE[3:0], PD[7:0], PC[6:0], PB[7:0]
PD3, PD2
PCINT[27:0]
INT[1:0]
TC 1 (16-bit)
PB1, PB2
PD5
PB0
OC1A/B
T1
ICP1
TC 2 (8-bit async)
PB3
PD3
OC2A
OC2B
TC 3 (16-bit)
PD0, PD2
PE3
PE2
OC3A/B
T3
ICP3
TC 4 (16-bit)
PD1, PD2
PE1
PE0
OC4A/B
T4
ICP4
USART 0
RxD0
TxD0
XCK0
PD0
PD1
PD4
USART 1
RxD1
TxD1
XCK1
PB4
PB3
PB5
TWI 0
SDA0
SCL0
PC4
PC5
TWI 1
SDA1
SCL1
PE0
PE1
SPI 1
MISO1
MOSI1
SCK1
SS1
PC0
PE3
PC1
PE2

Arduino

ATMega 328P





Datasheet

- Un microcontroler este un circuit integrat complex care are un manual. Acest manual se numește fișă de date și poate fi găsit online:

https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

pentru cipul pe care il vom folosi

Datasheet

- Manualele conțin fiecare detaliu despre cipul pe care încercați să îl utilizați::
  - Opțiuni de ambalare (de exemplu, DIP, QFN, TQFP)
  - Diagrame de pin (unde sunt pinii de alimentare, unde sunt pinii pentru perifericele pe care doriți să-l utilizați)
  - Caracteristici electrice (de exemplu, tensiune maximă, curenți maximi ai pinii, pragurile tensiunii de intrare a pinii)
  - Periferice pe care le are MCU și cum să le accesați (de exemplu, ADC-uri, temporizatoare, periferice de comunicare)

Datasheet

- Atenție: acest manual are 294 de pagini! Nu trebuie să le citiți pe TOATE înainte de a putea folosi cipul.
  - Folosind o placă de dezvoltare, este posibil să nu vă pese de alimentarea MCU. S-a făcut pentru tine de către dezvoltatorul plăcii
  - Nu veți folosi toate perifericele pe care le are dispozitivul
  - Inițial nu vă va păsa de diagramele de timp, curenții sau alte detalii
- Dar ar ajuta să vă cunoașteți mai bine dispozitivul ;)
  - Este posibil să încercați să faceți în software ceva ce un periferic ar putea face cu ușurință pentru dvs. (de exemplu, comunicare, cronometrare)
  - S-ar putea să vă depanați programul deoarece un periferic este diferit de altul (de exemplu, Timer 1 vs Timer 2)
  - ...

Regiștrii cu funcții speciale

- Special Function Registers (SFRs) sunt regiștrii care:
  - controlează un comportament al MCU sau perifericelor sau
  - oferă un schimb de date cu resursele MCU
- Exemple
  - DDRB, PORTB - direcția portului și valoarea de ieșire
  - CLKPR - Clock Prescaler Register
  - SPDR0 - SPI Data Register 0

Regiștrii cu funcții speciale

- Ele pot fi considerate „memorii conectate la un alt hardware”
- Hardware-ul are conexiuni directe la acești biți și îi folosește pentru configurare, control și transfer de date.

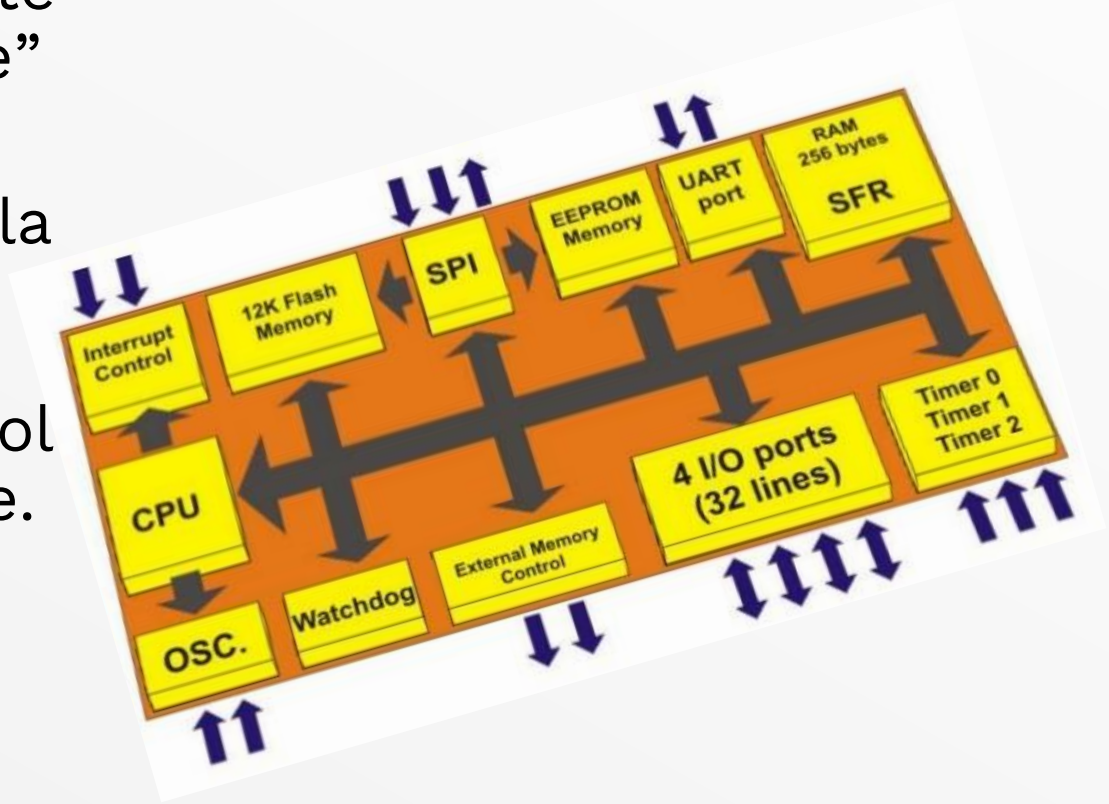


Image source: http://mcuhq.com/25/atmega-avr-code-example/

Regiștrii cu funcții speciale

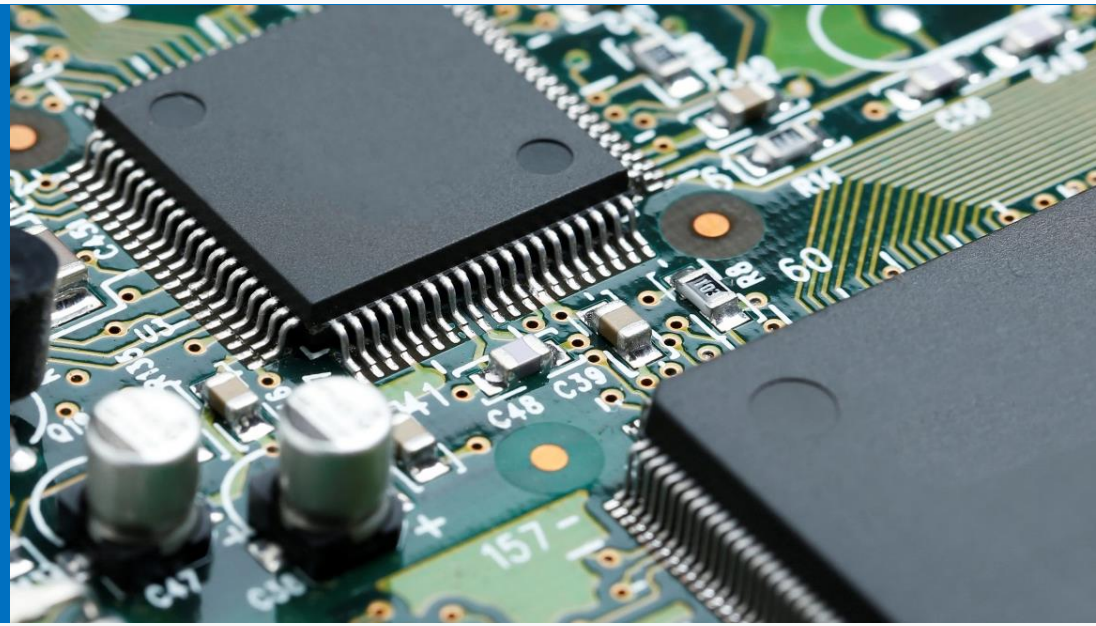
- Spre deosebire de memorie, acestea pot avea biți neimplementați sau biți care pot fi doar citiți (hardware-ul îi stabilește)
- Fișa tehnică explică fiecare registru în detaliu.

Clock Prescaler Register

Name: CLKPR
Offset: 0x61
Reset: Refer to the bit description
Property: -

Bit	7	6	5	4	3	2	1	0
	CLKPCE				CLKPS3	CLKPS2	CLKPS1	CLKPS0
Access	R/W				R/W	R/W	R/W	R/W
Reset	0				x	x	x	x

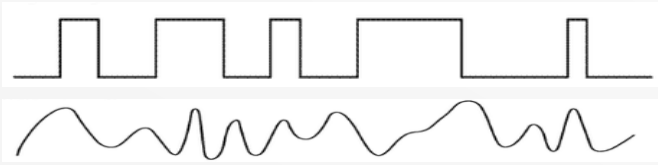
Recap - logică digitală



PM

Aplicații ale domeniului digital

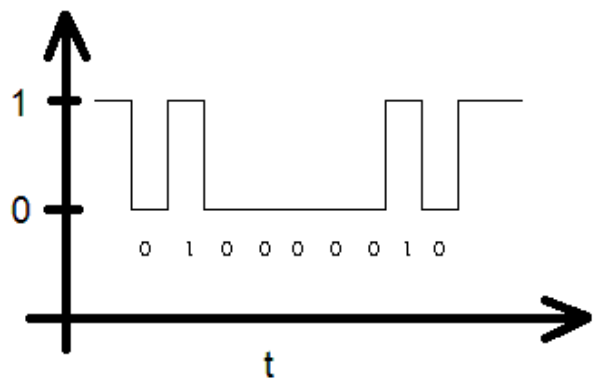
- O aplicație de bază pentru microcontrolere constă în
  - Citirea unor intrări
  - Procesări simple sau comunicare cu un sistem central
  - Controlul unor ieșiri
- Intrările și ieșirile pot fi fie
  - Digitale
  - Analogice

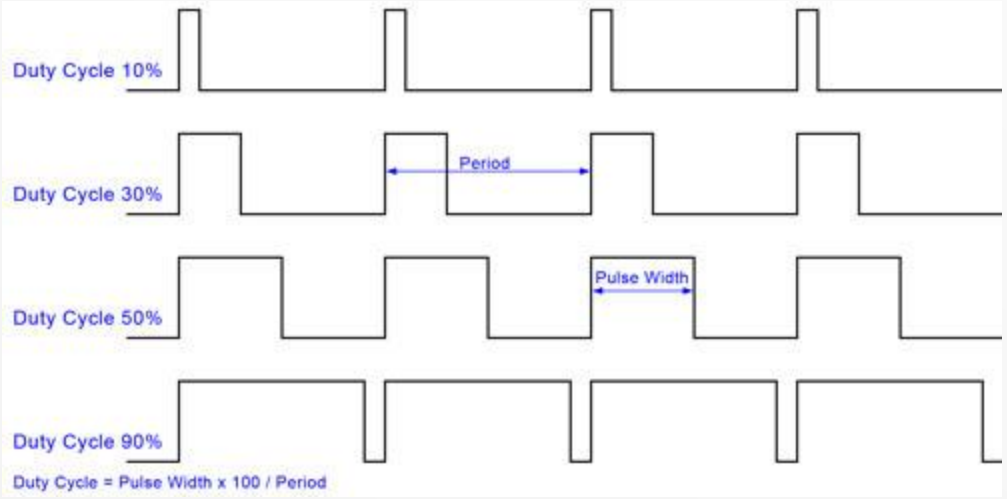


- Procesarea în cadrul ALU are loc întotdeauna în domeniul digital
  - Unele operațiuni pe semnale analogice se pot întâmpla și în MCU, dar acest lucru este foarte rar (de exemplu, cu amplificatoare operaționale integrate)

Aplicații ale domeniului digital

- Semnalele digitale pot transmite doar o singură valoare binară în fiecare moment.
  - Acest lucru sugerează posibilitatea de a folosi timpul ca dimensiune suplimentară pentru a transmite mai multe date folosind o valoare digitală.
  - Comunicațiile digitale seriale folosesc acest fapt pentru a transmite date.
  - Modularea în lățime a impulsurilor(Pulse-Width Modulation/PWM) folosește acest fapt pentru a transmite informații analogice: media de timp a semnalului digital poate fi transformată într-o valoare analogică.





PWM image from http://www.protostack.com/blog/2011/06/atmega168a-pulse-width-modulation-pwm/

Aplicații ale domeniului digital

- În timp ce interfața cu domeniul analogic poate fi necesară în unele cazuri, multe aplicații au nevoie doar de interfața digitală:
  - Intrări digitale:
    - butoane (de exemplu, comutatoare, jumperi, comutatoare cu lame(reed switches))
    - codificatoare rotative
    - senzori de prag (de exemplu, lumină, temperatură, greutate)









Keypad EZ

Aplicații ale domeniului digital

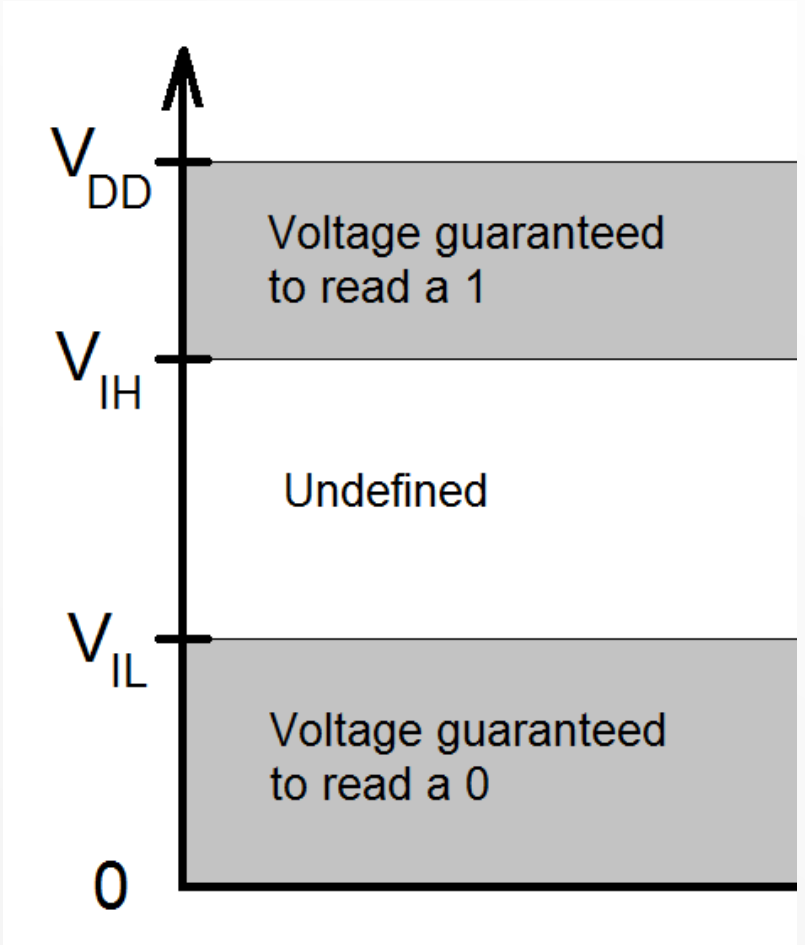
- În timp ce interfața cu domeniul analogic poate fi necesară în unele cazuri, multe aplicații au nevoie doar de interfața digitală:
  - Intrări digitale:
    - butoane (de exemplu, comutatoare, jumperi, comutatoare cu lame(reed switches))
    - codificatoare rotative
    - senzori de prag (de exemplu, lumină, temperatură, greutate)
  - Ieșiri digitale:
    - Indicatoare luminoase (de exemplu, LED-uri, becuri)
    - Afișaje (de exemplu, LCD, OLED, TFT)
    - Controlul pornit/oprit al echipamentelor (de exemplu, relee pentru motoare, iluminat)

Intrări digitale

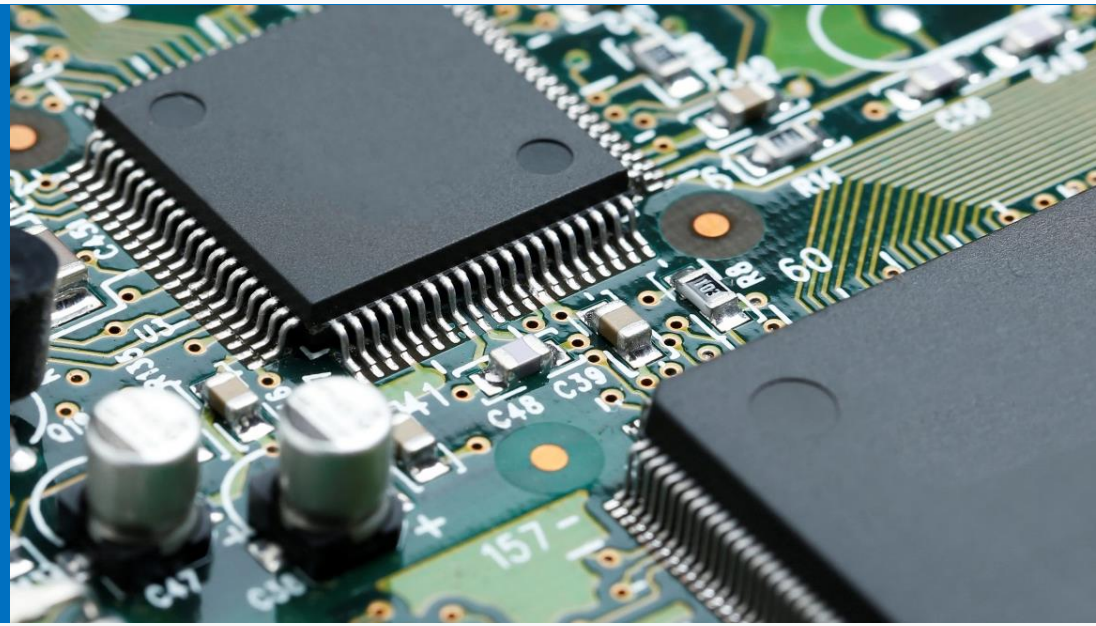
- O intrare digitală este un pin configurat să accepte un semnal digital
- Aceasta este de obicei o tensiune proprie
  - 0 volți pentru a semnala un 0 digital
  - V_{DD} volți pentru semnal digital 1
    - adică 3,3 V pentru un MCU furnizat la 3,3 V
    - unele piese alimentate la mai puțin de 5V au pini „toleranți la 5V” care pot fi utilizați ca intrări cu tensiuni de până la 5V

Intrări digitale

- Un MCU nu se așteaptă la EXACT 0 V sau 3,3 V (dacă este alimentat de la 3,3 V) pentru a citi intrarea ca 1 sau 0
- Intervalele garantate pentru a produce citiri corecte sunt date în fișa de date sub „Caracteristici electrice”, de obicei:
  - Se obține un 0 digital pentru o tensiune joasă de intrare:
    - V_{IL} = 0V to 0.2 * V_{DD}
  - Se obține un 1 digital pentru o tensiune de intrare ridicată:
    - V_{IH} = 0.8 * V_{DD} to V_{DD}

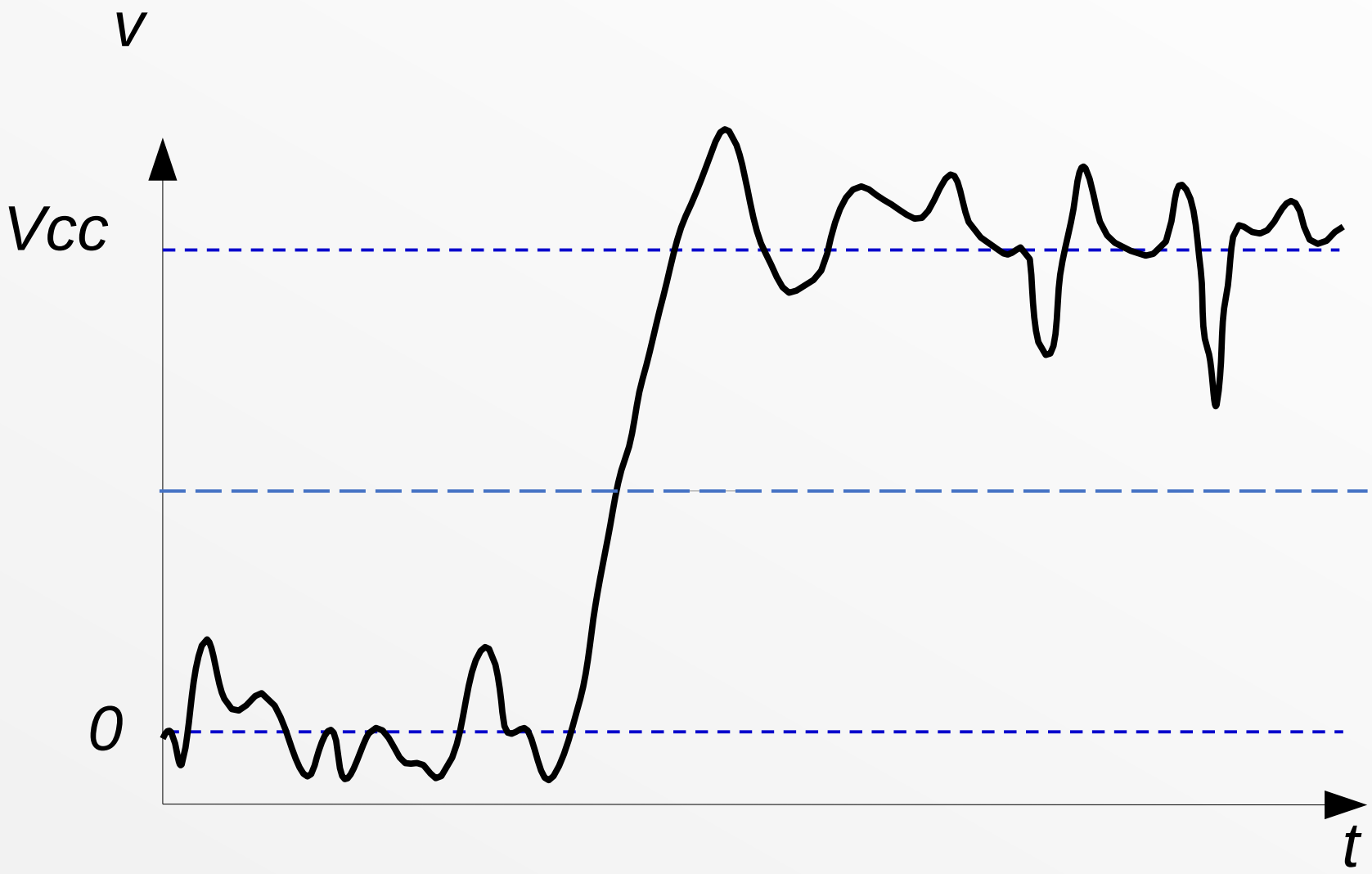


Recap – praguri logice



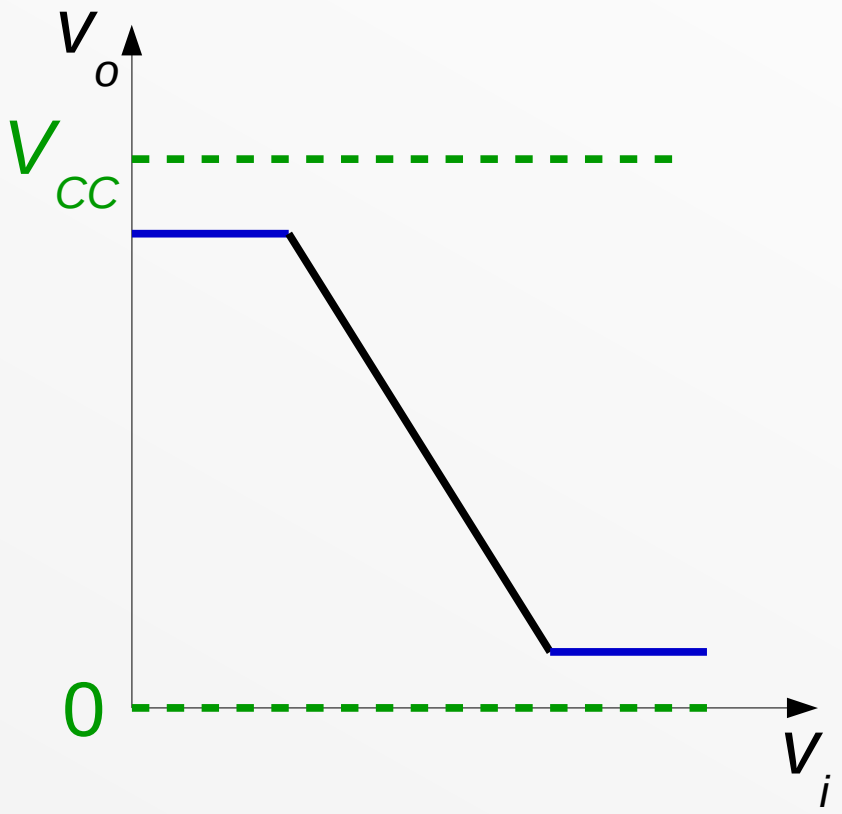
PM

Intro

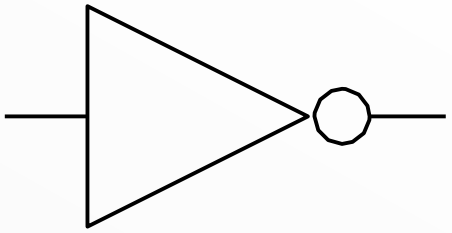


Intro

- este dezirabil ca nivelele logice să fie cât mai depărtate – egale cu tensiunile de alimentare – dar acest lucru **nu se întâmplă întotdeauna!**



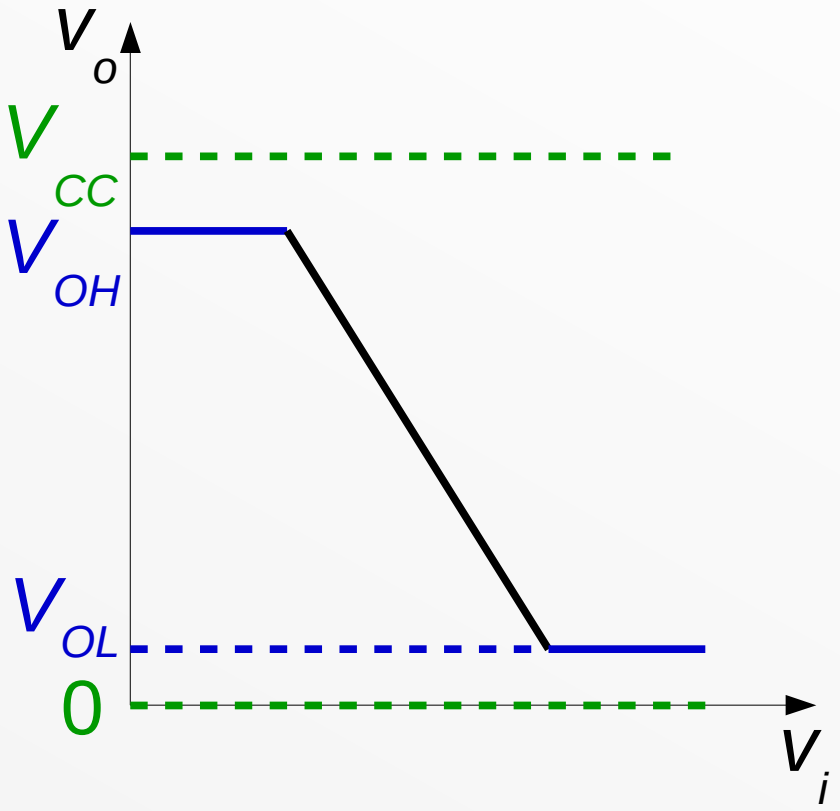
Nivelele logice pe ieșire



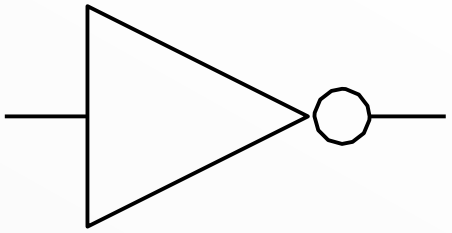
tensiunile de ieșire corespunzătoare celor două stări logice

nu sunt constante, sunt afectate de:

- tensiunea de alimentare
- curentul de ieșire
- temperatură
- variațiile procesului de fabricație



Nivelele logice pe intrare

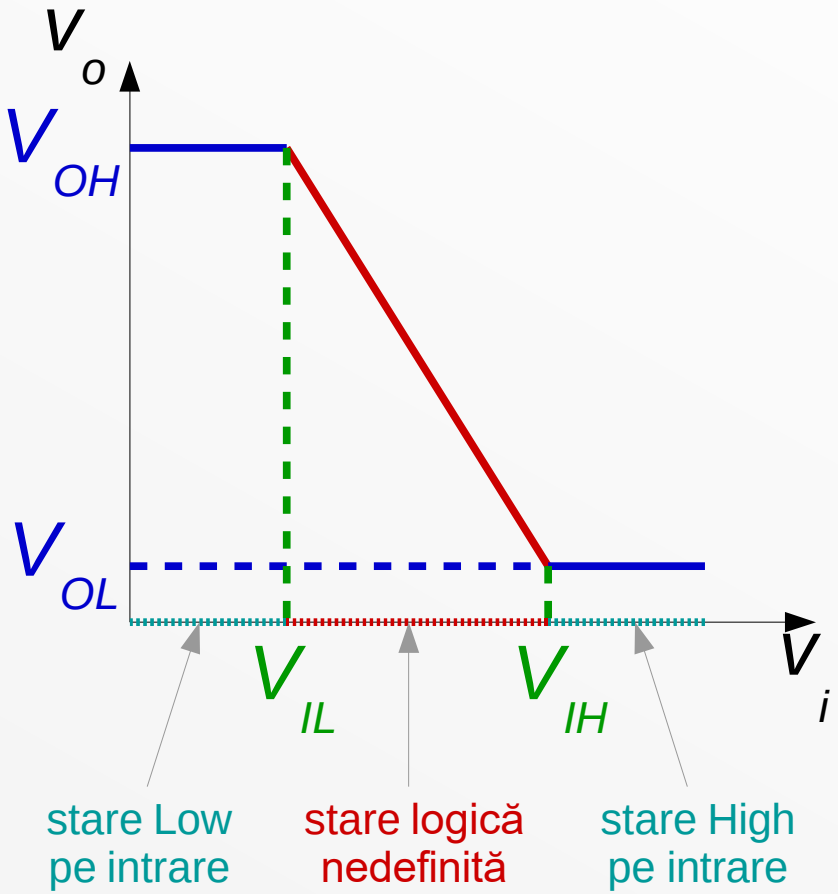


tensiunile de intrare corespunzătoare celor două stări logice ?

de fapt sunt două domenii de intrare

limitele interioare ale domeniilor de intrare corespunzătoare celor două stări logice

= limitele zonei de tranziție dintre stări



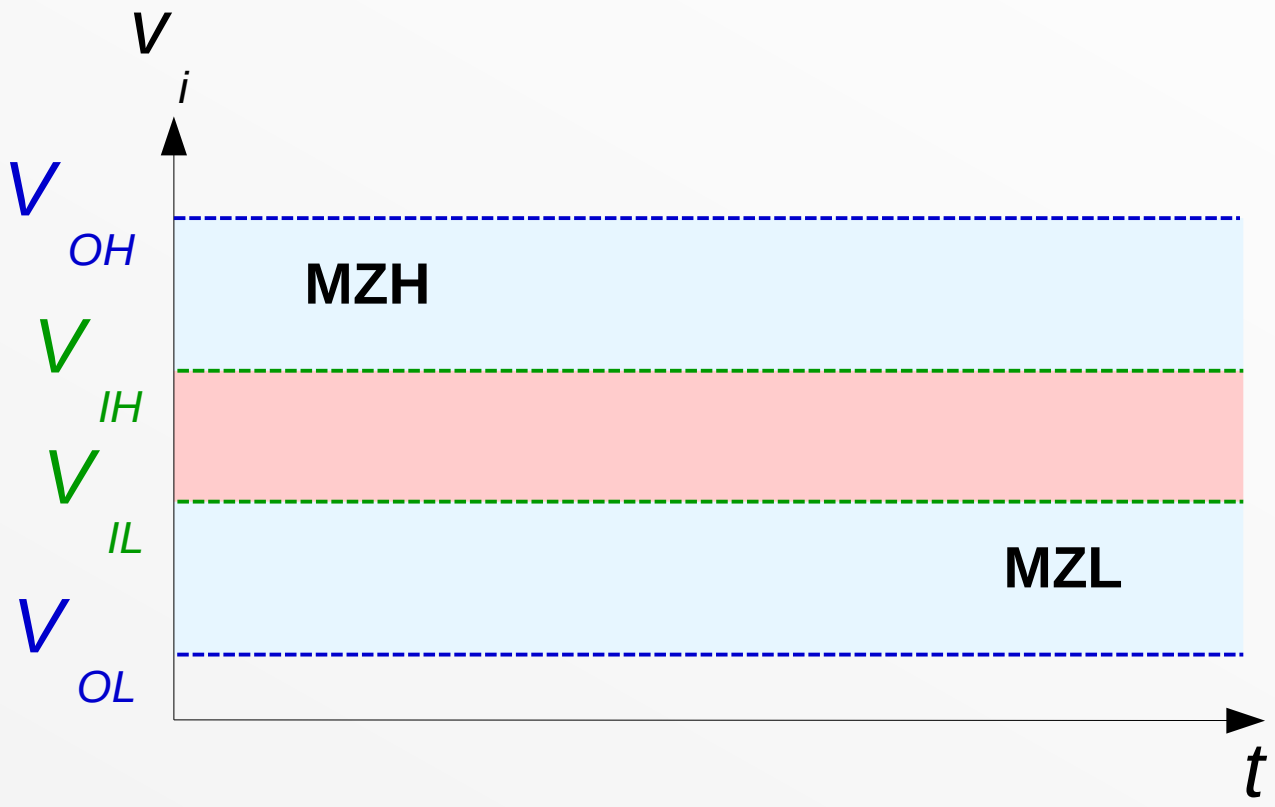
Marginile de zgomot

- Marginea de zgomot Low: V_{IL} - V_{OL}
  - cel mai defavorabil:

    V_{ILmin} - V_{OLmax}

- Marginea de zgomot High: V_{OH} - V_{IH}
  - cel mai defavorabil:

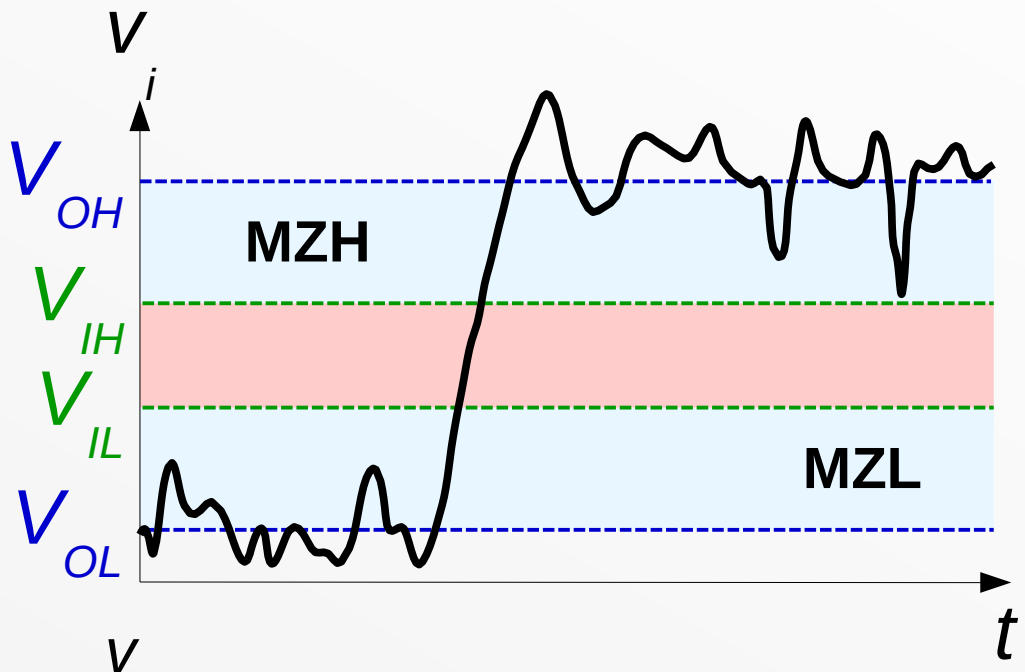
    V_{OHmin} - V_{IHmax}

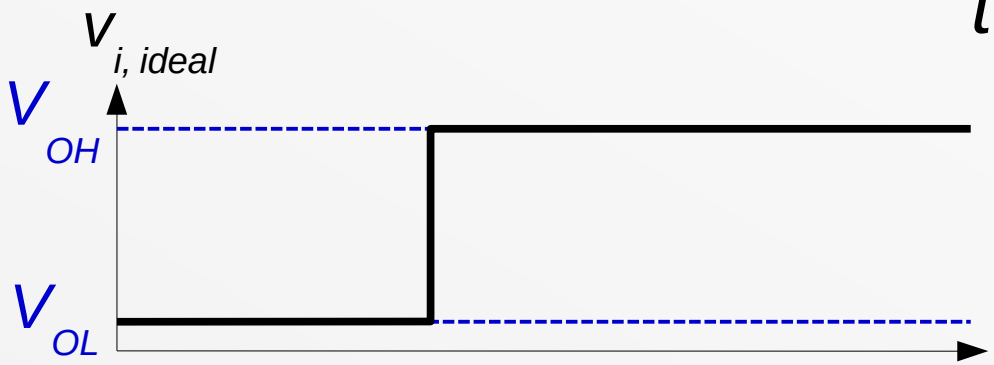


Marginile de zgomot

- Interpretare:

  Marginea de zgomot = amplitudinea maximă a zgomotului tolerat în starea *low / high* (fără a modifica starea citită de intrarea porții)

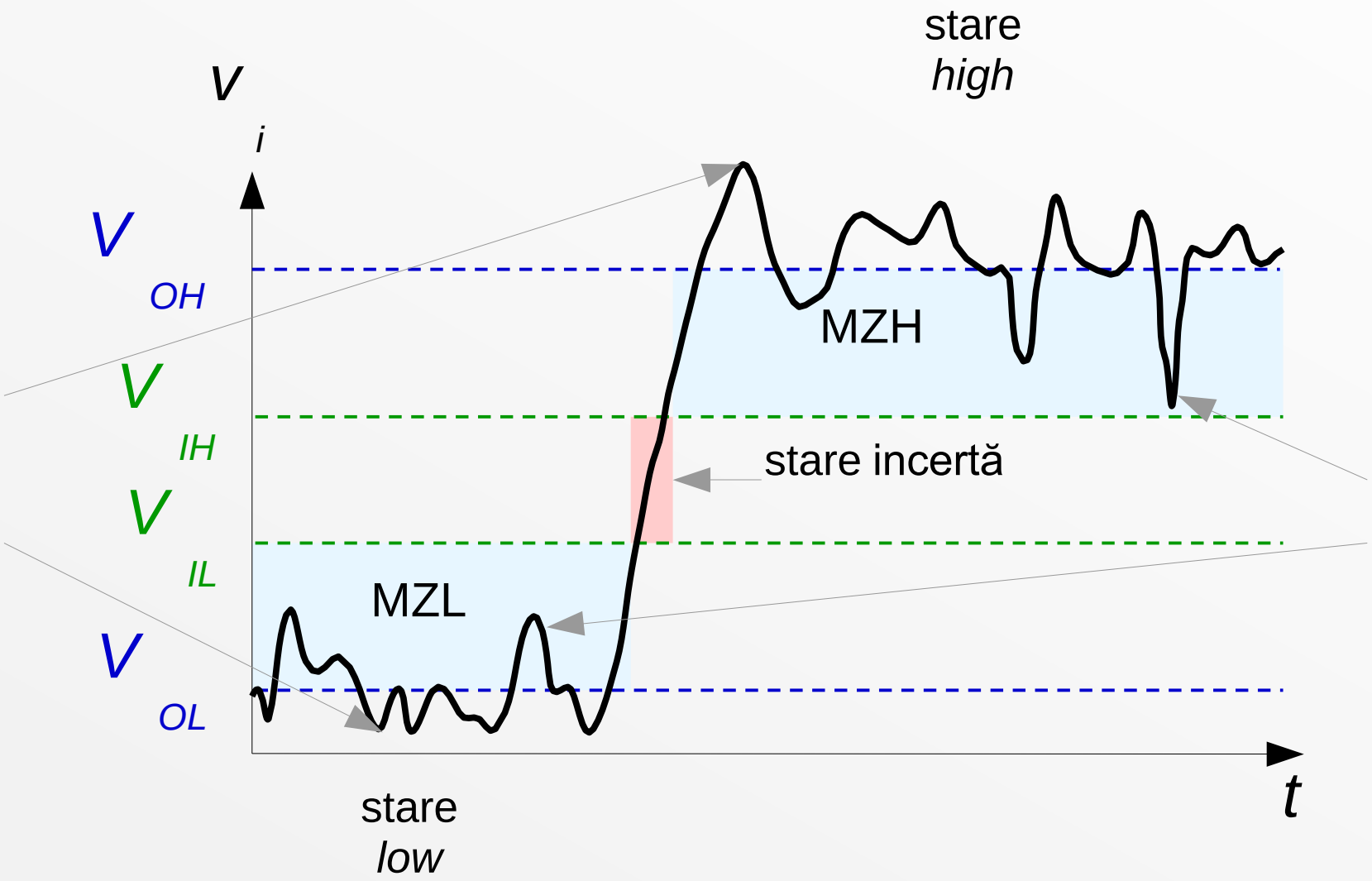




Situație ideală – semnalul:
- comută imediat
- nu este afectat de zgomot

– imposibilă în practică!

Marginile de zgomot

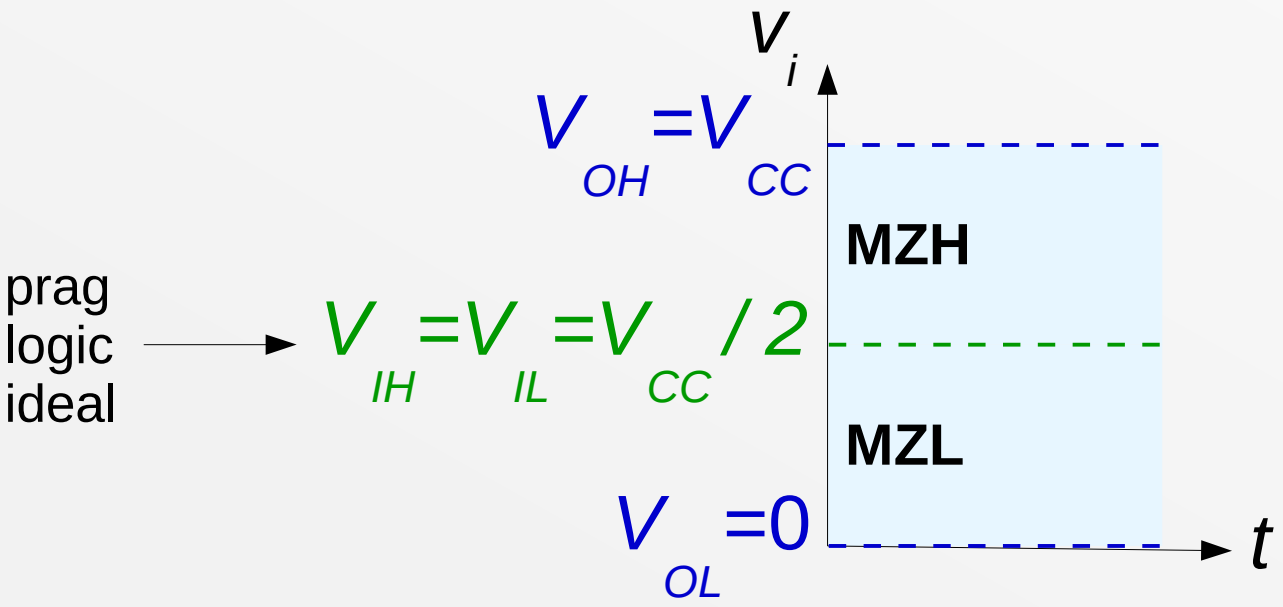


exemple de deviații care **nu** riscă să schimbe starea logică

exemple de deviații care **riscă** să schimbe starea logică dacă depășesc marginea de zgomot corespunzătoare stării low/high

Importanța marginilor de zgomot

- cuantifică (exprimă numeric) **robustețea** unei conexiuni între două porți sau module logice
  - dezirabil: cât mai mari
  - ideal: sunt maxime simultan când sunt egale cu jumătate din tensiunea de alimentare:



Interconectarea modulelor

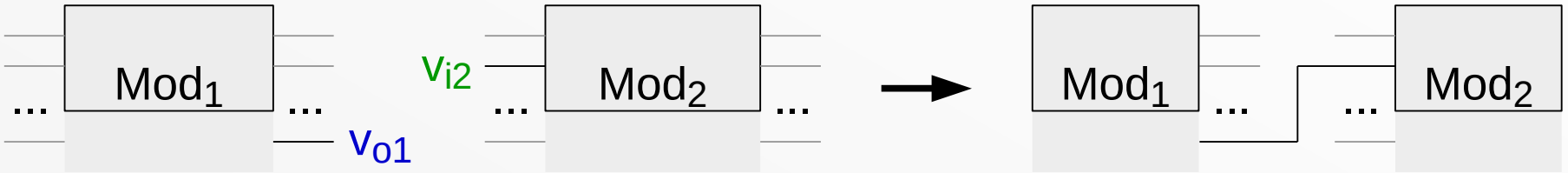
Condiție minimală la interconectarea a două module / subsisteme cu proprietăți diferite: margini de zgomot pozitive

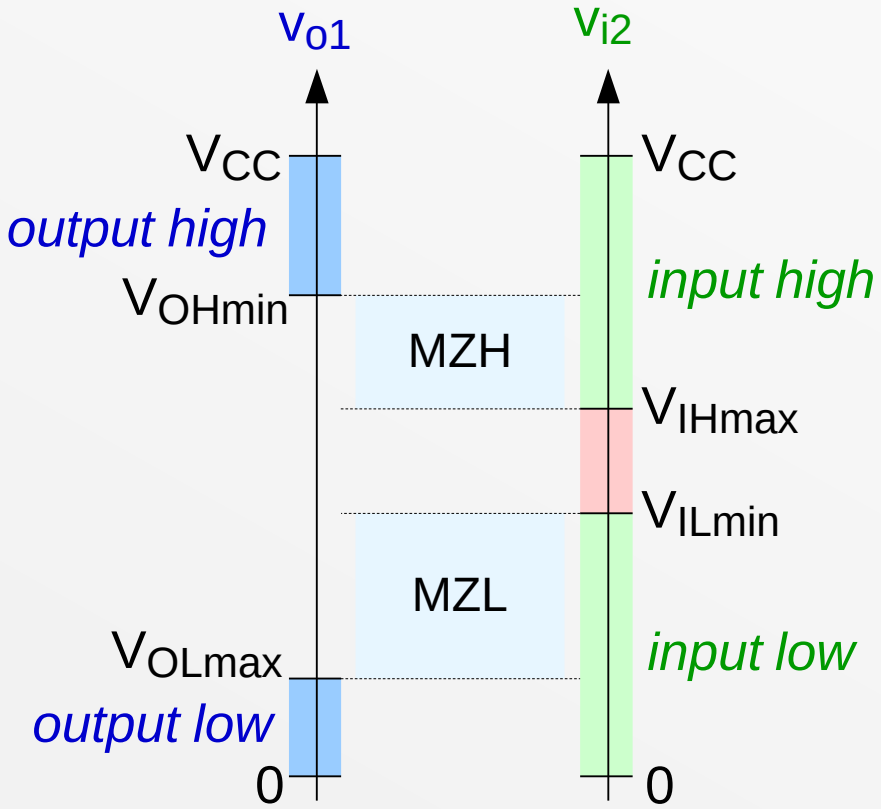
- insuficient în practică – deoarece nu ar oferi toleranță la pertubații
- principiu: trebuie asigurate margini de zgomot **comparabile** cu cele ale sistemelor individuale (dintre porțile / blocurile lor componente)

Condiții minimale:

$$V_{OH} > V_{IH}$$
$$V_{OL} < V_{IL}$$

Exemplu: module identice*

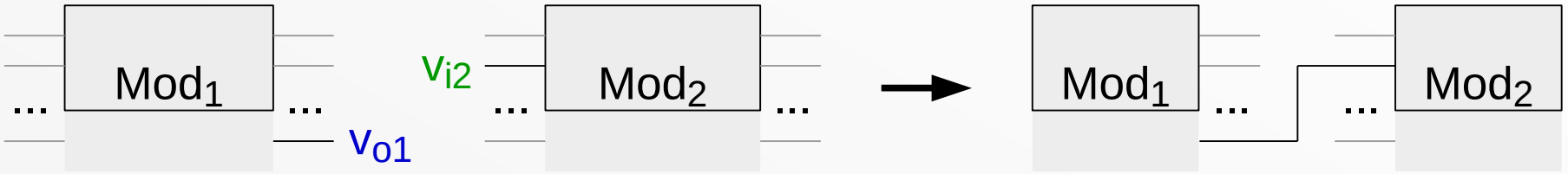


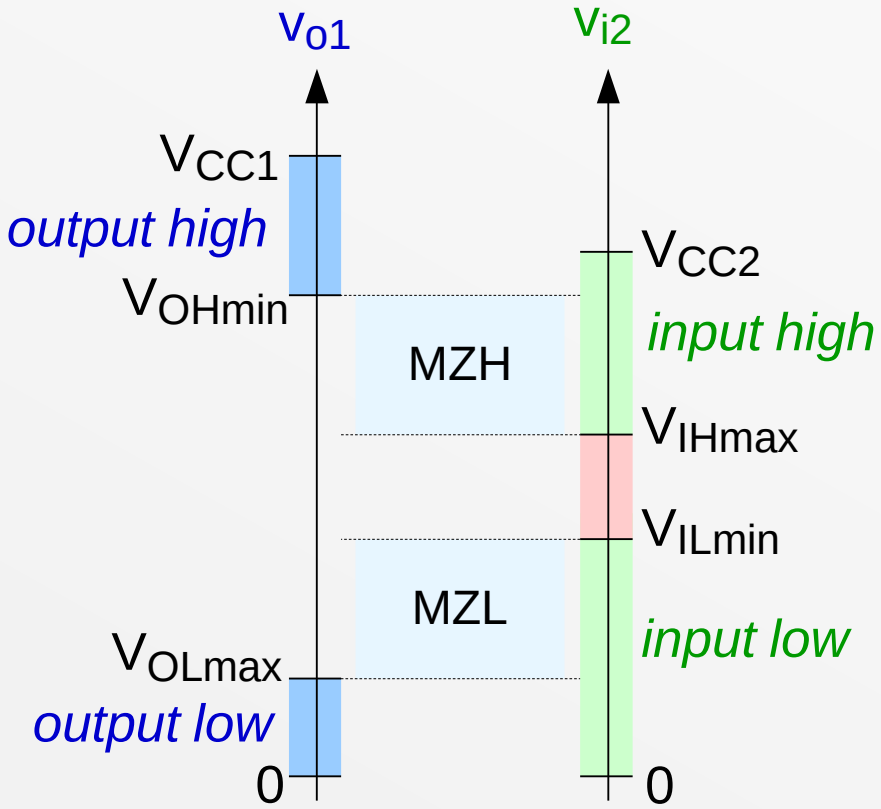


* realizate în aceeași tehnologie și utilizate în aceleași condiții
=> vor fi automat compatibile *(by design)* – vor fi respectate
Condițiile minimale:

$$V_{OH2} > V_{IH1}$$
$$V_{OL2} < V_{IL1}$$

Exemplu: VCC1 > VCC2





Condiție suplimentară
impusă de obicei pentru a nu distruge modulul 2:

$$V_{OH1} \leq V_{CC2}$$

ceea ce înseamnă practic:

$$V_{CC1} \leq V_{CC2}$$

Discuție: VCC1 > VCC2

În general dacă tensiunea de intrare o depășește pe cea de alimentare poarta se poate distruge (există o diodă de limitare internă între intrare și alimentare, prin care poate apărea curent excesiv)

- ex.: porți logice din familia 74HC

Există porți speciale care tolerează tensiuni de intrare mai mari decât cea de alimentare

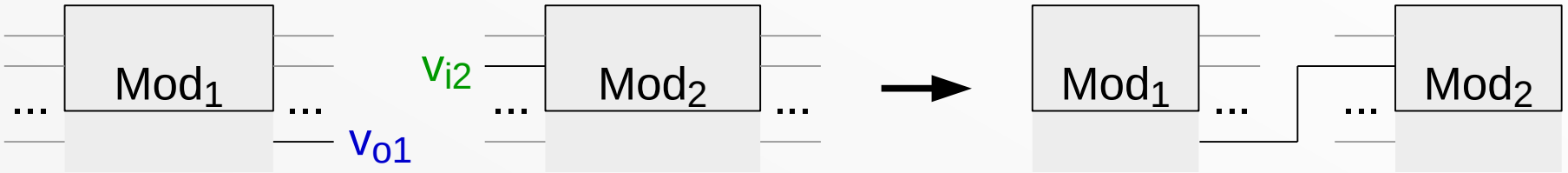
- ex.: porți logice din familiile 74LVC, 74AUP

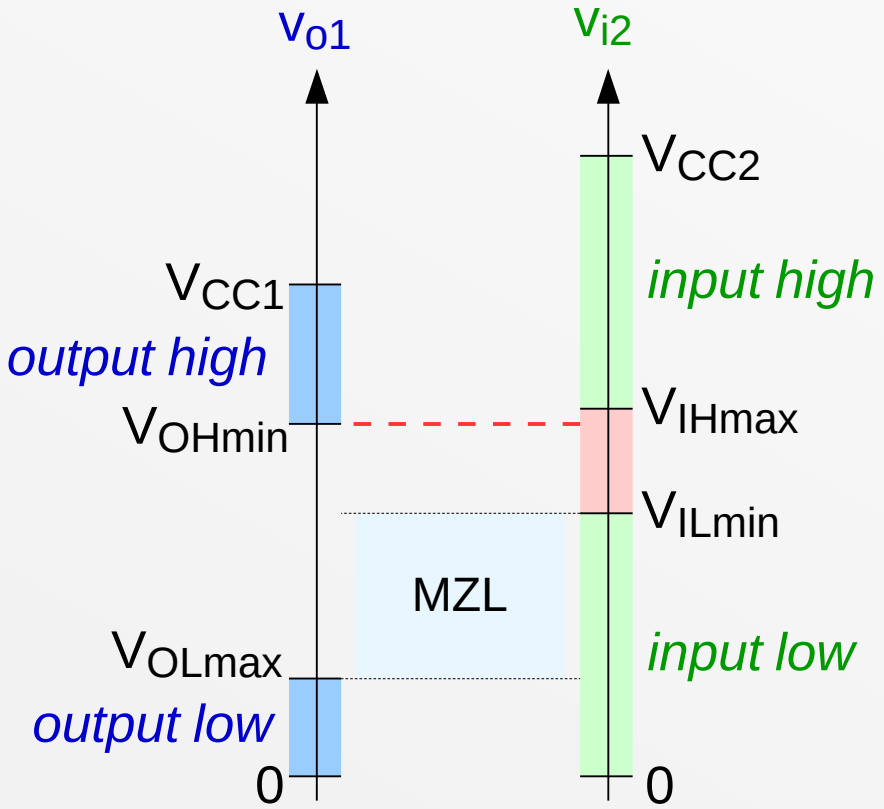
Soluții pentru VCC1 > VCC2

Soluții pentru evitarea depășirii tensiunii maxime tolerate pe intrare:

- utilizarea unui adaptor de nivel *(level shifter)*:
  - cu circuit integrat specializat, de ex. 74HC4050 cu tranzistor cu
  - rezistoare în bază și colector
- utilizarea unui divizor de tensiune sau a unui limitator de tensiune cu rezistor și diode
  - se va avea în vedere evitarea apariției unui curent excesiv prin dimensionarea corectă a rezistoarelor

Exemplu: VCC1 < VCC2





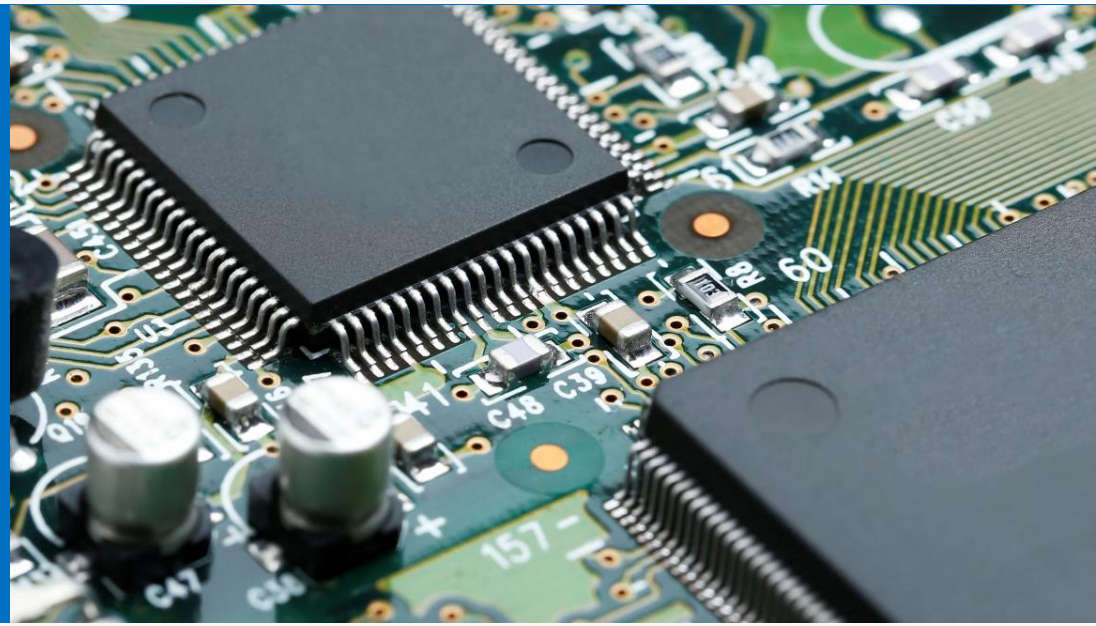
Potențială problemă:
nivelul high al ieșirii este insuficient pentru a garanta citirea ca stare high de către intrare.

Soluție:
adaptor de nivel
(level shifter)

Discuție: VCC1 < VCC2

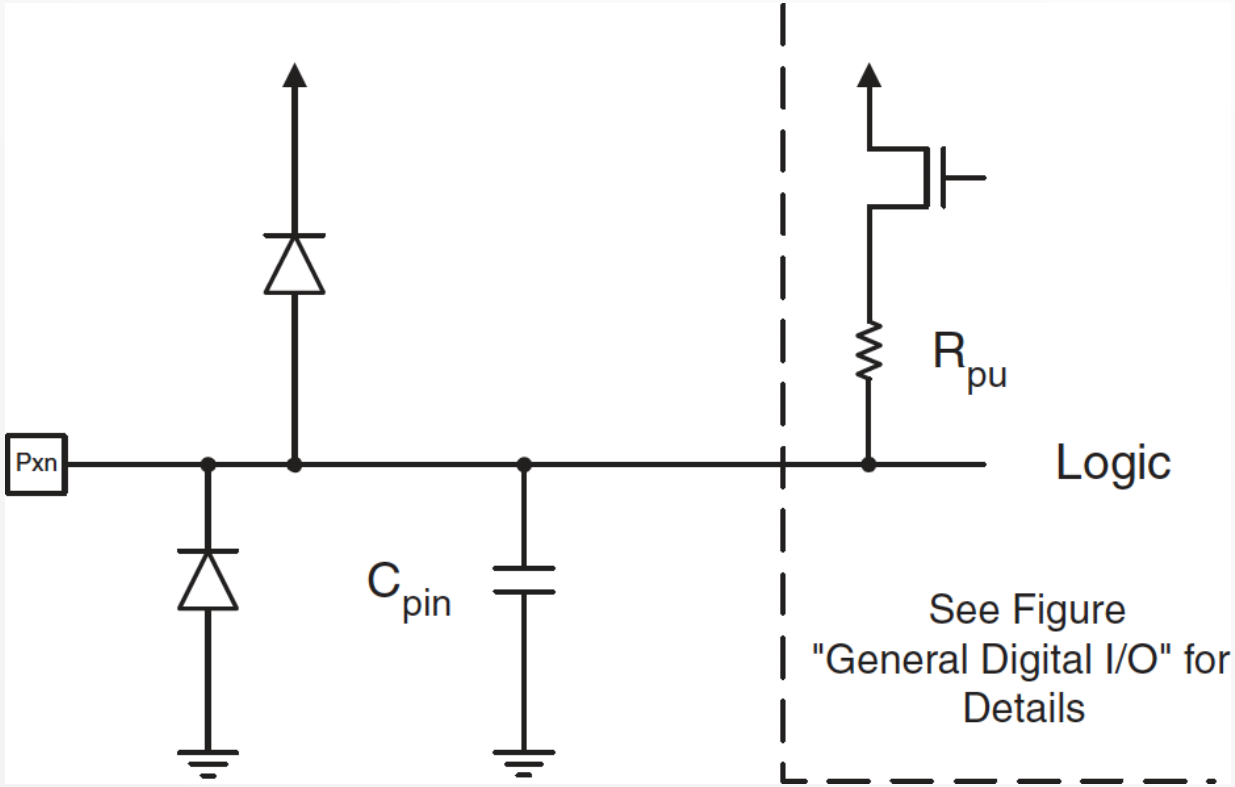
- Ipoteză: condiția **nu** se respectă pe cazul cel mai defavorabil (nu se poate asigura margine de zgomot *high* în toate situațiile posibile).
- În practică este totuși posibil ca sistemul să funcționeze (să avem un caz favorabil sau funcționare la limită).
- Situația poate produce defecte intermitente datorate variației condițiilor de funcționare sau perturbațiilor tranzitorii
  - foarte dificil de depanat!
  - proiectare pe cazul cel mai defavorabil, cu rezervă

Intrări/ieșiri digitale



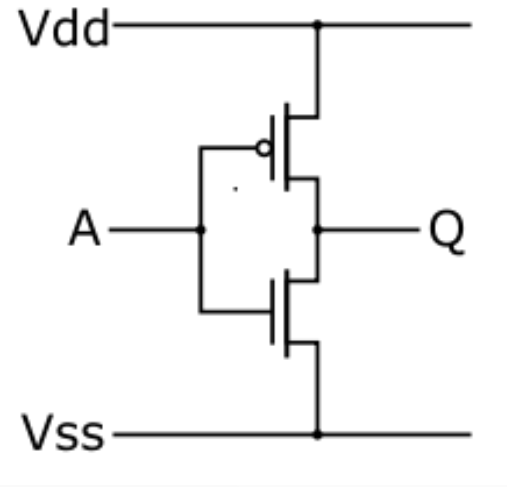
PM

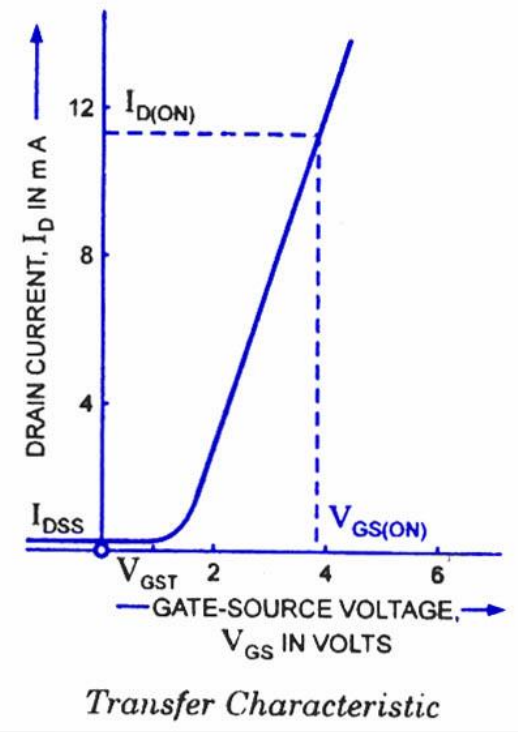
Structura generală pentru in pin I/O



Pini digitali de intrare

- Pinii care sunt configurați ca intrare și lăsați neconectați pot avea, în general, un impact negativ asupra sistemului
- De obicei, aceasta înseamnă doar citiri nedefinite ale portului și un consum mai mare de energie
  - Un port de intrare în interior arată similar cu imaginea de sus: două tranzistoare complementare cu porțile legate împreună la intrarea A
  - Intrarea A este un condensator (mic) care poate fi ușor influențat de cantități mici de încărcare
  - Dacă tensiunea la intrarea A atinge ~ (V_{DD} + V_{SS})/2 ambele tranzistoare conduc un anumit curent de la V_{DD} la V_{SS}





Top image source: http://embeddedfreak.files.wordpress.com/2010/09/220px-cmos_inverter-svg.png

Bottom image source: http://www.globalspec.com/learnmore/semiconductors/discrete/transistors/power_mosfet

Pini digitali de intrare

- Practic, asta înseamnă că acesta nu este un design bun pentru o intrare prin buton
- Când butonul nu este apăsat, acesta lasă pinul de intrare în aer

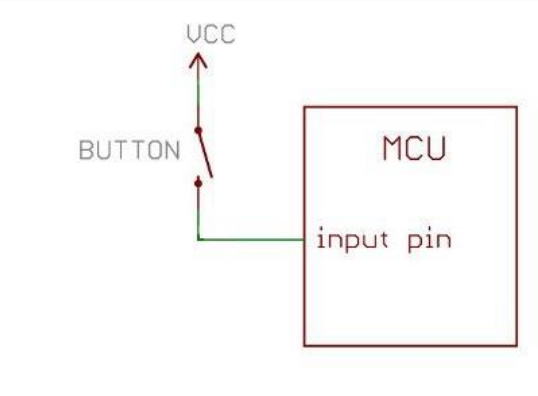
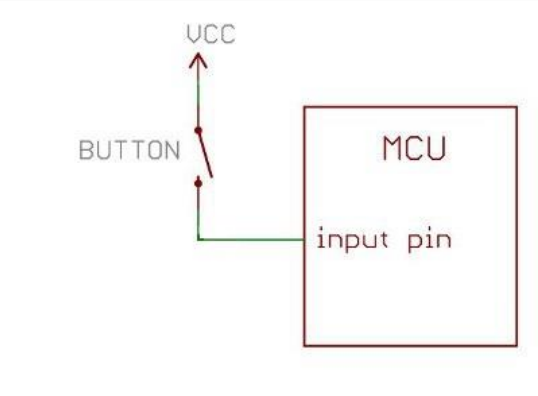


Image source: https://learn.sparkfun.com/tutorials/pull-up-resistors

Pini digitali de intrare

- Practic, asta înseamnă că acesta nu este un design bun pentru o intrare prin buton
- Când butonul nu este apăsat, acesta lasă pinul de intrare în aer
- Designul din dreapta asigură că știftul are o stare bine definită, indiferent de starea butonului
- R1 se numește rezistor „pull-down”



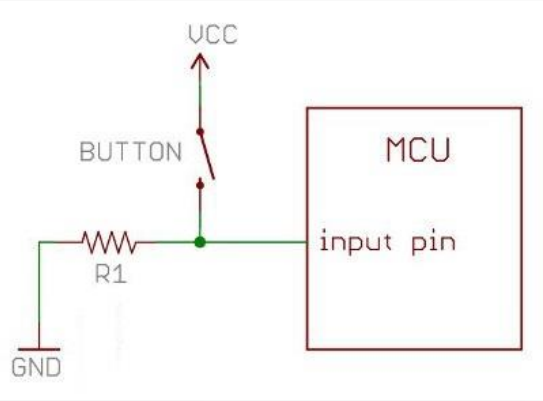
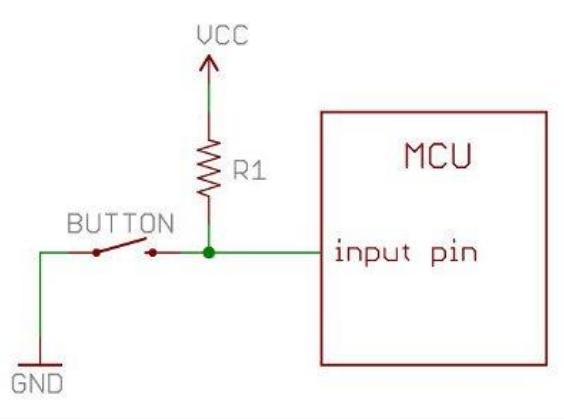


Image source: https://learn.sparkfun.com/tutorials/pull-up-resistors

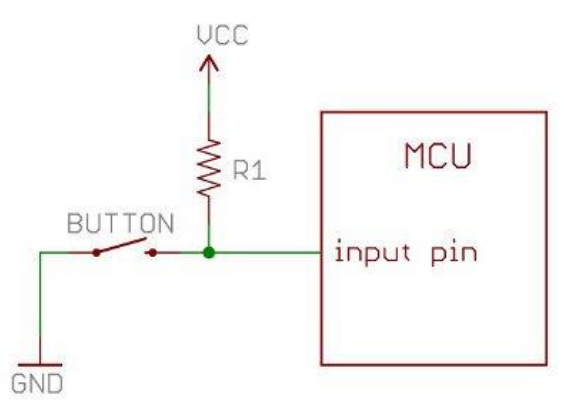
Pini digitali de intrare

- Topologii similare pot fi utilizate cu rezistențe de tip pull-up
- În acest caz, valoarea digitală citită de MCU va fi inversată
  - Butonul apăsat înregistrează valoarea 1
  - Butonul eliberat înregistrează valoarea 0



Pini digitali de intrare

- Topologii similare pot fi utilizate cu rezistențe de tip pull-up
- În acest caz, valoarea digitală citită de MCU va fi inversată
  - Butonul apăsat înregistrează valoarea 1
  - Butonul eliberat înregistrează valoarea 0
- Curentul este disipat prin R1 numai atunci când butonul este apăsat
  - Curent de scurgere de intrare este de ~50nA



Pini digitali de intrare

- Multe MCU-uri includ pull-up-uri care pot fi activate sau dezactivate pentru fiecare pin
- Acest lucru ajută la reducerea numărului de piese și a costului total al sistemului

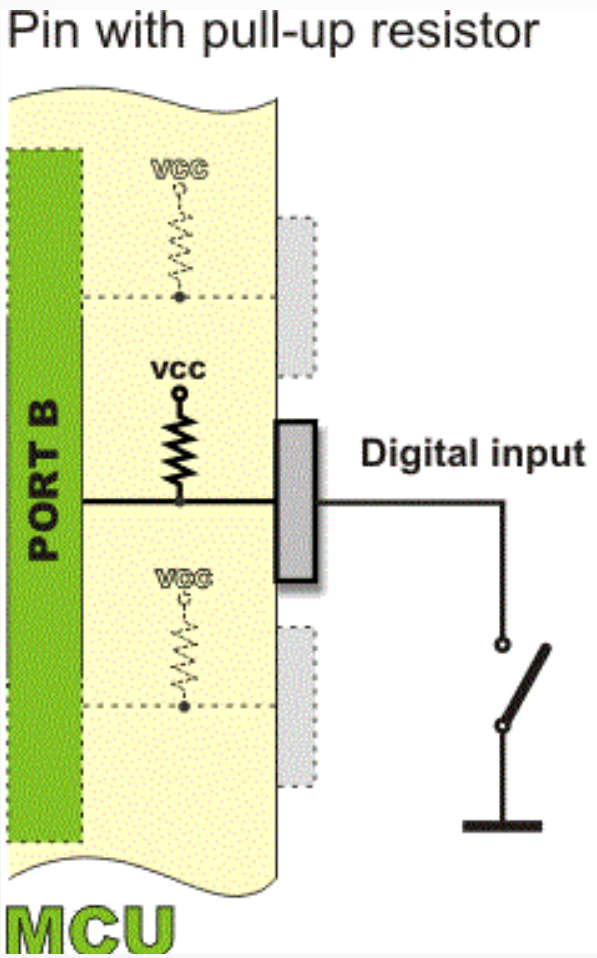
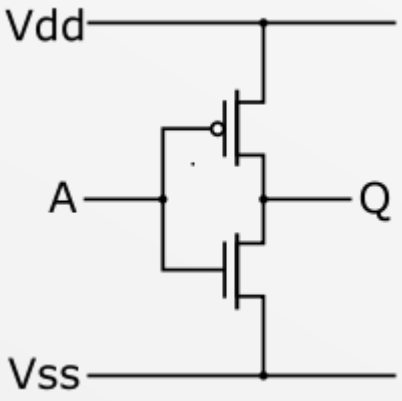


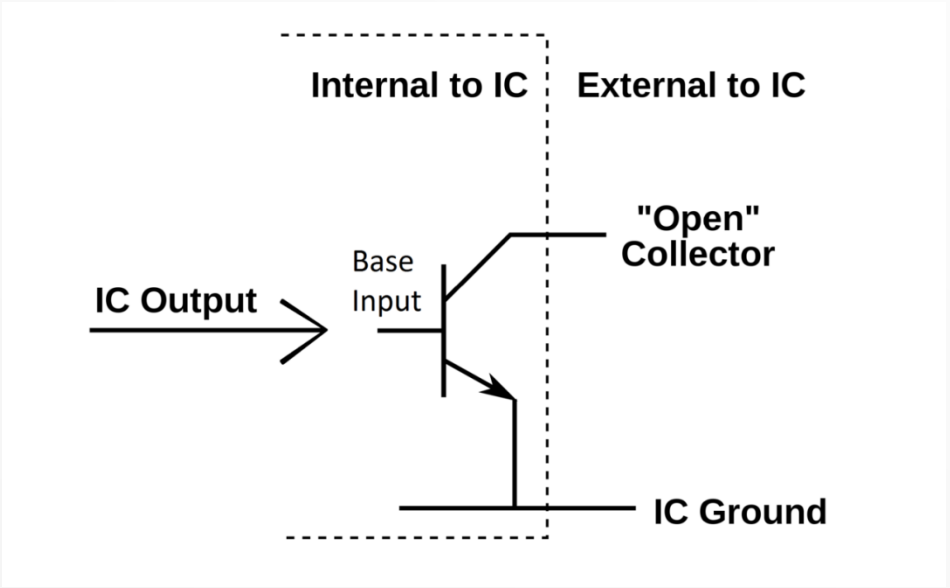
Image source: http://www.mikroe.com/chapters/view/4/chapter-3-i-o-ports/

Pini digitali de ieșire

- O ieșire digitală este un pin configurat pentru a scoate un semnal digital
- Configurațiile tipice de ieșire sunt
  - Push-Pull
  - Open drain / Open collector (ex: I2C)



Q is a push-pull output

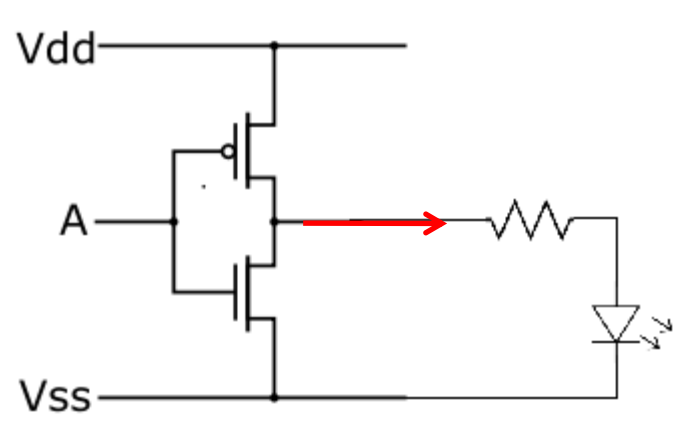


Open collector output

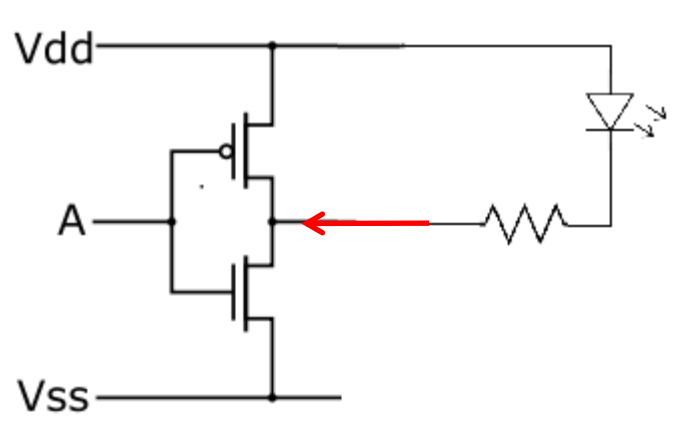
Image source: http://en.wikipedia.org/wiki/Open_collector

Pini digitali de ieșire

- O ieșire push-pull este de obicei o tensiune a fiecăreia
  - Un 0 digital va scoate ~0 volți
  - Un 1 digital va scoate ~V_{DD} volți
    - adică 3.3 V pentru un MCU furnizat la 3.3V
- Pot genera atât curenți de sursă, cât și curenți de scurgere



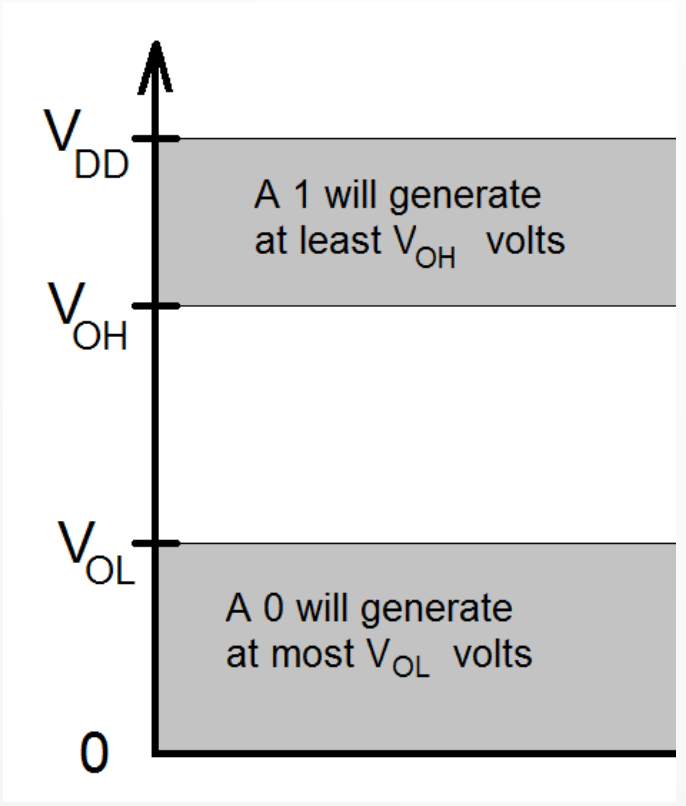
Output sourcing current



Output sinking current

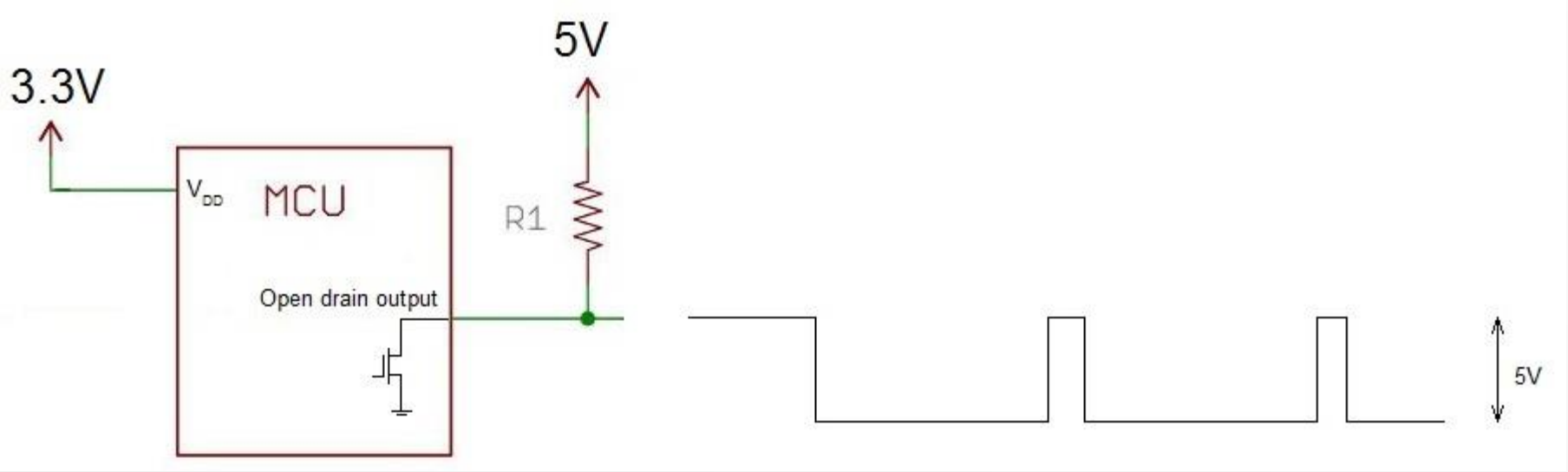
Pini digitali de ieșire

- Similar intrărilor digitale, MCU nu poate genera EXACT 0 V sau 3.3 V (dacă este alimentat de la 3.3 V) la ieșiri
- Intervalele pe care MCU este garantat să le producă sunt date în datasheet pentru diferite valori V_{DD} și curenți sursă/scurgere (exemplu pentru V_{DD} = 3.6V)
  - V_{OH} = min. 3V, sursă 3mA
  - V_{OL} = max. 0.4V, scurgere 8.5mA



Pini digitali de ieșire

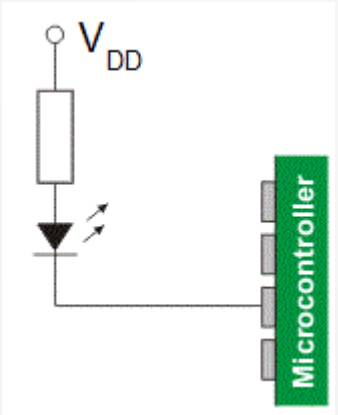
- Ieșirile Open drain / Open collector folosesc un singur tranzistor la ieșire
- În unele cazuri, acest lucru permite MCU să fie alimentat de la 3.3 V și să genereze semnale de 5V

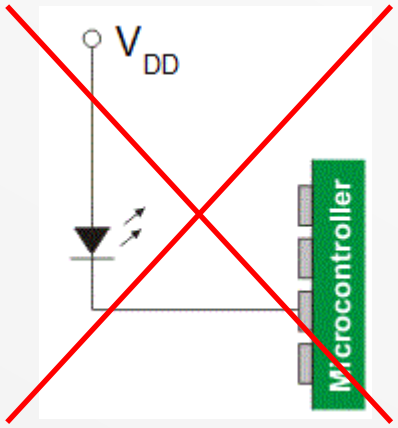


Pini digitali de ieșire

- AVERTISMENT: O ieșire digitală nu poate genera/scurge o cantitate nelimitată de curent. Nu este furnizat niciun circuit de limitare în MCU!
- Trebuie avut grijă pentru a proteja pinii de ieșire de curenți mari.

This is OK. Resistor limits LED and pin current

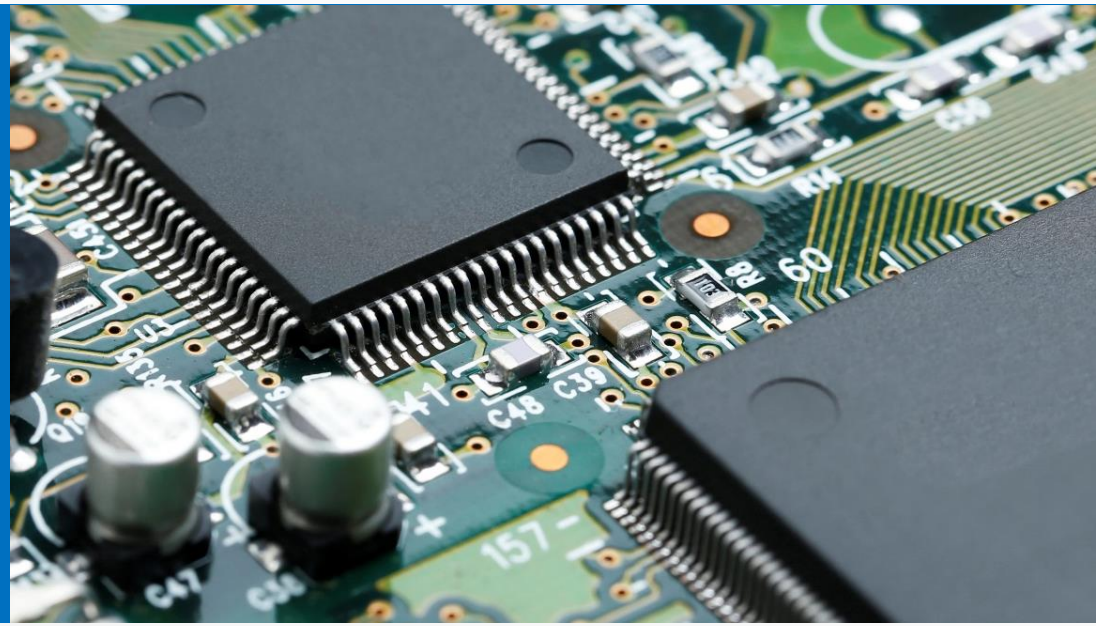




This is NOT OK. LED does not limit current and failure of LED, MCU or both can occur.

Image source: http://www.mikroe.com/chapters/view/69/chapter-6-examples/

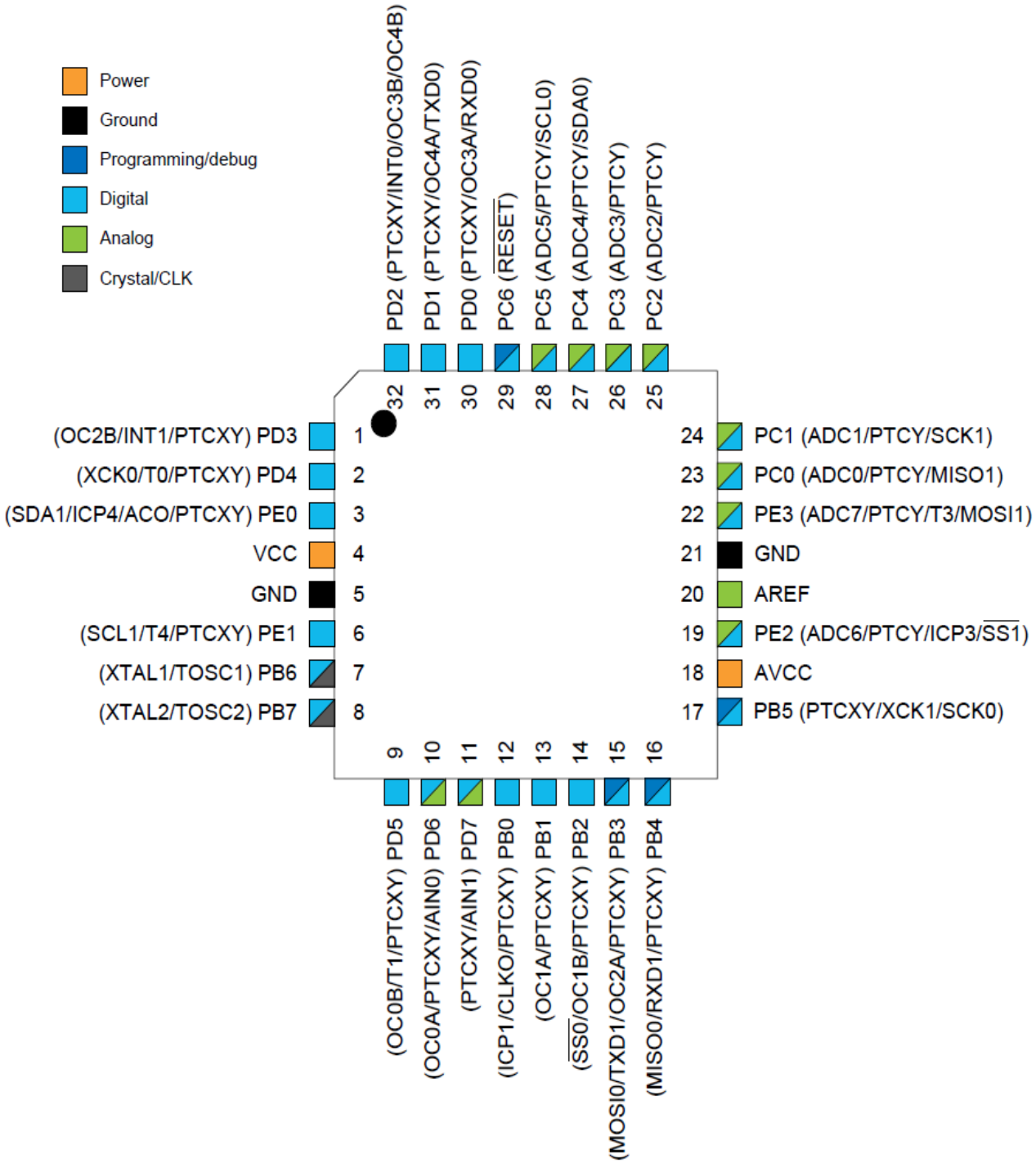
328P GPIO



PM

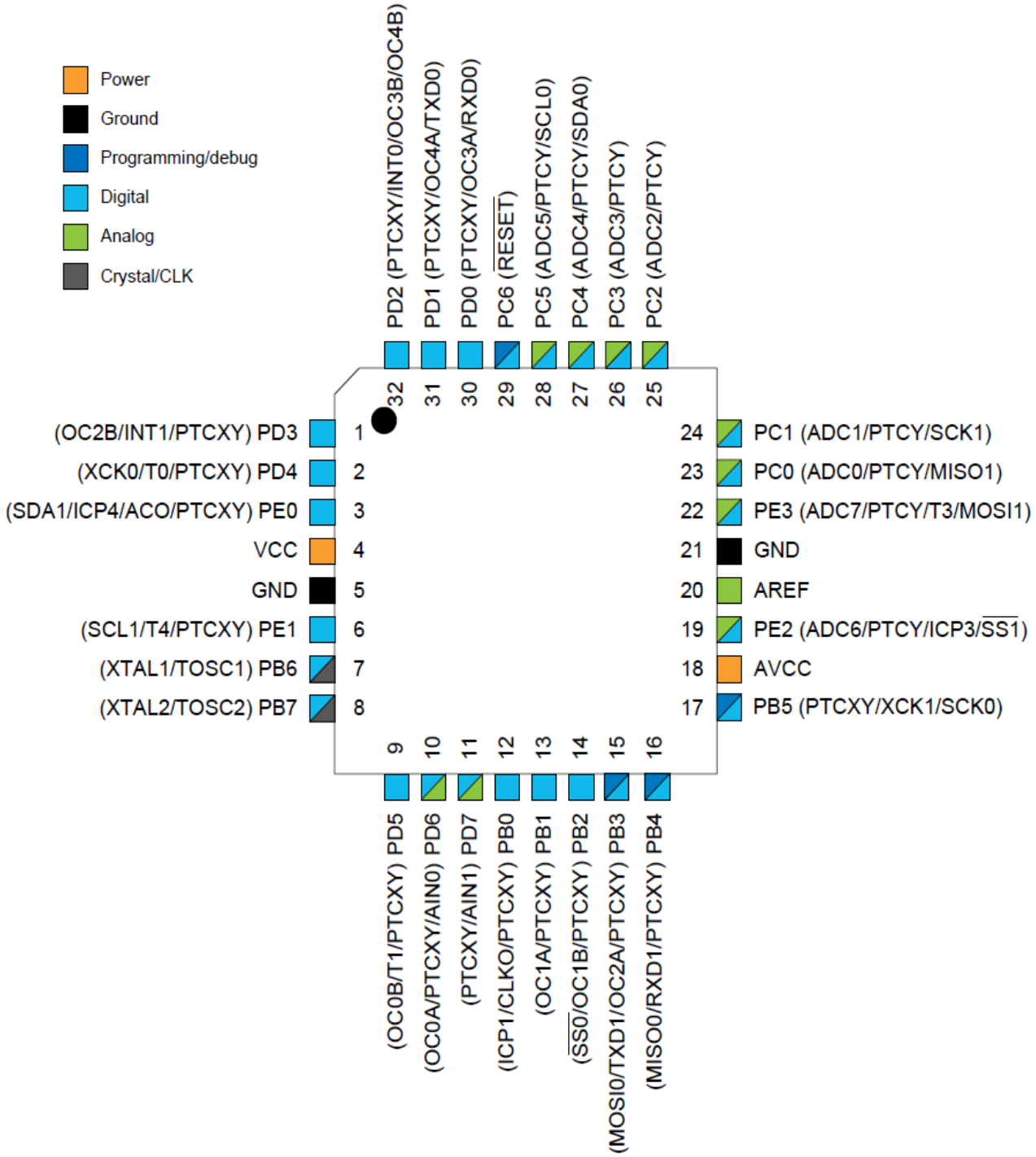
Pini GPIO pentru microcontroler

- Majoritatea pinilor de pe orice microcontroler pot fi configurați ca intrare/ieșire generală (GPIO).
- Pe ATmega328P (în dreapta) toți pinii albaștri pot fi configurați pentru intrare sau ieșire digitală (GPIO).
- Acest lucru prezintă o problemă pe măsură ce configurația devine mai complexă. Pinii trebuie selectați pentru funcționalitatea GPIO dacă sunt partajați cu alte periferice.
  - Toată configurația este sub control software.

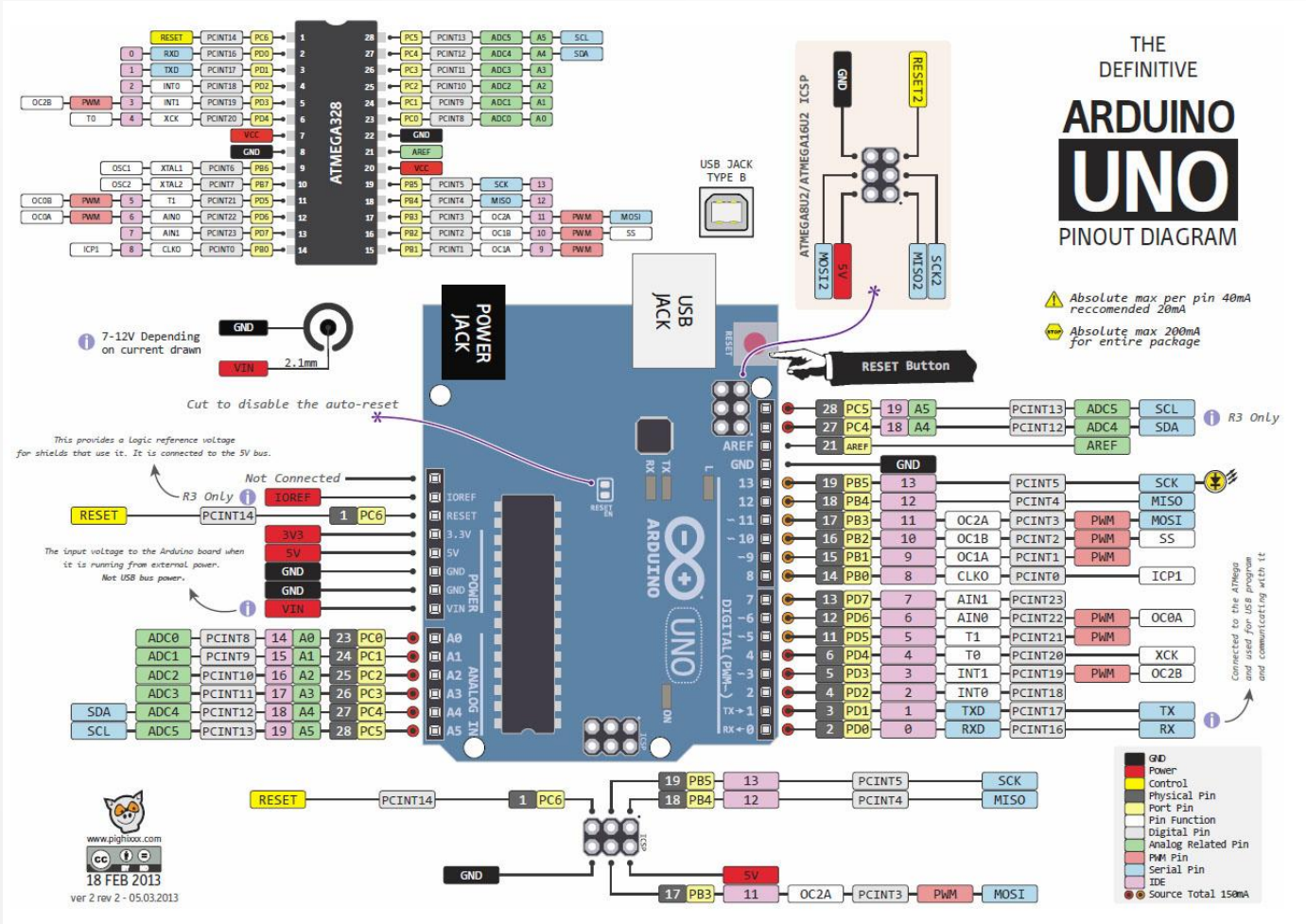


Pini GPIO pentru microcontroler

- Un pin care este selectat pentru funcționalitatea I/O digitală poate fi configurat pentru
  - Input
    - Pull-up
    - No pull-up
  - Output
    - Push-pull
    - Open-drain



Pini GPIO Arduino



Pini GPIO pentru microcontroler

- Există trei registre asociate cu funcționarea pinii portului ca I/O-uri digitale:
  - **DDRx** - registru de direcție a datelor. Dacă bitul de direcție a datelor este 1, pinul este o intrare
    - DDRB &= ~(1<<DDB7); // Face pinul PB7 de intrare
    - DDRB |= (1<<DDB5); // Face pinul PB5 de ieșire
  - **PORTx** – (1<<DDB5); // Face pinul PB5 o ieșire
  - PORTx – registru de date port. Dacă un pin este configurat pentru ieșire, setarea bitului de blocare la 1 face ca ieșirea pinului să fie de ~V_{DD} volți
    - PORTB |= (1<<PORTB5); // Aprinde LED-ul
  - **PINx** - citirea pinii portului.
    - if (PINB & (1<<PINB7) == 1) {...} // Este apăsat un buton.

Pini GPIO pentru microcontroler

13.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

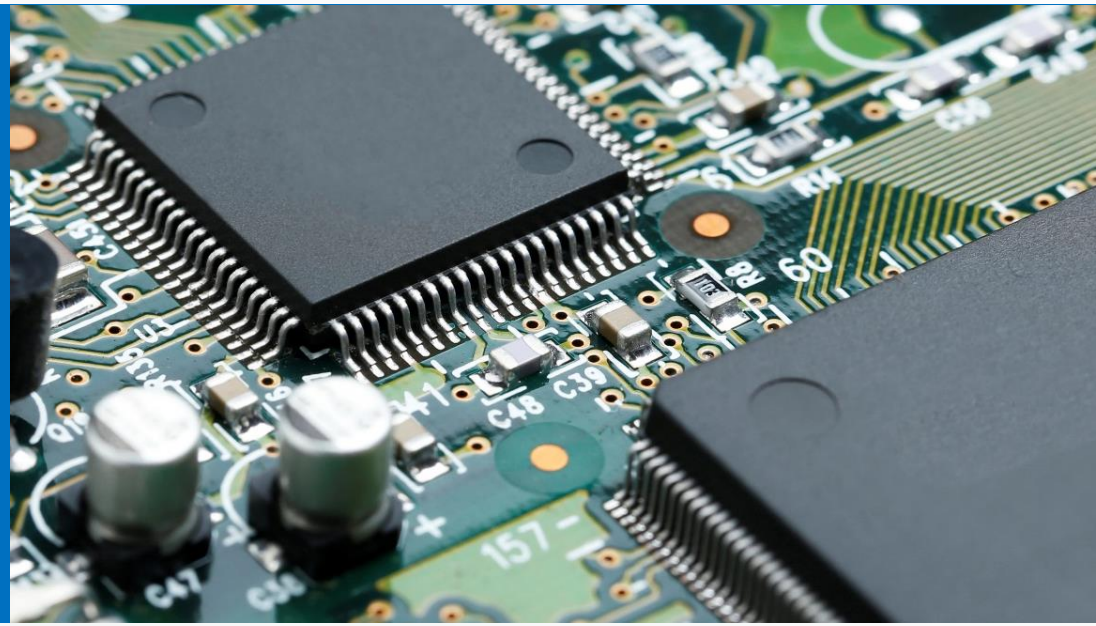
13.4.4 PINB – The Port B Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

Controlul registrilor vs digitalWrite



Introducere în software

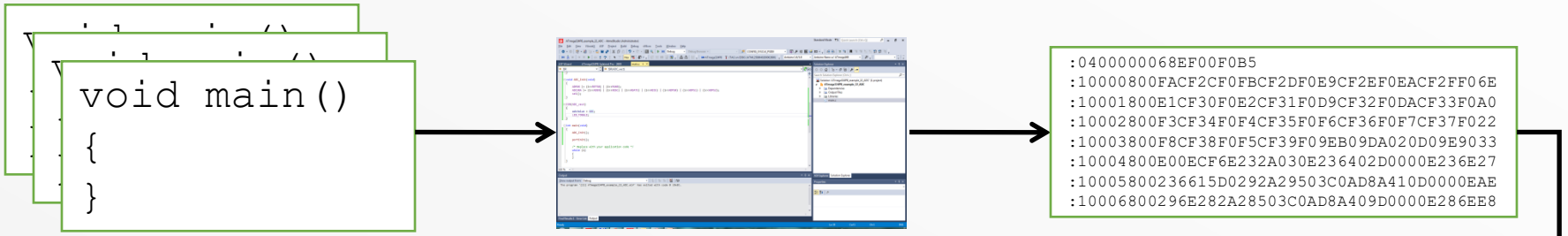


PM

Software încorporat

- Tinde să fie foarte specific aplicației
  - Vine sub formă de blob, care conține date, configurație, aplicație și drivere
  - În timp ce unele sisteme de operare (în sensul adevărat, gestionat de un package manager) există pentru dispozitive încorporate, ele sunt foarte rare
- Utilizează hardware specializat pentru a-și atinge scopul
  - DSP-uri pentru procesarea audio/video
  - Periferice on-chip/off-chip (ADC-uri/DAC-uri pentru achiziție de date, redare audio, atingere capacitivă)
  - Afișaje, butoane pentru interfețele utilizator
- Este mult mai strâns cuplat la hardware decât software-ul PC/server
  - Acest lucru permite fișiere binare mai mici (mai puține abstracții), dar compromisul este un cod mai puțin portabil (mai puține abstracții)
  - Trebuie proiectat în paralel cu hardware-ul

Software încorporat



project source files

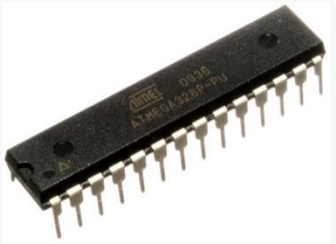
IDE, compiler, MCU specific definitions

binary file (executable)

- Dezvoltarea software-ului încorporat implică atât instrumente software, cât și hardware:
  - IDE, compilator
  - Dispozitiv de programare
  - Dispozitiv și placa țintă



programming device



target device

Flow-ul unui program pe AVR

- Încărcarea programului

  Cu ajutorul unui programator extern sau a unui bootloader, binarul de AVR trebuie să ajungă în memoria de program

- Lansarea în execuție

  La resetarea microcontroller-ului, se pornește execuția începând cu o adresa prestabilită

- Fire de execuție

  Unul singur, dat de registrul PC (Program Counter)

- Interacțiune cu exteriorul

  Doar prin instrucțiuni in/out (port-mapped I/O)

Metode de programare

- Programul poate fi pus în memoria de program cu ajutorul unui In-System Programmer sau a unui debugger extern
- Alternativ, putem folosi un bootloader, un program care funcționează doar dintr-o parte a memoriei de program și care poate scrie în restul memoriei
- Fiind arhitectură Harvard, în mod normal nu există instrucțiuni de acces la memoria de program pentru scriere
- În loc de instrucțiuni de **ld** și **st** (load și store), avem **lpm** și **spm**

Provocările unui bootloader

- Bootloaderul ocupă spațiu din memoria de program
- Dacă bootloader-ul ocupă 2kB, rămân 30kB pentru programul nostru
- Memoria de program este în tehnologie Flash, deci trebuie scrise datele în pagini (dimensiunea depinzând de chip)

Mediul de programare

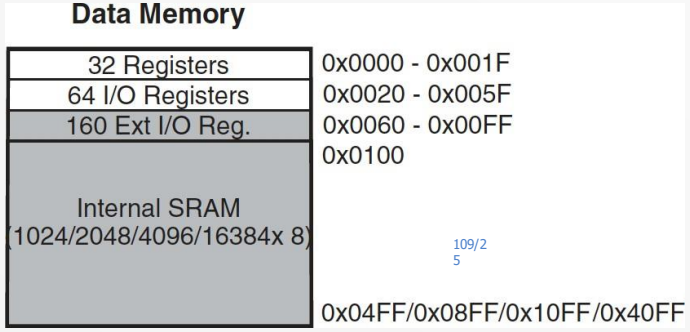
- În C, foarte similar cu programarea pe PC
- Input/Output
  - Nu avem consolă
  - Poate avem LED-uri conectate la microcontroller
  - Putem avea un LCD
  - Serială, etc.
- Variabile
  - Le putem avea în RAM (2kB)
  - Le putem avea și în Flash (32kB, împreună cu codul)
- Pornire la reset (sau după bootloader)
  - GCC este configurat pentru un sistem pe 8 biț i (int și pointerii sunt pe 16 biți)
  - Avem macrodefiniții pentru toate adresele relevante de I/O, compilatorul știe automat să folosească instrucțiunile dedicate pentru scrierile și citirile de la aceste adrese

```
#define PORTC *(volatile uint8_t *) (0x020)
```

Variabile

- Cum clasificăm variabilele într-un mediu de programare pe PC
  - În funcție de alocarea memoriei lor?
- Variabilele pot fi locale funcției → pe stivă
- Variabilele pot fi statice → secțiunea .bss
- Variabilele pot fi alocate dinamic → heap
  - Presupune adăugarea codului pentru alocatorul de memorie
    - Memory overhead pentru un dispozitiv care are foarte puțină memorie în general
- Constante! Le putem ține în memoria de program
- Stocare non-volatilă există și în EEPROM
  - Citire și scriere greoaie
  - 1kB de memorie
  - număr destul de limitat de scrieri (100 000x)
  - util pentru ce trebuie citit/scris doar o dată per restart

Memoria



- 32 registre general purpose accesibile și ca memorie
- 64 registre I/O accesibile și cu in/out
- 160 de registre I/O extra
- 2048 de octeți de memorie SRAM

Tipuri de date C

- Tipul de date integer din limbajul C nu este suficient de bine definit pentru utilizare în dispozitive încorporate
  - int trebuie să aibă „dimensiunea de cel puțin 16 biți”
  - long int trebuie să aibă „dimensiunea de cel puțin 32 de biți”
- În dispozitive încorporate folosim tipuri definite de stdint-gcc.h
  - int8_t, uint8_t
  - int16_t, uint16_t
  - int32_t, uint32_t
- Acestea au dimensiuni fixe, astfel încât știm exact pentru ce intervale pot fi utilizate și care este utilizarea memoriei.
  - MCU-urile au de obicei cantități foarte limitate de RAM, așa că utilizarea trebuie menținută la minimum
  - Registrele au dimensiuni fixe: valorile transferate trebuie să aibă sens.

Tipuri de date C

- Proiectarea software-ului încorporat necesită uneori mutarea de biți
  - (adică operatorii „<<” și „>>” din C)
- Aceasta poate fi o problemă. Luați în considerare următorul cod:
- Q1: Ce valoare are variabila b?

```
uint8_t a;
uint32_t b;

a = 0x01;
b = a << 24;
```

Tipuri de date C

- Proiectarea software-ului încorporat necesită uneori mutarea de biți
  - (adică operatorii „<<” și „>>” din C)
- Aceasta poate fi o problemă. Luați în considerare următorul cod:
- Q1: Ce valoare are variabila b?
- A1:
  - Depinde de arhitectură și compilator
  - Modul corect de a face acest lucru este
    - b = (uint32_t) a << 24;
    - apoi b == 0x01000000 pe orice arhitectură și compilator

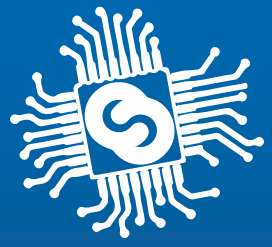
```
uint8_t a;
uint32_t b;

a = 0x01;
b = a << 24;
```

La cursul următor

- Software programming
- Debugging
- Button de-bounce
- Interrupt requirements
- ISR & atomicity
- Timers
- PWM

Cursul #4



PM

Disclaimer

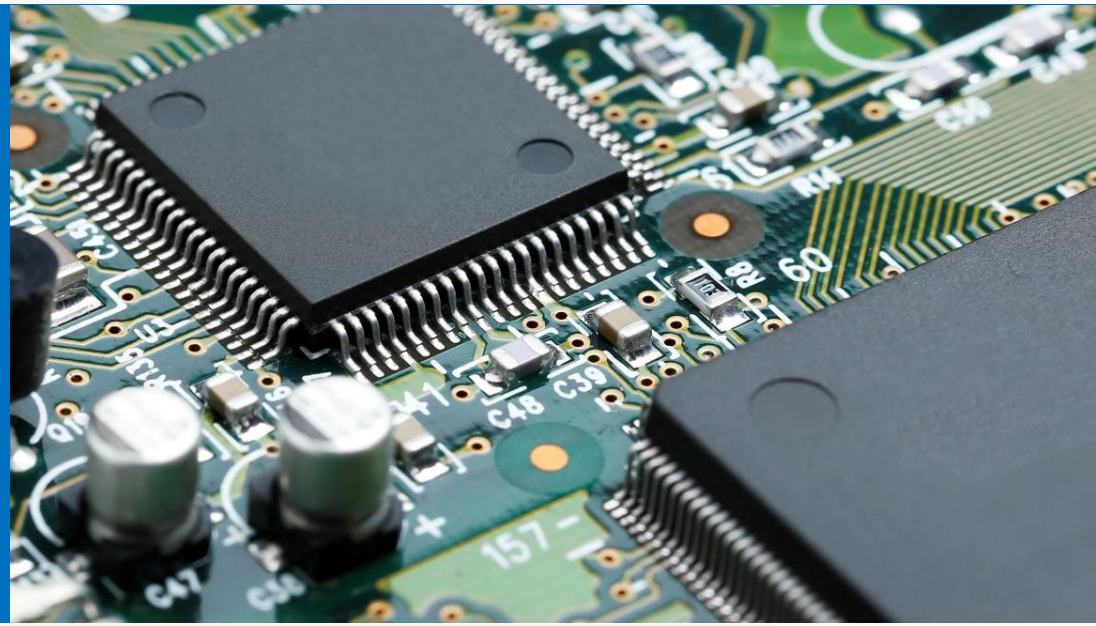
- *Slide-urile acestea reprezintă un rezumat.*
- *Slide-urile nu acoperă toate explicațiile, simulările, demonstrațiile oferite în timpul cursului.*
- *Slide-urile nu limitează în vreun fel materia cerută la examen.*
- *Pentru varianta completă, sunteți așteptați la curs.*

© Elemente din curs preluate din:
- *notițe curs PM 2021, Dan Tudose*
- *notițe curs PM 2021, Andrei Voinescu*
- *scoala de vara eap 2014 - Mihai Cuciuc & 2016, Marius Nicolae*
- *notite curs EEA/ED –Razvan Tataroiu, Daniel Rosner*

Cuprins

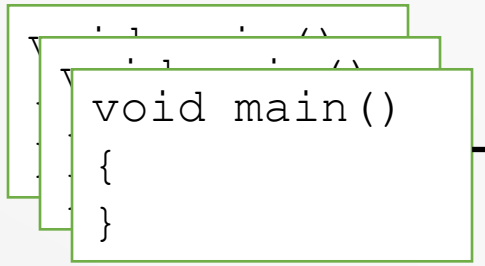
- Programare Software
- Depanarea
- Efectul de de-bounce
- Întreruperi
- ISR și atomicitate

Programare Software

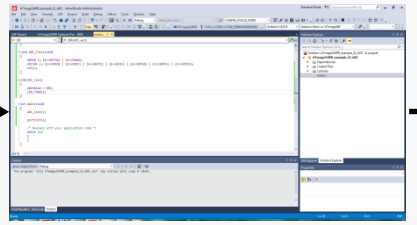


PM

Stiva software Arduino



project source files



IDE, compiler, MCU specific definitions

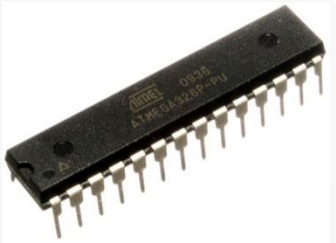
```
:0400000068EF00F0B5
:10000800FACF2CF0FBCF2DF0E9CF2EF0EACF2FF06E
:10001800E1CF30F0E2CF31F0D9CF32F0DACF33F0A0
:10002800F3CF34F0F4CF35F0F6CF36F0F7CF37F022
:10003800F8CF38F0F5CF39F09EB09DA020D09E9033
:10004800E00ECF6E232A030E236402D0000E236E27
:10005800236615D0292A29503C0AD8A410D0000EAE
:10006800296E282A28503C0AD8A409D0000E286EE8
```

binary file (executable)





programming device



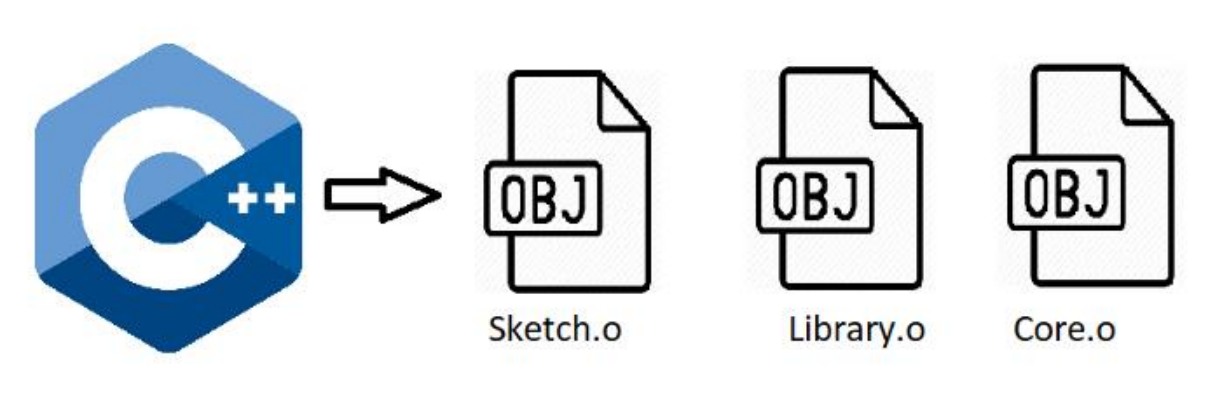
target device

Arduino - preprocesare

- .ino to .C++
- Adaugă Arduino.h
- Adaugă directiva #Line
- Adăugă prototipul funcției pentru toate funcțiile

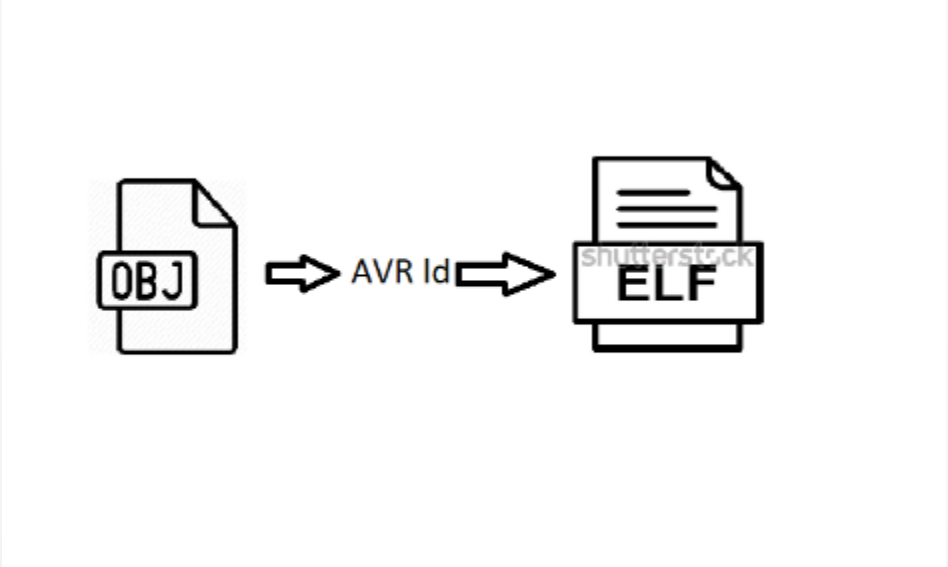
Arduino - compilare

- AVR-GCC Toolchain
- Compilator, asamblator, linker, biblioteca standard C, biblioteci de matematică și multe altele...



Arduino - linkare

- AVR-ld (GNU linker)
- Formatul ELF (Executable Linkable Format)



Arduino – extragere ELF

- AVR-Objcopy: convertește ELF la EEP și HEX
- Fișiere EEP – stochează informații pe medii EEPROM (Electrically Erasable Programmable Read-Only Memory)



Sumar pasi

- .ino
- .cpp
  - + #include Arduino.h
  - + prototip functie – pentru toate functiile
  - + int main
  - + #line (source map) //for warning or error messages
- .i
- .s
- .obj
- .elf (executable and Linkable Format)
- .hex (fisierul urcat in memoria flash) + .eep

Arduino - flashing

- AVRDUDE: descarcă și încarcă datele pe microcontrolerele AVR de la Atmel
- Funcționează cu memorie Flash și EEPROM
- Selectați placa corectă și portul serial pentru încărcare
  - Tools > Board
  - Tools > Port
- Resetați automat și începeți încărcarea
- Bootloader Arduino

Programare AVR fără IDE: cerințe preliminare

- Windows:
  - Atmel AVR Toolchain pentru Windows
  - pacman -S make mingw-w64-i686-avrdude
- Linux:
  - Ubuntu: sudo apt-get install gcc-avr avr-libc avrdude
  - Arch Linux: sudo pacman -S avr-gcc avr-libc avrdude
- Mac:
  - brew install avrdude
  - xcode-select --install
  - brew tap osx-cross/avr
  - brew install avr-gcc

Programare AVR fără IDE

- main.c – blink LED conectat la PD1
- Makefile – make; make program

```c
#define F_CPU 20000000    // AVR clock frequency in Hz, used by util/delay.h
#include <avr/io.h>
#include <util/delay.h>

int main() {
    DDRD |= (1<<1);        // set LED pin PD1 to output
    while (1) {
        PORTD |= (1<<1);   // drive PD1 high
        _delay_ms(100);    // delay 100 ms
        PORTD &= ~(1<<1);  // drive PD1 low
        _delay_ms(900);    // delay 900 ms
    }
}
```

```
MCU=atmega328p
PORT=$(shell pavr2cmd --prog-port)
CFLAGS=-g -Wall -mcall-prologues -mmcu=$(MCU) -Os
LDFLAGS=-Wl,-gc-sections -Wl,-relax
CC=avr-gcc
TARGET=main

all: $(TARGET).hex

clean:
	rm -f *.o *.elf *.hex

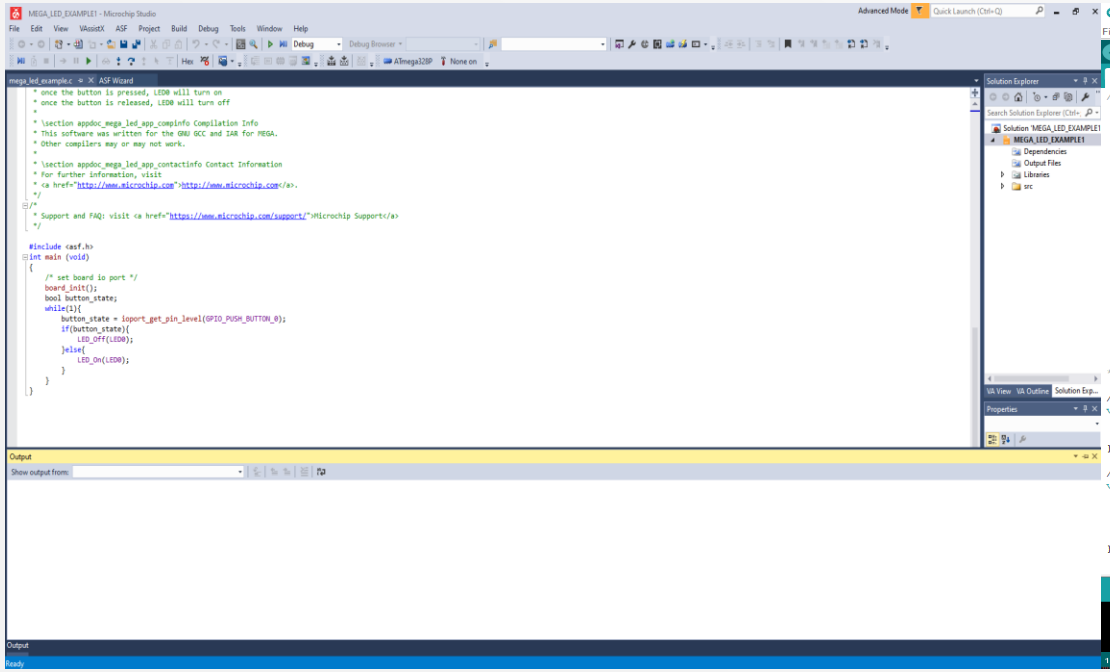
%.hex: %.elf
	avr-objcopy -R .eeprom -O ihex $< $@

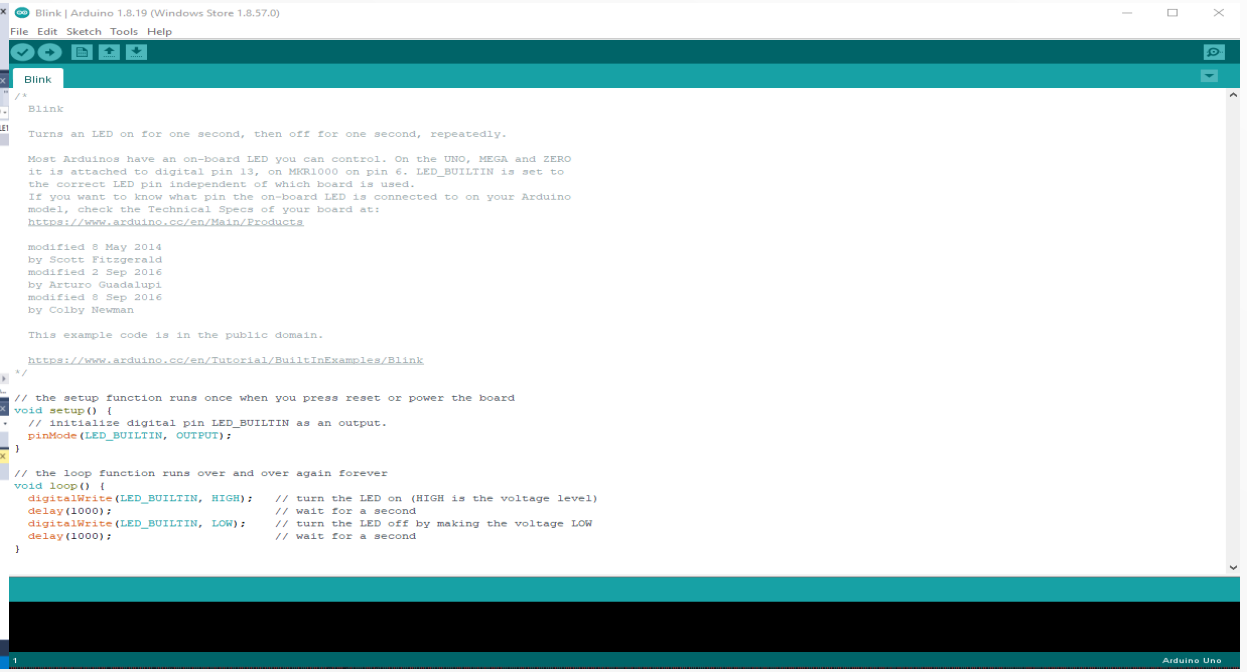
$(TARGET).elf: $(TARGET).o
	$(CC) $(CFLAGS) $(LDFLAGS) $^ -o $@

program: $(TARGET).hex
	avrdude -c stk500v2 -P "$(PORT)" -p $(MCU) -U flash:w:$<:i
```

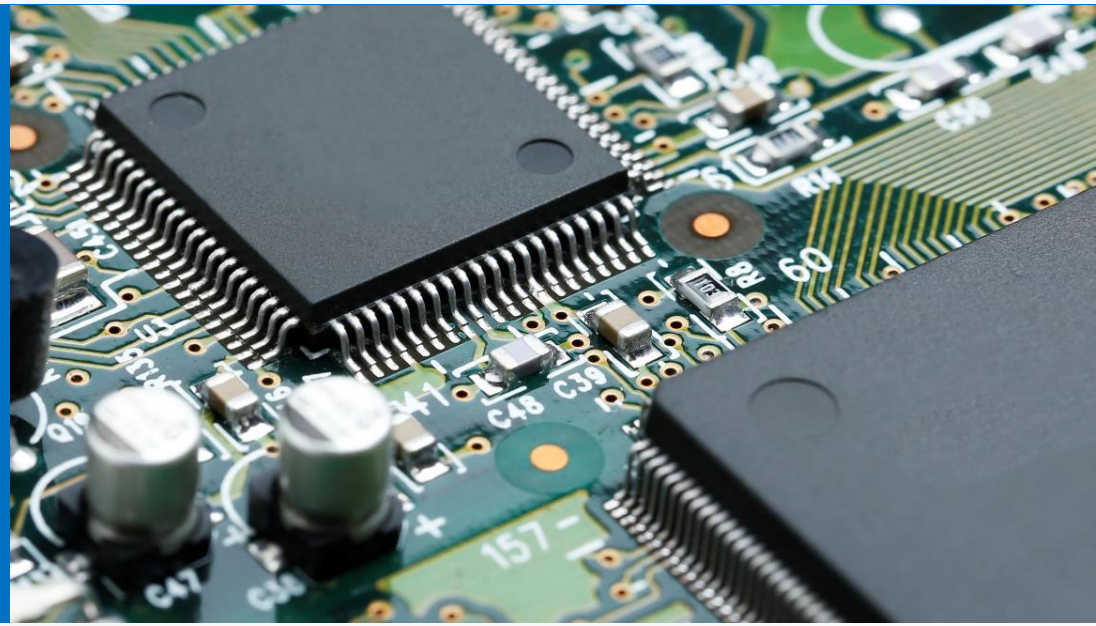
Programare AVR cu IDE

- Arduino IDE
- Microchip Studio
- ...





Depanarea în embedded



PM

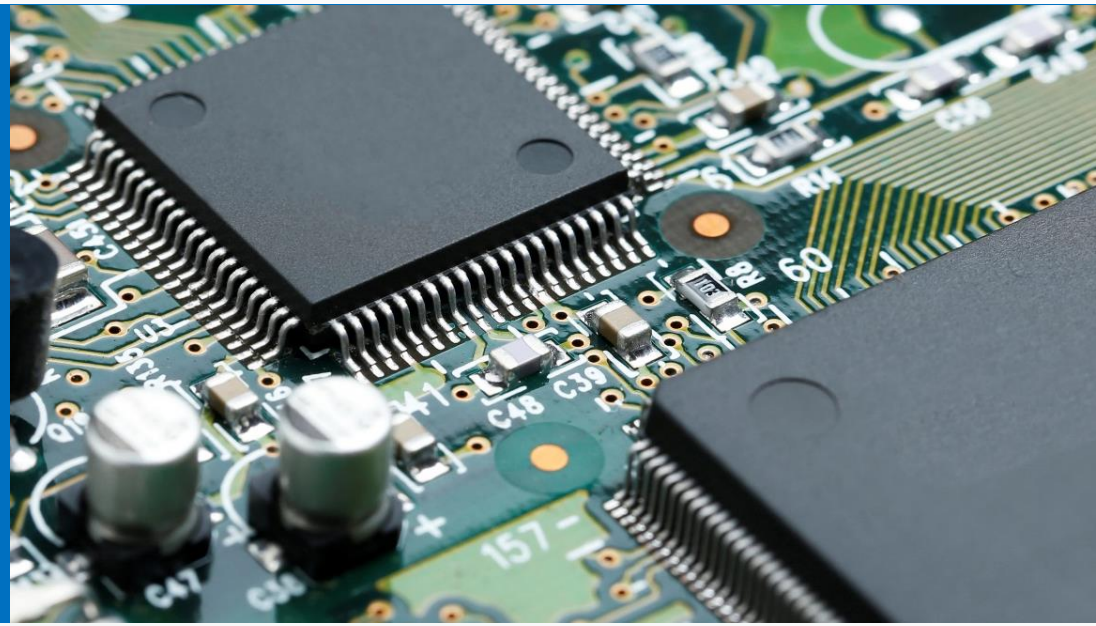
Diferențe față de depanarea uzuală

- Hardware și software specializat
- Debugger specializat (Lauterbach) + configurații particulare (propriul limbaj de configurare)
- Comportament invaziv hardware
- Erori hardware
- Lipsă interfață seriala/grafică

Instrumente specializate

- LED
- Mesaje USART, Bluetooth etc.
- Debugger JTAG
- Loopbacking (conectarea ieșirilor la intrări)
- Multimetre/osciloscoape
- Analizatoare logice/de protocol

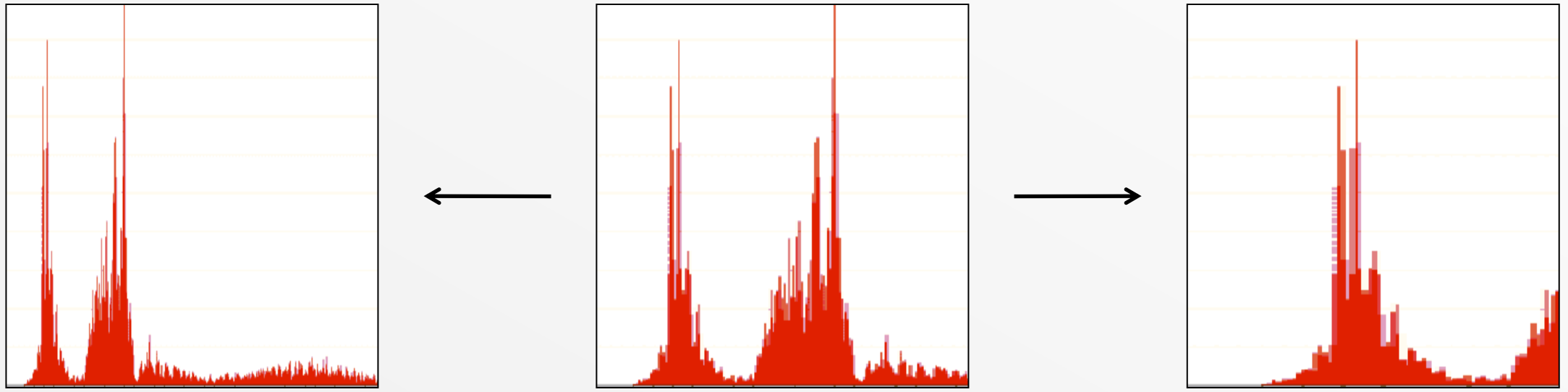
Contextul unei întreruperi



PM

Discutie

- Transformata Fourier a unui semnal
- Durata ~ 700 ms
- Urmărește intrarea utilizatorului pentru a selecta intervalul de frecvență (mărire/micșorare)



User pressed "zoom out" Initial view User pressed "zoom in"

Image source: http://www.dosits.org/science/soundmeasurement/viewed/

Discutie

- Folosim doua butoane pentru zoom in/out
- doFFT() – dureaza ~700ms
- Codul este următorul:

```
void main()
{
    TRISBbits.TRISB7 = 1;
    TRISBbits.TRISB8 = 1;

    while(1)
    {
        if (PORTBbits.PORTB7 == 0)
        {
            fftZoomIn();
        }

        if (PORTBbits.PORTB8 == 0)
        {
            fftZoomOut();
        }

        doFFT();
    }
}
```

Discutie

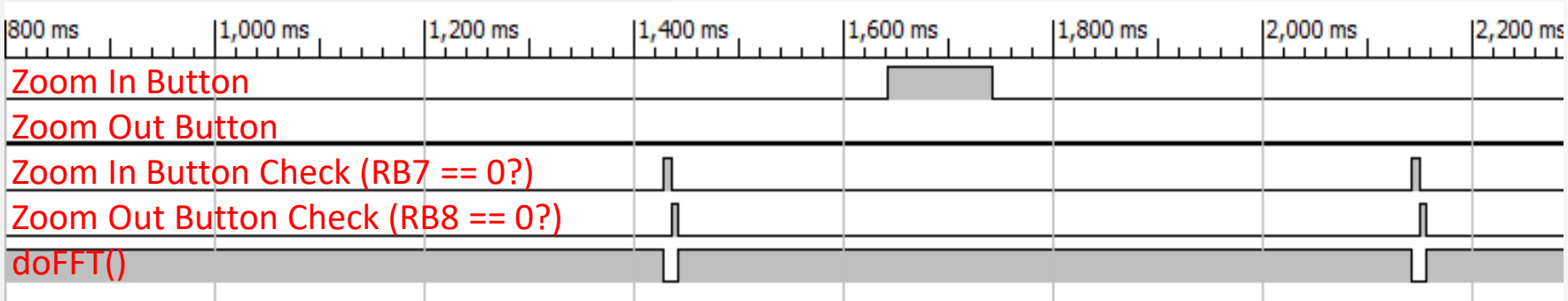
- Folosim doua butoane pentru zoom in/out
- doFFT() – dureaza ~700ms
- Codul este următorul:

```
void main()
{
    TRISBbits.TRISB7 = 1;
    TRISBbits.TRISB8 = 1;

    while(1)
    {
        if (PORTBbits.PORTB7 == 0)
        {
            fftZoomIn();
        }

        if (PORTBbits.PORTB8 == 0)
        {
            fftZoomOut();
        }

        doFFT();
    }
}
```



Discutie

- Cineva și-a cronometrat apăsările de buton și a avut o medie de ~110 ms

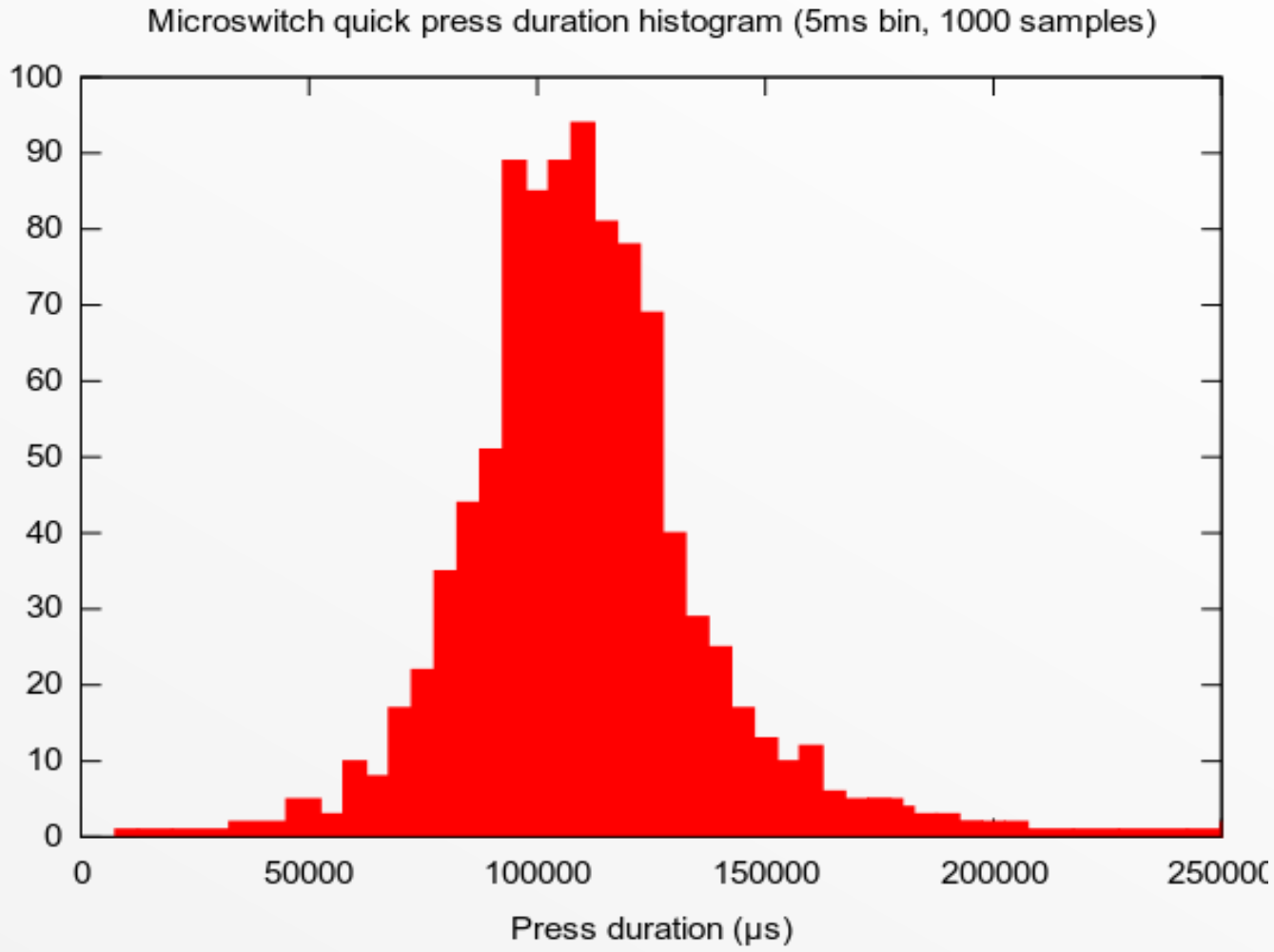
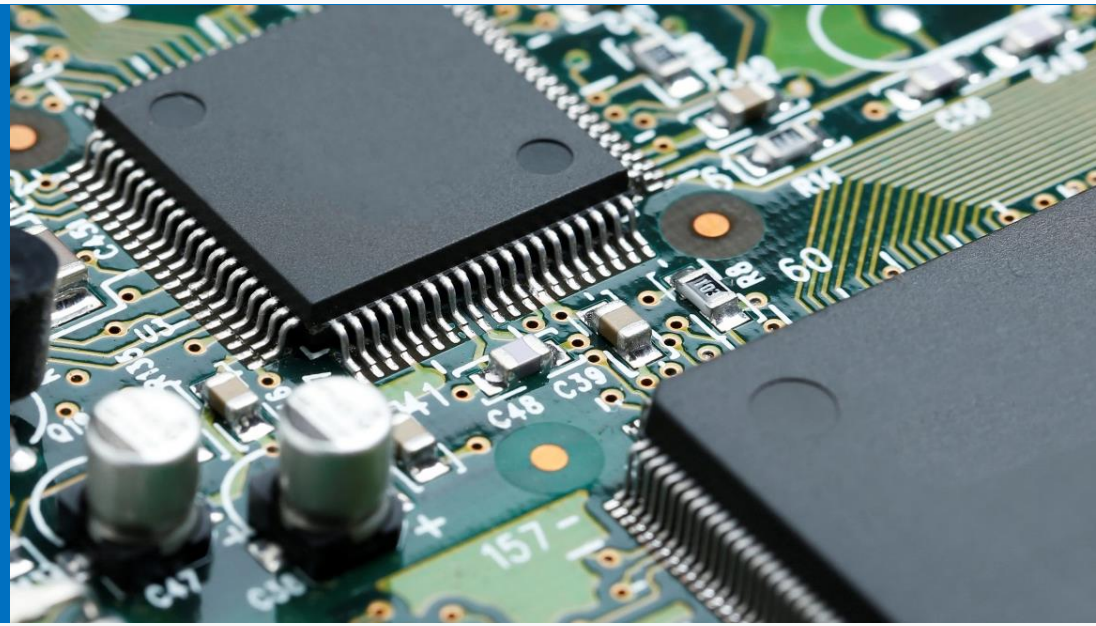


Image source: http://jdesbonnet.blogspot.ro/2010/07/unique-address-for-wireless-sensors.html

Discutie

- O soluție ar fi să facem ca doFFT() să calculeze doar rezultate parțiale la fiecare apel, astfel încât să revină mai des la main()
  - De exemplu, doFFT() ar putea dura doar 20 ms pentru a se executa o dată, dar ar trebui să îl apelați de 35 de ori pentru a termina toată munca.
  - Cu toate acestea, verificarea multor condiții în bucla while(1) ar putea face ca codul să reacționeze mai lent
- O altă soluție presupune întreruperi. Acestea pot întrerupe codul care se execută în prezent și pot executa o altă funcție.

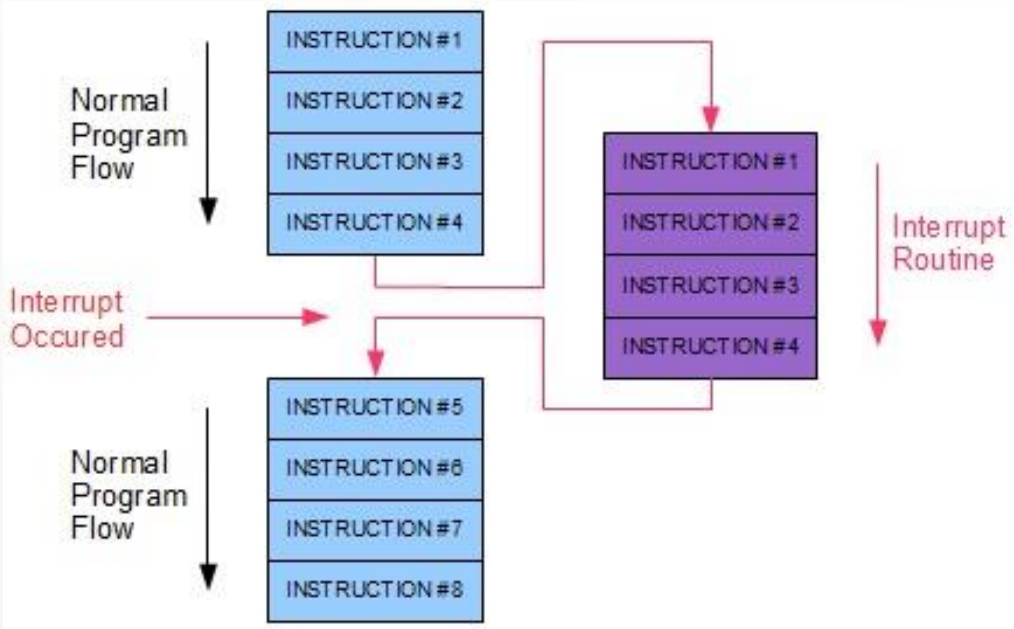
Proprietățile întreruperilor



PM

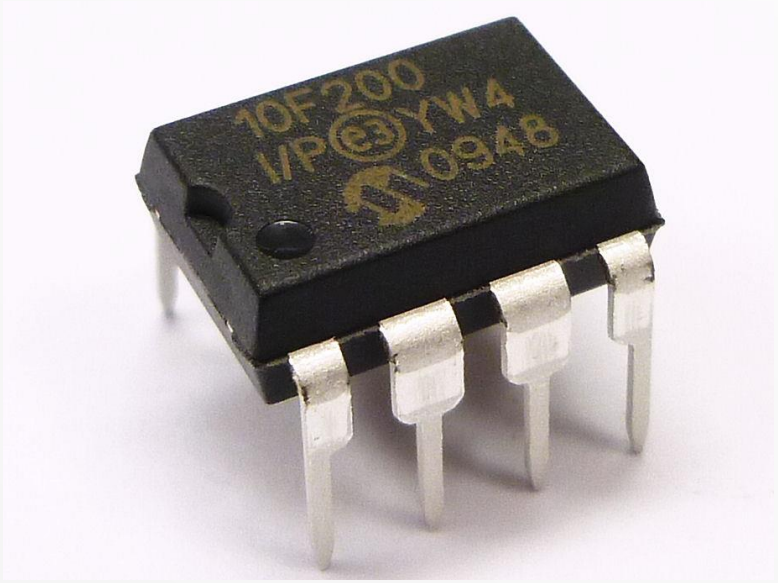
Rutina de întrerupere

- Semnal special către procesor care permite oprirea execuției curente și executarea codului într-o altă locație
- Odată ce codul din această locație diferită se termină, execuția normală este reluată.



Rutina de întrerupere

- Microcontrolere care nu au întreruperi există (foarte mici)
- Întreruperile sunt dezactivate implicit. Dacă dorim să folosim funcționalitatea adăugată, trebuie să ne exprimăm dorința configurându-le.



Rutina de întrerupere

- Din punctul de vedere al software-ului, întreruperile pot fi văzute ca „funcții care sunt executate la evenimente speciale”, ținând cont de următoarele:
  - Au o prioritate mai mare decât programul principal
  - Opresc programul principal, se execută până la finalizare și trec controlul înapoi programului principal
  - Acest mecanism poate fi folosit pentru a simula fire paralele de execuție
  - Aplicațiile care folosesc întreruperi trebuie să se asigure că integritatea datelor și a configurației hardware sunt păstrate.
- Acestea pot fi declanșate atât de surse interne, cât și externe:
  - Periferice externe
  - Periferice interne
  - Software

Discutie – rezolvare

- Uneori nu va functiona ok:
  - fftZoomIn() poate seta param diferiti in timp de doFFT() se executa
  - doFFT() va rula, potential, cu param modificati in timpul rularii => potential de crash

```
ISR(PCINT0_vect)
{
        fftZoomIn();
}

void main()
{
    DDRC |= (1<<DDC1);

        PCMSK0 |= (1<<PCINT7);
        sei();

    while(1)
    {
                doFFT();
    }
}
```

Discutie – rezolvare

- Rutina de tratare a intreruperii – trebuie sa fie scurta!
- `zoomInPressed – e ok?`

```
uint8_t zoomInPressed;

ISR(PCINT0_vect)
{
    zoomInPressed = 1;
}

void main()
{
    [...]

    zoomInPressed = 0;
    while(1)
    {
       if (zoomInPressed==1)
       {
          fftZoomIn();
          zoomInPressed = 0;
       }
       doFFT();
    }
}
```

Discutie – rezolvare

- Fără declarație de volatile

```
uint8_t zoomInPressed;

ISR(PCINT0_vect)
{
    zoomInPressed = 1;
}

void main()
{
    [...]

    zoomInPressed = 0;
    while(1)
    {
       if (zoomInPressed==1)
       {
          fftZoomIn();
          zoomInPressed = 0;
       }
       doFFT();
    }
}
```

Discutie

- În acest caz, compilatorul observă că variabila zoomInPressed din while(1) nu poate săfie modificată și optimizează codul în felul următor:

```
[...]
while(1)
{
    doFFT();
}
```

```
uint8_t zoomInPressed;

ISR(PCINT0_vect)
{
    zoomInPressed = 1;
}

void main()
{
    [...]

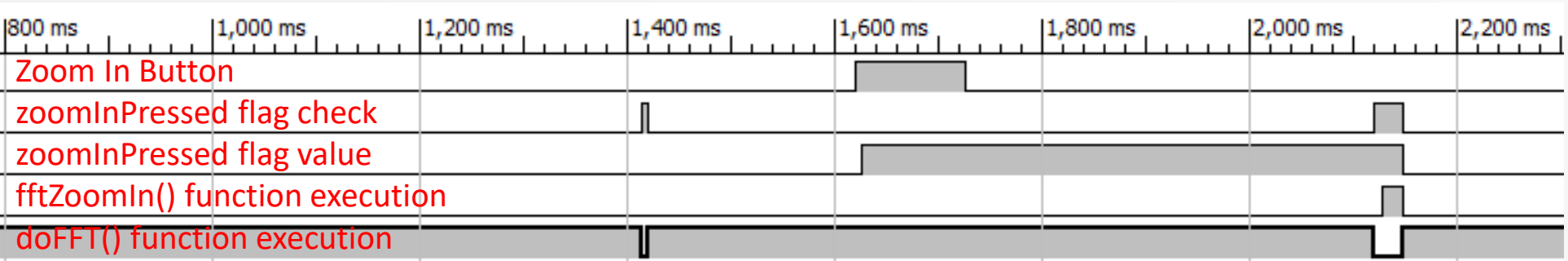
    zoomInPressed = 0;
    while(1)
    {
        if (zoomInPressed==1)
        {
            fftZoomIn();
            zoomInPressed = 0;
        }
        doFFT();
    }
}
```

Discutie – rezolvare

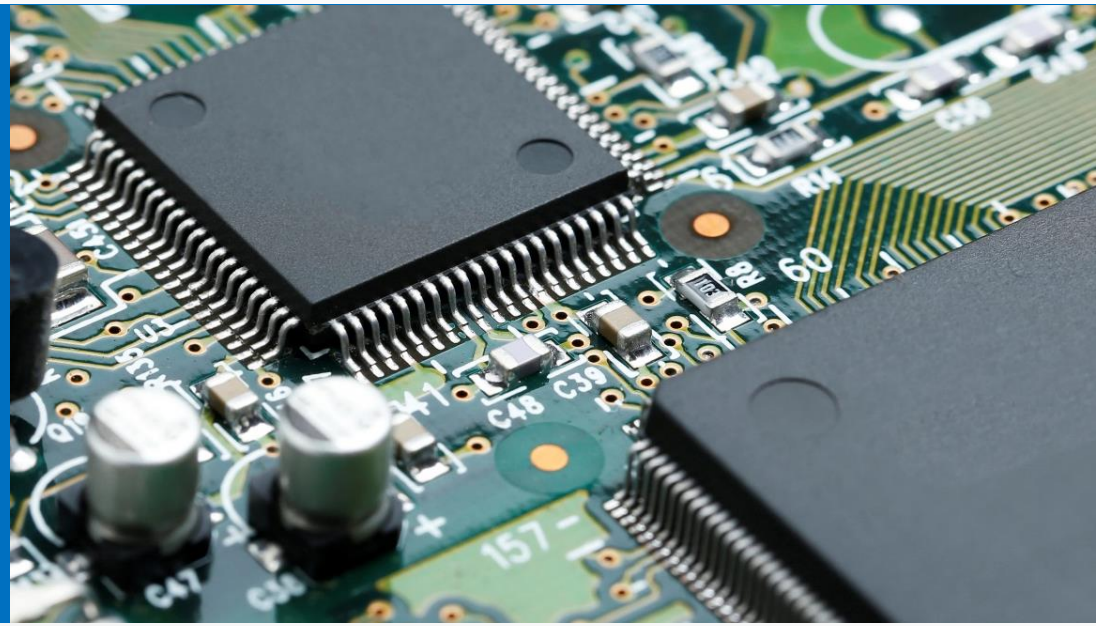
```
Volatile uint8_t zoomInPressed;

ISR(PCINT0_vect)
{
    zoomInPressed = 1;
}
```

```
void main()
{
    [...]
    zoomInPressed = 0;
    while(1)
    {
        if (zoomInPressed==1)
        {
            fftZoomIn();
            zoomInPressed = 0;
        }
        doFFT();
    }
}
```



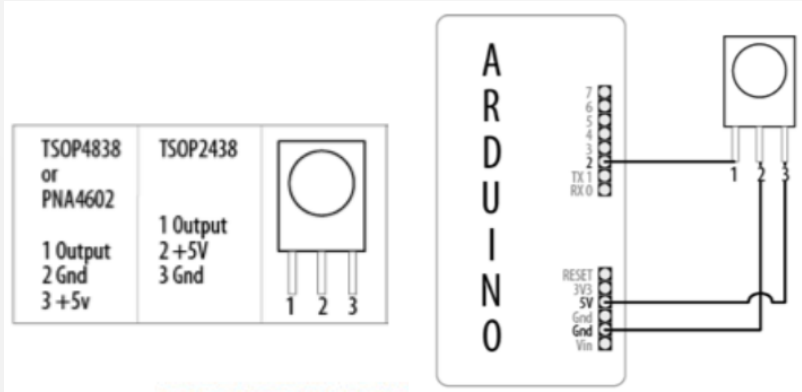
Exemplu de întrerupere



PM

Întreruperi în Arduino

- Avem un detector IR conectat la pinul 2
- Programul monitorizează pulsul pe acest pin și stochează durata fiecărui impuls într-o matrice
- Când matricea este atașată, durata este afișată pe monitorul serial




```
const int irReceiverPin = 2;             //pin the receiver is connected to
const int numberOfEntries = 64;

volatile unsigned long microseconds;
volatile byte index = 0;
volatile unsigned long results[numberOfEntries];

void setup()
{
  pinMode(irReceiverPin, INPUT);
  Serial.begin(9600);
  attachInterrupt(0, analyze, CHANGE);  // encoder pin on interrupt 0 (pin 2);
  results[0]=0;
}

void loop()
{
  if(index >= numberOfEntries)
  {
    Serial.println("Durations in Microseconds are:") ;
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    while(1)
      ;
  }
  delay(1000);
}

void analyze()
{
  if(index < numberOfEntries  )
  {
    if(index > 0)
    {
      results[index] = micros() - microseconds;
    }
    index = index + 1;
  }
  microseconds = micros();
}
```

“volatile”

- Este un calificativ variabil
- Este folosit înaintea tipului de date al unei variabile, pentru a modifica modul în care compilatorul și programul ulterior tratează variabila.
- Compilatorul va încărca variabila din RAM și nu dintr-un registru de stocare.
- În anumite condiții, cum ar fi prin întreruperi, valoarea pentru o variabilă stocată în registre poate fi inexactă.

```
const int irReceiverPin = 2;            //pin the receiver is connected to
const int numberOfEntries = 64;

volatile unsigned long microseconds;
volatile byte index = 0;
volatile unsigned long results[numberOfEntries];

void setup()
{
  pinMode(irReceiverPin, INPUT);
  Serial.begin(9600);
  attachInterrupt(0, analyze, CHANGE);  // encoder pin on interrupt 0 (pin 2);
  results[0]=0;
}

void loop()
{
  if(index >= numberOfEntries)
  {
    Serial.println("Durations in Microseconds are:") ;
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    while(1)
      ;
  }
  delay(1000);
}

void analyze()
{
  if(index < numberOfEntries  )
  {
    if(index > 0)
    {
      results[index] = micros() - microseconds;
    }
    index = index + 1;
  }
  microseconds = micros();
}
```

Schimbarea variabilelor

- Variabilele care sunt modificate într-o funcție de întrerupere sunt declarate ca volatile.
- Acest lucru permite compilatorului să știe că valorile se pot schimba în orice moment (prin un handler de întreruperi).
- Fără a utiliza cuvântul cheie “volatile”, compilatorul ar crede că aceste variabile nu sunt modificate de niciun cod apelat și ar înlocui aceste variabile cu valori constante.

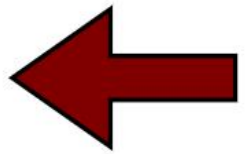
```
const int irReceiverPin = 2;             //pin the receiver is connected to
const int numberOfEntries = 64;

volatile unsigned long microseconds;
volatile byte index = 0;
volatile unsigned long results[numberOfEntries];

void setup()
{
  pinMode(irReceiverPin, INPUT);
  Serial.begin(9600);
  attachInterrupt(0, analyze, CHANGE);  // encoder pin on interrupt 0 (pin 2);
  results[0]=0;
}

void loop()
{
  if(index >= numberOfEntries)
  {
    Serial.println("Durations in Microseconds are:") ;
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    while(1)
      ;
  }
  delay(1000);
}

void analyze()
{
  if(index < numberOfEntries  )
  {
    if(index > 0)
    {
      results[index] = micros() - microseconds;
    }
    index = index + 1;
  }
  microseconds = micros();
}
```



attachInterrupt()

- attachInterrupt(0, analyze, CHANGE); permite programului să gestioneze întreruperi.
- Primul număr din apel specifică întreruperea care trebuie inițializată.
- Pe o placă standard Arduino (cum ar fi UNO), sunt disponibile două întreruperi: numărul 0, care folosește pinul 2 și numărul 1 pe pinul 3.
- Întreruperea 0 și întreruperea 1 au aceleași priorități (cu Wiring).

```
const int irReceiverPin = 2;            //pin the receiver is connected to
const int numberOfEntries = 64;

volatile unsigned long microseconds;
volatile byte index = 0;
volatile unsigned long results[numberOfEntries];

void setup()
{
  pinMode(irReceiverPin, INPUT);
  Serial.begin(9600);
  attachInterrupt(0, analyze, CHANGE);  // encoder pin on interrupt 0 (pin 2);
  results[0]=0;
}

void loop()
{
  if(index >= numberOfEntries)
  {
    Serial.println("Durations in Microseconds are:") ;
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    while(1)
      ;
  }
  delay(1000);
}

void analyze()
{
  if(index < numberOfEntries  )
  {
    if(index > 0)
    {
      results[index] = micros() - microseconds;
    }
    index = index + 1;
  }
  microseconds = micros();
}
```

attachInterrupt()

- Al doilea parametru specifică ce funcție să apeleze (handler-ul de întrerupere) când are loc evenimentul de întrerupere.
- Parametrul final specifică ce ar trebui să declanșeze întreruperea:
  - CHANGE: ori de câte ori se schimbă nivelul pinului (de la mic la mare sau invers).
  - LOW: când pinul este jos.
  - RISING: când pinul merge de jos spre sus.
  - FALLING : când pinul merge de sus spre jos.

```
const int irReceiverPin = 2;             //pin the receiver is connected to
const int numberOfEntries = 64;

volatile unsigned long microseconds;
volatile byte index = 0;
volatile unsigned long results[numberOfEntries];

void setup()
{
  pinMode(irReceiverPin, INPUT);
  Serial.begin(9600);
  attachInterrupt(0, analyze, CHANGE);  // encoder pin on interrupt 0 (pin 2);
  results[0]=0;
}

void loop()
{
  if(index >= numberOfEntries)
  {
    Serial.println("Durations in Microseconds are:") ;
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    while(1)
      ;
  }
  delay(1000);
}

void analyze()
{
  if(index < numberOfEntries  )
  {
    if(index > 0)
    {
      results[index] = micros() - microseconds;
    }
    index = index + 1;
  }
  microseconds = micros();
}
```

loop()

- Bucla principală verifică doar variabila de index pentru a vedea dacă toate intrările au fost setate de handler-ul de întrerupere.
- Acesta va imprima conținutul rezultatelor matricei o singură dată.
- Nimic din buclă nu modifică valoarea indexului.
- Indicele este schimbat în interiorul funcției de analiză atunci când apare condiția de întrerupere.

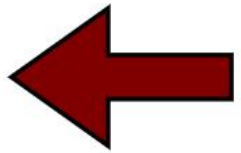
```
const int irReceiverPin = 2;            //pin the receiver is connected to
const int numberOfEntries = 64;

volatile unsigned long microseconds;
volatile byte index = 0;
volatile unsigned long results[numberOfEntries];

void setup()
{
  pinMode(irReceiverPin, INPUT);
  Serial.begin(9600);
  attachInterrupt(0, analyze, CHANGE);  // encoder pin on interrupt 0 (pin 2);
  results[0]=0;
}

void loop()
{
  if(index >= numberOfEntries)
  {
    Serial.println("Durations in Microseconds are:") ;
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    while(1)
      ;
  }
  delay(1000);
}

void analyze()
{
  if(index < numberOfEntries  )
  {
    if(index > 0)
    {
      results[index] = micros() - microseconds;
    }
    index = index + 1;
  }
  microseconds = micros();
}
```



while(1)

- Codul rămâne în bucla while la sfârșitul blocului interior, așa că trebuie să resetați placa atunci când doriți să faceți o altă rulare.
- Bad practice

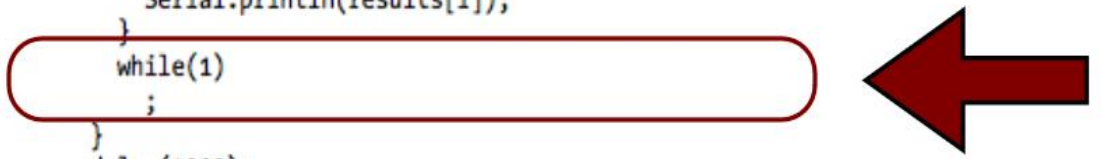
```
const int irReceiverPin = 2;            //pin the receiver is connected to
const int numberOfEntries = 64;

volatile unsigned long microseconds;
volatile byte index = 0;
volatile unsigned long results[numberOfEntries];

void setup()
{
  pinMode(irReceiverPin, INPUT);
  Serial.begin(9600);
  attachInterrupt(0, analyze, CHANGE);  // encoder pin on interrupt 0 (pin 2);
  results[0]=0;
}

void loop()
{
  if(index >= numberOfEntries)
  {
    Serial.println("Durations in Microseconds are:") ;
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    while(1)
      ;
  }
  delay(1000);
}

void analyze()
{
  if(index < numberOfEntries  )
  {
    if(index > 0)
    {
      results[index] = micros() - microseconds;
    }
    index = index + 1;
  }
  microseconds = micros();
}
```



micros()

- Funcția micros() returnează numărul de microsecunde de când Arduino a început să ruleze programul curent.
- Acest număr va face overflow (reveni la zero), după aproximativ 70 de minute.
- Pe plăcile Arduino de 16 MHz (de exemplu, UNO), această funcție are o rezoluție de patru microsecunde (adică valoarea returnată este întotdeauna un multiplu de patru).

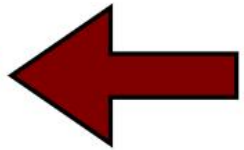
```
const int irReceiverPin = 2;            //pin the receiver is connected to
const int numberOfEntries = 64;

volatile unsigned long microseconds;
volatile byte index = 0;
volatile unsigned long results[numberOfEntries];

void setup()
{
  pinMode(irReceiverPin, INPUT);
  Serial.begin(9600);
  attachInterrupt(0, analyze, CHANGE);  // encoder pin on interrupt 0 (pin 2);
  results[0]=0;
}

void loop()
{
  if(index >= numberOfEntries)
  {
    Serial.println("Durations in Microseconds are:") ;
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    while(1)
      ;
  }
  delay(1000);
}

void analyze()
{
  if(index < numberOfEntries  )
  {
    if(index > 0)
    {
      results[index] = micros() - microseconds;
    }
    index = index + 1;
  }
  microseconds = micros();
}
```



analyze()

- Valoarea indexului este utilizată pentru a stoca timpul de la ultima schimbare de stare în următorul slot din matricea de rezultate.
- Timpul se calculează scăzând ultima dată, starea schimbată din timpul curent în microsecunde.
- Ora curentă este apoi salvată ca ultima dată când a avut loc o modificare.

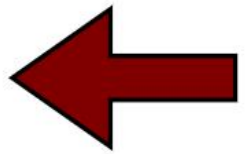
```
const int irReceiverPin = 2;             //pin the receiver is connected to
const int numberOfEntries = 64;

volatile unsigned long microseconds;
volatile byte index = 0;
volatile unsigned long results[numberOfEntries];

void setup()
{
  pinMode(irReceiverPin, INPUT);
  Serial.begin(9600);
  attachInterrupt(0, analyze, CHANGE);  // encoder pin on interrupt 0 (pin 2);
  results[0]=0;
}

void loop()
{
  if(index >= numberOfEntries)
  {
    Serial.println("Durations in Microseconds are:") ;
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    while(1)
      ;
  }
  delay(1000);
}

void analyze()
{
  if(index < numberOfEntries  )
  {
    if(index > 0)
    {
      results[index] = micros() - microseconds;
    }
    index = index + 1;
  }
  microseconds = micros();
}
```



Privire de ansamblu

- De fiecare dată când este declanșată o întrerupere, indexul este incrementat și ora curentă este salvată.
- Diferența de timp este calculată și salvată în matrice (cu excepția primei declanșări a întreruperii, când indicele este 0).
- Când a avut loc numărul maxim de intrări, blocul interior în buclă rulează și imprimă toate valorile pe portul serial.

```
const int irReceiverPin = 2;             //pin the receiver is connected to
const int numberOfEntries = 64;

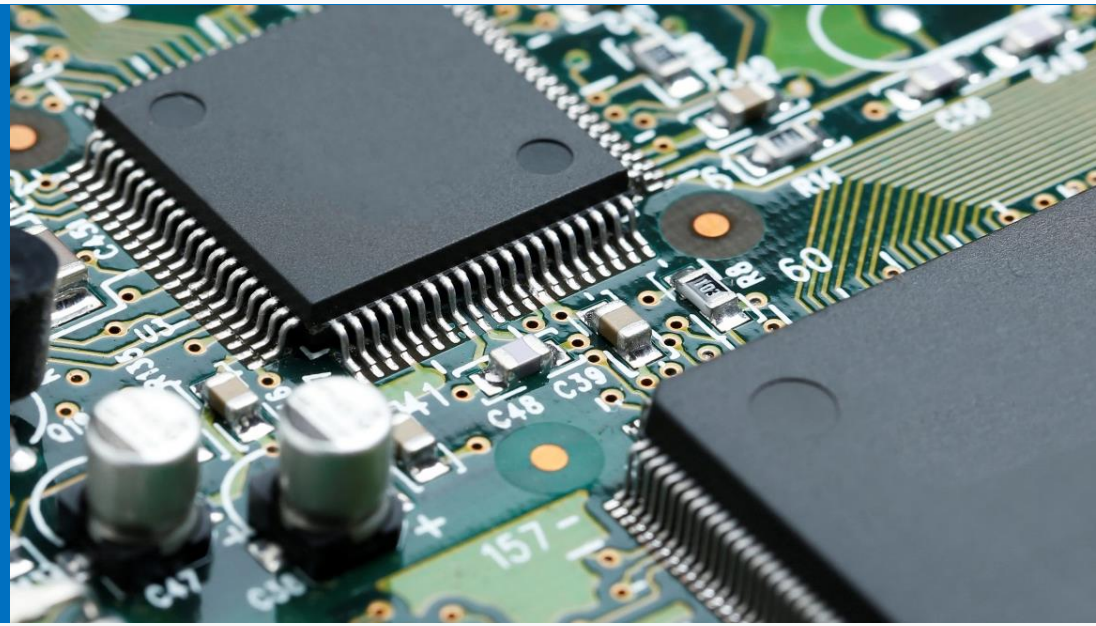
volatile unsigned long microseconds;
volatile byte index = 0;
volatile unsigned long results[numberOfEntries];

void setup()
{
  pinMode(irReceiverPin, INPUT);
  Serial.begin(9600);
  attachInterrupt(0, analyze, CHANGE);  // encoder pin on interrupt 0 (pin 2);
  results[0]=0;
}

void loop()
{
  if(index >= numberOfEntries)
  {
    Serial.println("Durations in Microseconds are:") ;
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    while(1)
      ;
  }
  delay(1000);
}

void analyze()
{
  if(index < numberOfEntries  )
  {
    if(index > 0)
    {
      results[index] = micros() - microseconds;
    }
    index = index + 1;
  }
  microseconds = micros();
}
```

Lista de întreruperi



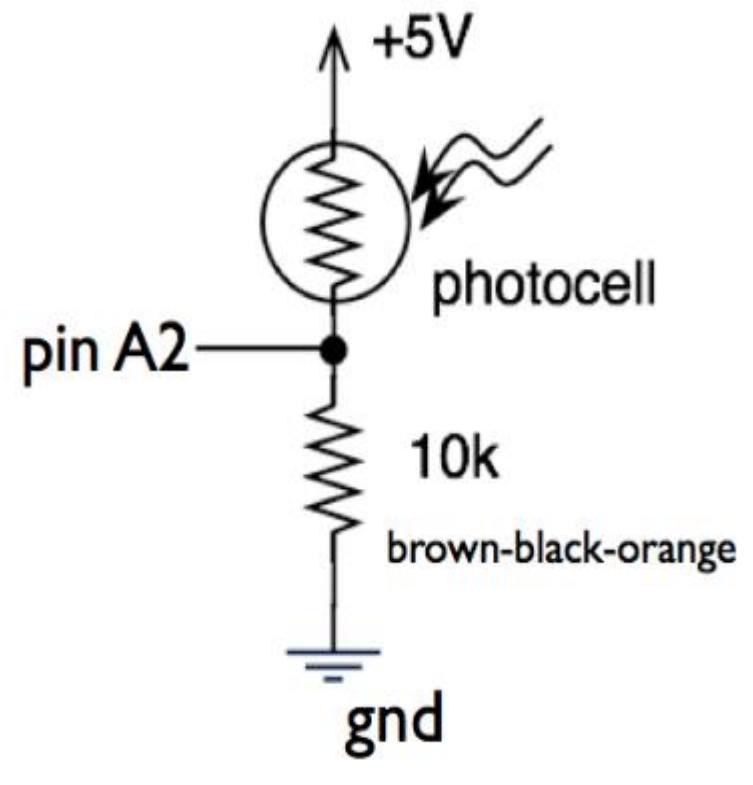
PM

Lista întreruperi - prioritati

Vector no.	Program address	Source	Interrupt definition
1	0000	RESET	External Pin, Power-on Reset, Brown-out Reset, Watchdog Reset and JTAG AWR Reset
2	0002	INT0	External Interrupt Request 0
3	0004	INT1	External Interrupt Request 1
4	0006	PCINT0	Pin Change Interrupt Request 0
5	0008	PCINT1	Pin Change Interrupt Request 1
6	000A	PCINT2	Pin Change Interrupt Request 2
7	000C	WDT	Watchdog Time-out Interrupt
8	000E	TIMER2_COMPA	Timer/Counter2 Compare Match A
9	0010	TIMER2_COMPB	Timer/Counter2 Compare Match B
10	0012	TIMER2_OVF	Timer/Counter 2 Overflow
11	0014	TIMER1_CAPT	Timer/Counter1 Capture Event
12	0016	TIMER1_COMPA	Timer/Counter1 Compare Match A
13	0018	TIMER1_COMPB	Timer/Counter1 Compare Match B
14	001A	TIMER1_OVF	Timer/Counter1 Overflow
15	001C	TIMER0_COMPA	Timer/Counter0 Compare Match A
16	001E	TIMER0_COMPB	Timer/Counter0 Compare Match B
17	0020	TIMER0_OVF	Timer/Counter0 Overflow
18	0022	SPI_STC	SPI Serial Transfer Complete
19	0024	USART0_RX	USART0 Rx Complete
20	0026	USART0_UDRE	USART0 Data Register Empty
21	0028	USART0_TX	USART0 Tx Complete
22	002A	ADC	ADC Conversion Complete
23	002C	EE_READY	EEPROM Ready
24	002E	ANALOG_COMP	Analog Comparator
25	0030	TWI	Two-Wire Serial Interface
26	0032	SPM_READY	Store Program Memory Ready

Ce face următorul cod?

- Dar ce culoare/valoare are rezistenta din schemă?



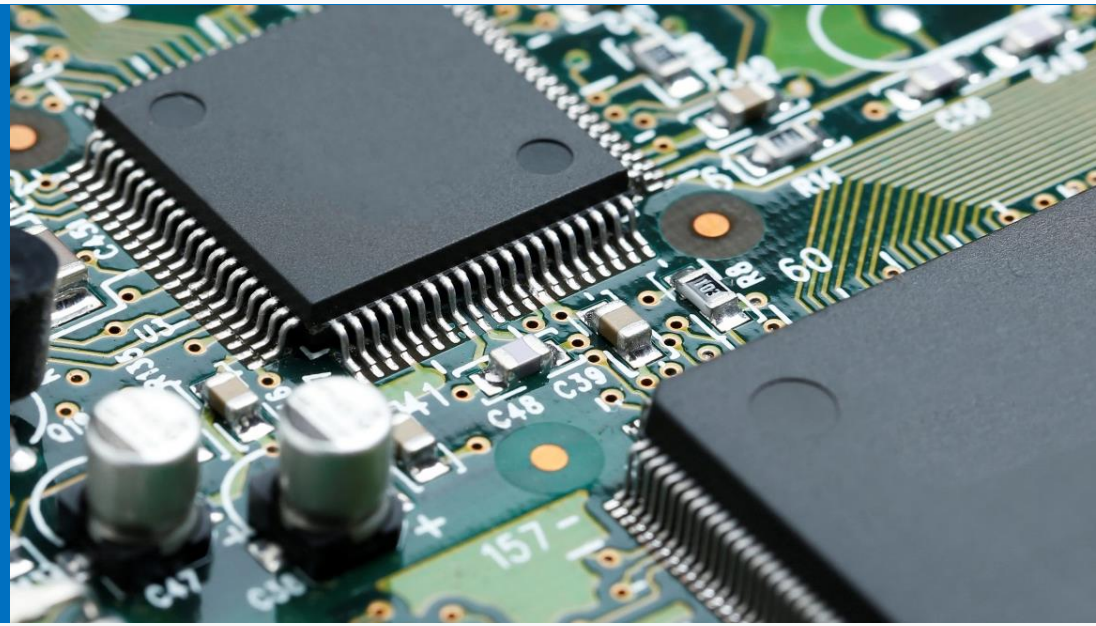

```
int pin = 13;
volatile int state = LOW;

void setup()
{
  pinMode(pin, OUTPUT);
  attachInterrupt(0, blink, CHANGE);
}

void loop()
{
  digitalWrite(pin, state);
}

void blink()
{
  state = !state;
}
```

ISR & atomicitate



PM

Limitări Arduino C (C-Wiring)

- În interiorul funcției de întrerupere, delay() nu va funcționa
- Valorile returnate de funcțiile millis() și micros() nu vor crește.
- Comunicațiile seriale în timp ce sunteți în funcție se pot pierde!
- Ar trebui să declarați ca volatile orice variabile pe care le modificați în cadrul funcției atașate.
- În mod implicit, întreruperile sunt atomice.

Secțiuni atomice în Arduino

- Putem activa/dezactiva întreruperile pe anumite secțiuni de cod cu interrupts() și noInterrupts().
- Întreruperile în Arduino sunt activate implicit.
- Unele funcții nu vor funcționa în timp ce întreruperile sunt dezactivate, iar comunicațiile de intrare pot fi ignorate.
- Cu toate acestea, întreruperile pot perturba ușor sincronizarea codului și pot fi dezactivate pentru secțiunile de cod deosebit de critice.

```
void setup() {}

void loop()
{
  noInterrupts();
  // critical, time-sensitive code here
  interrupts();
  // other code here
}
```

Starea implicită a întreruperilor

- În mod implicit, nu puteți avea întreruperi în interiorul întreruperilor.
- Cu C-Wiring, atât întreruperii 0 cât și întreruperii 1 li se atribuie aceeași prioritate.
- *Activarea întreruperilor în interiorul întreruperilor este un „hack”: nu este recomandat, deoarece ridică tot felul de probleme cu păstrarea stării mașinii înainte ca întreruperea de întrerupere să fie reparată.*

Activarea întreruperilor în întrerupere

```
volatile int i,j,z;

void artificialdelay()
{
  for (i=0; i<900; i++){
    for (j=0; j<900; j++){
      z=i*10;}}
}

void setup()
{
  Serial.begin(9600);
  attachInterrupt(0, interrupt0, CHANGE);
  attachInterrupt(1, interrupt1, CHANGE);
  pinMode(13, OUTPUT);
  pinMode(12, OUTPUT);
}

void loop()
{
  Serial.println("entering main loop");
  artificialdelay();
  Serial.println("leaving main loop");
}
```

```
void interrupt0()
{
  interrupts();
    digitalWrite(13,HIGH);
    artificialdelay();
    digitalWrite(13,LOW);
  noInterrupts(); //not really needed
}

void interrupt1()
{
  interrupts();
    digitalWrite(12,HIGH);
    artificialdelay();
    digitalWrite(12,LOW);
   noInterrupts(); //not really needed
}
```

- Dacă apăs pe pin2, va intra în interrupt0().
- În timp ce se procesează, pot apăsa pin3 și pot să sară în funcția interrupt1().

Scenariul 1

```
volatile int i,j,z;

void artificialdelay()
{
  for (i=0; i<900; i++){
    for (j=0; j<900; j++){
      z=i*10;}}
}

void setup()
{
  Serial.begin(9600);
  attachInterrupt(0, interrupt0, CHANGE);
  attachInterrupt(1, interrupt1, CHANGE);
  pinMode(13, OUTPUT);
  pinMode(12, OUTPUT);
}

void loop()
{
  Serial.println("entering main loop");
  artificialdelay();
  Serial.println("leaving main loop");
}
```

```
void interrupt0()
{
  interrupts();
    digitalWrite(13,HIGH);
    artificialdelay();
    digitalWrite(13,LOW);
  noInterrupts(); //not really needed
}

void interrupt1()
{
  interrupts();
    digitalWrite(12,HIGH);
    artificialdelay();
    digitalWrite(12,LOW);
   noInterrupts(); //not really needed
}
```

- Ce se întâmplă dacă declanșați întreruperea 0 când în interiorul întreruperii 0?

Scenariu

```
volatile int i,j,z;

void artificialdelay()
{
  for (i=0; i<900; i++){
    for (j=0; j<900; j++){
      z=i*10;}}
}

void setup()
{
  Serial.begin(9600);
  attachInterrupt(0, interrupt0, CHANGE);
  attachInterrupt(1, interrupt1, CHANGE);
  pinMode(13, OUTPUT);
  pinMode(12, OUTPUT);
}

void loop()
{
  Serial.println("entering main loop");
  artificialdelay();
  Serial.println("leaving main loop");
}
```

```
void interrupt0()
{
  //interrupts();
    digitalWrite(13,HIGH);
    artificialdelay();
    digitalWrite(13,LOW);
  //noInterrupts();
}

void interrupt1()
{
  //interrupts();
    digitalWrite(12,HIGH);
    artificialdelay();
    digitalWrite(12,LOW);
   //noInterrupts();
}
```

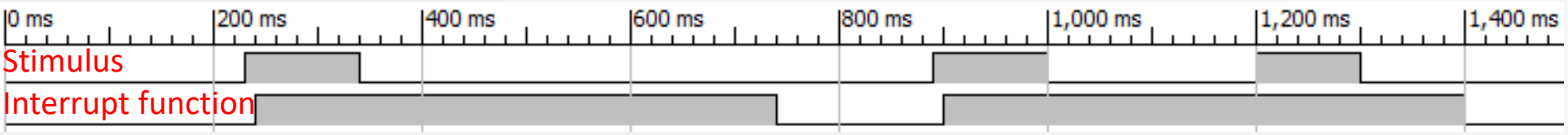
- Ce se întâmplă dacă declanșați interrupt1 când în interiorul interrupt0?

Cerințe pentru întreruperi

- De luat în calcul:
  - data pasate între întreruperi și codul principal – variabile "volatile"
  - Rutinele de întrerupere (funcțiile) trebuie să fie foarte scurte
  - Atomocitate – variabilele folosite între intreruperi și codul principal – trebuie să fie folosite în mod atomic

Cerințe - short ISRs

- Rutinele de întrerupere (funcțiile) trebuie să fie foarte scurte:
  - În mod normal, o întrerupere nu poate întrerupe o altă întrerupere
  - Dacă o întrerupere durează mult, alte întreruperi riscă să fie amânate =>
    - Timpi mai mari de raspuns
    - Input ratat



Discutie - atomicitate

- Q: ce se întâmplă dacă apare întreruperea în acel punct?

```
volatile uint16_t a, b, c;

ISR(PCINT0_vect)
{
    c = (a + b) / 2;
}

void main()
{
    [...]
    zoomInPressed = 0;
    c = 0;
    while(1)
    {
        a = readADC(channel_1);
        b = readADC(channel_2);
        __delay_ms(1000);
        displayLCD(c);
    }
}
```

Discutie - atomicitate

- Solutie

```
volatile uint16_t a, b, c;

ISR(PCINT0_vect)
{
    c = (a + b) / 2;
}

void main()
{
    [...]
    zoomInPressed = 0;
    c = 0;
    while(1)
    {
        disableInterrupts();
        a = readADC(channel_1);
        b = readADC(channel_2);
        enableInterrupts();
        __delay_ms(1000);
        displayLCD(c);
    }
}
```

Operations are atomic

Discutie - atomicitate

- Discutie – o variabilă uitn32_t va produce pe o arhitectură embedded (de ex pe o arh pe 16b) mai multe instrucțiuni
- Întreruperile poat avea efect între aceste instricțiuni => pot modifica valorea corectă
- ! Foare greu de făcut debug

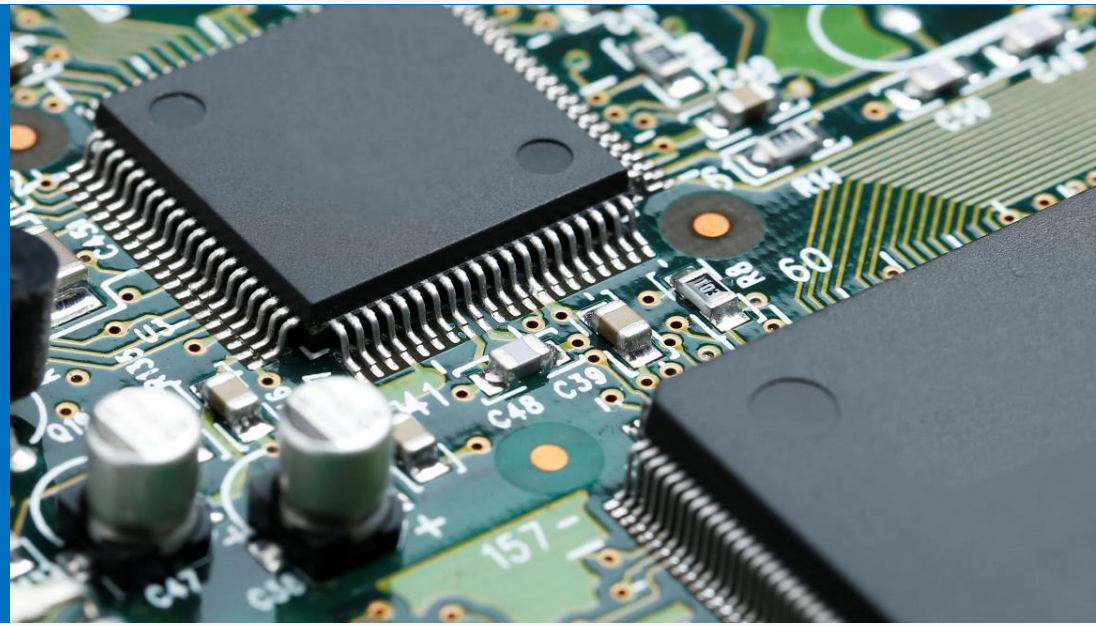
```
volatile uint32_t a;

ISR(PCINT0_vect)
{
    displayLCD(a);
}

void main()
{
    [...]
    while(1)
    {
        a = 0x00112233;
        a = 0xAABBCCDD;
    }
}
```

LCD could show 0x00112233, 0xAABBCCDD
but also 0x0011CCDD or 0xAABB2233

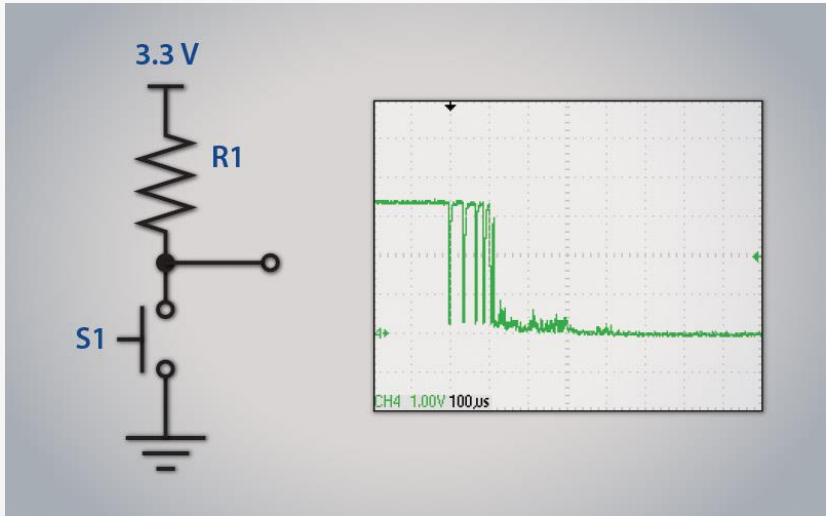
Efectul de de-bounce



PM

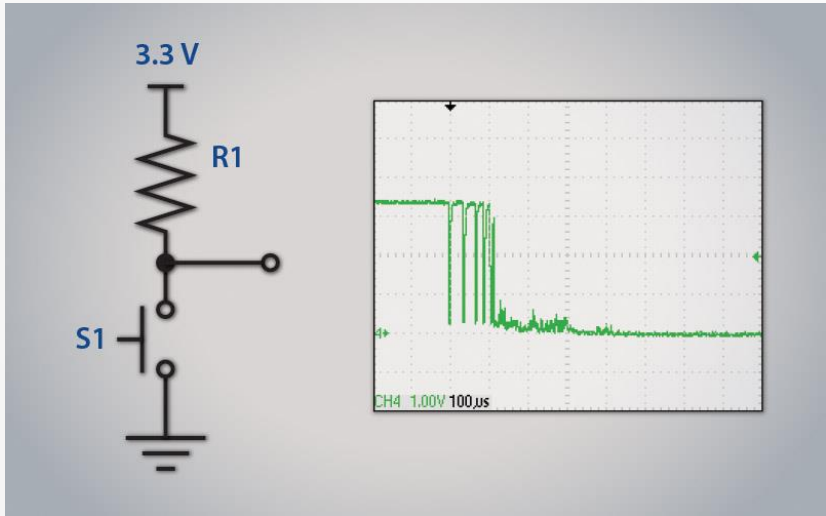
Deservirea întreruperilor

- S-ar putea să fi observat că nu toate apăsările de butoane produc rezultatul dorit (comutarea fiabilă). Ce ar putea cauza asta?
- Butonul (sau conexiunea prin cablu) nu pare foarte curat pe o scară scurtă de timp
- Aceasta se numește „saritură de buton” (button bounce)



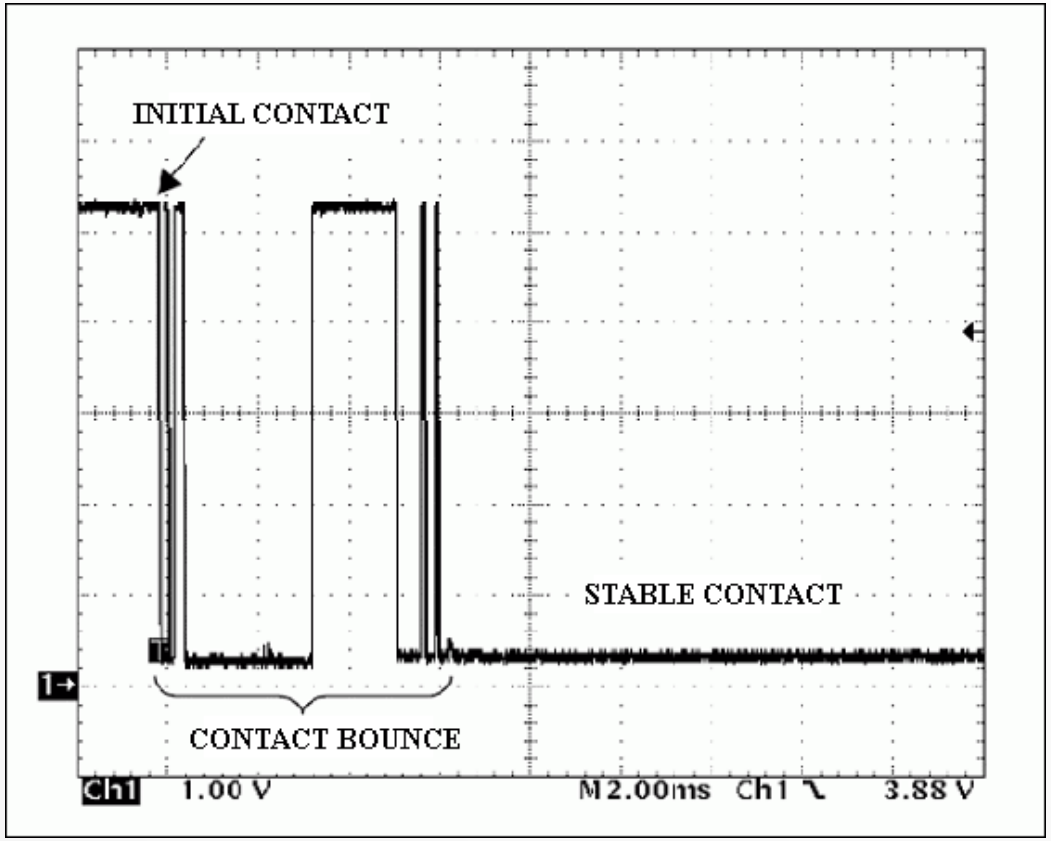
Deservirea întreruperilor - rezultat

- MCU este foarte rapid și va genera o întrerupere pe fiecare dintre aceste margini multiple
- Pentru o experiență de utilizator acceptabilă, este necesar un mecanism de deblocare a butoanelor
- Debouncing poate fi gestionat atât în hardware, cât și în software.



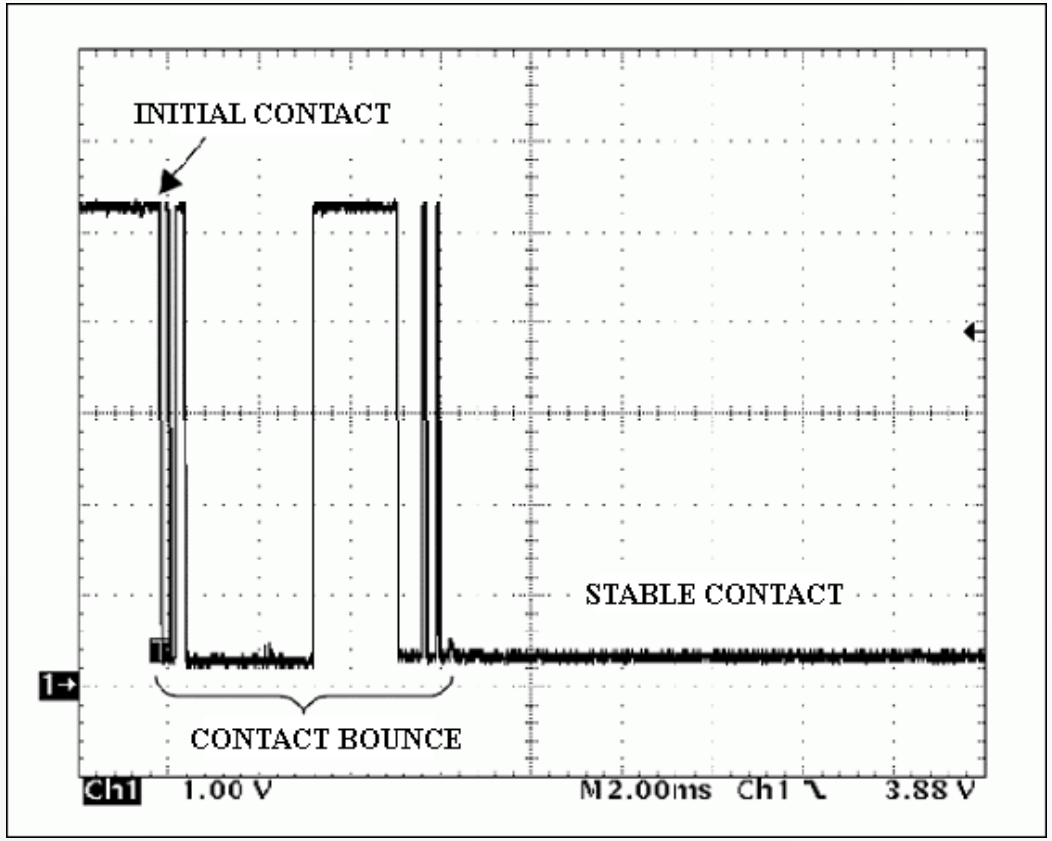
Efectul de debounce

- Acesta este un răspuns tipic al unui buton atunci când este apăsat
- Fiecare margine de cădere a semnalului ar declanșa o întrerupere și am primi ~6 apăsări pentru acest semnal
- Putem face debouncing în software pentru a ignora acești declanșatori multipli

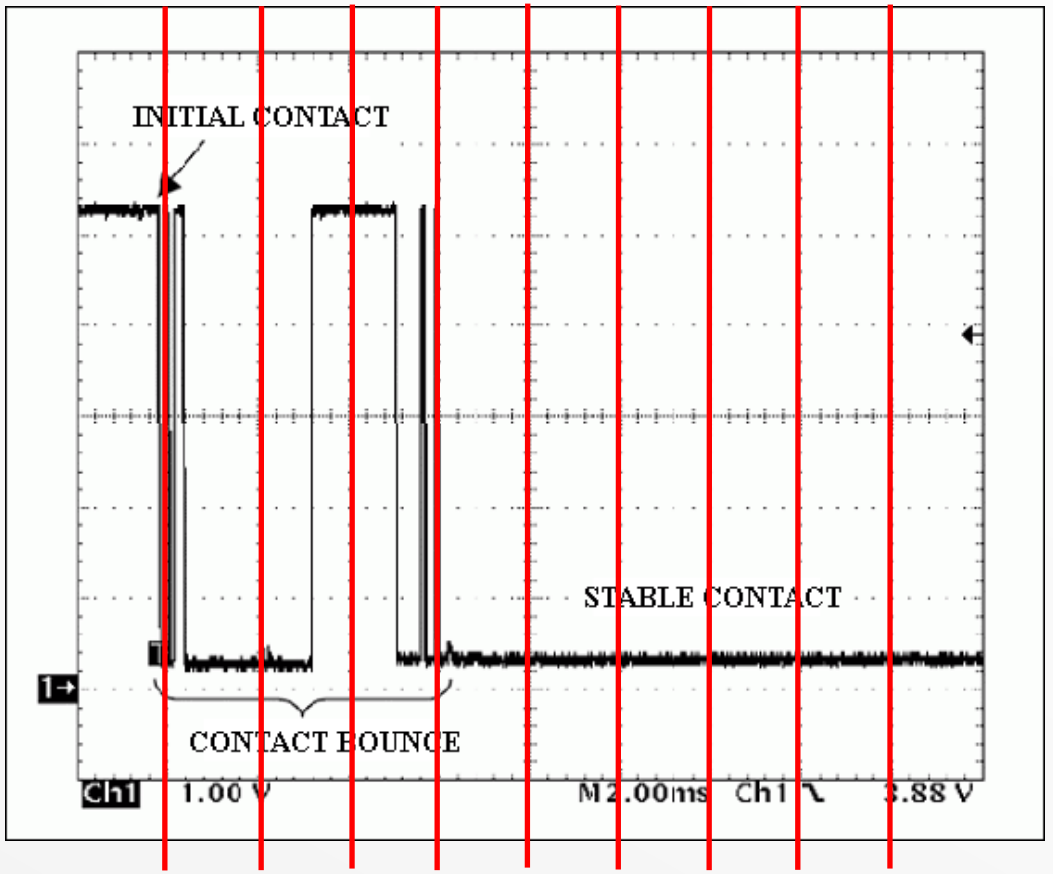


Efectul de debounce - rezolvări

- Sunt disponibile multe metode. Acesta este una simplă:
  - Utilizați un cronometru pentru a citi portul necondiționat (nu mai există întreruperi pe INT0)
  - Vom citi portul la fiecare 2 ms
  - Vom considera că butonul a fost apăsat atunci când obținem un șir de 10 zerouri
- Soluția alternativă este descrisă în laborator



Efectul de debounce - discutie



1 0 1 1 0 0 0 0 0

Individual samples

Image source: http://arduino.stackexchange.com/questions/408/why-does-my-sketch-report-too-many-button-presses

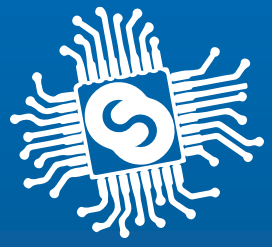
Hardware debounce

- https://www.onsemi.com/pdf/datasheet/mc14490-d.pdf
- https://datasheets.maximintegrated.com/en/ds/MAX6816-MAX6818.pdf
- FTJ
- FTJ + Schmitt trigger https://www.ti.com/lit/an/scea094/scea094.pdf?ts=1648841063443

Cuvinte cheie

- Întrerupere
- Atomicitate
- ISR
- Volatile
- Debounce
- Arduino IDE
- Microchip Studio

Cursul #5



PM

Disclaimer

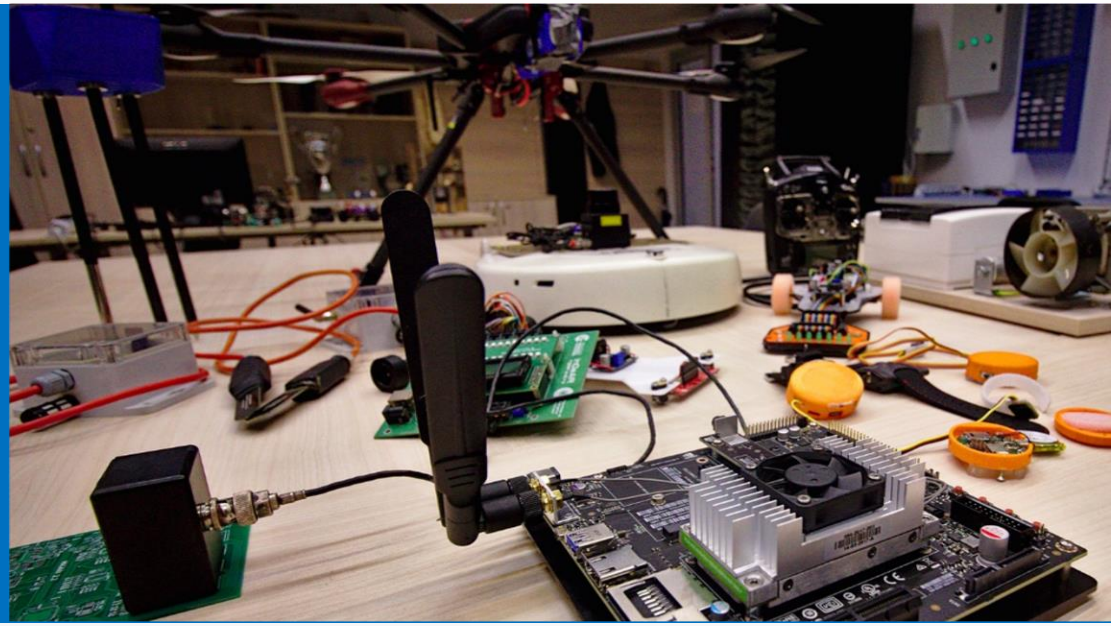
- *Slide-urile acestea reprezintă un rezumat.*
- *Slide-urile nu acoperă toate explicațiile, simulările, demonstrațiile oferite în timpul cursului.*
- *Slide-urile nu limitează în vreun fel materia cerută la examen.*
- *Pentru varianta completă, sunteți așteptați la curs.*

© Elemente din curs preluate din:
- *notițe curs PM 2021, Dan Tudose*
- *notițe curs PM 2021, Andrei Voinescu*
- *scoala de vara EAP 2016, Marius Nicolae*
- *notite curs EEA/ED, Razvan Tataroiu & Daniel Rosner*

Cuprins

- Timere
- Moduri de lucru
- PWM
- ADC
- Watchdog
- Next time

Timere



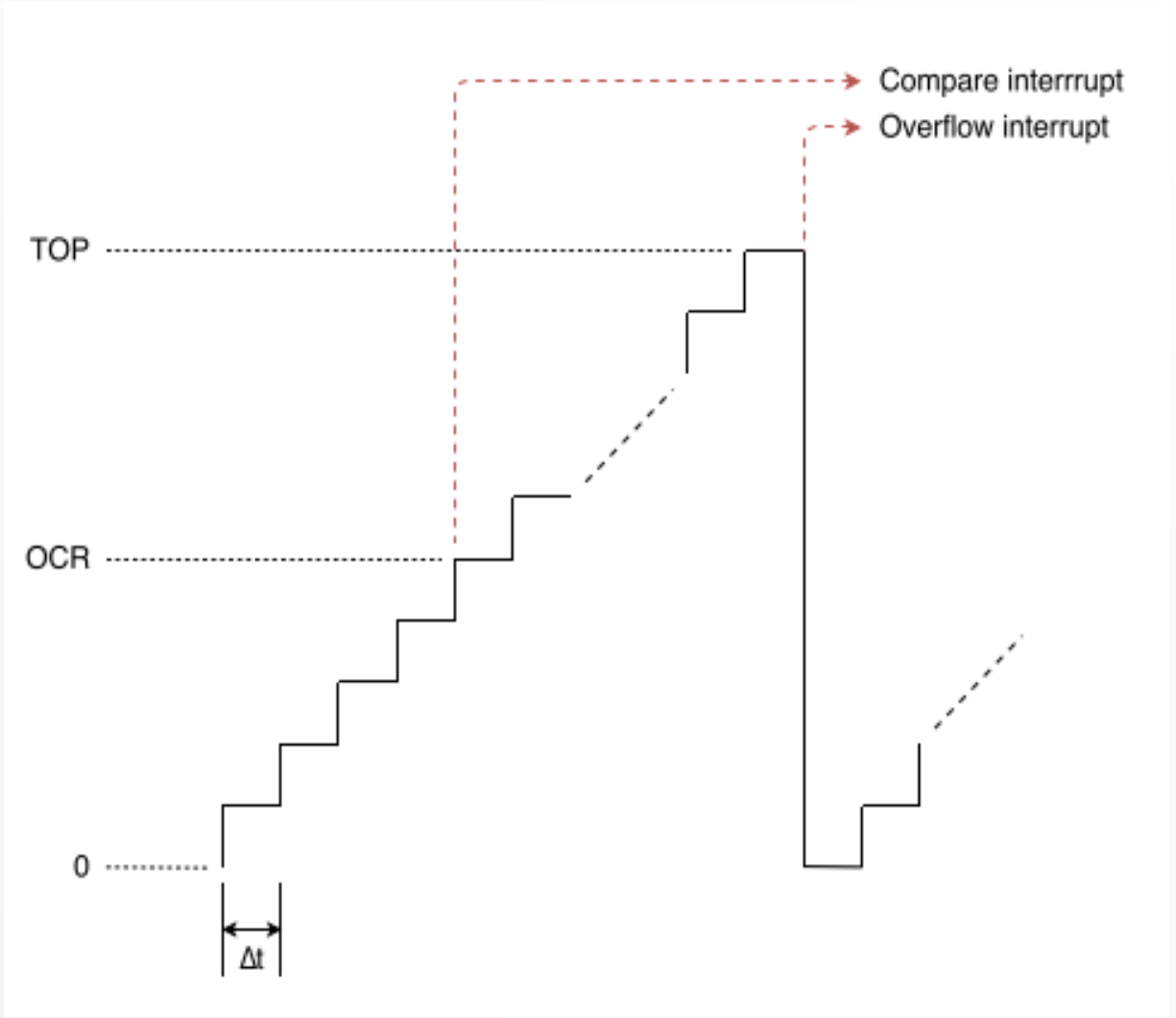
PM

Elemente introductive

- Timerele sunt dispozitive care ne permit sa
  - Știm cât timp a trecut între două evenimente
  - Numărăm evenimente externe
  - Să executăm cod/acțiuni după un anumit timp sau la intervale regulate (PWM)
- Timerele sunt implementate ca niște contoare
- Ca orice periferic I/O, vor avea registre I/O unde sunt configurate
  - Cum numără (până la cât, crescător/descrescător)
  - Cât de repede numără
  - Când sunt asociate acțiuni
  - Ce se execută în acele acțiuni

Reprezentare

- TOP este valoare până la care se numără
- OCR este un prag la care se poate întâmpla o acțiune
  - Întrerupere
  - PWM
- Δt este dat de viteza cu care numără
- $\Delta t = \frac{PS}{f_{CPU}}$



Organizare registre

- Pe ATMega328P timerele apar sub forma Timer/Counter
- Timer-ele 0 și 2 sunt timere pe 8 biți
- Timer1 este timer pe 16 biți
- TCCRxA și TCCRxB (x=0, 1, 2) - Control Register
  - Bitfield CS - Clock Select (cât de repede numără)
  - Bitfield WGM - Waveform Generation Mode (cum numără)
  - Bitfield COM - Compare Output (pentru PWM) Mode
- OCRxA, OCRxB - Output Compare Register (praguri)
- TCNTx - Counter (contorul efectiv)
- TIMSKx - interrupt mask register

Biți CS

- No clock source înseamnă că nu va număra
- Prescaler-ul are valori fixe
- Poate fi legat un semnal extern de ceas pe Timer0 și 1
- Pe timer 2 poate fi legat un alt cristal de cuarț

Table 16-6. Clock Select bit description.

CSn2	CSn1	CSn0	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$clk_{I/O}$/1 (No prescaling)
0	1	0	$clk_{I/O}$/8 (From prescaler)
0	1	1	$clk_{I/O}$/64 (From prescaler)
1	0	0	$clk_{I/O}$/256 (From prescaler)
1	0	1	$clk_{I/O}$/1024 (From prescaler)
1	1	0	External clock source on Tn pin. Clock on falling edge.
1	1	1	External clock source on Tn pin. Clock on rising edge.

Biți WGM

Table 16-5. Waveform Generation mode bit description [(1)].

Mode	WGMn3	WGMn2 (CTCn)	WGMn1 (PWMn1)	WGMn0 (PWMn0)	Timer/Counter mode of operation	TOP	Update of OCRnx at	TOVn flag set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCRnA	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICRn	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCRnA	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICRn	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCRnA	TOP	BOTTOM
12	1	1	0	0	CTC	ICRn	Immediate	MAX
13	1	1	0	1	(Reserved)	–	–	–
14	1	1	1	0	Fast PWM	ICRn	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCRnA	BOTTOM	TOP

Note: 1. The CTCn and PWMn1:0 bit definition names are obsolete. Use the WGMn2:0 definitions. However, the functionality and location of these bits are compatible with previous versions of the timer.

Biți COM

- Biții de COM activează semnalul de PWM pe canalul asociat (A sau B)
- Non-inverting generează un semnal cu duty cycle $D = \frac{1+prag}{1+TOP}$
- Inverting mode va avea $D = \frac{TOP-prag}{1+TOP}$
- Toggle mode va avea D = 50%

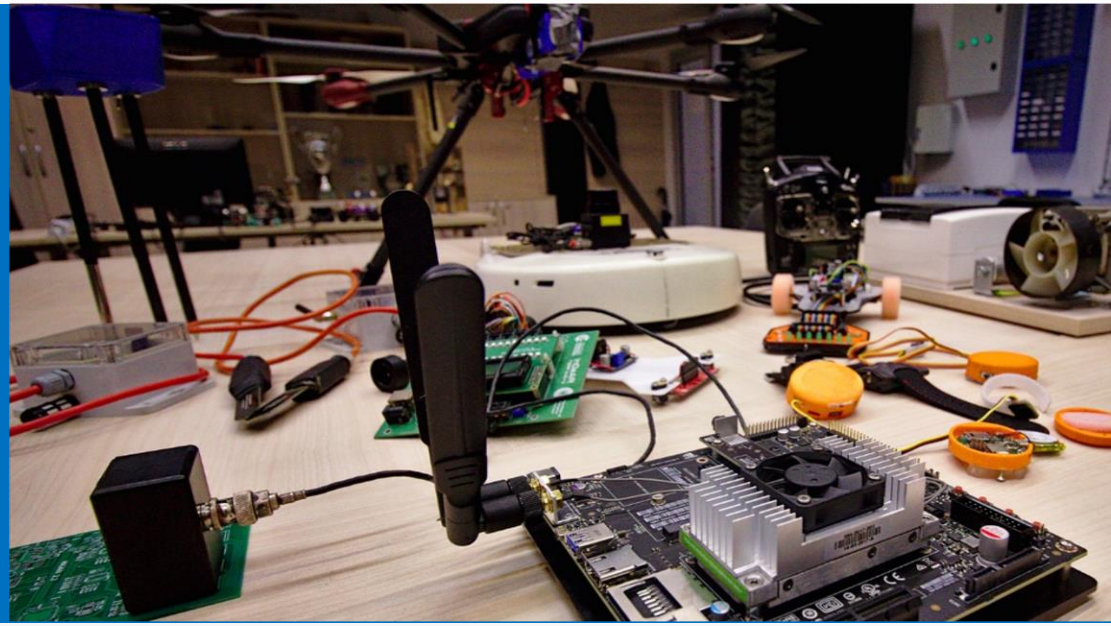
Table 16-3. Compare Output mode, fast PWM [(1)].

COMnA1/COMnB1	COMnA0/COMnB0	Description
0	0	Normal port operation, OCnA/OCnB disconnected.
0	1	WGMn3:0 = 14 or 15: Toggle OC1A on Compare Match, OC1B disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OCnA/OCnB on Compare Match, set OCnA/OCnB at BOTTOM (non-inverting mode)
1	1	Set OCnA/OCnB on Compare Match, clear OCnA/OCnB at BOTTOM (inverting mode)

Accesul la registre pe 16 biți

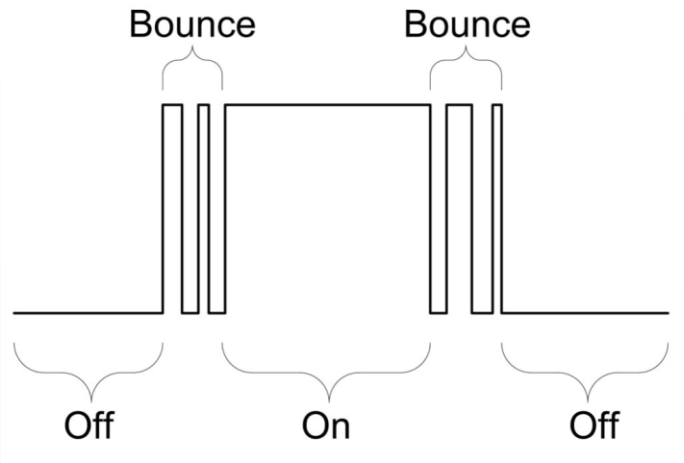
- AVR este o arhitectură pe 8 biți, dar se dorește totuși accesul ”atomic” la valori care depășesc 255
- Registrele pe 16 biți sunt împărțite într-un byte high și un byte low
- Pentru a face accesul atomic, perifericele intermediază accesul la aceste adrese cu un latch:
  - La citirea byte-ului low, se citește automat și byte-ul high și se stochează într-o locație temporară
  - La citirea byte-ului high valoarea citită este tot timpul cea din locația temporară
  - Scrierea se face în ordinea opusă (întâi byte-ul high și apoi low)
  - Motivația este dată de variația în timp, vrem byte-ul low să corespundă în timp ultimei citiri/scrieri
- Pentru C compilatorul poate face singur aceste operații în ordinea corectă

Moduri de lucru



PM

Modul normal



- Utilizări
  - Modul default pentru toate timerele
  - TOP fixat – frecvența fixă pentru evenimente generate
    - Pentru un timer de 8 biți, frecvența la care va face overflow va fi $f_{OVF} = \frac{f_{CPU}}{PS \cdot 256}$
    - Controlul frecvenței este limitat la prescaler sau la evenimente externe
- Folosiri → aplicații care nu au nevoie de o anumită frecvență
- Exemplu: o variantă de debouncing pentru butoane
- Cum am putea face debouncing cu ajutorul unui timer?
  - Nu avem nevoie de o frecvență anume, trebuie doar să știm cât timp a trecut de la ultima apăsare

Exemplu mod normal

```
volatile uint16_t    lastEvent = 0;
volatile uint8_t     buttonState = 0;   // 0 normal, 1 apasat

// 250 ms
#define DELAY    (16000000UL / 1024 / 4)

ISR(PCINT2_vect)
{
    if (buttonState != ((PINC & (1 << PC0)) == 0))
    { // daca butonul e apasat si noi credeam ca nu este
        if ((TCNT1 - lastEvent > DELAY))
        {
            buttonState = ((PINC & (1 << PC0)) == 0);
            buttonState? button_press() : button_release();
            lastEvent = TCNT1;
        }
    }
}

int main()
{
    TCCR1B = (5 << CS10); // prescaler 1024

    DDRC    &= ~(1 << PC0);       // PC0 ca intrare
    PCICR   |= (1 << PCINT2);     // PC0 este PCINT16, deci e in al
    PCMSK2  |= (1 << PCMSK16);    // treilea registru PCMSK

    sei();
    while (1);
    return 0;
}
```

Modul CTC

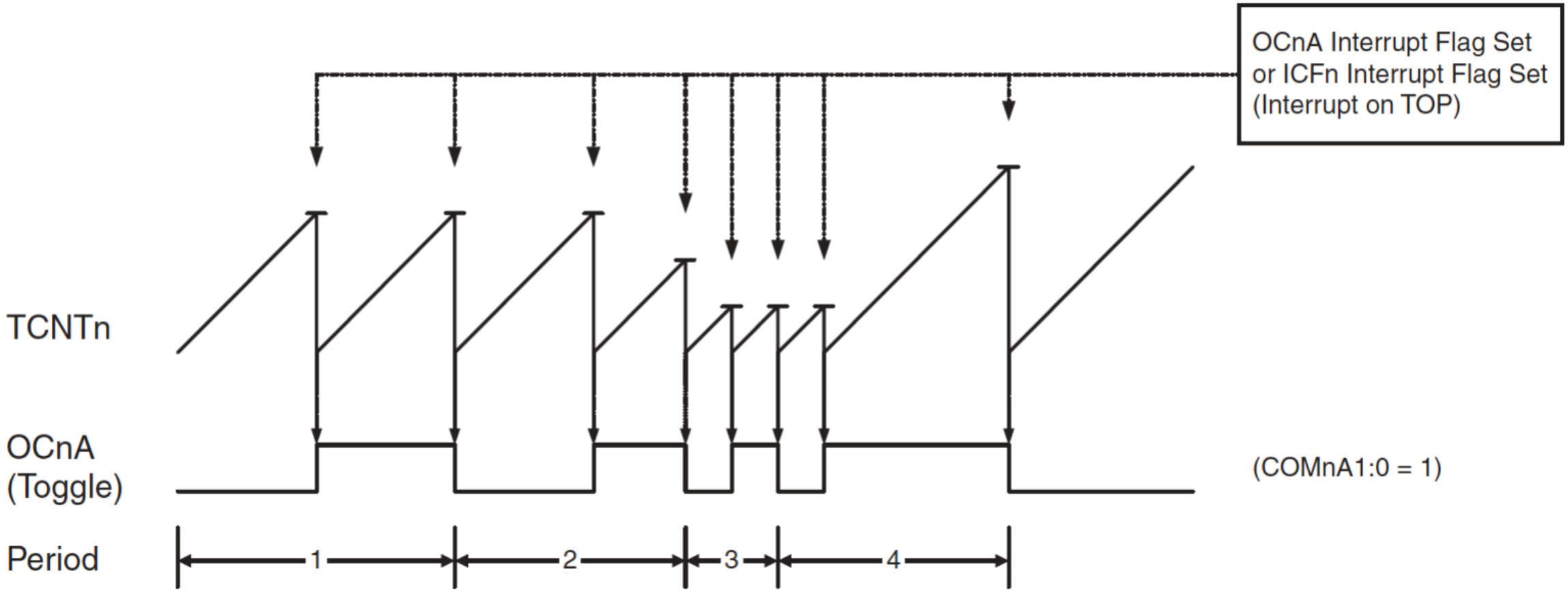
- Modul de bază în care contorul se resetează când atinge un prag (fie OCR, fie ICR)
- Putem controla frecvența cu care ajungem la Compare match
- Dar nu ajungem aproape niciodată la overflow

$$f_{\mathrm{CMP}} = \frac{f_{CPU}}{\mathrm{PS}\cdot(1+\mathrm{OCRxA})}$$

- Spre deosebire de alte moduri, modificările la prag se fac instantaneu
  - Ex: pragul este 100, contorul este 50, mutăm pragul la 25
  - La ce valoare se va reseta?
  - Pentru că update-ul se face imediat, nu la fiecare reset, contorul va ajunge la valoarea maximă (0xFF sau 0xFFFF)

Modul CTC

Figure 16-6. CTC mode, timing diagram.



Exemplu mod CTC

- Un ceas are nevoie să numere secundele
- Vrem să numărăm o secundă în hardware, iar numărul de secunde să îl numărăm în software
- Cu prescalerul maxim de 1024, $f_{CMP_{\mathrm{MAX}}} = \frac{f_{CPU}}{1024 \cdot 256} \simeq 61$ Hz
- Nu putem număra o secundă cu timerele de 8 biți
- Cu un timer de 16 biți → OCRxA = $\frac{f_{CPU}}{\mathrm{PS} \cdot 1\mathrm{Hz}} - 1 = 15624$

Exemplu mod CTC

```
...

volatile uint32_t nrSeconds = 0;

ISR(TIMER1_COMPA_vect)
{
    nrSeconds++;
}

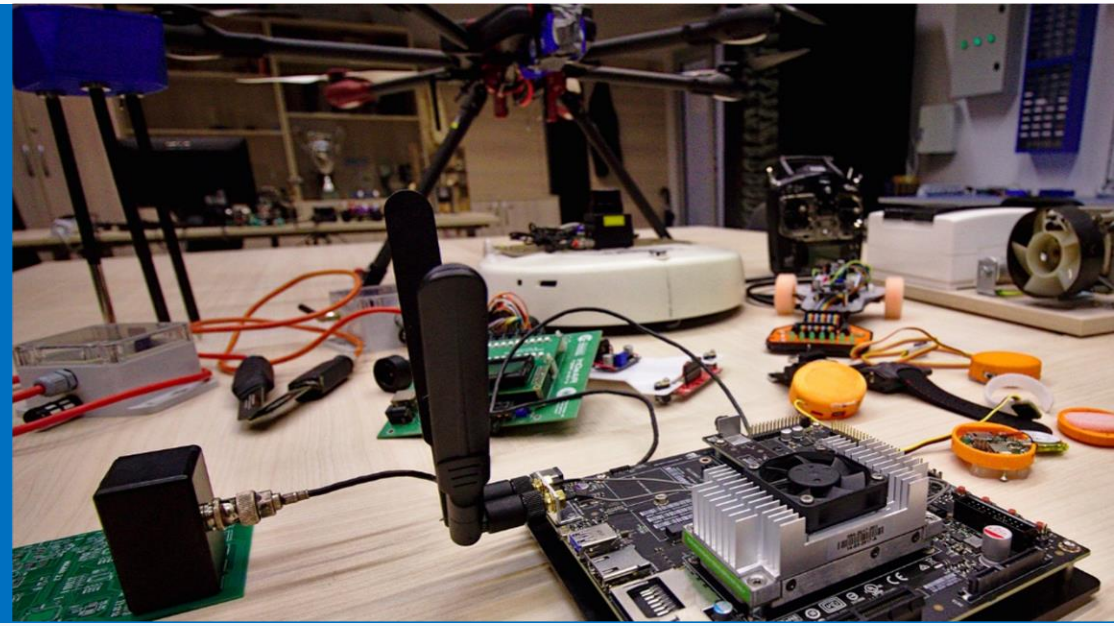
int main()
{
    OCR1A  = (F_CPU / 1024) - 1;          // 15624
    TCCR1B = (5 << CS10) | (1 << WGM12); // prescaler 1024 si mod CTC cu TOP la OCR1A

    TIMSK |= (1 << OCIE1A);               // intrerupere pe OCR1A

    sei();

    while (1);
    return 0;
}
```

PWM



PM

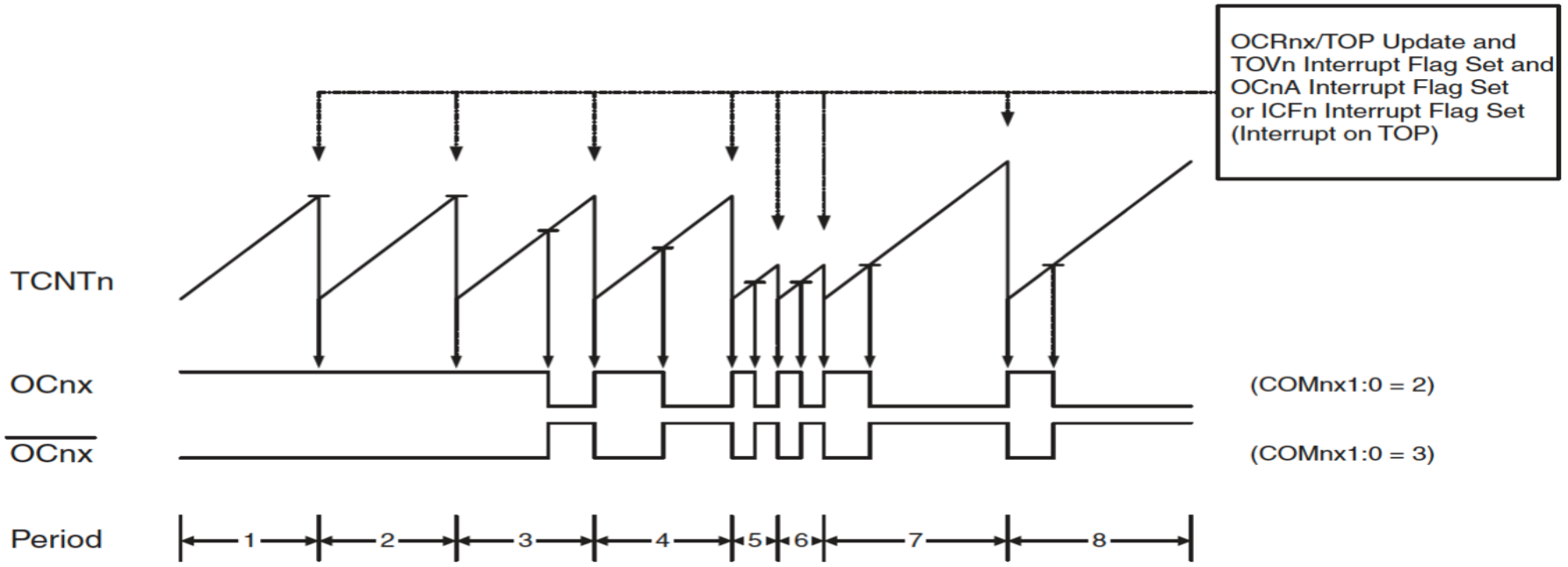
Semnale PWM

- Semnale pătratice periodice
  - Duty cycle
  - Frecvență
- De ce ne trebuie
  - Pot controla foarte multe dispozitive
    - Drivere de LED-uri
    - Drivere de motoare
- Cum sunt implementate
  - Timer-ele au căi de output → canale
  - Timer-ele produc output PWM configurabil
  - Restul funcționalității de timer nu sunt afectate

Modul FastPWM

- Moduri similare cu modul normal sau CTC, dar cu update de prag la reset
  - Acest mod oferă garanții pentru semnalele PWM generate
- Modul cel mai folosit de la laborator!

Figure 16-7. Fast PWM mode, timing diagram.



Exemplu mod FastPWM

- Vom da ca exemplu folosirea unui singur timer pentru două scopuri
  - Un canal de PWM este folosit pentru aprinderea unui LED
  - Alt canal de PWM este folosit pentru speaker
- Nevoi:
  - Canalul 1 (LED): Vrem să putem modifica ușor duty-cycle-ul, frecvența poate fi oricât atât timp cât este mai mare de 500Hz
  - Canalul 2 (Speaker): Vrem să putem modifica frecvența, duty cycle-ul trebuie să fie fixat la 50%

Exemplu mod FastPWM

- 2 semnale → 2 canale PWM
- Pentru a modifica frecvența putem folosi doar OCR1A sau ICR
  - LEDul este pe canalul A, deci avem nevoie de OCR1A pentru acel canal (prag → duty cycle)
  - **trebuie** să folosim ICR ca prag (ICR este Input Capture Register, folosit pentru a reține momentul apariției unor evenimente externe)
- Concluzii:
  - Frecvența este setată de ICR1
  - pragul pentru canalul A este OCR1A și determină luminozitatea pe LED
  - duty cycle-ul pe canalul B trebuie să fie 50%, putem face asta în două moduri diferite

Exemplu mod FastPWM

```
...

uint32_t nrSeconds = 0;

void set_speaker_frequency(int fr)
{
    uint32_t threshold = F_CPU / 2 / fr - 1; // inversul formulei de mai devreme

    threshold = MIN(threshold, 0xFFFF);
    threshold = MAX(threshold, 3);

    ICR = threshold;     // cast implicit la 16 biti
}

int main()
{
    // prescaler 1 si mod CTC cu TOP la ICR1
    TCCR1B = (1 << CS10) | (1 << WGM13) | (1 << WGM12);
    TCCR1A = (1 << WGM11);

    TCCR1A |= (2 << COM1A0);     // non-inverting mode pe canalul A
    TCCR1A |= (3 << COM1B0);     // toggle mode pe canalul B

    DDRD |= (1 << PD4) | (1 << PD5); // canalele de PWM trebuie setate ca iesiri

    while (1);
    return 0;
}
```

Probleme cu codul precedent

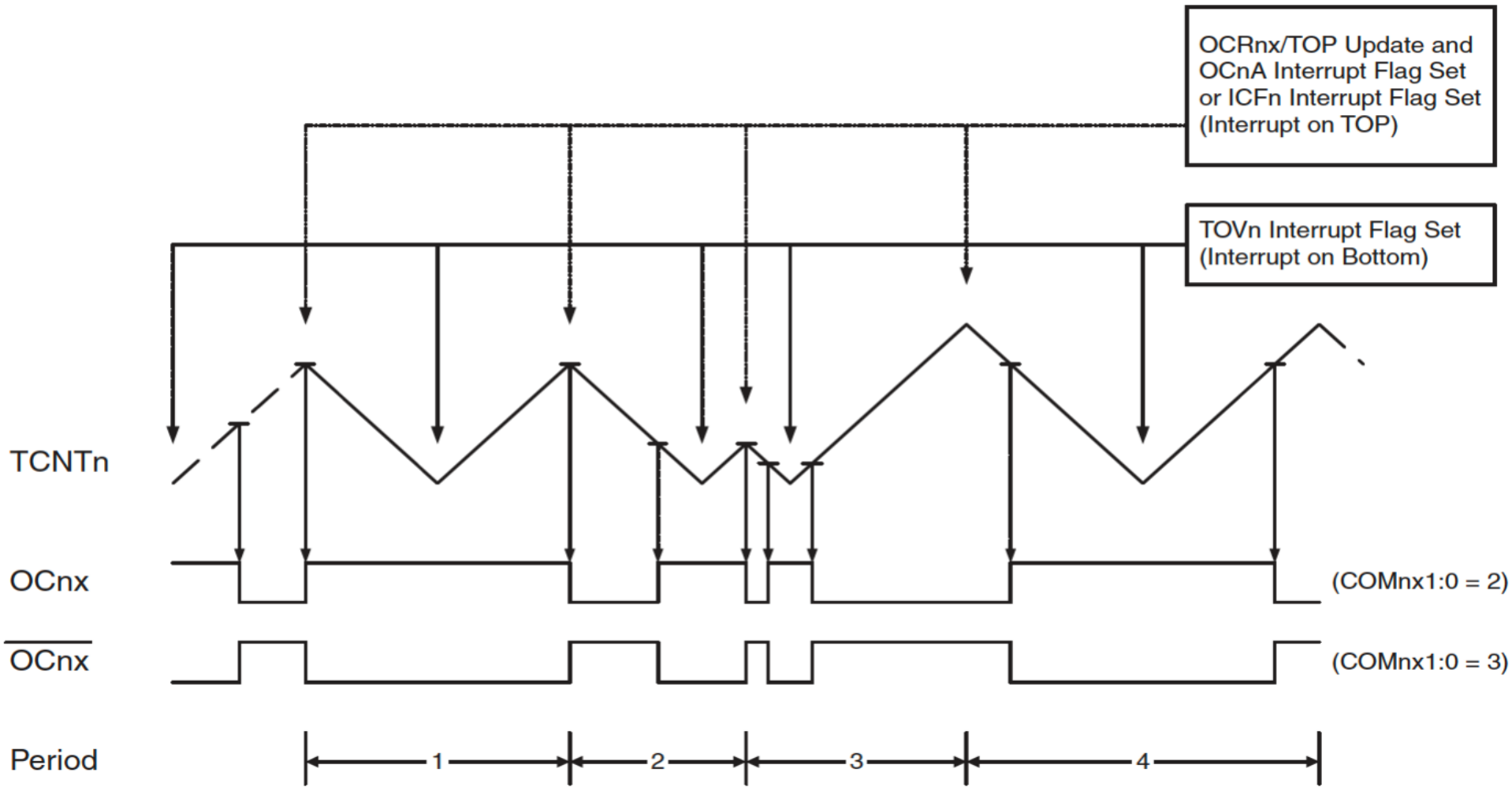
- Care este gama de frecvențe posibilă pe speaker-ul din exemplul precedent?
  - Gama de praguri valide este [3, 65535]
  - $f_{\text{Speaker}} = \frac{f_{CPU}}{2 \cdot 1 \cdot (1+\text{prag})}$
  - Gama de frecvențe este atunci [122Hz, 2MHz]
  - Cum putem schimba această gamă?
  - Dacă am fi folosit prescaler-ul de 1024, gama era [0.12Hz, 1.95kHz]
- Trebuie să folosim un prescaler care ne permite să avem întreaga gamă de frecvențe audibile [20Hz, 20kHz], dacă este posibil

Modul PWM, Phase correct

- Folosit tot pentru generarea semnalelor PWM
- Acolo unde tranzițiile semnalului sunt în general nedorite
- Contorul numără între 0 și TOP, apoi înapoi la 0
  - Dacă facem tranziția pe semnalul PWM la prag, obținem un semnal cu același duty cycle ca fastPWM, dar cu frecvență de 2 ori mai mică
- Clear when down-counting, set on up-counting
- Set when down-counting, clear on up-counting

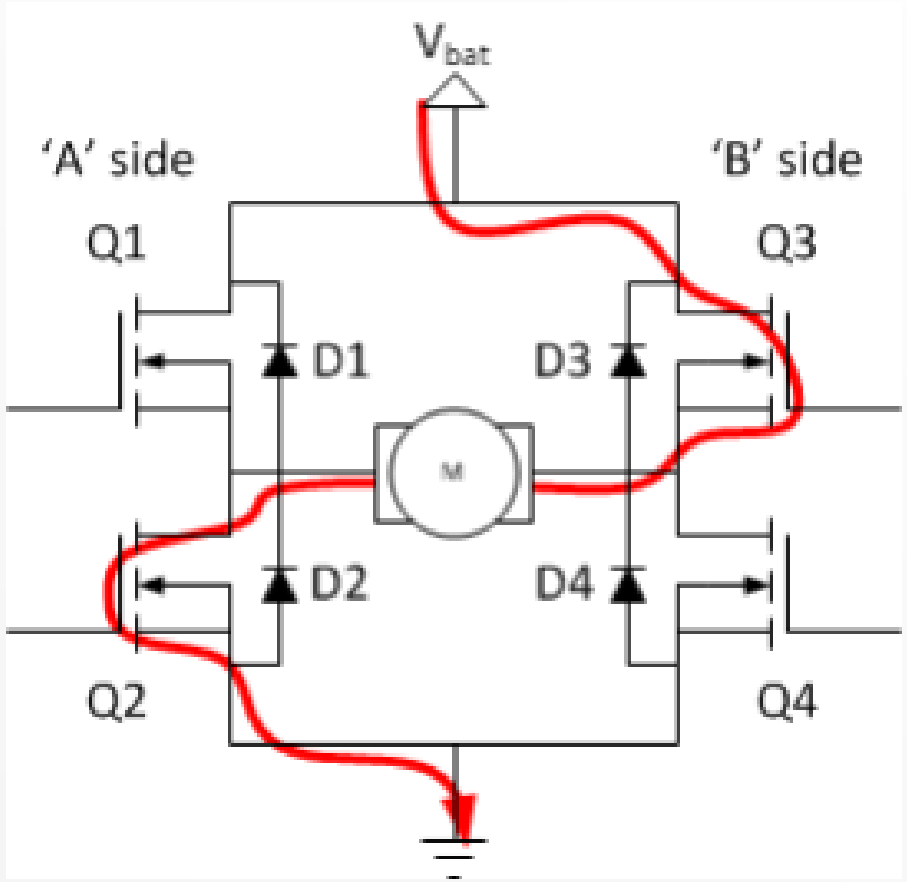
Modul PWM, Phase correct

Figure 16-8. Phase Correct PWM mode, timing diagram.



Drivere de motoare DC

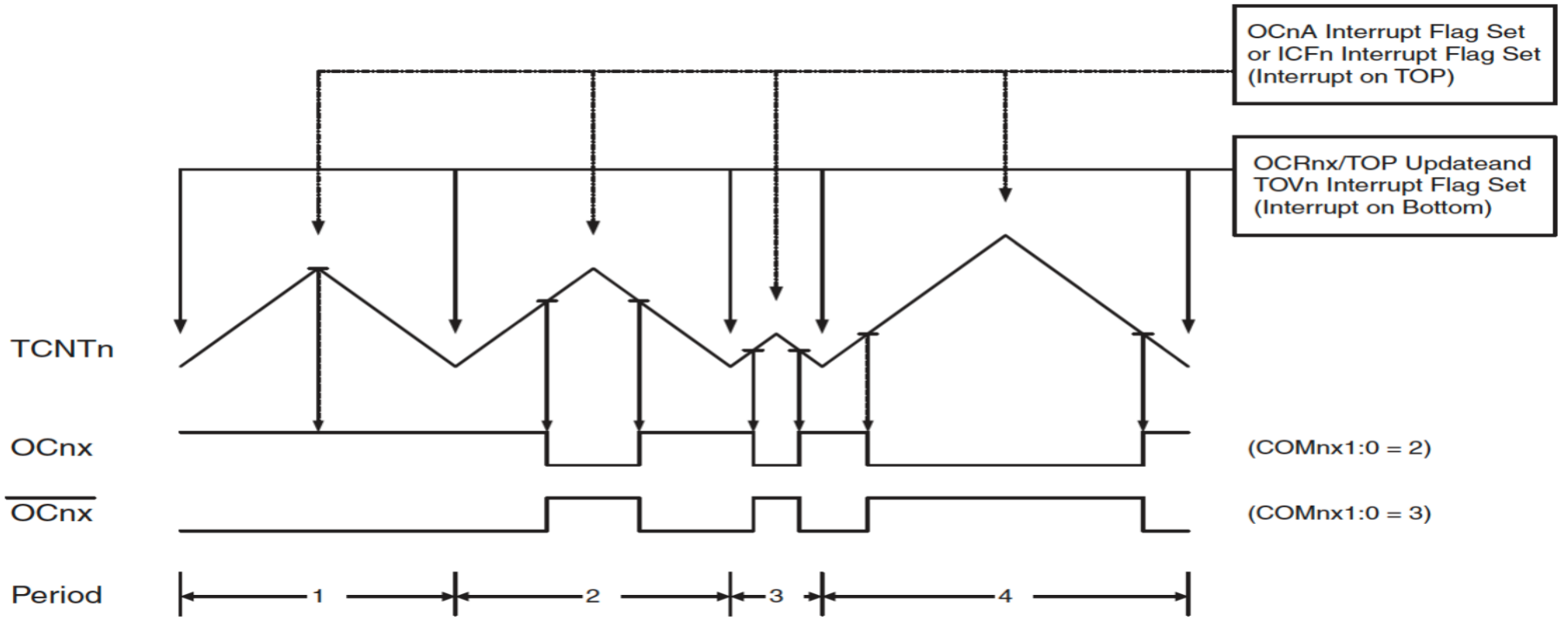
- Un motor nu poate fi controlat direct din pini
- Dacă folosim un singur tranzistor (driver), putem porni motorul într-un singur sens
- Pentru a porni motorul DC în ambele sensuri este nevoie de o punte H



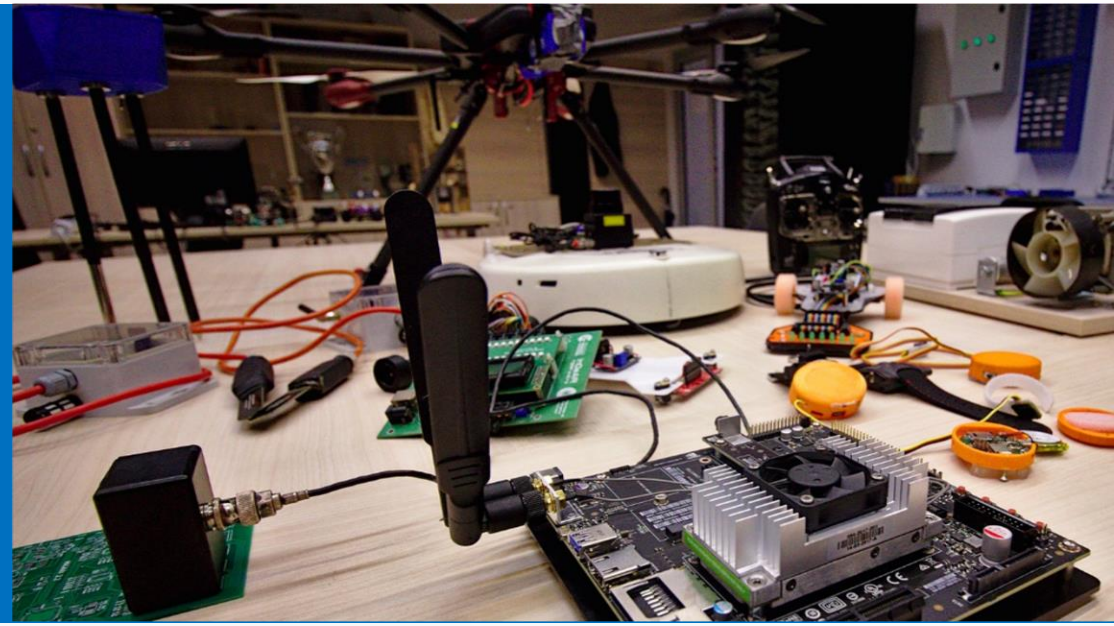
Modul PWM, Phase and frequency correct

- La fel ca modul PWM phase correct, dar update-ul pe prag se întâmplă când se ajunge la 0 → pulsurile generate sunt întotdeauna consistente

Figure 16-9. Phase and Frequency Correct PWM mode, timing diagram.



ADC

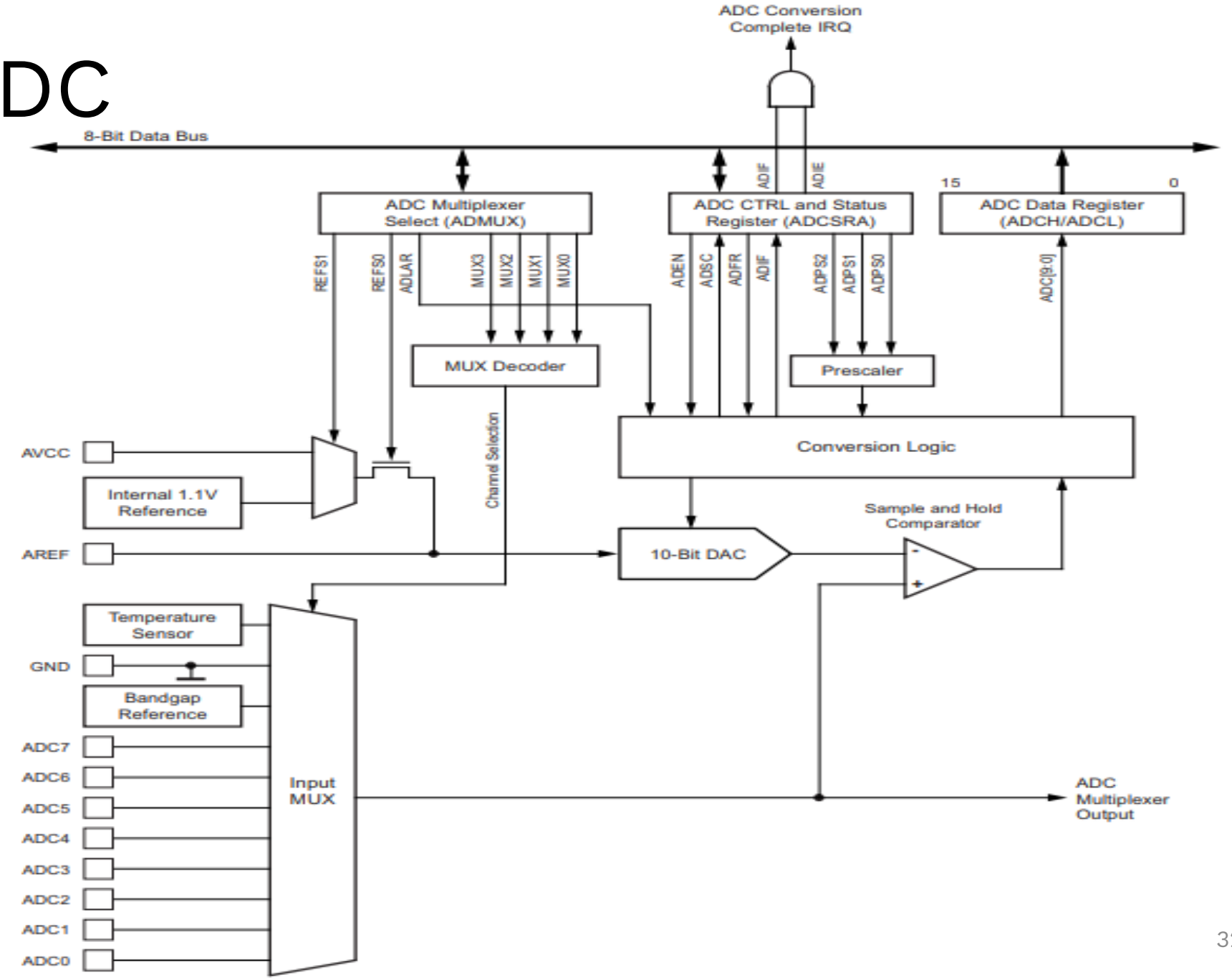


PM

ADC

- Analog to Digital Converter
  - Periferic care se ocupă de conversia unor tensiuni pe pini în valori digitale.
  - Util pentru senzori analogici (temperatură, presiune, lumină, curent etc.)
- Rezoluție de 10 biți pe ATMega328P
  - Rezoluție vs precizie
  - Rezoluție 10 biți - 1024 valori
  - Domeniu [0..5V] →4.88mV
  - Precizie 2LSB = 9.76mV

Schema ADC


ADC Conversion Complete IRQ
8-Bit Data Bus
ADIF
ADIE
15
0
ADC Multiplexer Select (ADMUX)
ADC CTRL and Status Register (ADCSRA)
ADC Data Register (ADCH/ADCL)
REFS1
REFS0
ADLAR
MUX3
MUX2
MUX1
MUX0
ADEN
ADSC
ADFR
ADIF
ADPS2
ADPS1
ADPS0
ADC[9:0]
MUX Decoder
Prescaler
Channel Selection
Conversion Logic
AVCC
Internal 1.1V Reference
AREF
Sample and Hold Comparator
10-Bit DAC
Temperature Sensor
GND
Bandgap Reference
ADC7
ADC6
ADC5
ADC4
ADC3
ADC2
ADC1
ADC0
Input MUX
ADC Multiplexer Output

Configurare ADC

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul [0..x]?
  - Vcc
  - Referință externă
  - 2.56V generați intern
  - 1.1V generați intern
- De unde luăm tensiunea
  - Un pin - PA0..PA7
  - Diferența între doi pini amplificată
- Când facem conversia
  - La comandă
  - La un anumit semnal

Implementare ADC

- Să definim initial o functie pentru inițializarea ADC înainte de main

```
void ADC_Init(void)
{

}
```

Implementare ADC - ADMUX

- Va trebui să definim o serie de registrii

ADC Multiplexer Selection Register

ADMUX

Bit	7	6	5	4	3	2	1	0
	REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 7:6 – REFSn: Reference Selection [n = 1:0]

These bits select the voltage reference for the ADC. If these bits are changed during a conversion, the change will not go in effect until this conversion is complete (ADIF in ADCSRA is set). The internal voltage reference options may not be used if an external reference voltage is being applied to the AREF pin.

Table 27-3. ADC Voltage Reference Selection

REFS[1:0]	Voltage Reference Selection
00	AREF, Internal V_{ref} turned off
01	AV_{CC} with external capacitor at AREF pin
10	Internal 1.1V Voltage Reference with external capacitor at AREF pin
11	Internal 2.56V Voltage Reference with external capacitor at AREF pin

Note: If differential channels are selected, only 2.56V should be used as Internal Voltage Reference.

Implementare ADC - ADMUX

- Va trebui să definim o serie de registrii

ADC Multiplexer Selection Register

ADMUX

Bit	7	6	5	4	3	2	1	0
	REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 4:0 – MUXn: Analog Channel and Gain Selection Bits [n = 4:0]

The value of these bits selects which combination of analog inputs are connected to the ADC. These bits also select the gain for the differential channels. If these bits are changed during a conversion, the change will not go in effect until this conversion is complete. (ADIF in ADCSRA is set).

Table 27-4. Input Channel and Gain Selections

MUX[4:0]	Single Ended Input	Positive Differential Input	Negative Differential Input	Gain
00000	ADC0	N/A		
00001	ADC1			
00010	ADC2			
00011	ADC3			

Implementare ADC - ADMUX

- Așa arată implementarea registrului ADMUX

```
void ADC_Init(void)
{
   ADMUX |= (1<<REFS0) | (1<<MUX0);
}
```

Implementare ADC - ADCSRA

- V-a trebui să continuăm setarea de regiștrii

ADC Control and Status Register A
ADCSRA

Bit	7	6	5	4	3	2	1	0
	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit 7 – ADEN: ADC Enable
Writing this bit to one enables the ADC. By writing it to zero, the ADC is turned off. Turning the ADC off while a conversion is in progress, will terminate this conversion.

Implementare ADC - ADCSRA

- V-a trebui să continuăm setarea de regiștrii

ADC Control and Status Register A
ADCSRA

Bit	7	6	5	4	3	2	1	0
	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit 6 – ADSC: ADC Start Conversion

In Single Conversion mode, write this bit to one to start each conversion. In Free Running mode, write this bit to one to start the first conversion. The first conversion after ADSC has been written after the ADC has been enabled, or if ADSC is written at the same time as the ADC is enabled, will take 25 ADC clock cycles instead of the normal 13. This first conversion performs initialization of the ADC.

ADSC will read as one as long as a conversion is in progress. When the conversion is complete, it returns to zero. Writing zero to this bit has no effect.

Implementare ADC - ADCSRA

- V-a trebui să continuăm setarea de regiștrii

ADC Control and Status Register A
ADCSRA

Bit	7	6	5	4	3	2	1	0
	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit 5 – ADATE: ADC Auto Trigger Enable
When this bit is written to one, Auto Triggering of the ADC is enabled. The ADC will start a conversion on a positive edge of the selected trigger signal. The trigger source is selected by setting the ADC Trigger Select bits, ADTS in ADCSRB.

Implementare ADC - ADCSRA

- V-a trebui să continuăm setarea de regiștrii

ADC Control and Status Register A
ADCSRA

Bit	7	6	5	4	3	2	1	0
	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit 3 – ADIE: ADC Interrupt Enable
When this bit is written to one and the I-bit in SREG is set, the ADC Conversion Complete Interrupt is activated.

Implementare ADC - ADCSRA

- V-a trebui să continuăm setarea de regiștrii

ADC Control and Status Register A
ADCSRA

Bit	7	6	5	4	3	2	1	0
	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Table 27-5. Input Channel Selection

ADPS[2:0]	Division Factor
000	2
001	2
010	4
011	8
100	16
101	32
110	64
111	128

Implementare ADC - ADCSRA

- Așa arată implementarea registrului ADMUX

```
void ADC_Init(void)
{
   ADMUX |= (1<<REFS0) | (1<<MUX0);
   ADCSRA |= (1<<ADEN) | (1<<ADSC) | (1<<ADATE) | (1<<ADIE);
   ADCSRA |= (1<<ADPS0) | (1<<ADPS1) | (1<<ADPS2);
}
```

Implementare ADC – Global Interrupts

- Activare Global Interrupts

Status Register

When addressing I/O Registers as data space using LD and ST instructions, the provided offset must be used. When using the I/O specific commands IN and OUT, the offset is reduced by 0x20, resulting in an I/O address offset within 0x00 - 0x3F.

Name: SREG
Offset: 0x5F
Reset: 0x00
Property: When addressing as I/O Register: address offset is 0x3F

Bit	7	6	5	4	3	2	1	0
	I	T	H	S	V	N	Z	C
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit 7 – I: Global Interrupt Enable

The Global Interrupt Enable bit must be set for the interrupts to be enabled. The individual interrupt enable control is then performed in separate control registers. If the Global Interrupt Enable Register is cleared, none of the interrupts are enabled independent of the individual interrupt enable settings. The I-bit is cleared by hardware after an interrupt has occurred, and is set by the RETI instruction to enable subsequent interrupts. The I-bit can also be set and cleared by the application with the SEI and CLI instructions, as described in the instruction set reference.

Implementare ADC - Global Interrupts

- Așa arată implementarea registrului ADMUX

```
void ADC_Init(void)
{
   ADMUX |= (1<<REFS0) | (1<<MUX0);
   ADCSRA |= (1<<ADEN) | (1<<ADSC) | (1<<ADATE) | (1<<ADIE);
   ADCSRA |= (1<<ADPS0) | (1<<ADPS1) | (1<<ADPS2);
   sei();
}
```

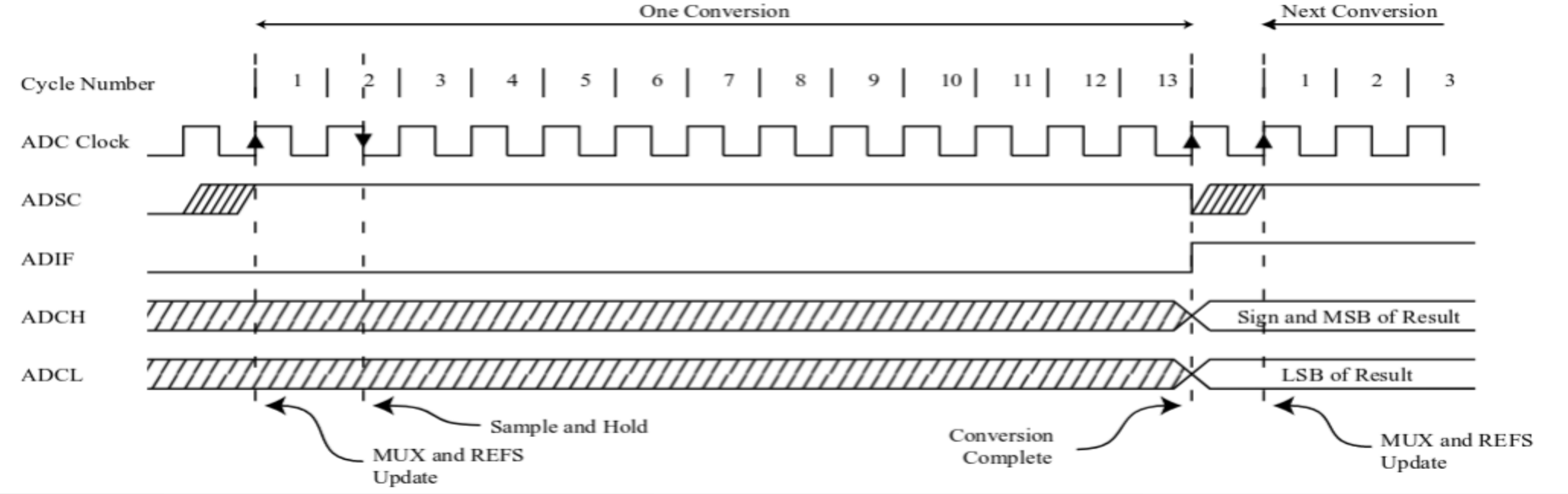
Implementare ADC - ADCSRA

- Să definim apoi rutina de întreruperi și să păstrăm valoarea conversiei ADC într-o variabilă
- Când o conversie este completă, bitul ADIF este setat și nu trebuie să fie șters de software

```
uint16_t adcValue;

ISR(ADC_vect)
{
   adcValue = ADC;
}
```

Timing conversie ADC

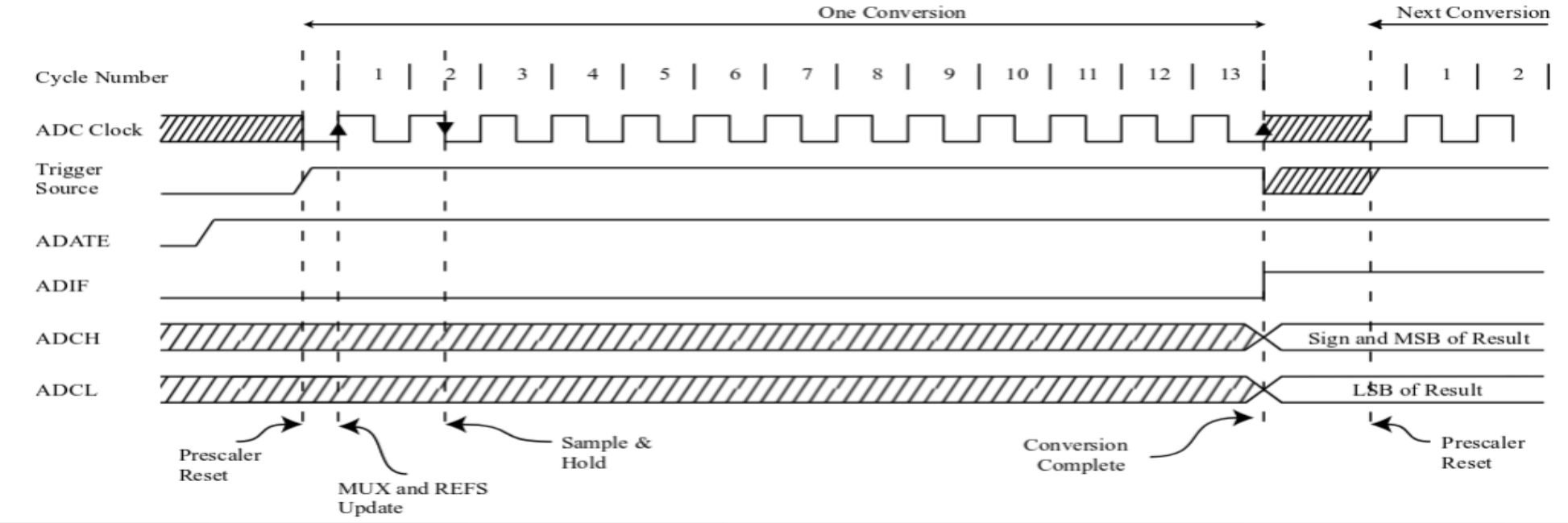


- O conversie durează 13.x cicli ADC (prima 25)
- Timing special pentru schimbarea de canal/referință
  - Trebuie evitate schimbări de la startul conversiei până la un ciclu ADC după (maxim 8µs sau 128 cicli procesor)
- După 1.5 cicli ADC se face Sample&Hold

Auto-triggering ADC

- Permite pornirea automată de conversii
- Surse posibile:
  - La terminarea conversiei precedente
  - Întrerupere externă INT0
  - Timer0 compA sau overflow
  - Timer1 compB/overflow/input capture
  - Comparatorul analogic
  - Metodă rapidă de a porni conversii

Timing auto-triggering ADC



- Durează 13.5 cicli ADC
- Mai sensibilă la schimbări de canal/referință
  - Schimbăm canalul în timpul unei conversii, la un ciclu ADC după pornire

Exemplu complex ADC

- Folosire cu mai mulți senzori
- Avem mai multe variante:
  - Pornim o conversie de fiecare dată când avem nevoie de o valoare
  - Facem conversiile pentru toți senzorii și stocăm cea mai recentă valoare
- Dacă vrem să facem toate conversiile, putem face în mai multe moduri
  - Auto-running mode: Lăsăm fiecare conversie să ”cascadeze” în următoarea. Avem grijă de fiecare dată să schimbăm canalul pentru a face conversiile pentru toți senzorii
  - Timer care pornește o cascadă de conversii (o dată la X ms se pornește prima conversie, care o antrenează pe următoarea etc.)

Exemplu complex ADC

```
volatile uint8_t channel = 0;
volatile uint8_t adc_values[NRSENSORS];

ISR(TIMER1_COMPA_vect)
{
    ADCSRA |= (1 << ADSC);
}
ISR(ADC_vect)
{
    adc_values[channel] = ADC;                  // luam rezultatul precedent
    channel = (channel + 1) % NRSENSORS;        // trecem la urmatorul canal
    ADMUX = (ADMUX & 0xE0) | channel;           // schimbam canalul si punem in ADMUX
    if (channel != 0)                           // pornim urmatoarea conversie daca nu
    {                                           // este ultima
        ADCSRA |= (1 << ADSC);
    }
}
init_adc()
{
    ADMUX = (1 << REFS0);                       // referinta de tensiune AVCC
    ADCSRA = (1 << ADEN) | (1 << ADIE) | (5 << ADPS0);
}
init_timer()
{
    TCCR1B = (1 << WGM12) | (5 << CS10);        // CTC, TOP OCR1A, prescaler 1024
    TIMSK1 = (1 << OCIE1A);                     // activare intrerupere compare

    OCR1A = 16000000UL/1024/10 - 1;             // 10Hz - 100ms
}
int main()
{
    ...
}
```

Exemplu complex ADC

```
volatile uint8_t channel = 0;
volatile uint8_t adc_values[NRSENSORS];

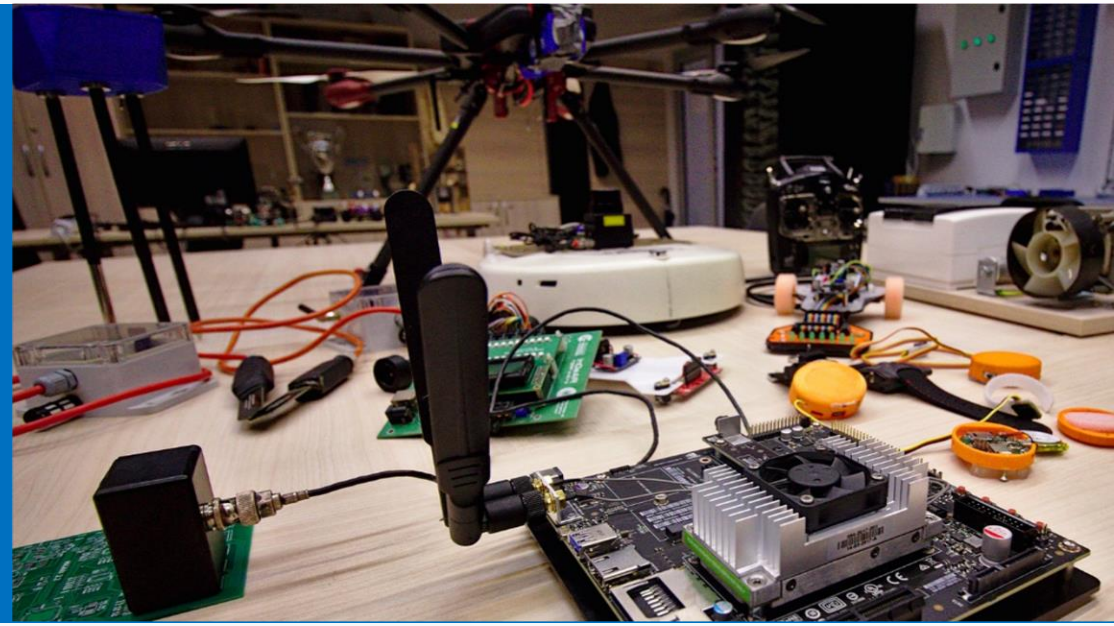
ISR(ADC_vect)
{
    adc_values[channel] = ADC;                  // luam rezultatul precedent
    channel = (channel + 1) % NRSENSORS;        // trecem la urmatorul canal
    ADMUX = (ADMUX & 0xE0) | channel;           // schimbam canalul si punem in ADMUX
    if (channel != 0)                           // pornim urmatoarea conversie daca nu
    {                                           // este ultima
        ADCSRA |= (1 << ADSC);
    }
}
init_adc()
{
    ADMUX = (1 << REFS0);                       // referinta de tensiune AVCC
    ADCSRA = (1 << ADEN) | (1 << ADATE) | (1 << ADIE) | (5 << ADPS0);
    ADCSRB = (6 << ADTS0);                      // trigger pe timer 1 overflow
}
init_timer()
{
    TCCR1A = (3 << WGM11);
    TCCR1B = (3 << WGM12) | (5 << CS10);        // Fast PWM, TOP OCR1A, prescaler 1024

    OCR1A = 16000000UL/1024/10 - 1;             // 10Hz - 100ms
}
int main()
{
    ...
}
```

Concluzii ADC

- Putem converti o tensiune de pe pinii Analogici
- Precizie vs. Rezoluție
- Proces lung → trebuie evitat busy-waiting
- Update-ul de configurație are perioade critice neprotejate
- Conversiile pot dura diferit în funcție de stare/configurare

Watchdog



PM

Ce este un watchdog?

- Watchdog-ul este un timer special al sistemului
- Când timer-ul ajunge la o anumită valoare timer-ul ”expiră”
  - Controller-ul este fie resetat fie intră într-o întrerupere de watchdog
- Pentru a preveni asta, programul de pe microcontroller trebuie să reseteze contorul periodic petting the watchdog
- Astfel se garantează că programul nu intră într-o stare de deadlock sau o altă stare invalidă

Exemplu watchdog

```
...

ISR(WDT_vect)
{
    do_stuff();                                 // retine informatii relevante despre
                                                // starea anormala in care se gaseste
}

int main()
{
    WDTCSR = (1 << WDTON) | (1 << WDIE); // watchdog enable, cu intrerupere
    WDTCSR |= (9 << WDP0);               // timeout de 8s

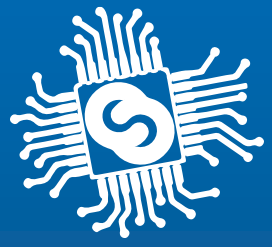
    sei();

    while (1)
    {
        wdt_reset();                    // pet/reset the watchdog
        do_normal_stuff();              // bucla normala
        _delay_ms(250);
    }
    return 0;
}
```

La cursul următor

- UART/USART
- SPI
- I2C
- OneWire
- Aplicații/utilizări

Cursul #6&7



PM

Disclaimer

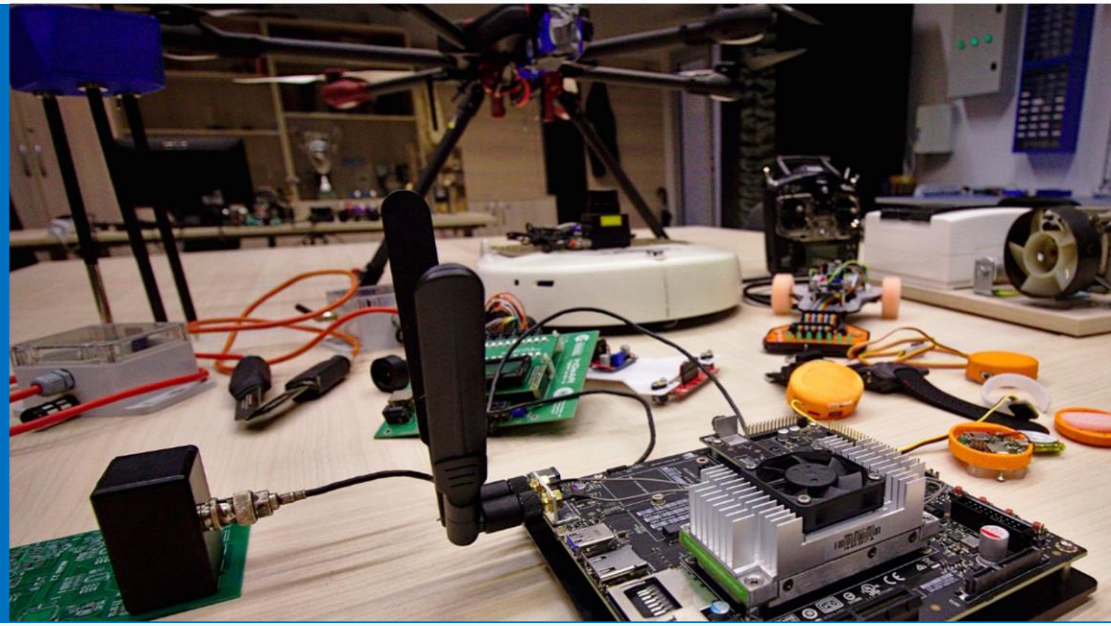
- *Slide-urile acestea reprezintă un rezumat.*
- *Slide-urile nu acoperă toate explicațiile, simulările, demonstrațiile oferite în timpul cursului.*
- *Slide-urile nu limitează în vreun fel materia cerută la examen.*
- *Pentru varianta completă, sunteți așteptați la curs.*

© Elemente din curs preluate din:
- *notițe curs PM 2021, Dan Tudose*
- *notițe curs PM 2021, Andrei Voinescu*
- *scoala de vara EAP 2016, Marius Nicolae*
- *notite curs EEA/ED, Razvan Tataroiu & Daniel Rosner*

Cuprins

- UART/USART
- SPI
- I2C
- OneWire
- USB
- Aplicații/utilizări
- Next time

UART



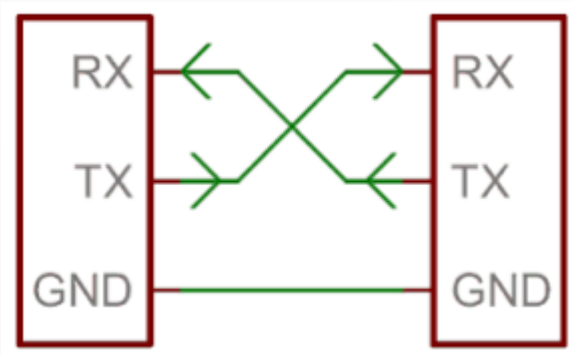
PM

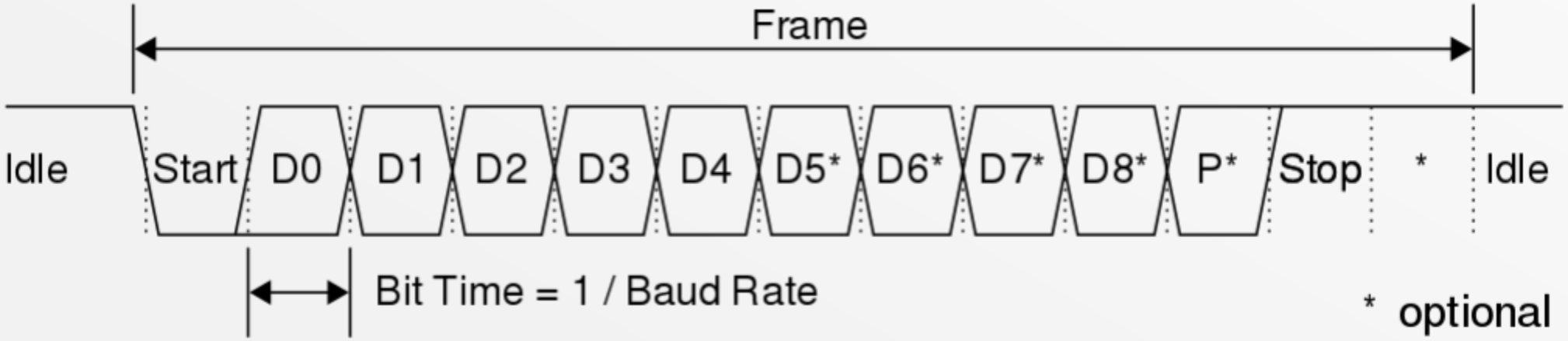
Seriale

- Modalități de interconectare între dispozitive, sau între chip-uri din cadrul aceluiași dispozitiv
- În contrast cu interfețele paralele, presupune înșiruirea mai multor biți pe aceeași linie de date
- Duplex/ Half-duplex / Full-duplex
  - Duplex se referă la posibilitatea de a avea comunicație în 2 sensuri pe 2 canale diferite.
  - Comunicația Full-duplex permite comunicației să aibă loc simultan în ambele sensuri
  - Comunicația Half-duplex nu permite comunicația într-un singur sens la un moment dat

Conexiune UART

- Start-ul este dat de o tranziție a liniei între 1 și 0
- Datele pot avea între 5 și 9 biți
- Datele pot fi verificate cu un bit de paritate
- Sunt 1 sau 2 biți de STOP
- Conexiuni doar între două dispozitive
- Semnalele sunt legate în cruce

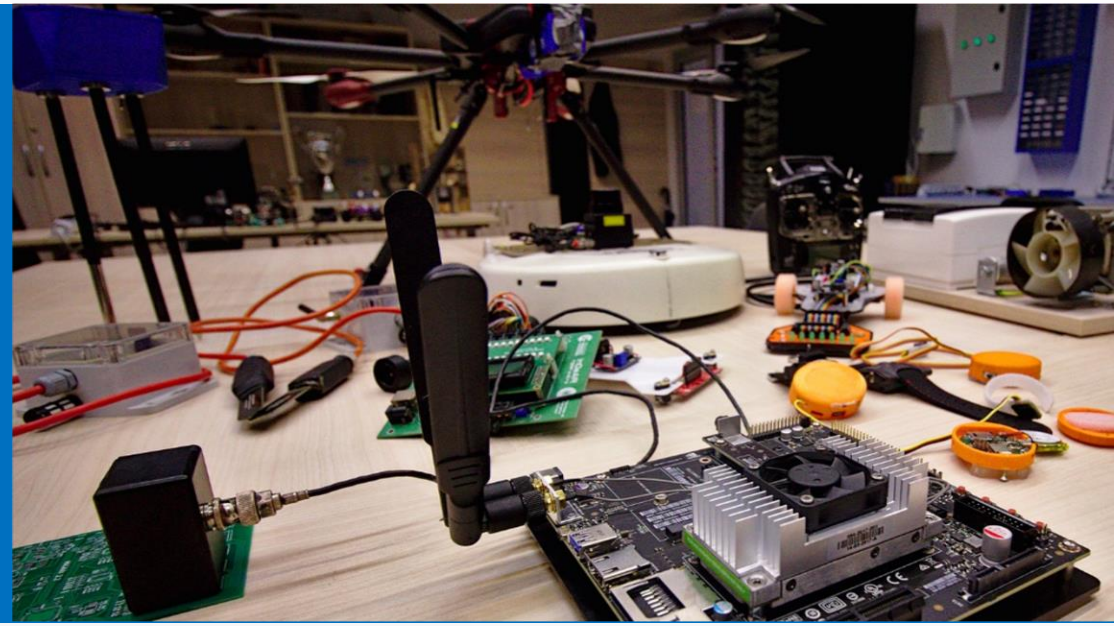




U(S)ART

- Universal Synchronous/Asynchronous Receiver Transmitter
- Comunicație full-duplex
- UART - Protocol cu ceas implicit
  - Ambii parteneri în comunicație (cele 2 dispozitive) folosesc același ceas în transmisie/recepție
  - Similar, toți parametrii comunicației trebuie să fie identici
    - Frame size
    - Stop bits
    - Parity bit
- USART - ceas explicit - XCK
  - Generat de master
  - Slave-ul trebuie să se sincronizeze

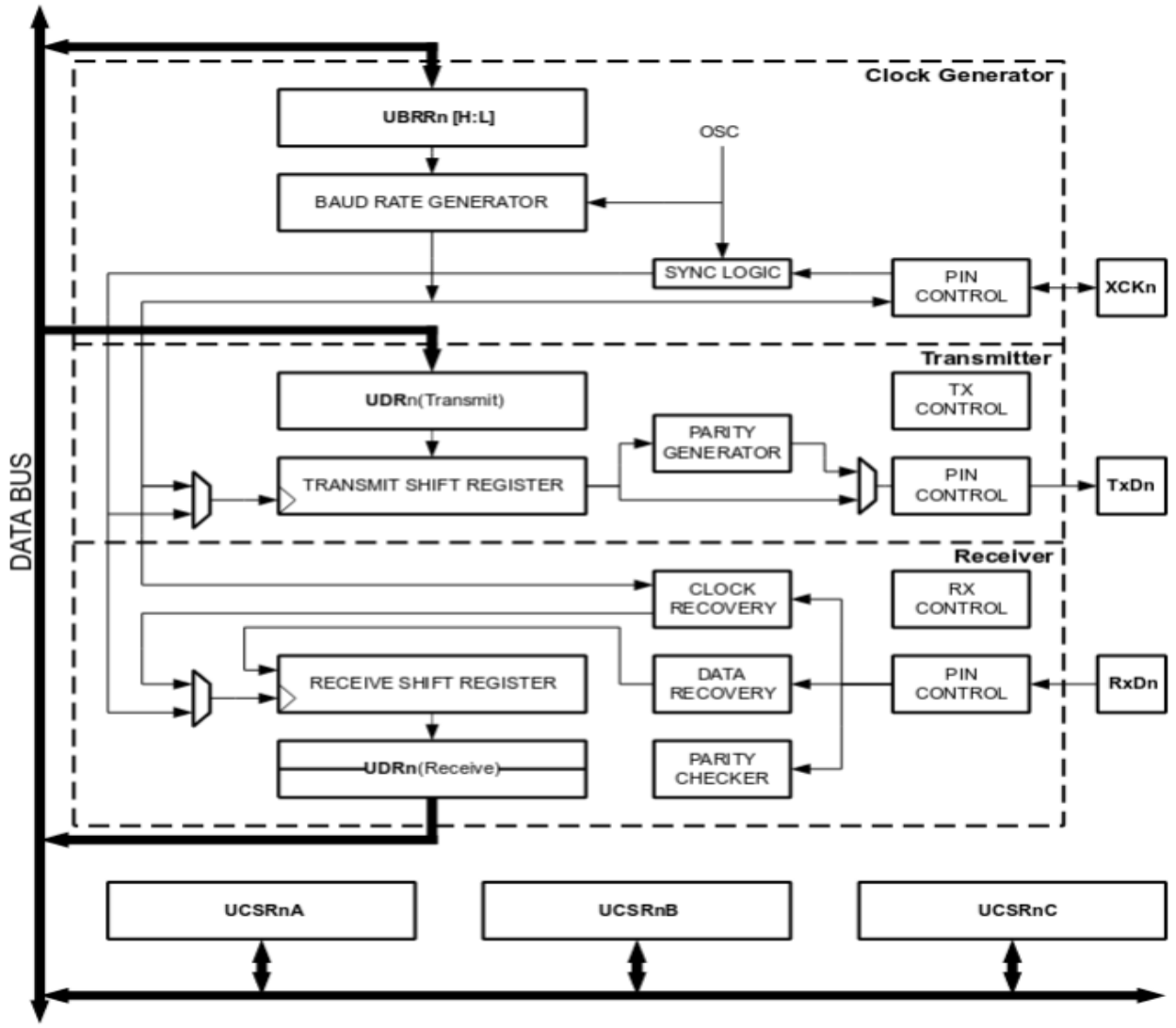
UART pe ATMega



PM

Arhitectura

Figure 21-1. USART Block Diagram



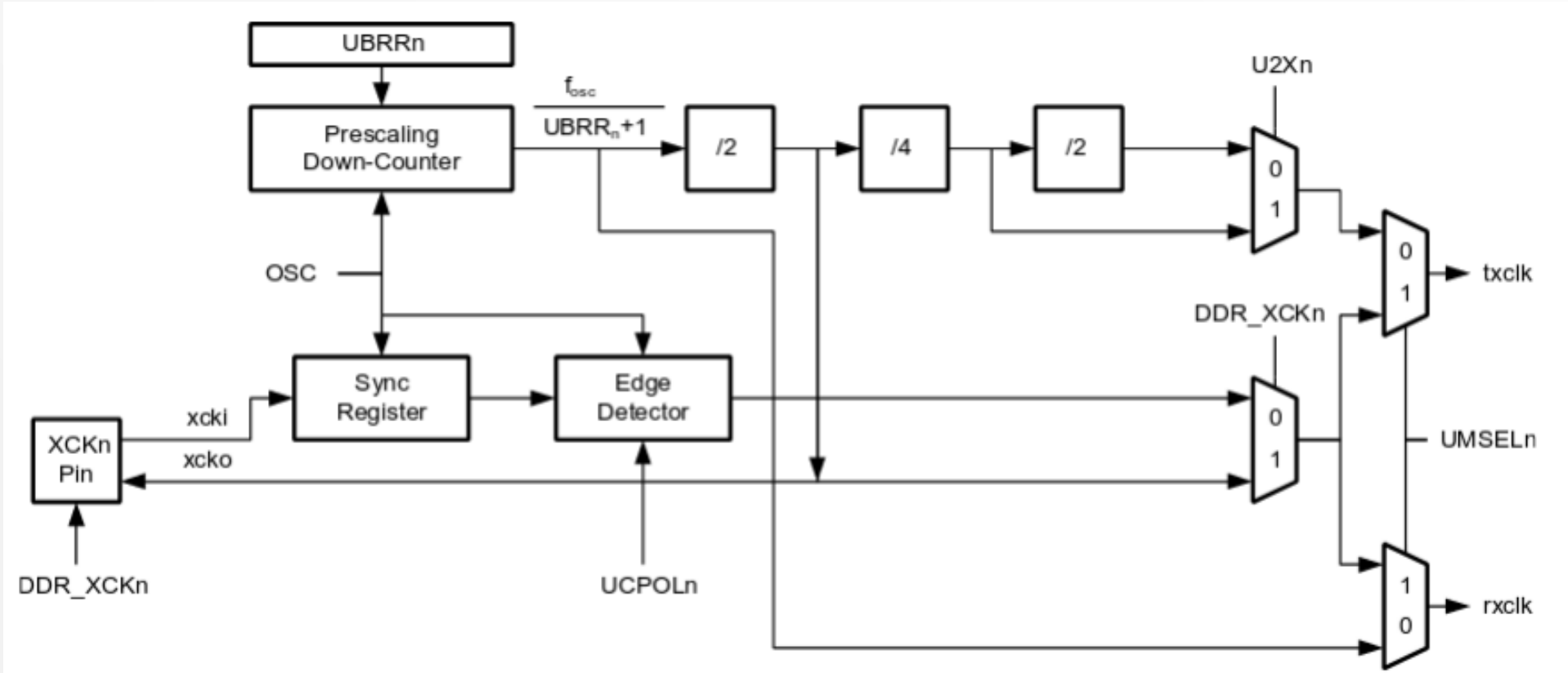
Programare

- Ce trebuie să controlăm
  - Structura unui cadru (frame)
  - Viteza de tx/rx
- Ce trebuie să aflăm
  - Flow date în/din buffer
  - Verificări paritate

Registre

- Datorită sincronizării între domenii de timp

$$f_{XCK} < \frac{f_{osc}}{4}$$



Sincronizare

- Receptorul caută START
  - Detectează o tranziție 1→0
  - Citiri repetate (3) în mijlocul bitului
  - 3 x 0 → START valid
- Citește fiecare bit din frame
- Citește STOP bit ca 1 (altfel Frame Error)

- Surse de erori:
  - Frecvențele de tx/rx nu sunt egale
  - Frecvențe instabile
  - Glitch-uri pe linia de transmisie (START fals)
  - All of the above
- Frame Error - bit în registru de status
- Parity Error - bit în registru de status

USART pe ATMega

- Format cadru - identic
- Două câmpuri de biți de configurare în plus
  - UMSEL - mode select
  - UCPOL - clock polarity
- O linie de semnal în plus
- Frecvențe de comunicație mai mari
- Sursele de eroare mult diminuate

Detalii registre

- UBRR0
  - Registru prescaler pentru generarea ceasului de transmisie
  - Prescaler-ul de la fCPU este 16 · UBRR0
  - UBRR0 poate fi calculat sau poate fi extras din tabele existente în datasheet

$$\text{UBRR} = \frac{f_{CPU}}{16 \cdot \text{baud}} - 1$$

  - putem dubla viteza cu același UBRR

$$UBRR_{U2X} = \frac{f_{CPU}}{8 \cdot \text{baud}} - 1$$

$$\text{Error}_{[\%]} = \left(\frac{\text{Baudrate}_{closest}}{\text{Baudrate}} - 1\right) \cdot \%$$

Detalii registre

- UCSRnA (status)
  - UDRE - 1 când buffer-ul de ieșire este liber
  - RXC - 1 când am primit un frame nou
  - U2X - 1 dacă micșorăm prescalerul la 8
  - FE - 1 dacă a fost Frame Error
  - UPE - 1 dacă a fost Parity Error
- UCSRnB (biți enable)
  - TXEN - Activarea transmițătorului
  - RXEN - Activarea receptorului
- RXCIE, TXCIE, UDRIE - Activarea întreruperilor pentru RX complete, TX complete, buffer empty
- UCSRnC (parametri frame)
  - USBS - numărul de biți de stop
  - UCSZ - dimensiunea unui frame
  - UPM - configurarea bitului de paritate

Snippet cod

- Inițializare USART0 9600,8N1
  - UCSR0C |= (1 <<TXEN0) | (1 << RXEN0) ;
  - UBRR0 = 1 0 3 ;

- Inițializare USART0 115200,8E2
  - UCSR0B |= (1 <<TXEN0) | (1 << RXEN0) ;
  - UCSR0C |= (1 <<UPM01) | (1 << USBS0) | (3 << UCSZ00) ;
  - UBRR0 = 8 ;

- Inițializare USART1 2000000 (2Mbaud), 8O2
  - UCSR1A |= (1 << U2X1) ;
  - UCSR1B |= (1 <<TXEN1) | (1 << RXEN1) ;
  - UCSR1C |= (1 <<UPM10) | (1 << USBS1) | (3 << UCSZ10) ;
  - UBRR1 = 0 ;

Snippet cod

- Receive simplu

```
loop_until_bit_is_set (UCSR0A , RXC0 ) ;
char c = UDR0 ;
```

  - Busy-waiting
- Receive pe întrerupere

```
volatile  char c ;
ISR ( USART0 RX vect ) {
        c = UDR0 ;
}
```

  - Fără busy-waiting, dar se pierd date dacă nu le citește nimeni din variabila shared

Exemplu

```
#define BUFSIZE     (100)

volatile uint8_t rx_buffer[BUFSIZE];
volatile uint8_t read_index = 0, write_index = 0;

ISR(USART0_RX_vect)
{
    uint8_t next_write = (write_index + 1) % BUFSIZE;

    if (next_write != read_index) /* daca nu e plin bufferul */
    {
        rx_buffer[write_index] = UDR0;
        write_index = next_write;
    }
}

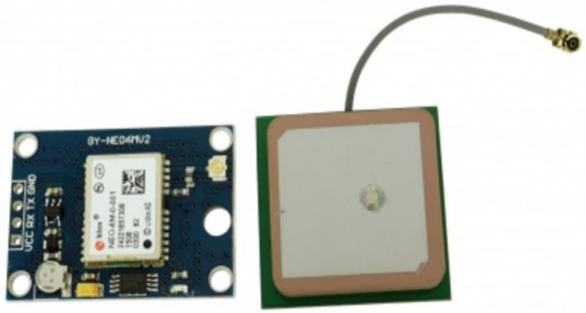
int main()
{
    init_stuff();
    sei();

    while(1)
    {
        while(read_index != write_index) /* cata vreme avem elemente */
        {
            LCD_putChar(rx_buffer[read_index]);
            read_index = (read_index + 1) % BUFSIZE;
        }
    }
    return 0;
}
```

Snippet

- Necesitatea unui buffer
  - Dacă folosim busy-waiting nu avem multă nevoie, dar avem multe alte dezavantaje
  - Bufferul din periferic reține un singur frame (5-9 biți)
  - Un buffer software ne va garanta că nu pierdem caractere

Exemplu GPS



- Modulele GSM/GPS de multe ori folosesc UART
- De ce?

CFG_COM1	CFG_COM0	Protocol	Messages	UARTBaud rate	USB power
1	**1**	**NMEA**	**GSV, RMC, GSA, GGA, GLL, VTG, TXT**	**9600**	**BUS Powered**
1	0	NMEA	GSV, RMC, GSA, GGA, GLL, VTG, TXT	38400	Self Powered
0	1	NMEA	GSV[14], RMC, GSA, GGA, VTG, TXT	4800	BUS Powered
0	0	UBX	NAV-SOL, NAV-STATUS, NAV-SVINFO, NAV-CLOCK, INF, MON-EXCEPT, AID-ALPSERV	57600	BUS Powered

- Configurația pinilor dictează protocoalele
- Pentru 1 și 1 avem UART 9600 baud cu formatul GGA

Protocolul NMEA

Table 1-3 contains the values for the following example:

$GPGGA,161229.487,3723.2475,N,12158.3416,W,1,07,1.0,9.0,M, , , ,0000*18

Table 1-3 GGA Data Format

Name	Example	Units	Description
Message ID	$GPGGA		GGA protocol header
UTC Time	161229.487		hhmmss.sss
Latitude	3723.2475		ddmm.mmmm
N/S Indicator	N		N=north or S=south
Longitude	12158.3416		dddmm.mmmm
E/W Indicator	W		E=east or W=west
Position Fix Indicator	1		See Table 1-4
Satellites Used	07		Range 0 to 12
HDOP	1.0		Horizontal Dilution of Precision
MSL Altitude	9.0	meters	
Units	M	meters	
Geoid Separation		meters	
Units	M	meters	
Age of Diff. Corr.		second	Null fields when DGPS is not used
Diff. Ref. Station ID	0000		
Checksum	*18		
<CR> <LF>			End of message termination

Checklist

- Conectăm modulul la ATMega324 (TX la RX, RX la TX)
- Masa și alimentarea, pinii de configurare pentru UART 9600 baud
- Configurăm seriala UART 0/1 pentru comunicație la 9600 baud
- Așteptăm frame-uri pe serială
- Citim frame-urile într-un buffer, parsăm atunci când am primit ”tot”

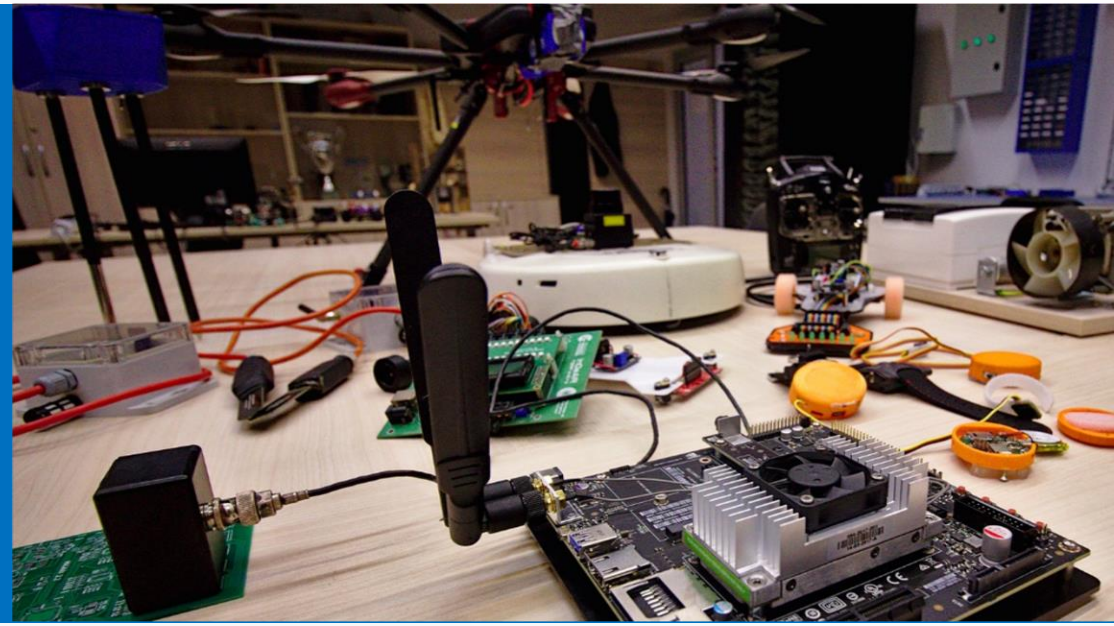
Intrebări relevante

- Cum se rezolvă?
- Cum se face inițializarea?
- Cum se face recepționarea datelor?
- Unde/Când sunt parsate datele?
- Ce facem cu rezultatul procesării datelor?

Concluzii

- Ceas implicit
- Transmițător independent de receptor
- Susceptibil la erori
- Error correction
- Mod sincron

SPI



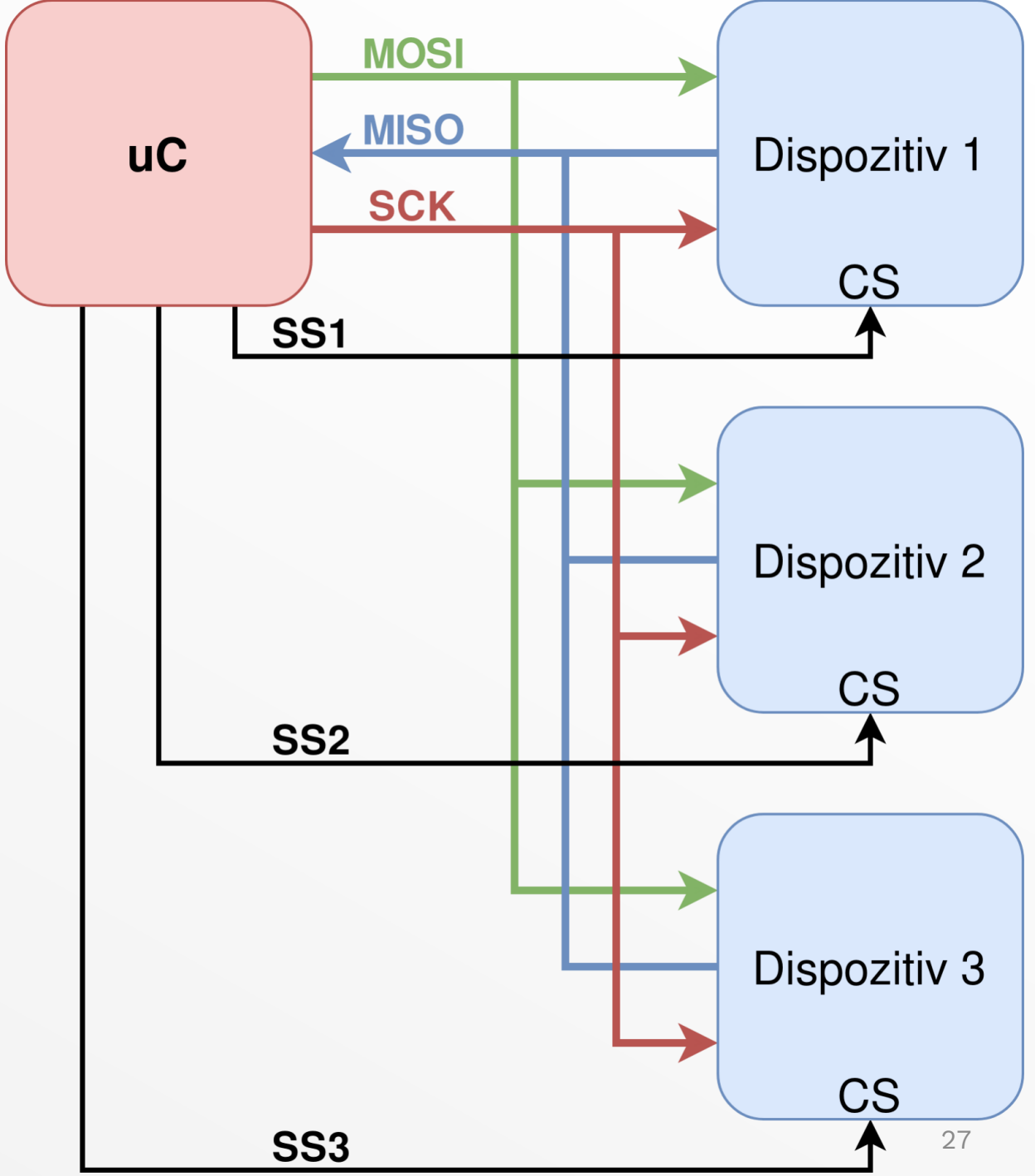
PM

SPI

- Serial Peripheral Interface
- Ce presupune
  - Comunicație sincronă 1:N
- Când este folosită
  - Dispozitive periferice externe cu bandwidth mare

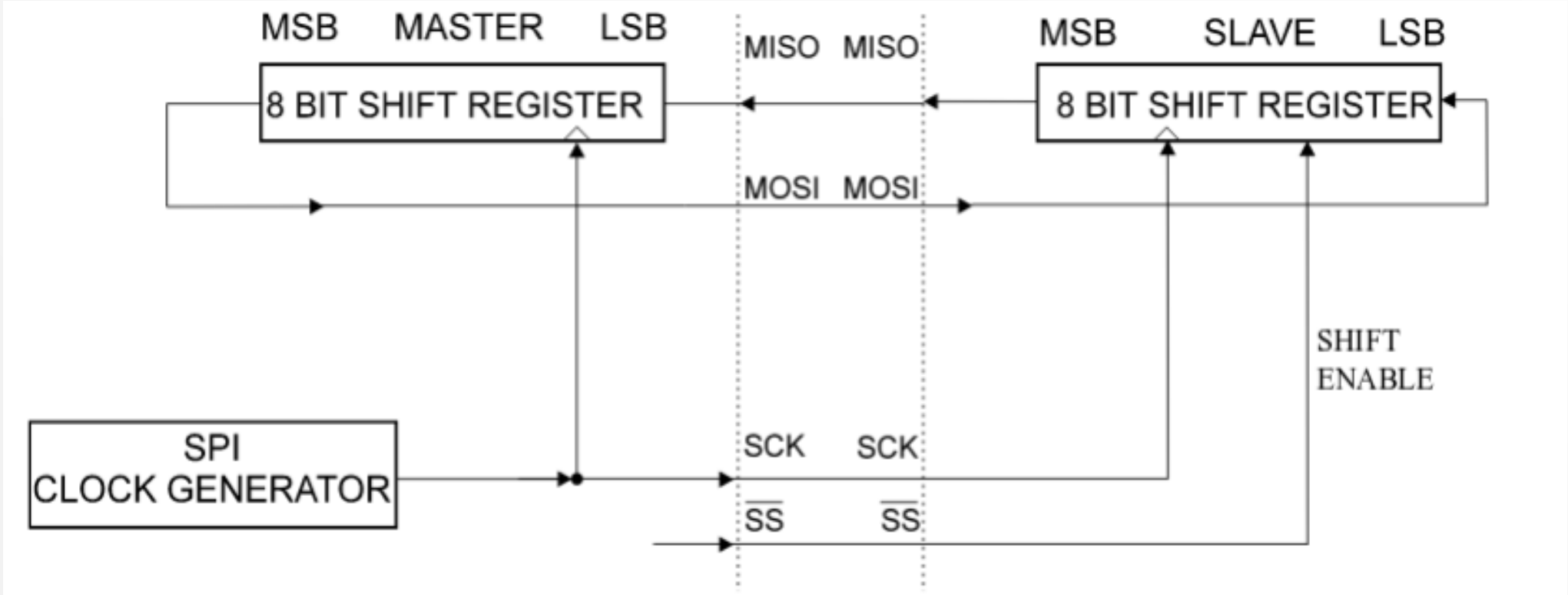
Conexiuni

- 4 linii de date pt device
- 3 linii comune
- câte o linie de select pt fiecare device
- linii unidirecționale
- MOSI - Master Output Slave Input
- MISO - Master Input Slave Output
- SCK - Serial Clock
- SS/CS - Slave/Chip Select

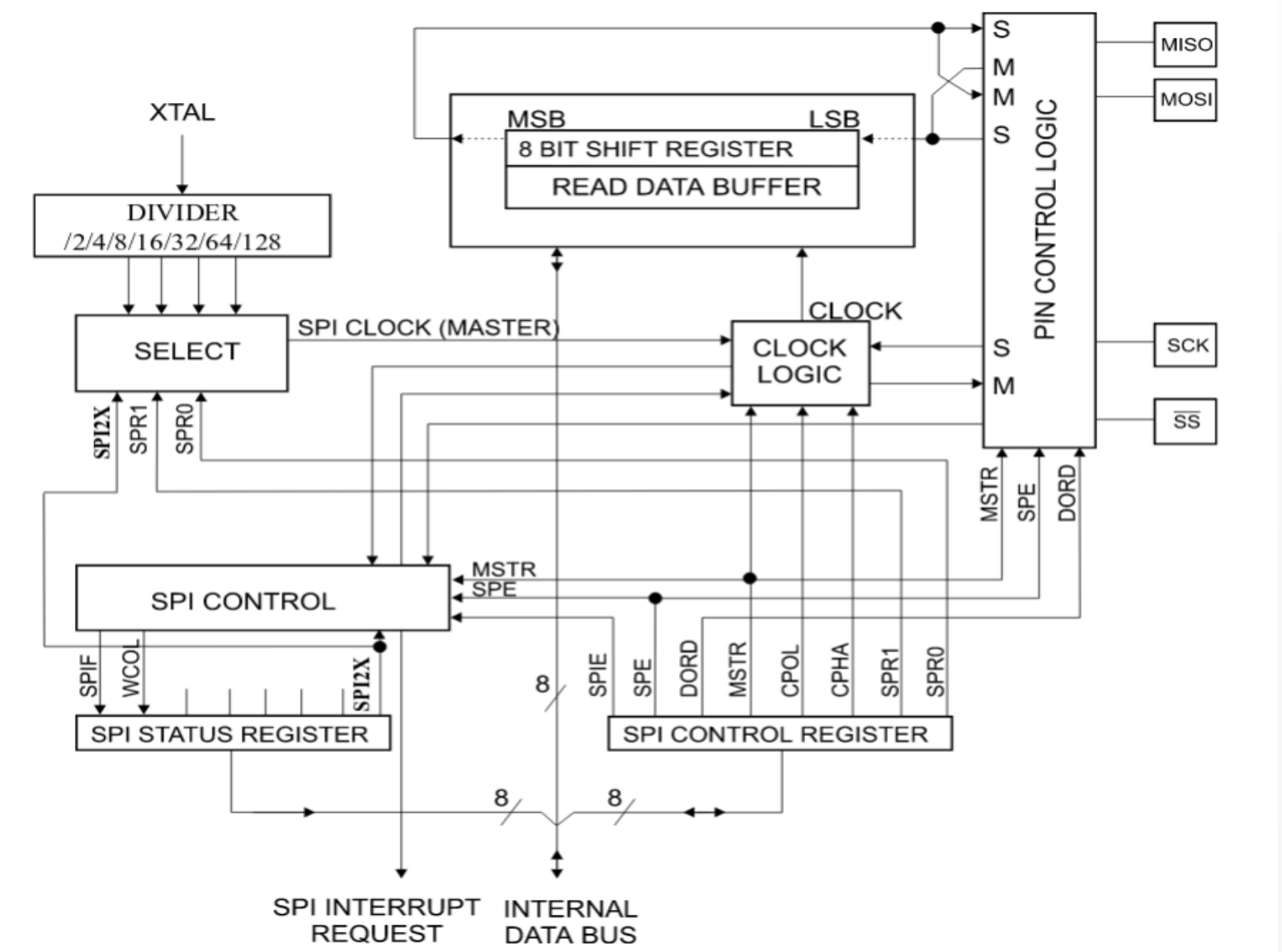


Conexiuni

- MSB de la master devine LSB la Slave (configurabil invers)
- Se formează practic un Rotate Left pe 16 biți
- Cu 8 biți pe Master și 8 biți pe Slave



Arhitectură



Programare

- Ce trebuie să controlăm
  - Modul Master/Slave
  - Viteza de transmisie/polaritate
  - Select Chip - se face manual cu GPIO (la fiecare tranzacție)
- Ce trebuie să aflăm
  - Dacă o transmisie s-a terminat

Registre

- SPCRn - registru de control
  - SPIE - 1 dacă întreruperea SPIF este activă
  - SPE - 1 dacă SPI este pornit
  - DORD - 1 dacă LSB se trimite primul
  - MSTR - 1 pentru master
  - CPOL:CPHA - Clock polarity & phase
  - SPR1:0 - SPI Data Rate
- SPSRn - registru de status
  - SPIF - 1 dacă un transfer e complet
  - SPI2X - dublează data rate-ul din SPR1:0
- SPDRn - buffer de date

Snippet

- Comunicație pe SPI full-duplex

```
uint8_t spi_xchg ( uint8_t date ) {
    SPDR = date ;
    loop_until_bit_is_set ( SPSR , SPIF ) ;
    return SPDR ;
}
```

- Citire pe SPI

```
uint8_t spi_read ( ) {
    return spi_xchg ( 0xFF ) ;
}
```

- Scriere pe SPI

```
Spi_write ( uint8_t in ) {
    spi_xchg ( in ) ;
}
```

Exemplu

- Full-duplex sau half-duplex?
  - Documentatie periferic (ex: Accelerometru ADXL362)

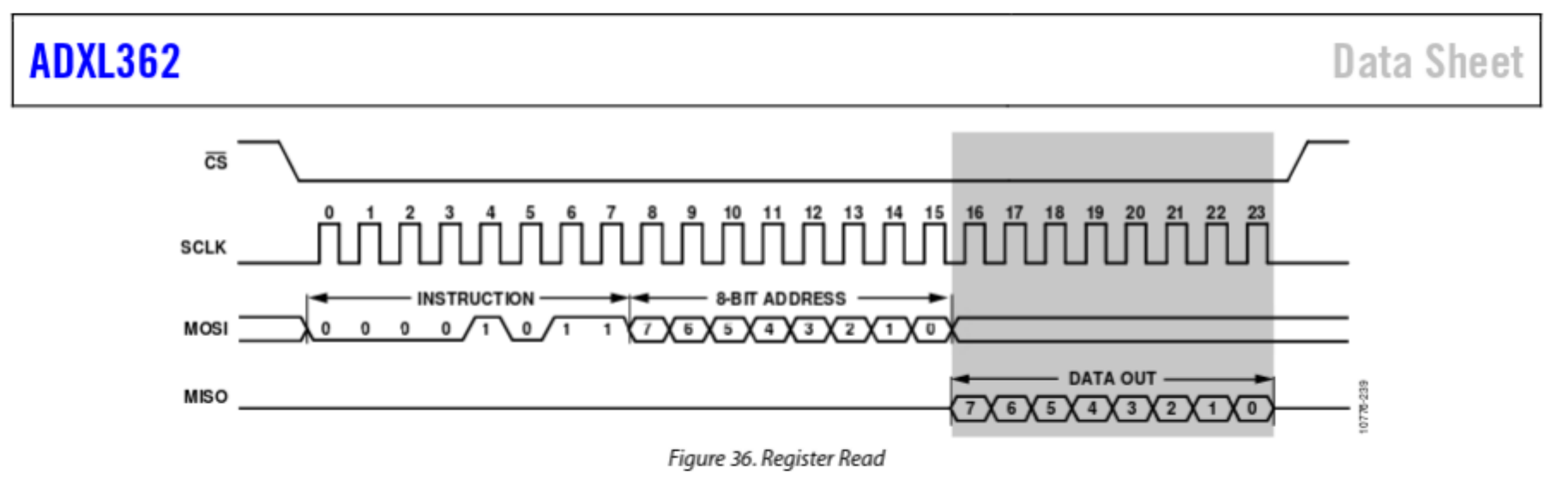


Figure 36. Register Read

  - Vrem sa citim de la adresa 0x08 din memoria perifericulului (XDATA)

```
spi_write ( 0x0B ) ; // comanda register read
uint8_t valoare = spi_xchg ( 0x08 ) ; // gresit !
```

  - Dispozitivul nu are cum să răspundă instant la adresă

```
spi_write ( 0x0B ) ; // comanda register read
spi_write ( 0x0B ) ; // adresa de citit
Uint8_t valoare = spi_read ( ) ; // corect
```

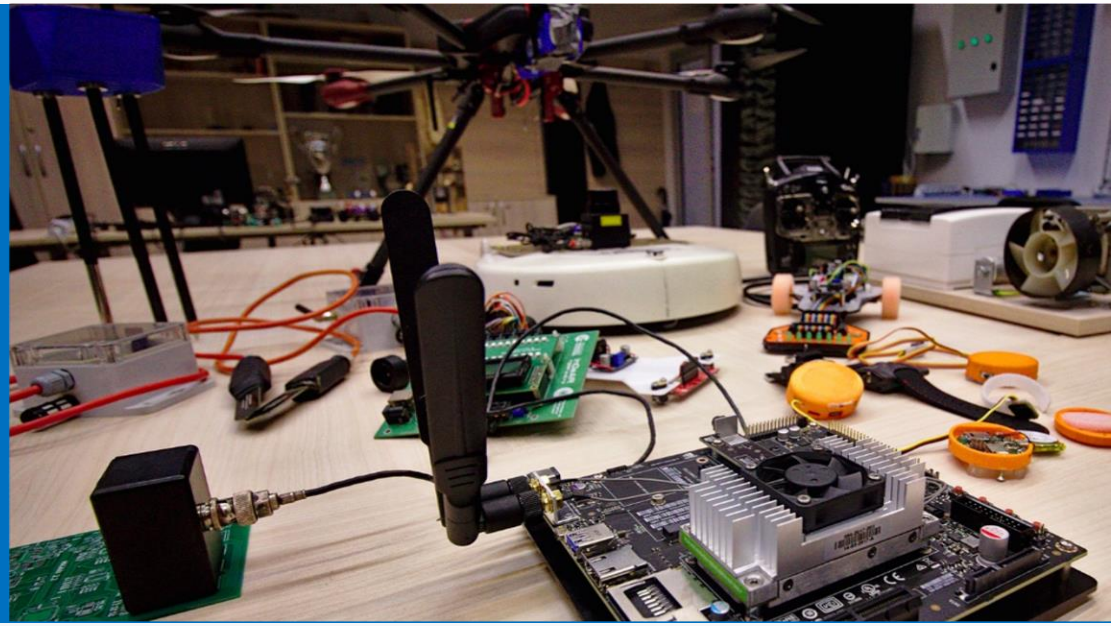
Utilizări

- Senzori (accelerometre, altimetre, giroscop etc.)
- Display-uri
- Memorii (Flash, carduri SD/MMC)
- Este extins pentru bandwidth
- Dual, Quad, Octo - SPI
- 2, 4, 8 liniile de date bidirecționale
- Inițial pe Single SPI

Concluzii

- Ceas explicit - fără erori
- Viteză de transmisie mare (f. mare pentru QSPI OSPI)
- Diferențe mici față de UART sincron
- Necesită multe linii pentru mai multe dispozitive (n+3)

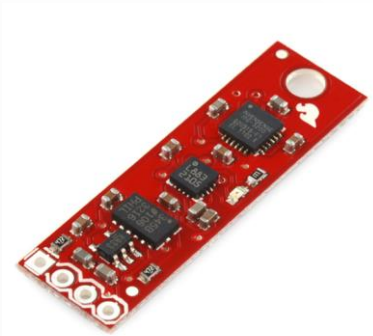
I2C



PM

I^2C

- Inter-Integrated Circuit, sau Two-Wire Interface (TWI)
  - Standard dezvoltat de Phillips, proprietar
- Comunicație între microcontrollere
- Comunicație cu senzori digitali
- Altele: memorii EEPROM seriale, ADC-uri, display-uri, ceasuri de timp real, date de monitorizare în PC-uri

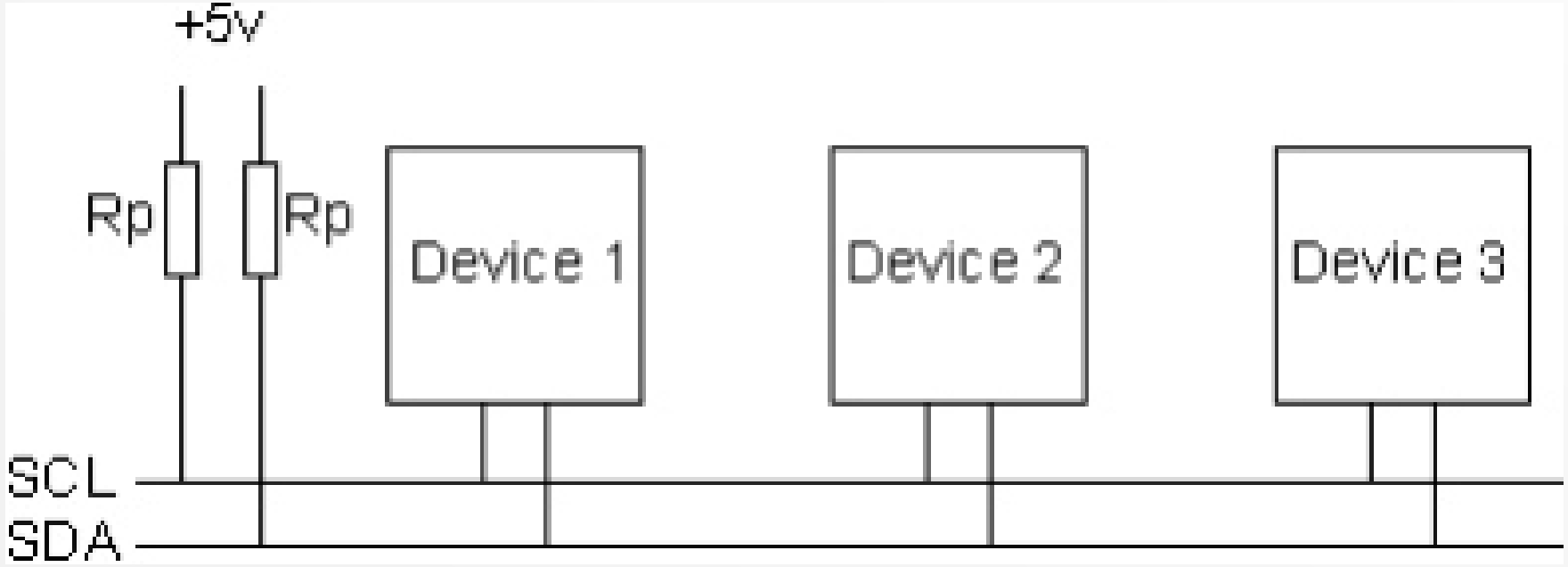






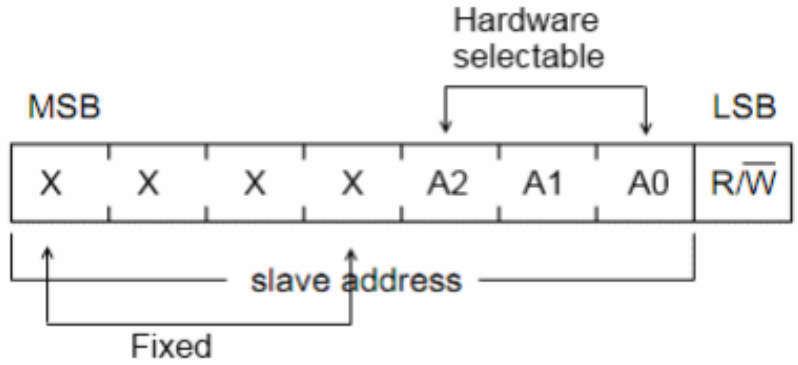
Caracteristici I^2C

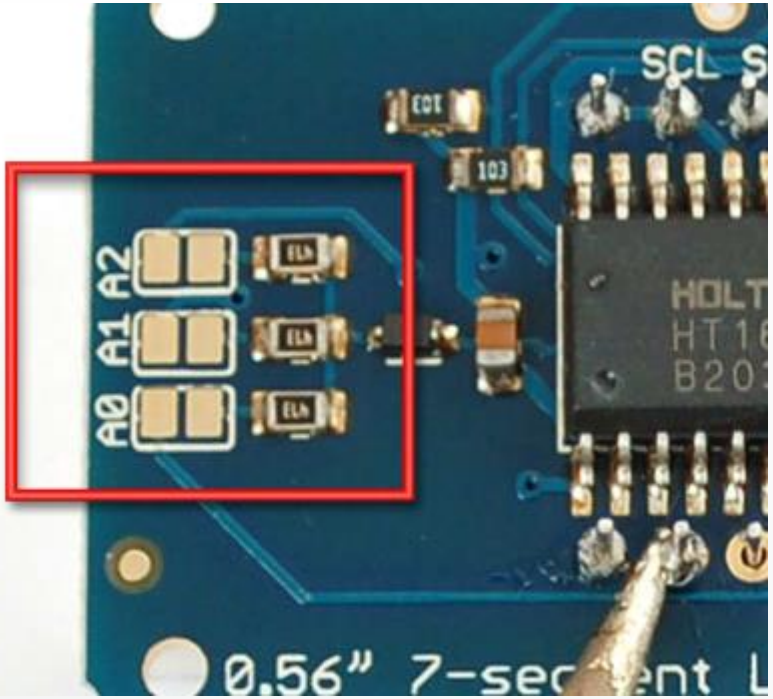
- Conexiuni open-collector → pull-up
- 1 - linie lăsată în aer, 0 - linie ”trasă” la masă
- Un singur master activ



Adresele în I^2C

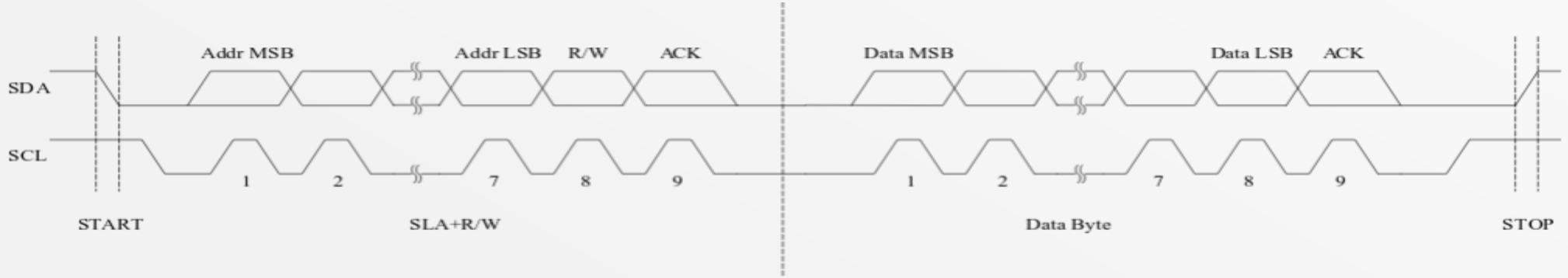
- Adrese pe 7 biți
  - Unii biți fixați
  - Alți biți configurabili hardware (imagine)
- Biții de configurare elimină conflicte între
  - Dispozitive de același fel
  - Dispozitive diferite cu aceeași adresă
- Ultimul bit (8) determină direcția comunicației



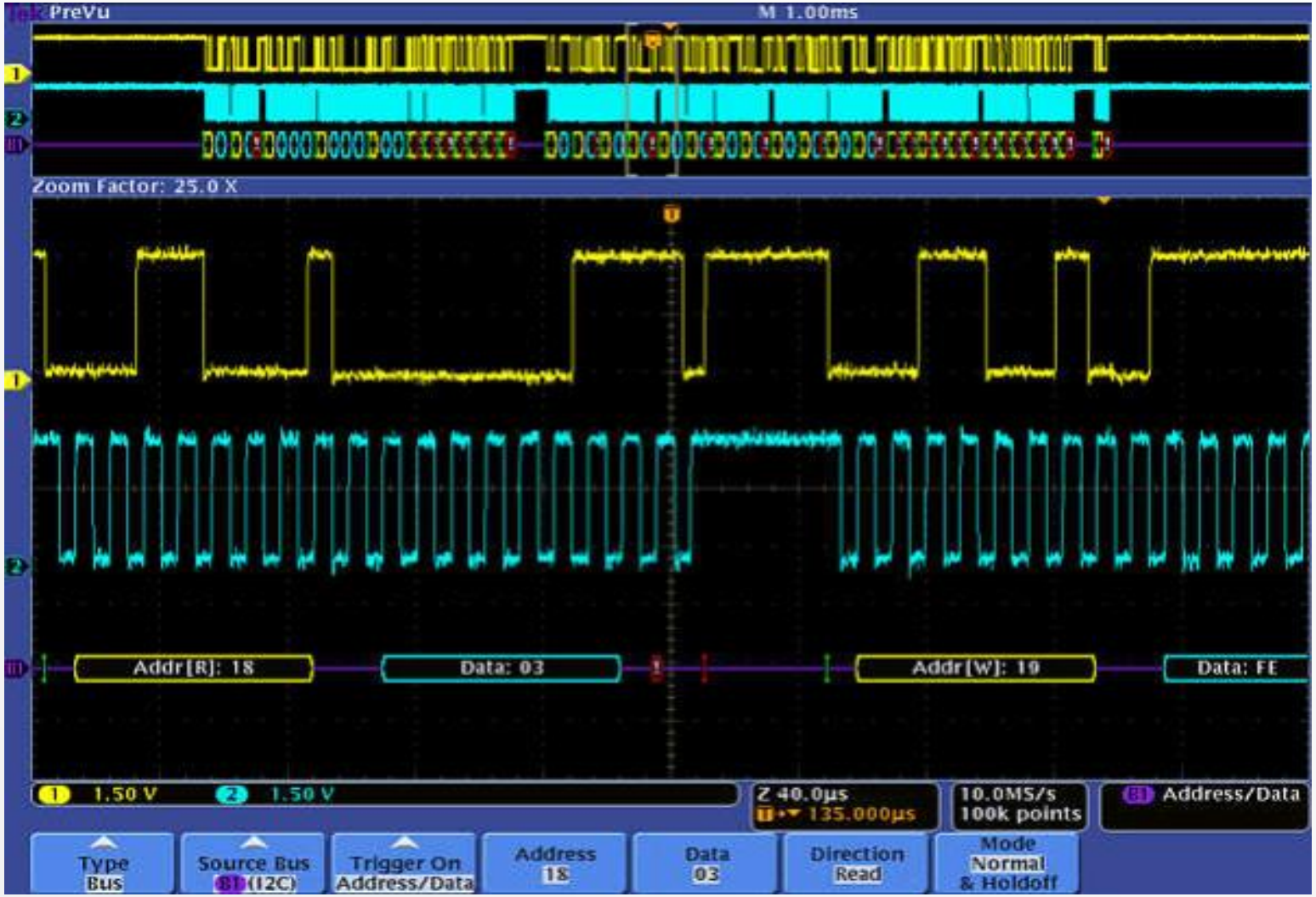


Protocolul de comunicație

- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1
- REPEATED START == START
- ACK
  - Receiver-ul comunicației după 8 biți de date
  - Receiver ține SDA pe 0 când și SCK este pe 0
- NACK
  - Dacă nu are nimeni adresa
  - Trimis înainte de oprirea comunicației
- Datele sunt valide pe frontul crescător al SCL



Exemplu semnal I^2C



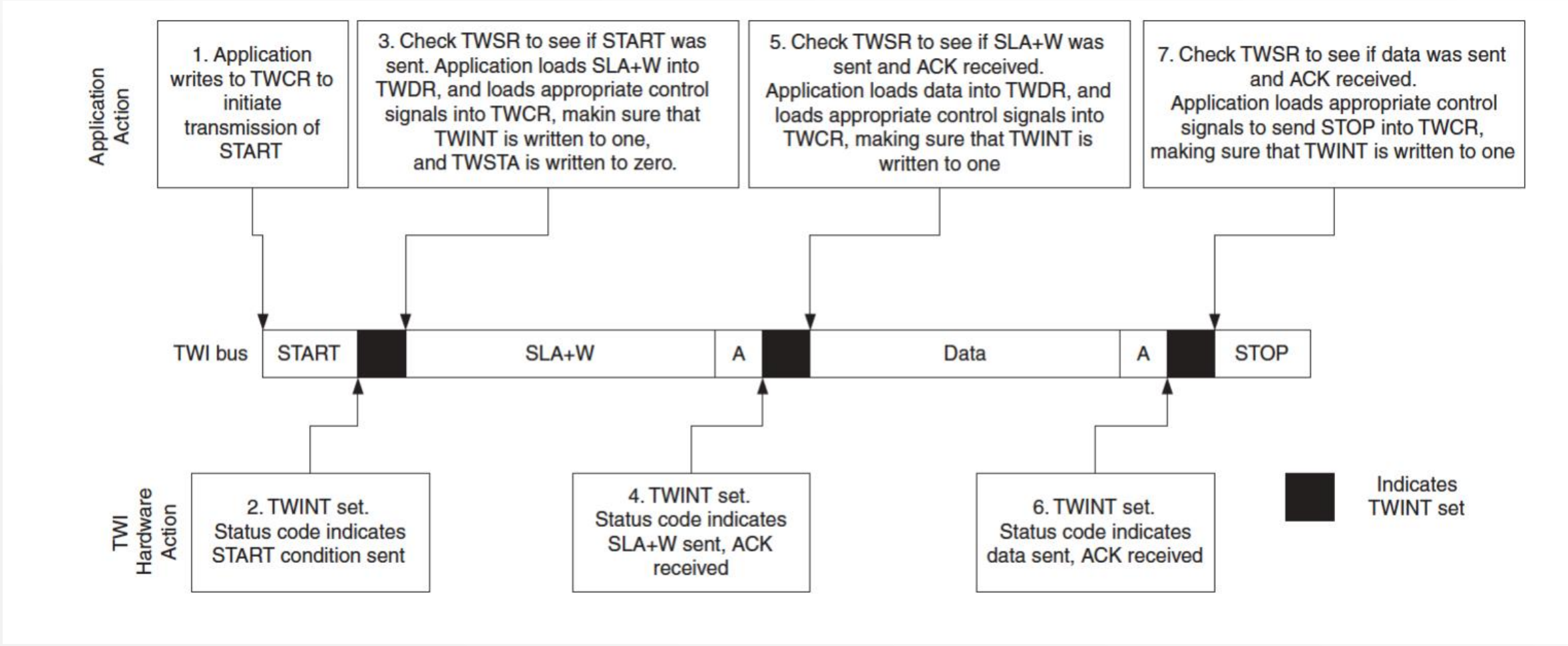
Arbitrarea în I^2C

- Liniile de I2C formează electric un ȘI logic
- Multi-master
  - Fiecare master monitorizează linia de clock (SCL)
  - În puține cazuri vor porni chiar în același timp
  - În momentul în care diferă comunicația ”câștigă” cel cu 0 (își vede propriul output)

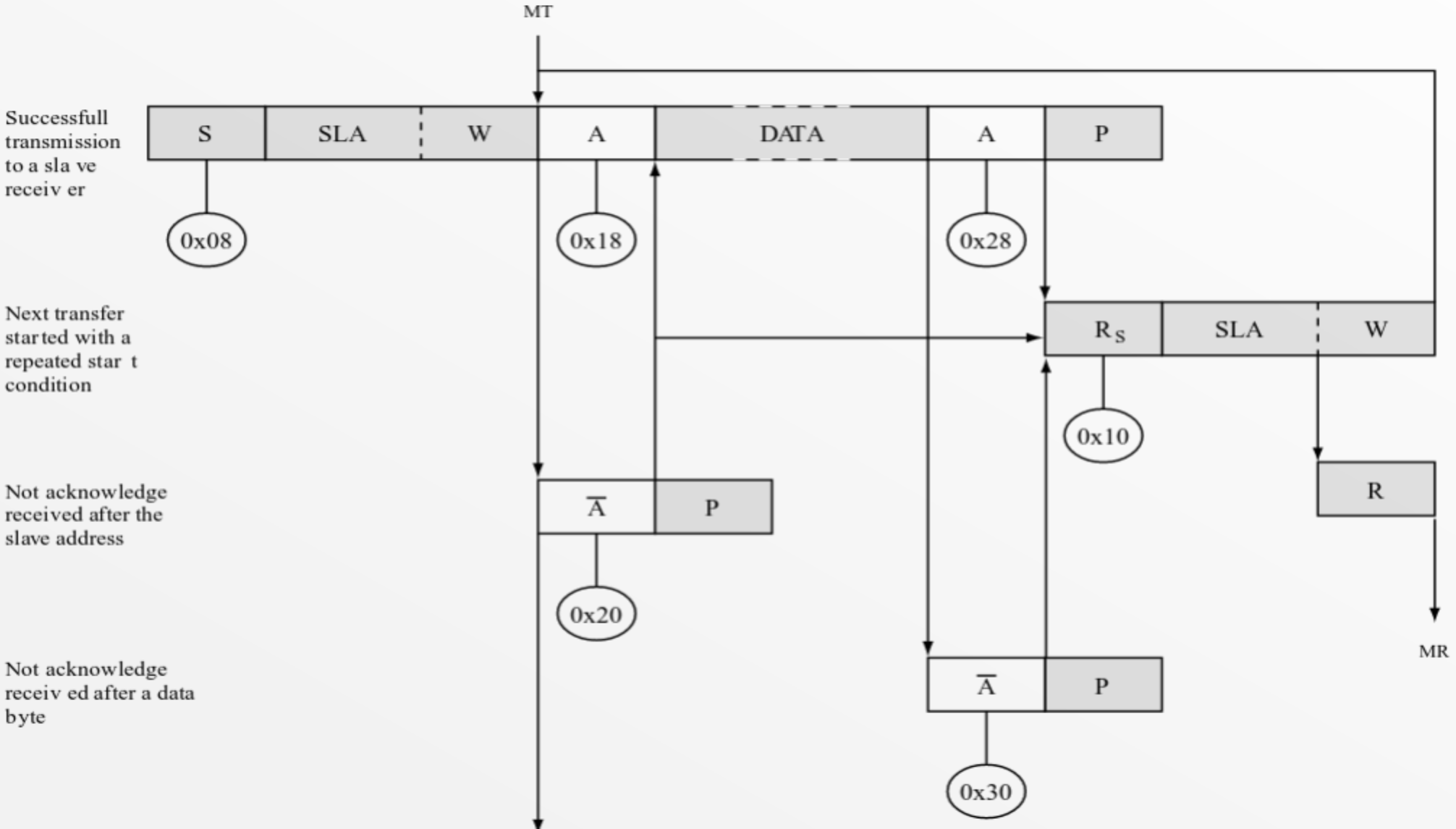
Interfața I^2C pe ATMega

- Registre:
  - TWCR - flag-urile de control
  - TWSR - cu biți de status
  - TWDR - bufferul de date
  - TWAR - adresa dispozitivului, dacă este slave
  - TWBR - viteza de transmisie
- Întrerupere atunci când perifericul a terminat operația curentă și așteaptă următoarea comandă

Exemplu flow transmisie



Exemplu flowchart



Exemplu cod I^2C

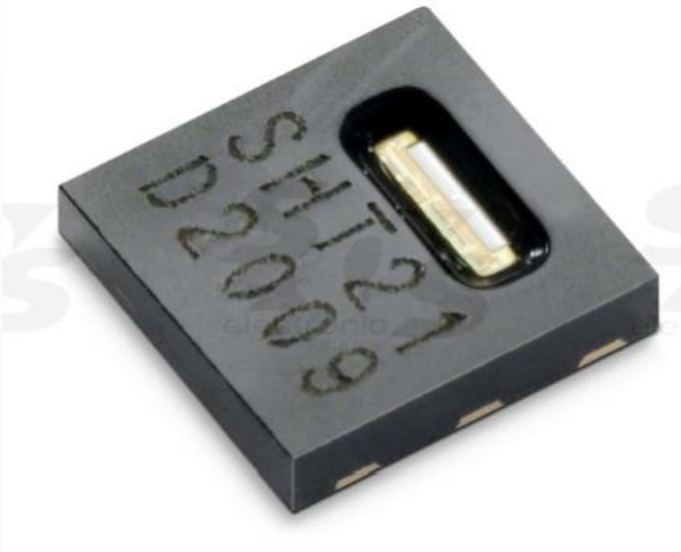
```
void i2c_writeb(uint8_t address, uint8_t value) {
  uint8_t state = STATE_START, next_state;

  TWCR0 = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);

  while(1) {
    loop_until_bit_is_set(TWCR0, TWINT);
    switch(state) {
      case STATE_START:
        if ((TWSR0 & 0xf8) != STATUS_START_OK) // 0x08
          return ERROR_NO_START;
        TWDR0 = (address << 1) + SLA_W; // 0
        next_state = STATE_SLA;
        break;
      case STATE_SLA:
        if ((TWSR0 & 0xf8) != STATUS_SLA_ACK) // 0x18
          return ERROR_NO_SLA_ACK;
        TWDR0 = value;
        next_state = STATE_DATA;
        break;
      case STATE_DATA:
        if ((TWSR0 & 0xf8) != STATUS_DATA_ACK) // 0x18
          return ERROR_NO_DATA_ACK;
        next_state = STATE_STOP;
        TWCR0 = (1 << TWINT) | (1 << TWEN) | (1 << TWSTO);
        return OK;
    }

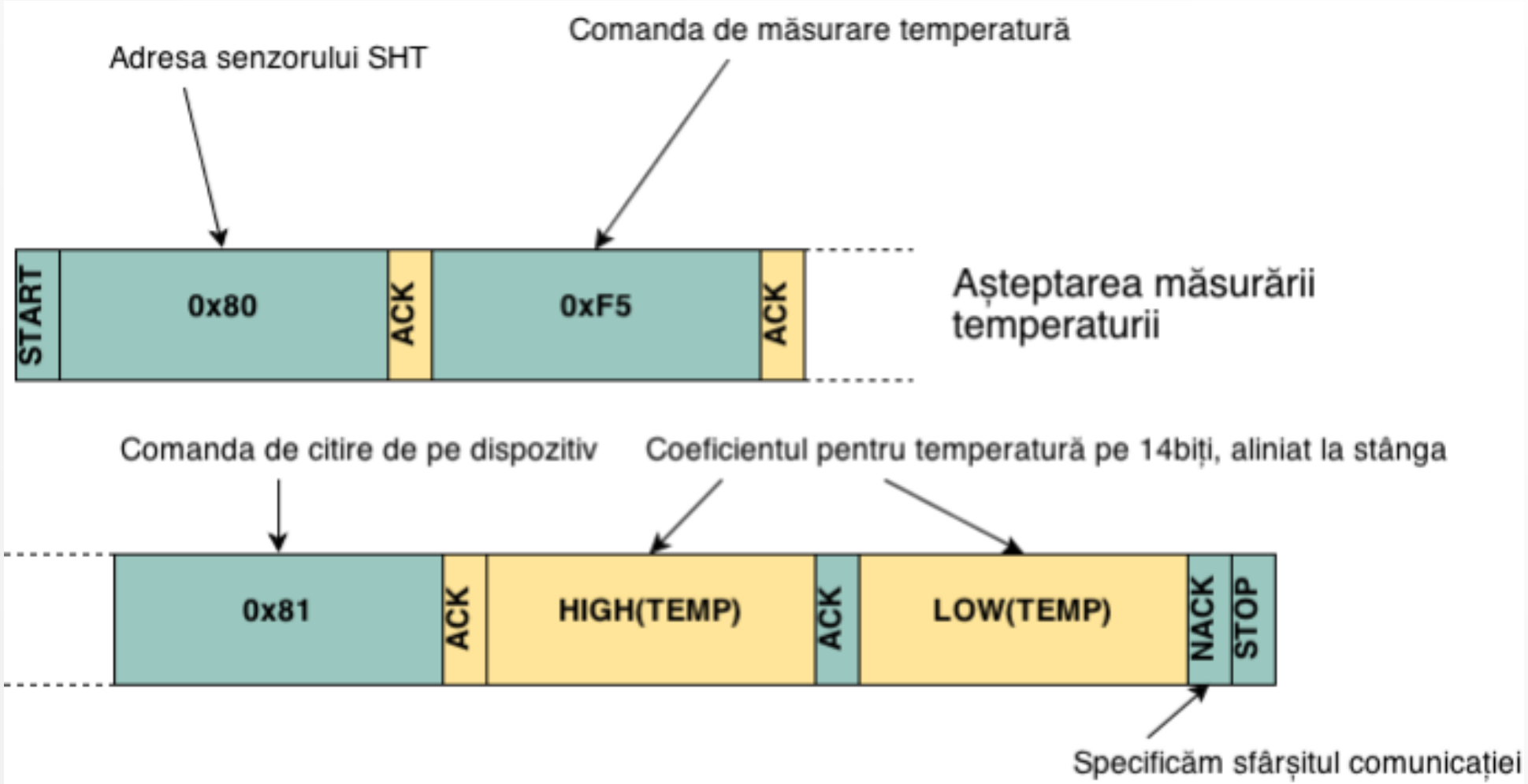
    TWCR0 = (1 << TWINT) | (1 << TWEN);
    state = next_state;
  }
}
```

Exemplu comunicație I^2C



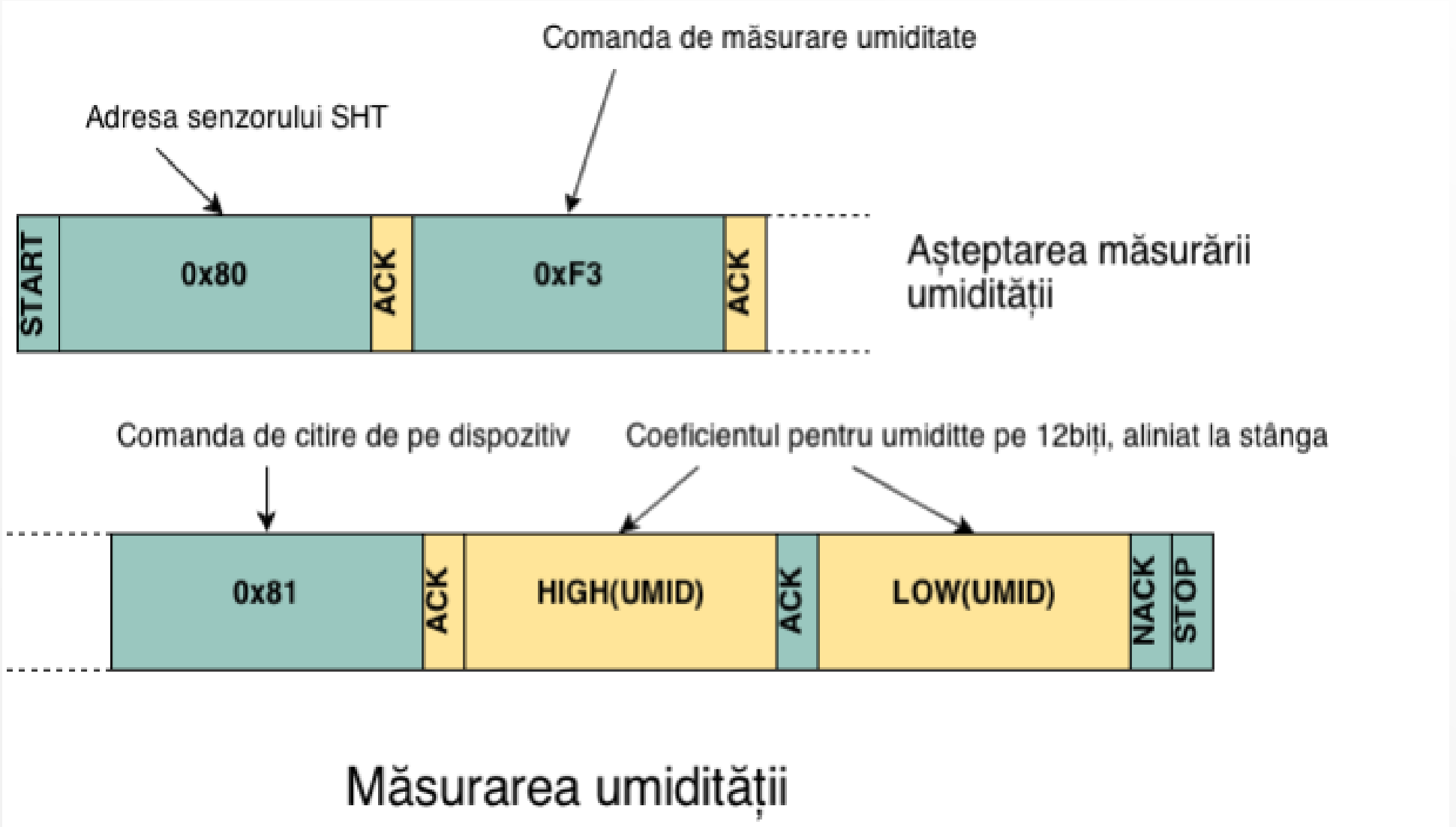
- SHT21
  - Senzor auto-calibrat de temperatură și umiditate
  - Adresa I2C pe 7 biți este 0x40
  - Măsurarea temperaturii durează cam 100ms (așteptare sau clock stretching)
    - Așteptarea presupune ca master-ul să știe cât este delay-ul între pornirea măsurătorii și terminarea ei, pentru a cere rezultatul după terminare
    - Clock stretching este metoda non-standard prin care slave-ul prelungește un bit 0 de pe SCK pentru a ține ocupat masterul și magistrala

Exemplu comunicație I^2C



Măsurarea temperaturii

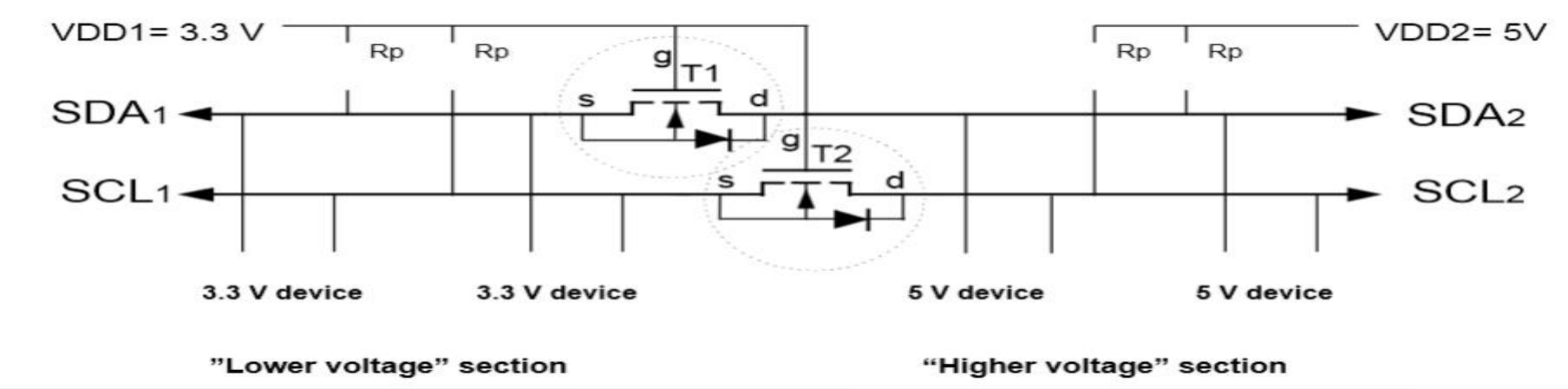
Exemplu comunicație I^2C



Măsurarea umidității

Ajustare nivele de tensiune pe I^2C

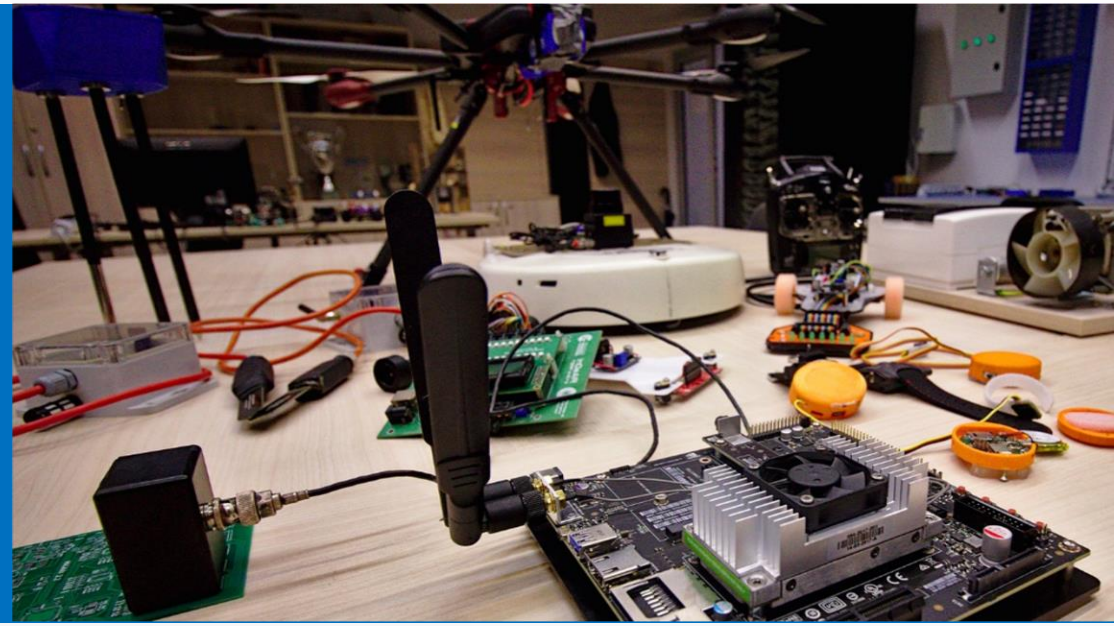
- Dacă avem dispozitive cu nivele logice diferite?
- Adaptarea nu se poate face cu divizor rezistiv
- Schema de mai sus garantează adaptarea nivelelor logice
  - În repaus este 1 logic pe ambele părți, MOSFET-ul închis
  - În 0 logic pe tensiunea mică va apărea un v_{gs} > prag, deci MOSFET-ul va fi deschis → 0 logic pe tensiunea mare
  - Cu 0 logic pe tensiunea mare va fi leakage current pe dioda de protecție → v_{gs} > prag



Concluzii I^2C

- Serială cu puține fire (2)
- Adresarea se face prin firul de date
- Suport multi-master
- Viteze relativ mici (400kHz vs. 8MHz la SPI)
- Necesită software mai complex

USB



PM

Universal Serial Bus

- Standard definit în anii ’90
- Unifică interfețe și alimentări
- USB1.0 1996
- USB2.0 2000
  - Viteze de transfer îmbunătățite (480Mbps)
  - Conectori mini
  - OTG în 2006
- USB3.0 2011
  - 5Gbps
  - Comunicație full-duplex
- USB3.1 2014
  - 10Gbps
  - Conector unic, simetric(!)
- USB4 2019
  - 40Gbps
  - Bazat pe protocolul Thunderbolt 3

Avantaje utilizator

- Număr redus de conectori (A/B/microAB/miniAB/C)
- Fără cablu de alimentare
- Număr de porturi extensibil
- Rate de transfer bune
- Hot plugging
- Fără configurări manuale

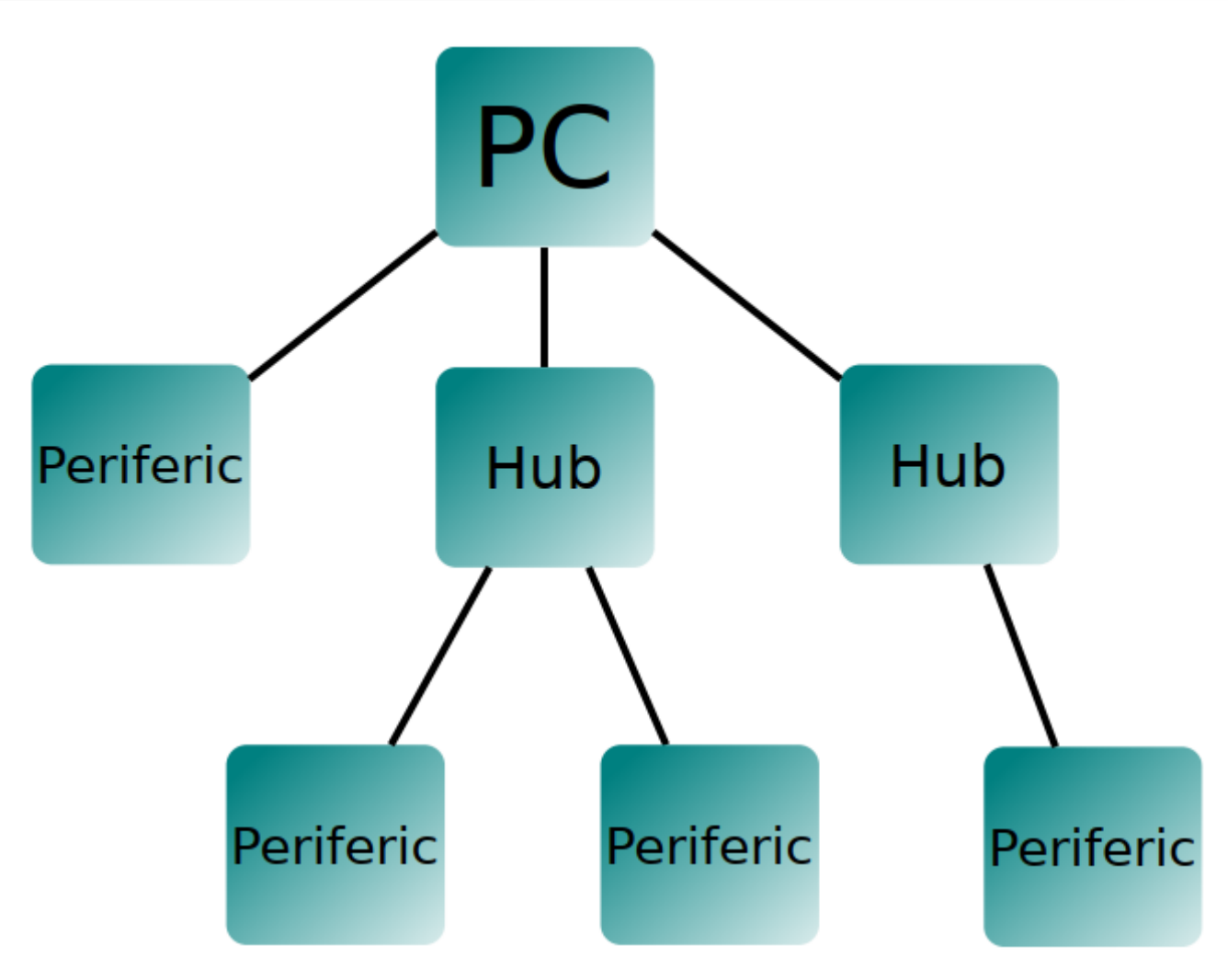
Avantaje dezvoltator

- Avantaje
  - Fără alimentare separată
  - Interfațare unică pentru dispozitive diferite
    - Se pot implementa chip-uri dedicate
  - Suport software în SO
  - Documentație din partea USB-IF

- Dezavantaje
  - Complexitate mărită (de ex. față de RS232)

Arhitectură

- Arbore:
  - Perifericele - frunze
  - Host, hub-uri - noduri
- Comunicație:
  - Half-duplex
  - Condiționată de host
  - Numai între host și un dispozitiv



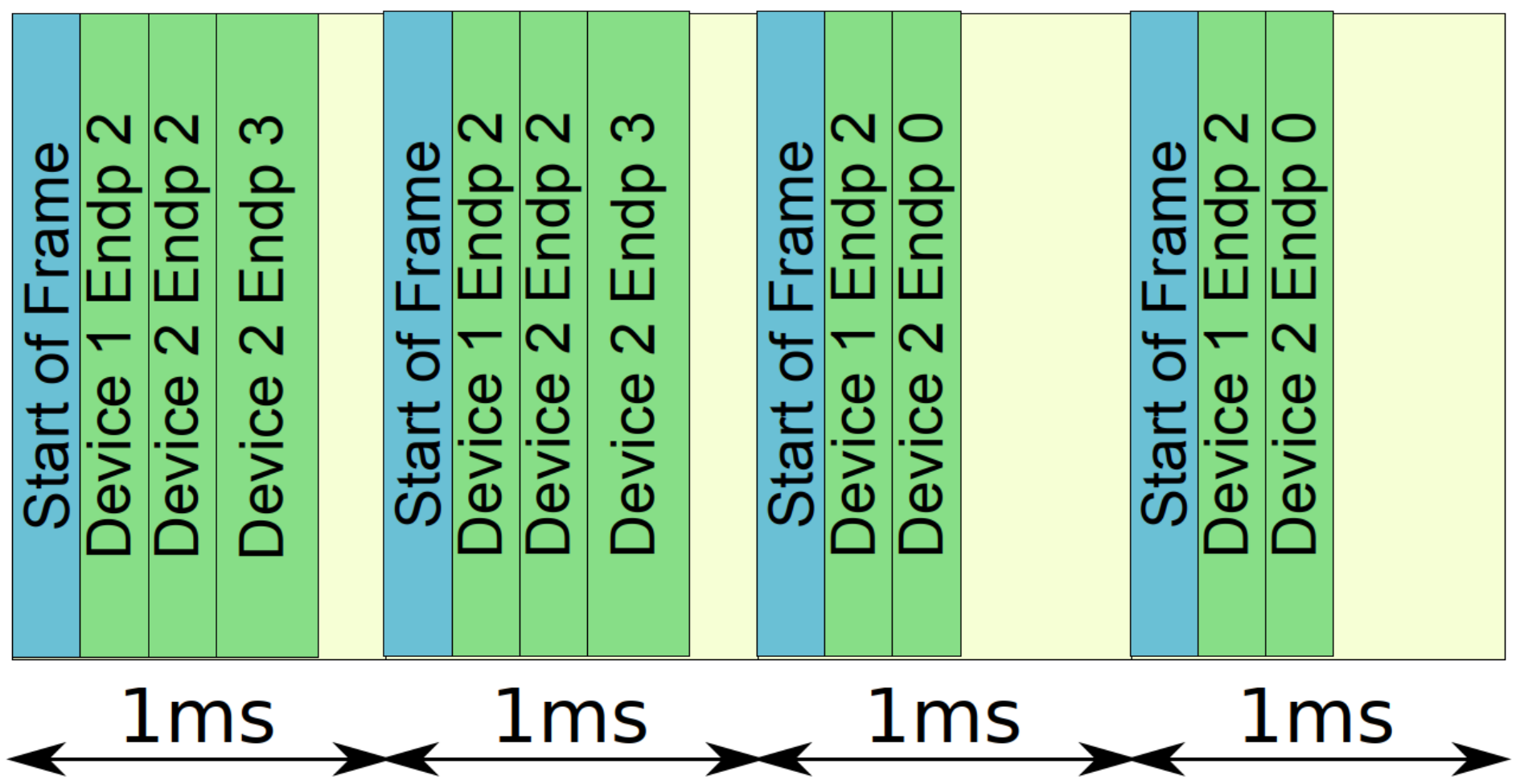
USB Host

- Cunoaște toate dispozitivele
  - Capabilitățile
  - nevoile de transfer de date
  - Cerințele de consum
- Administrează comunicația
  - Alocă lățime de bandă pentru anumite dispozitive
  - Anunță comunicația dispozitiv → host
  - Interoghează dispozitive

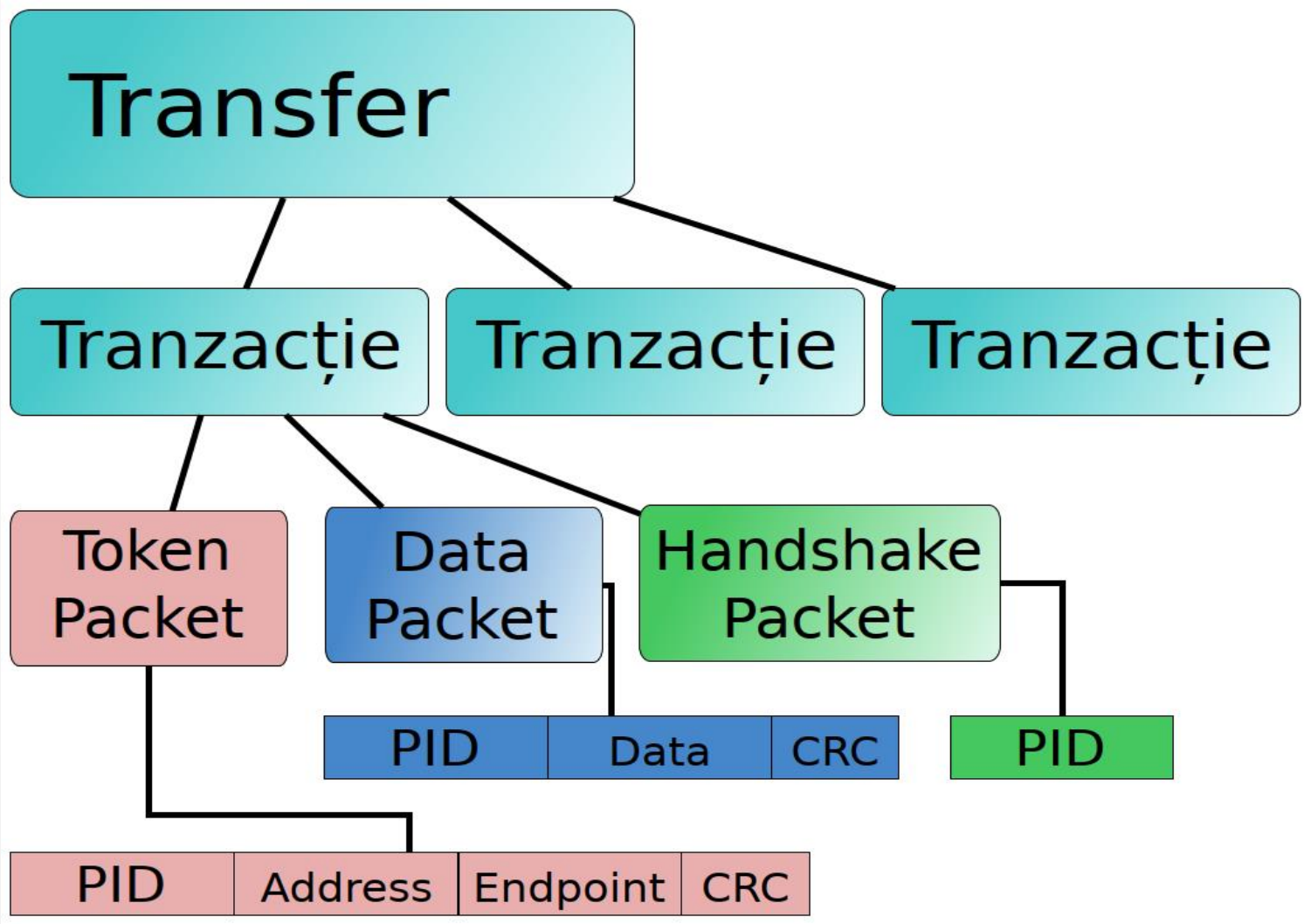
USB device

- Responsabil să se identifice host-ului
- Canale de comunicare (endpoints ← ports)
- Recunoaște comunicația adresată lui
- Răspunde la cererile host-ului
- Menține consumul în limitele declarate
  - Reduce consumul când bus-ul este în SLEEP
- Comunică cu host-ul atunci când acesta cere

Un frame USB


Start of Frame
Device 1 Endp 2
Device 2 Endp 2
Device 2 Endp 3
Start of Frame
Device 1 Endp 2
Device 2 Endp 2
Device 2 Endp 3
Start of Frame
Device 1 Endp 2
Device 2 Endp 0
Start of Frame
Device 1 Endp 2
Device 2 Endp 0
1ms
1ms
1ms
1ms

Un transfer USB



Tipuri de transferuri

Tip	**Control**	**Interrupt**	**Isochronous**	**Bulk**
Folosire	Identificare Configurare	Mouse Keyboard	Webcam	Storage
Obligatoriu	Da	Nu	Nu	Nu
Low-speed	Posibil	Posibil	Nu	Nu
Transfer	Medie	Mică	Medie	Mare
Trafic	Min 20%	-	Max 80%	-

Endpoints

- Analog porturilor de la socket
- Endpoint-ul este unidirecțional - poate fi IN sau OUT
- Poate avea un singur tip de transfer → tip endpoint
- Endpoint 0 - de control
- Fiecare funcție a dispozitivului - endpoints
- /sys/bus/usb/devices conține toate informațiile
- $ cat /sys/bus/usb/devices/usb1/ep_00/type

Enumerare

- Procedură de înregistrare a unui device la host
- Dispozitivul este atașat fizic
- Hub-ul
  - detectează dispozitivul
  - informează host-ul
  - detectează viteza cu care se poate comunica
  - resetează device-ul
  - Host-ul află dacă device-ul este high-speed
  - conectează dispozitivul la bus
- Host-ul
  - alocă o adresă
  - cere descriptorii device-ului
  - selectează un driver potrivit
- Driver-ul selectează o configurație din cele disponibile
- Încercați să urmăriți acasă cu Wireshark!

Descriptori

- Descriptorii sunt structuri care prezintă capabilitățile și configurările unui dispozitiv către host
- Device descriptor
  - Vendor ID, Product ID, link către string descriptors cu numele
  - Nivelul de compatibilitate USB
  - Numărul de configurații posibile
- Configuration Descriptor
  - Descrie un set de funcționalități care pot fi active simultan
  - Configurația curentă este selectată de Host bazându-se pe compatibilitate
- Interface Descriptor
- Endpoint Descriptor
- String Descriptor

Exemple descriptori

Type	Field	Value
uint8_t	bLength	18
uint8_t	bDescriptorType	0x01 (Device descriptor)
uint16_t	bscUSB	0x0200
uint8_t	bDeviceClass	0x00 (Composite)
uint8_t	bDeviceSubclass	0x00 (No specific subclass)
uint8_t	bDeviceProtocol	0x00 (No specific protocol)
uint8_t	bMaxPacketSize	64
uint8_t	idVendor	0x03eb (VID of Atmel)
uint8_t	idProduct	0x2018
uint16_t	bcdDevice	0x1000
uint8_t	iManufacturer	0 (no manufacturer string)
uint8_t	...	...

Exemple de clase

- Communication Device Class
  - Abstract Control Model
    - /dev/ttyACM0
- Mass Storage - stick-uri de memorie
- Monitor
- Video class - webcams
- Audio class - microfoane
- Printer

Interfața USB în ATMega

- ATMega32U4 este printre puținele chip-uri de AVR care au periferic USB, niciunul dintre controlerele care se găsesc în pachet DIP nu au
- Challenges:
  - Funcționarea full-speed necesită un ceas de 48MHz care este format din frecvența inferioară pe care o are controller-ul
  - Transmisia la viteze superioare necesită mult buffering → memorie dual-port
  - Endpoint-uri limitate ca memorie și număr, deoarece orice endpoint necesită un buffer
    - 1 control endpoint de 64 bytes
    - 1 endpoint de 256 bytes
    - 5 endpoint-uri de 64 bytes

Interfața USB în ATMega

- Dacă este prezent perifericul de USB
  - Management alimentare
  - Management conectare la bus (+enumerare)
  - Power management periferic
  - Management endpoint-uri
  - Management-ul memoriei alocate endpoint-urilor
- 7 + 8 posibile întreruperi * 7 endpoints = 63
- 20 de registre I/O pentru control/status
- Biblioteci:
  - LUFA - http://www.fourwalledcubicle.com/files/LUFA/Doc/120219/html/index.html
  - Atmel - http://atmel.com/dyn/products/app_notes.asp?family_id=607#USB

Exemplu cod LUFA

```
/* Exemplu HID - emulator mouse/tastatura */

// (...) Structuri configurare

void main()
{
  SetupHardware();
  GlobalInterruptEnable();

  for (;;)
  {
    HID_Device_USBTask(&Generic_HID_Interface);
    USB_USBTask();
  }
}

// (...) SetupHardware (apeleaza si USB_Init)

// (...) Event callbacks (e.g. EVENT_USB_Device_Disconnect

CALLBACK_HID_Device_CreateReport(USB_ClassInfo_HID_Device_t* const HIDInterfaceInfo,
                                 uint8_t* const ReportID,
                                 const uint8_t ReportType,
                                 void* ReportData,
                                 uint16_t* const ReportSize)
{
  /* TODO date puse in ReportData si ReportSize */
}

// (...) Callback host->device CALLBACK_HID_Device_ProcessHIDReport
```

Interfață pe ATMega324a

- Pe ATMega324(p)a putem folosi USB software:
  - V-USB - https://www.obdev.at/products/vusb/index.html
  - Implementează funcționalitate de low-speed device
    - USB HID - mouse/tastatură/joystick
    - USB ACM - serială
    - Clase custom
  - Folosește PD2 și PD3 (external interrupts)

Exemplu cod V-USB

```
/* Exemplu HID - emulator mouse/tastatura */

// (...) Structuri configurare ex: descriptorii

void main()
{
  (...) // initializare

  for(;;)
  {
    usbPoll();

    if(usbInterruptIsReady())
    {
      /* TODO pus date in reportBuffer */

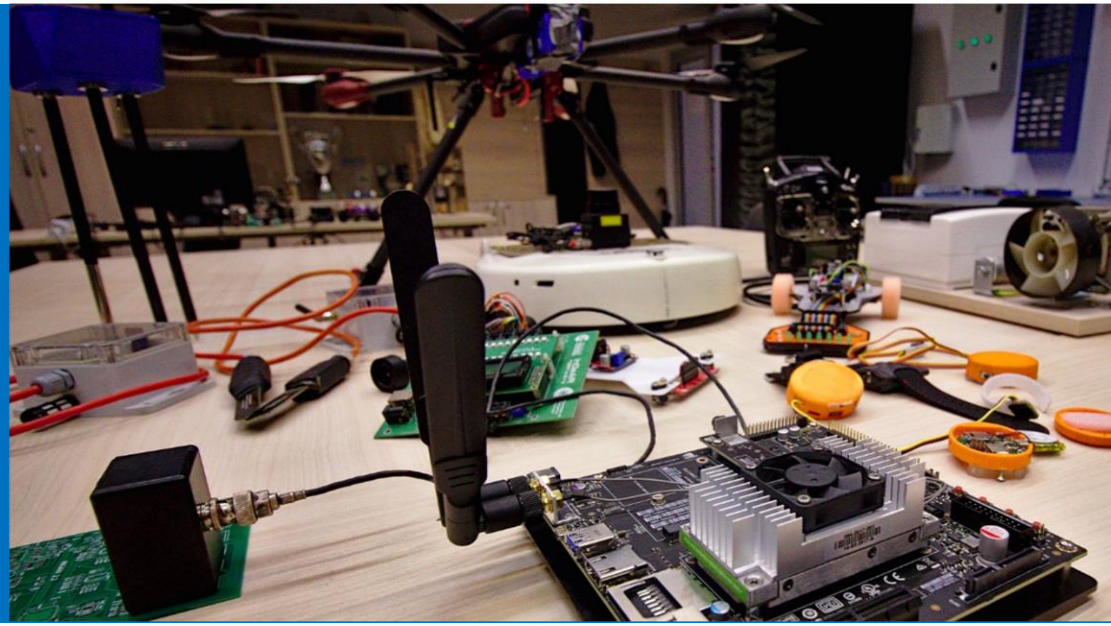
      usbSetInterrupt((void *) &reportBuffer, sizeof(reportBuffer));
    }
  }
}
```

Exemplu disponibil pe https://github.com/obdev/v-usb/blob/master/examples/hid-mouse/firmware/main.c

Concluzii USB

- Serială cu puține fire
- Hot-plugging
- Gestionare dificilă pe un control cu putere de procesare limitată
- Nu există periferic USB pe ATMega328P
- Folosim biblioteci pur software sau peste periferic

Alte protocoale de comunicație



PM

OneWire

- 1-Wire, proiectat de Dallas Semiconductor Corp.
  - Viteză mică (16,3 kbit/s[1])
  - Doua fire: date și masă
  - Date, semnalizare și putere pe un singur fir.
- Similar cu I^2C
- Permite construirea unei retele MicroLAN cu un master
- Extrem de versatil

SMBus

- SMBus (System Management Bus)
  - Încărcarea bateriei, temperaturi interne, ventilatoare
- I^3C (Improved I^2C)
  - Standard apărut 2017
  - Mai eficient energetic
  - Viteze de transfer mai mari decât I^2C
  - Feature-uri avansate

LIN

- LIN (Local Interconnect Network)
  - Protocol pe un singur fir
  - Viteză de 9,2 Kbit/s la o lungime de magistrală de 40 de metri
  - 16 noduri: un master, 15 slave
- Consorțiul LIN a fost fondat de producătorii de automobile
- Prima implementare in 2002
- Adresare pe bază de Node ADdress (NAD)
  - Valori pe 7 biți, deci în intervalul 1 la 127 (0x7F)
  - Compoziție a ID-ului furnizorului, ID-ului funcției și ID-ului variantei
  - CAN in Automation, autoritatea pentru atribuirea identificatorilor

CAN

- CAN (Controller Area Network)
- Comunicație bidirecțională cu magistrală serială cu două fire
- Dezvoltat inițial în 1989 de Bosch pentru industria auto
- Obiectiv principal de proiectare: Soluție economică pentru implementarea rețelelor de înaltă integritate în aplicații de control în timp real
- Acum standardizat la nivel internațional:
  - CAN 2.0A: ISO11519 — viteză mică
  - CAN 2.0B: ISO11898 — viteză mare
- Utilizare: >>100,000,000 nodes/an
  - Pest 80% în automotive
  - Multe oportunități actuale și potențiale de aplicații non-auto

I^2S

- I²S (Inter-IC Sound)
- Conectare dispozitive audio digitale
  - Date în format PCM
  - Folosește 3 fire:
    - continuous serial clock (SCK)
    - word select (WS)
    - serial data (SD)
- Nu are legătura cu I^2C
- Introdus în 1986 de Philips Semiconductor (acum NXP Semiconductors)

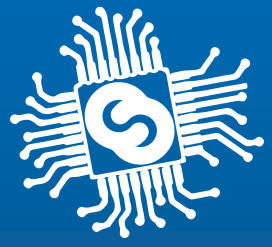
Concluzii

Name	Multidrop	Max nodes	Electrical type	Cable type	Max bitrate [kbit/s]	Length at max bitrate	Max length [m]	Bitrate at max length
EIA-485 (UART based)	Y	256	EIA-485	Twisted pair	1000 kbit/s		1330 m	64 kbit/s
SCSI-1/2	Y	8	Open collector	Ribbon cable	20 000 kbit/s	3 m	6 m	5 Mbit/s/ch
SCSI Ultra2	Y	16	EIA-485	Twisted pair	40 000 kbit/s	12 m		
LIN[1]	Y	16	Open collector	open collector with pull-up to 12V car supply	19.2 kbit/s	40 m		
SIOX	Y	62			19.2 kbit/s			
I²C	Y	127 or 1023	Open collector		5000 kbit/s		7.6 m	
SMBus	Y	128	Open collector		100 kbit/s			
PMBus	Y	128	Open collector		400 kbit/s			
10Base-2	Y	30	Differential	RG58 coaxial	10 000 kbit/s	185 m	185 m	
CAN	Y	128	ISO 11898-2 Differential	Twisted pair	1000 kbit/s	40 m	1000 m[2]	50 kbit/s
DMX512-A	N	32	EIA-485	Twisted pair 120 Ω[3]	250 kbit/s	548 m	548 m (1800ft.)[4]	250 kbit/s
DCC	Y	127	Single ended		5 kbit/s			
RS-232	N	2	Single ended		128 kbit/s	1.5 m (approx)	15 m	19.6 kbit/s
PSI5	Y			Twisted pair	189 kbit/s		12 m (approx)	
1-Wire	Y	2^{48}	Open collector	Single conductor, with ground	16.3 kbit/s		300 m	
X10	Y	256		Power line	0.05 kbit/s / 0.06 kbit/s			
FlexRay				Twisted pair	20 000 kbit/s			
SENT[5]	N	2	Single ended	Single conductor, with 5 V power supply and ground	async. transmission, min. 39.216 kbits/s	5 m		
SPC[6]	Y	5	Single ended or Open collector	Single conductor, with 5 V power supply and ground, shielded when using higher speeds	sync. and async. transmission, variable up to ~150 kbit/s	~0.5 m	5 m	same as SENT
I3C	Y	127						
SPI	Y				variable up to ~10000 kbit/s (depends strongly on physical setup)			

La cursul următor

- Consum de energie
- Setari de lucru
- Mod idle / power save
- Optimizari
- Regulatoare de tensiune
- Exemple implementari punte H
- Discutii despre proiect

Cursul #8&9



PM

Disclaimer

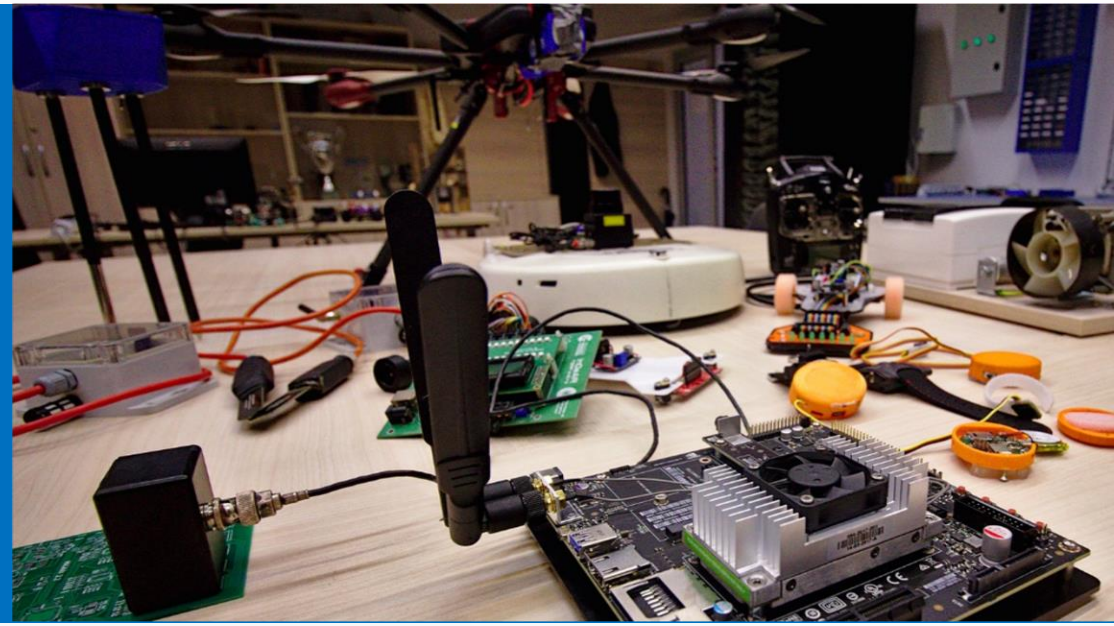
- *Slide-urile acestea reprezintă un rezumat.*
- *Slide-urile nu acoperă toate explicațiile, simulările, demonstrațiile oferite în timpul cursului.*
- *Slide-urile nu limitează în vreun fel materia cerută la examen.*
- *Pentru varianta completă, sunteți așteptați la curs.*

© Elemente din curs preluate din:
- *notițe curs PM 2021, Dan Tudose*
- *notițe curs PM 2021, Andrei Voinescu*
- *scoala de vara EAP 2016, Marius Nicolae*
- *notite curs EEA/ED, Razvan Tataroiu & Daniel Rosner*

Cuprins

- Stabilizatoare de tensiune
- LDO – Low Dropout
- Surse in comutație
- Alimentare AC
- Consum de energie
- Setari de lucru
- Mod idle / power save
- Optimizari
- Exemple implementari punte H
- Discutii despre proiect

Stabilizatoare de tensiune



PM

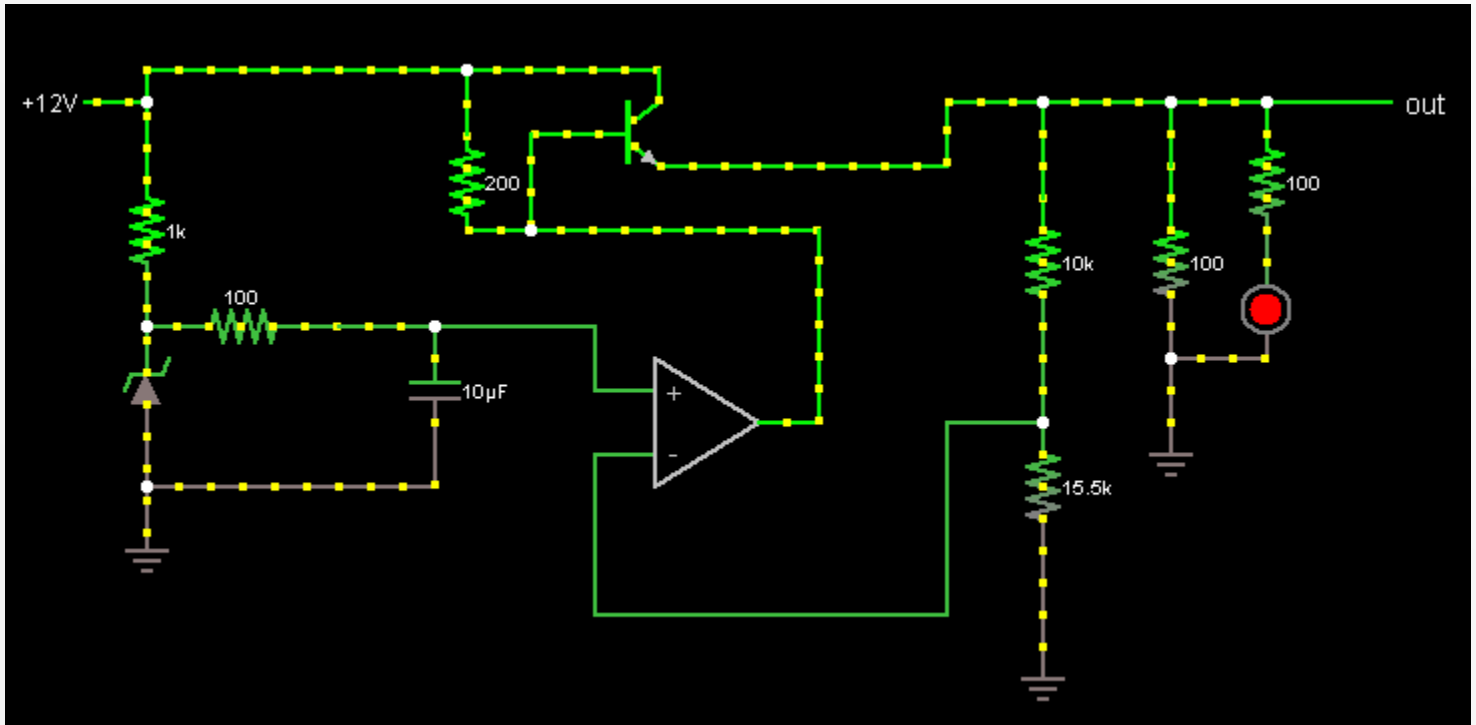
Scenariu de utilizare



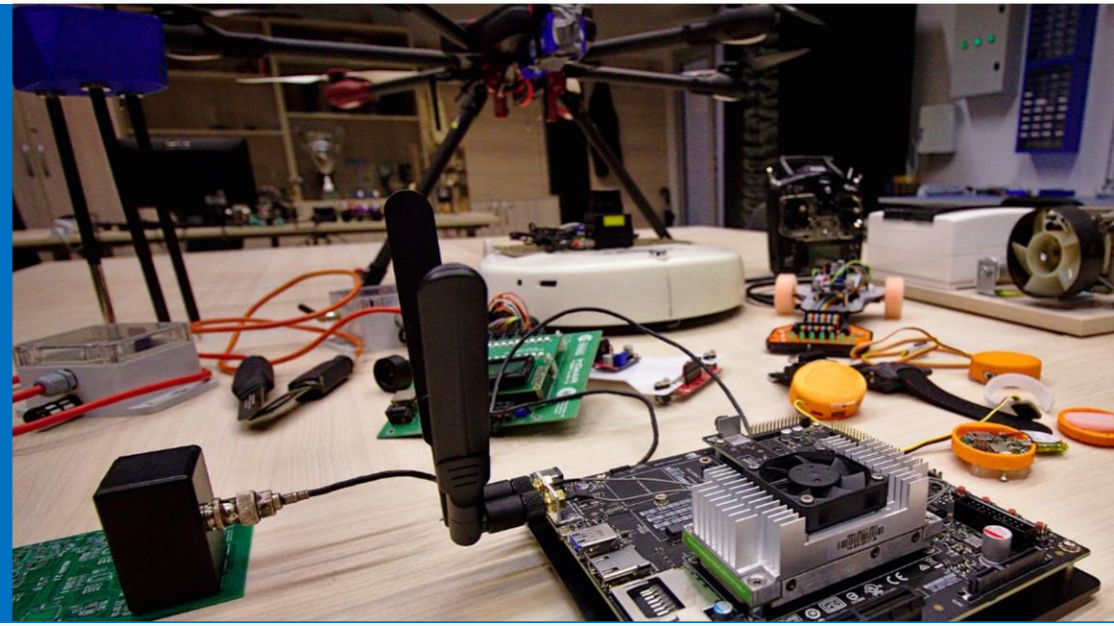
- Protejează împotriva fluctuațiilor de tensiune
- Le putem utiliza la dispozitivele din locuință
  - Centrale termice
  - Aer condiționat
  - Televizoare
  - Calculatoare
- Sau in cadrul proiectelor de la PM (DC-to-DC)
- Previn defecțiuni nedorite sau funcționări defectoase

Stabilizator de tensiune liniar

- Schemă interna simplificată
- Utilizează o dioda (Zener) sau tranzistor ca element activ
- Funcție de rezistor variabil ce permite reglarea tensiunii de ieșire
- Excesul de energie e disipat ca și căldură

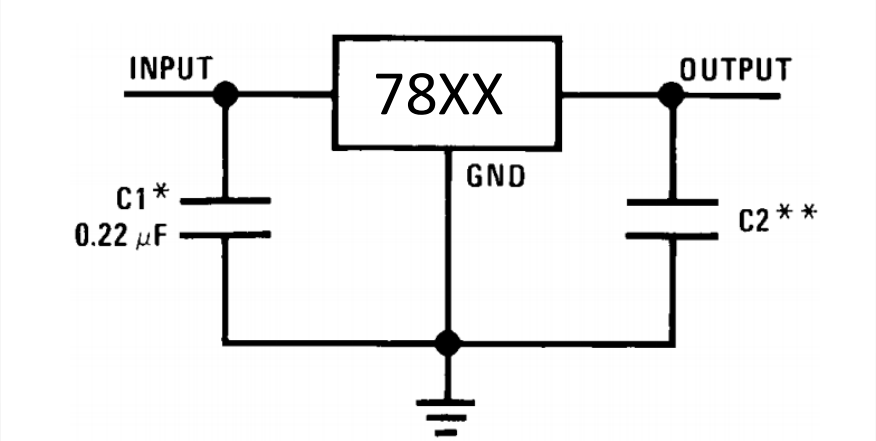


Stabilizatoare de tensiune – liniare integrate



PM

Descriere familia 78/79



- Stabilizator cu tensiunea la fixă
- Ultimele două cifre indică tensiunea de ieșire
- Cel mai comun este 7805 (5V)
- Disponibil si pe Arduino Uno

© https://www.ti.com/lit/ds/symlink/lm7800.pdf?ts=1610533958575

Familia 78/79

Tensiuni pozitive:

- LM78xx - http://www.fairchildsemi.com/ds/LM/LM7805.pdf
- uA78xx – http://www.ti.com/lit/ds/symlink/ua7805.pdf
- uA78lxx - http://www.datasheetcatalog.org/datasheets/208/402911_DS.pdf

Tensiuni negative:

- LM79xx - http://www.fairchildsemi.com/ds/LM/LM7905.pdf

Parametrii relevanți

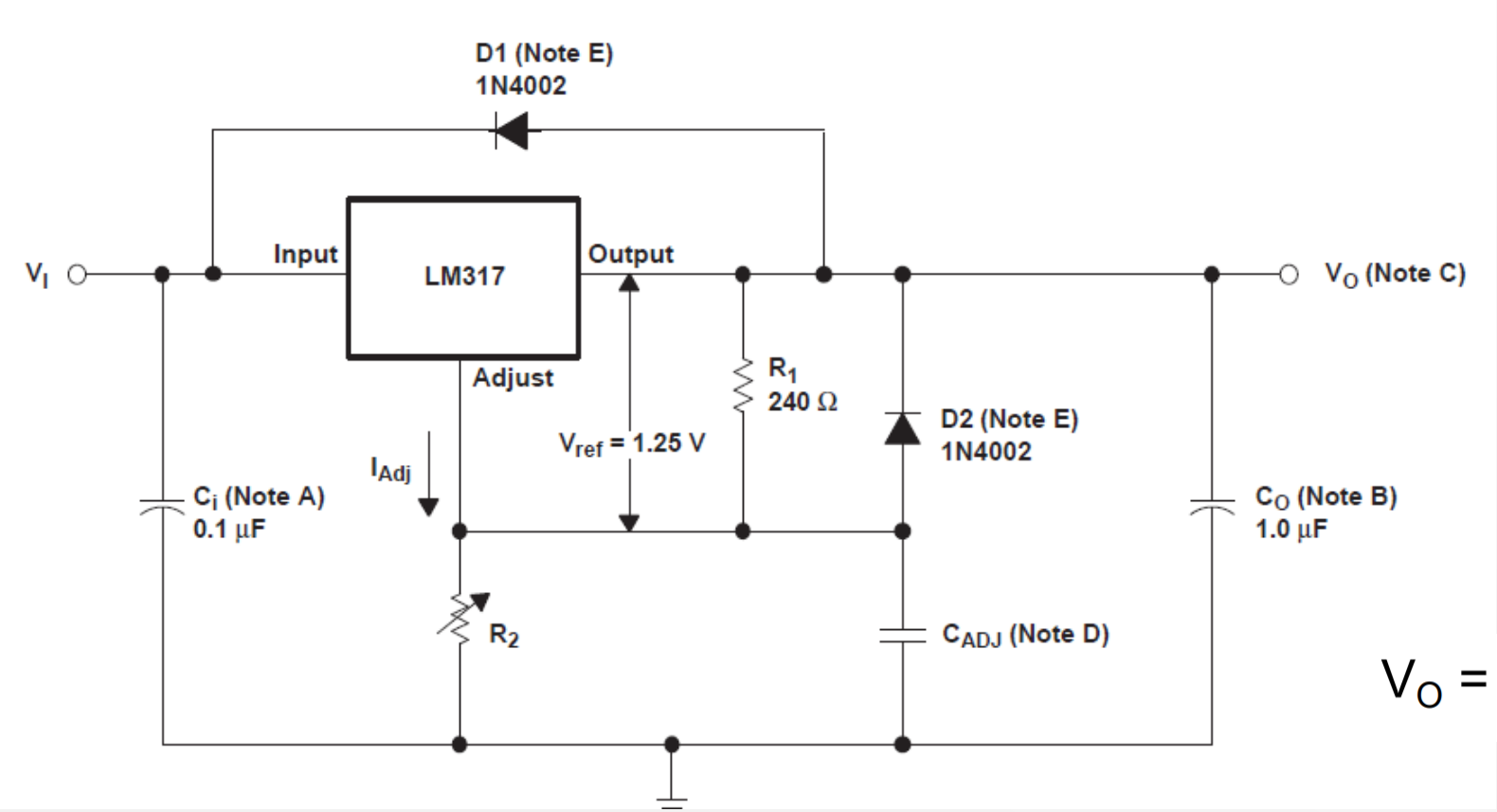
- Voltage dropout
- Quiescent current
- Current limit
- PSSR (Power-supply rejection ratio)
- Output noise

$$\mathrm{PSRR(dB)} = 20 \log \frac{V_{\mathrm{ripple(in)}}}{V_{\mathrm{ripple(out)}}}$$

Exemplu schematic LM317

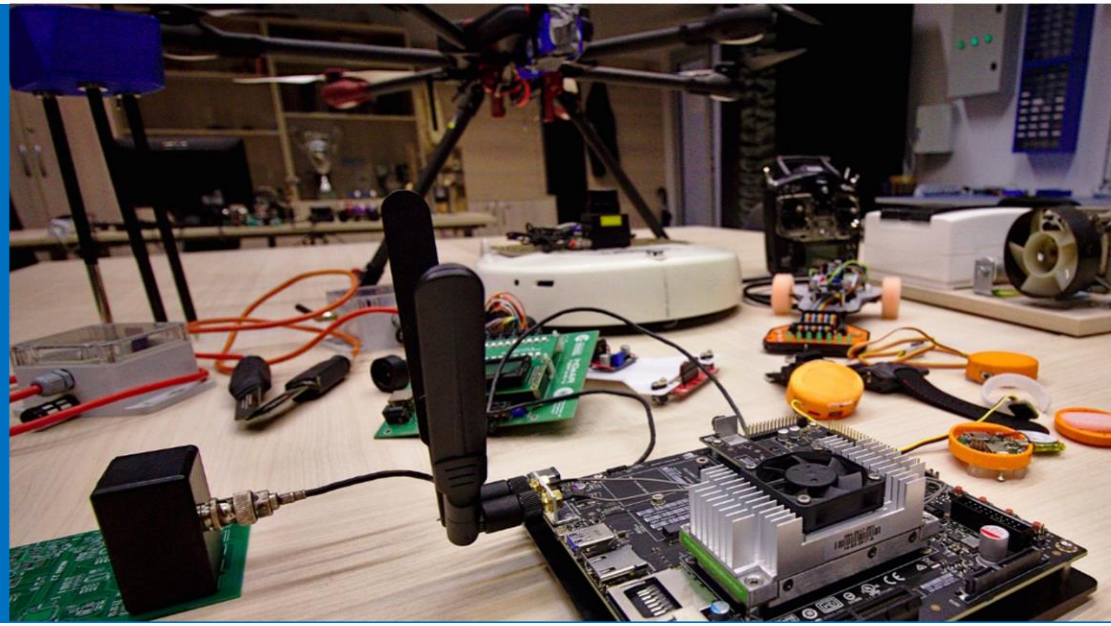
- http://www.ti.com/lit/ds/symlink/lm317.pdf

(pag7)



$$V_O = V_{REF} (1 + R2 / R1) + (I_{ADJ} \times R2)$$

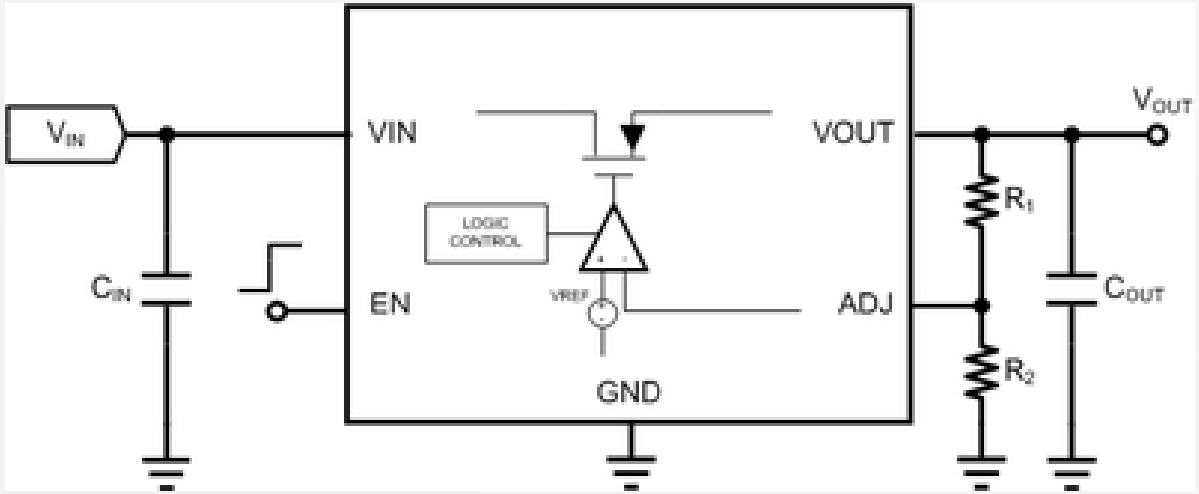
LDO – Low Dropout



PM

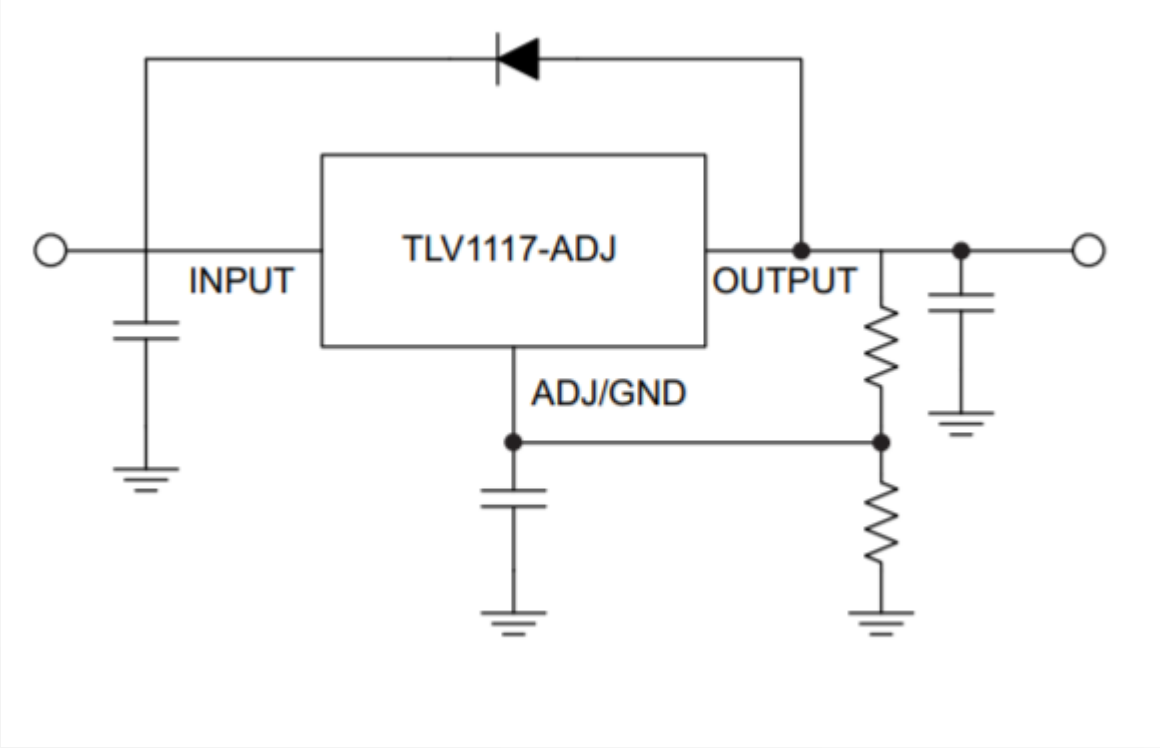
Motivație

- Poate regla tensiunea de ieșire chiar și atunci când tensiunea de alimentare e apropiată de cea de ieșire
- Zgomotul de comutare lipsește în cazul lor
- De asemenea disipa căldură în procesul de stabilizare a tensiunii de ieșire



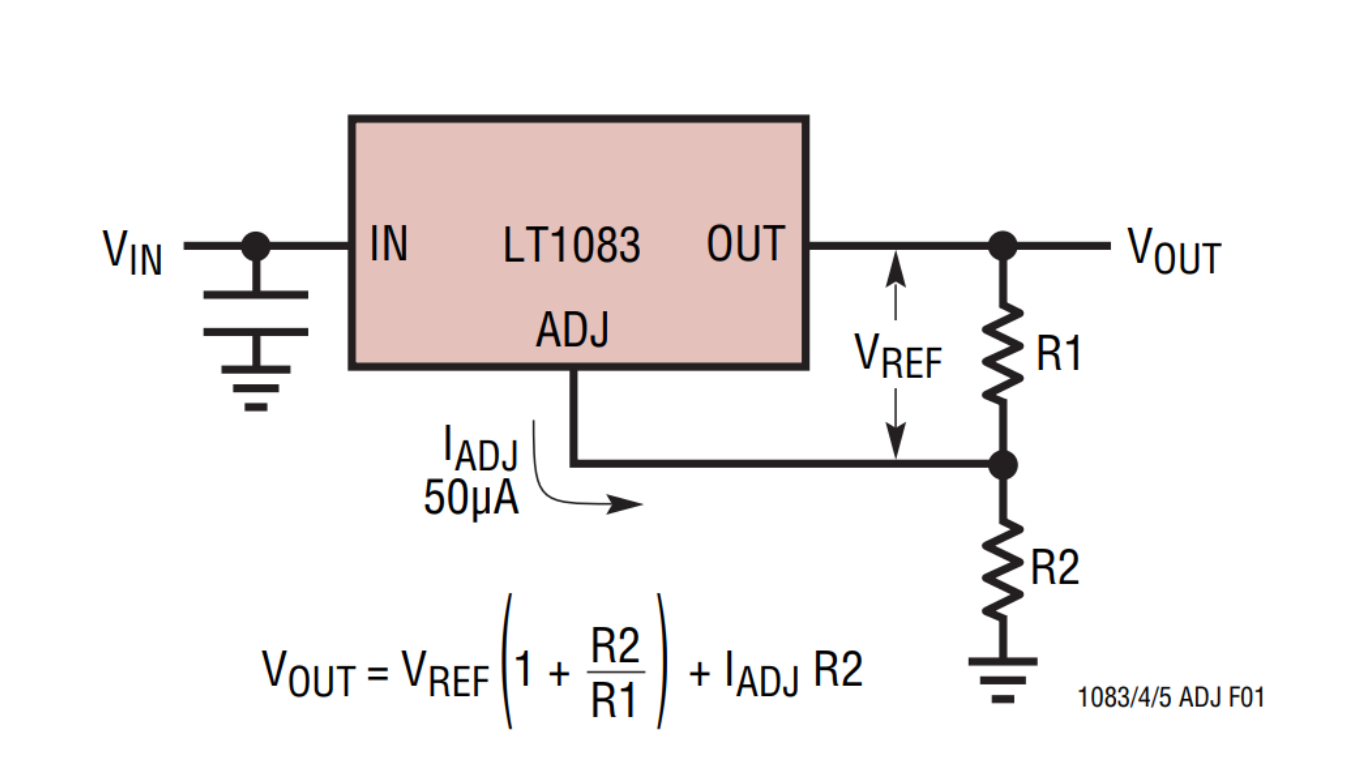
$$V_{\mathrm{OUT}} = \left(1 + \frac{R_1}{R_2}\right) V_{\mathrm{REF}}$$

TLV1117



https://www.ti.com/lit/ds/symlink/tlv1117.pdf

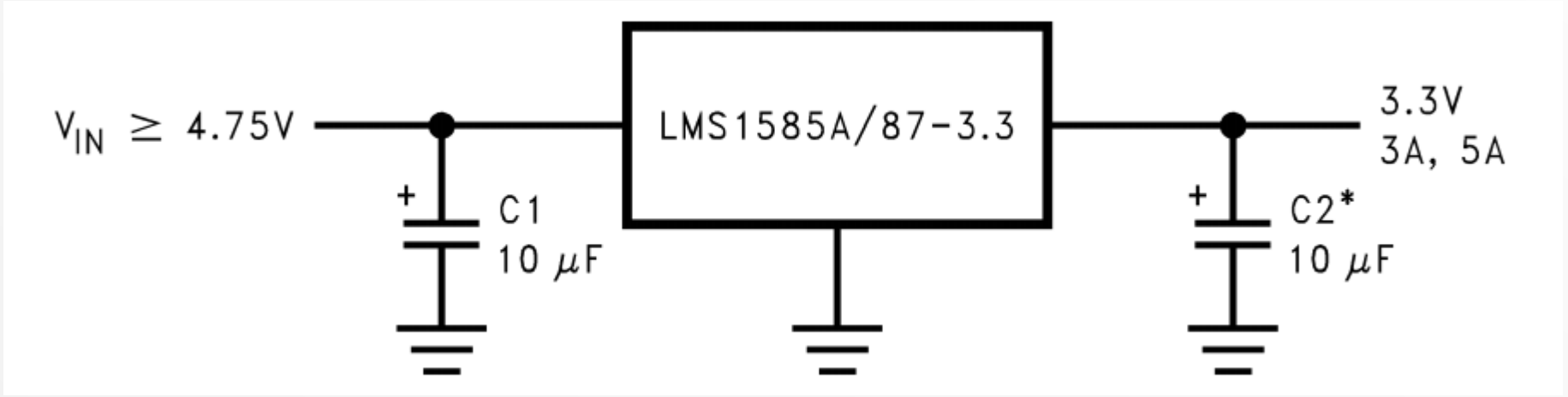
Discutie ADJ



https://www.analog.com/media/en/technical-documentation/data-sheets/108345fh.pdf

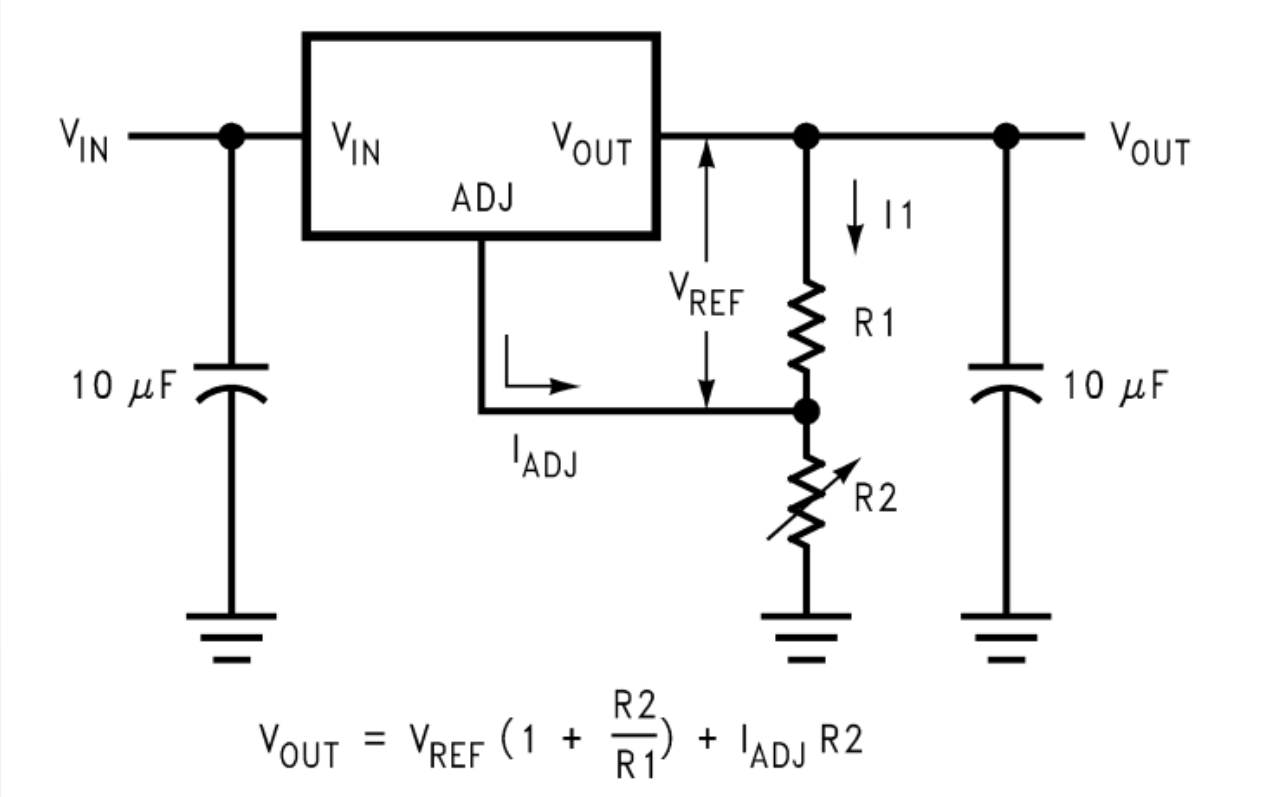
Variante pentru curenti mari

- Tensiuni mici (<3.3V) & curenti mari (5A) https://www.ti.com/lit/ds/symlink/lms1585a.pdf



Variante pentru curenti mari

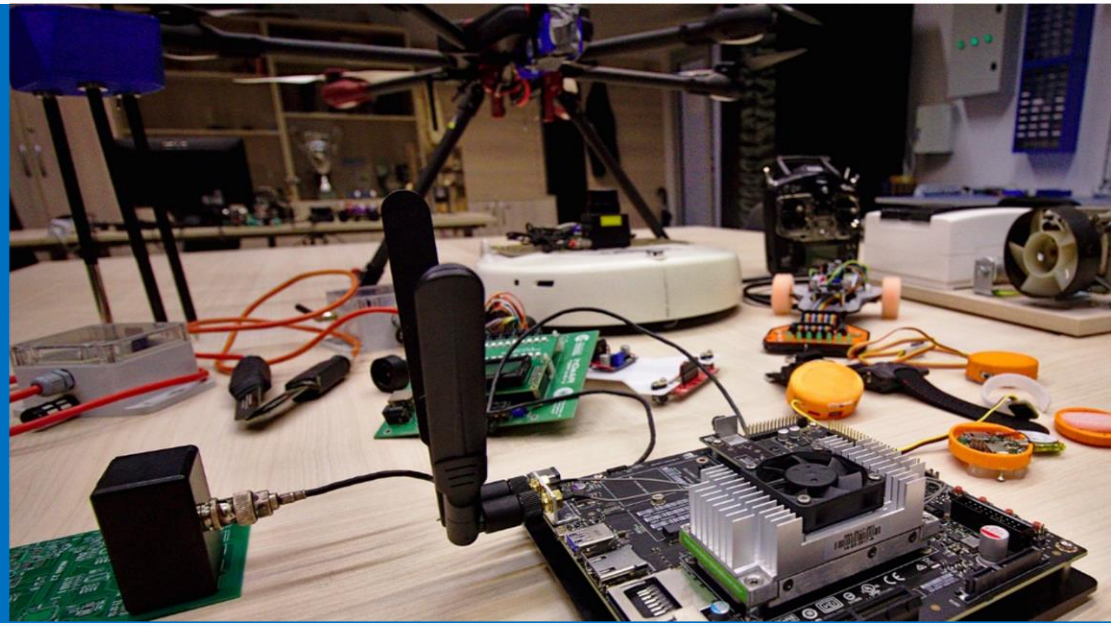
- Tensiuni mici (<3.3V) & curenti mari (5A) https://www.ti.com/lit/ds/symlink/lms1585a.pdf



Variante pentru curenti mici

- http://www.ti.com/lit/ds/symlink/lp3964.pdf
  - **dropout – 340mV la 800mA**

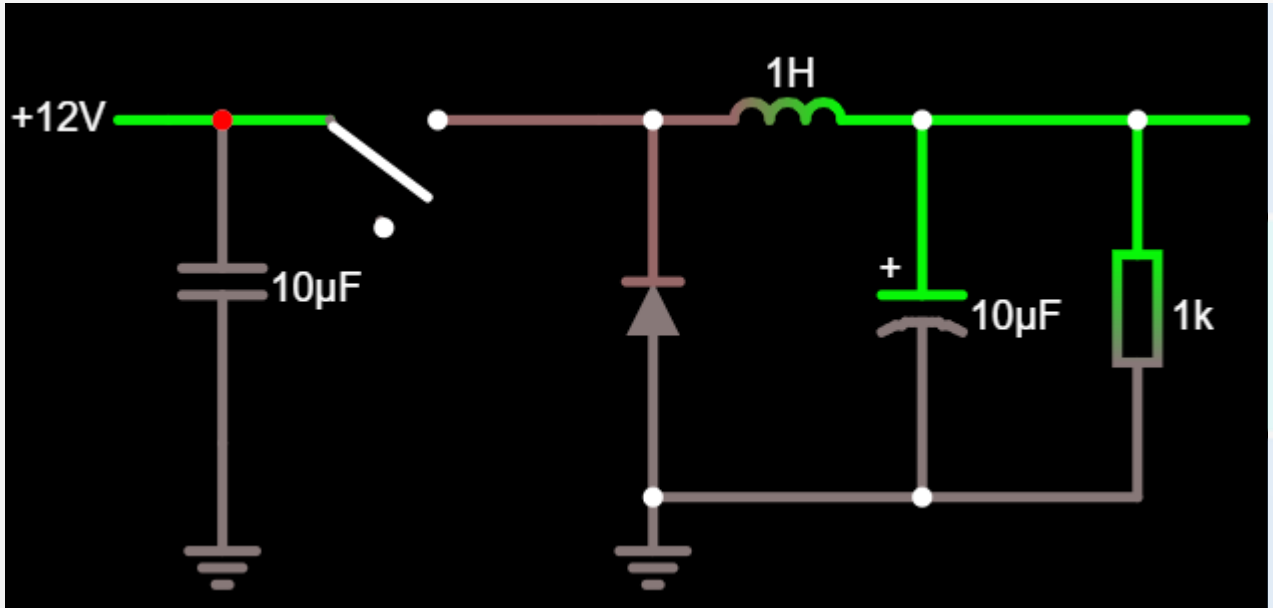
Surse în comutație

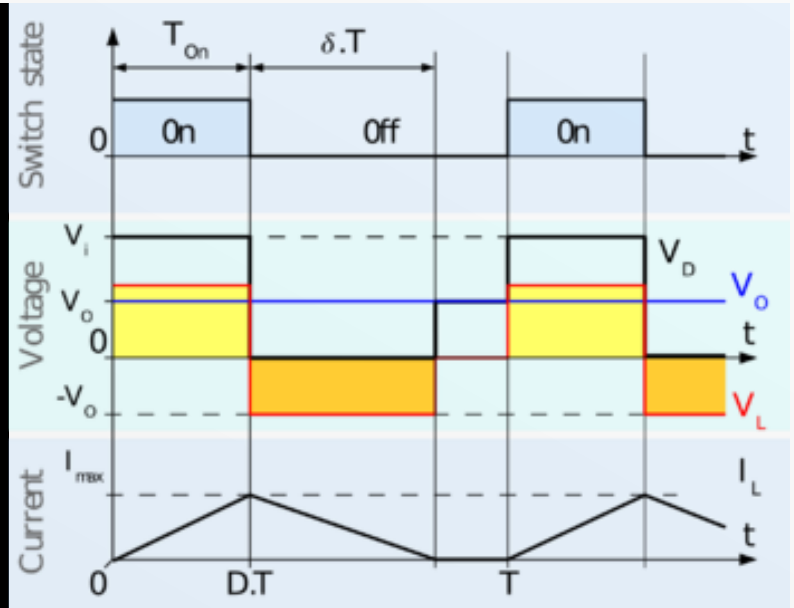


PM

Caracteristici

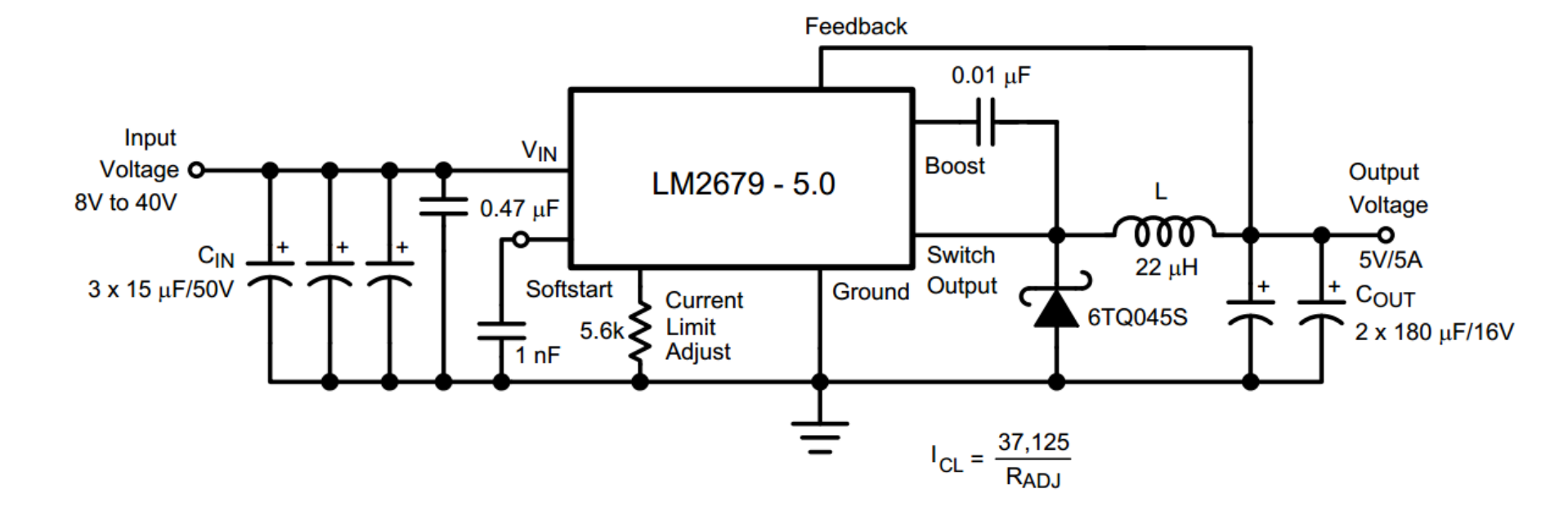
- Transfer de putere de la o sursă DC sau AC la una DC
- Reglarea tensiunii prin raportul dintre timpul de pornire și oprire (duty cycle)
- Eficiență mai ridicată decât regulatoarele de tensiune liniare
- Mai complexe și mai scumpe



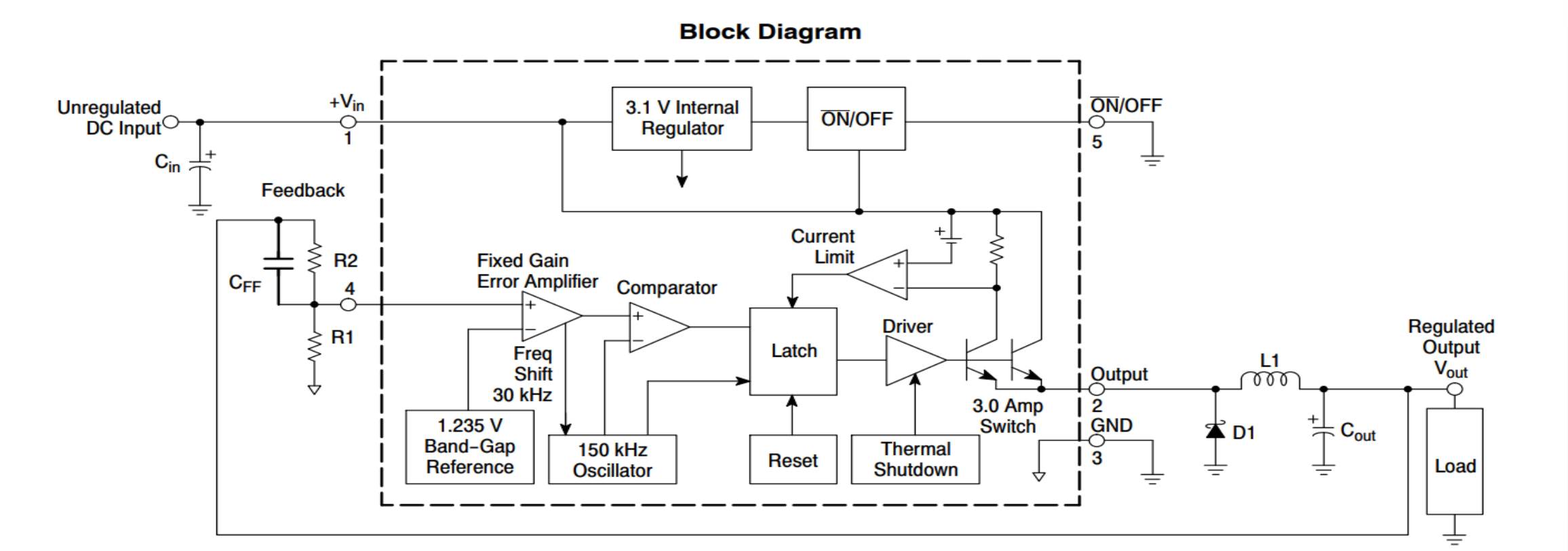


LM2679

- http://www.ti.com/product/lm2679



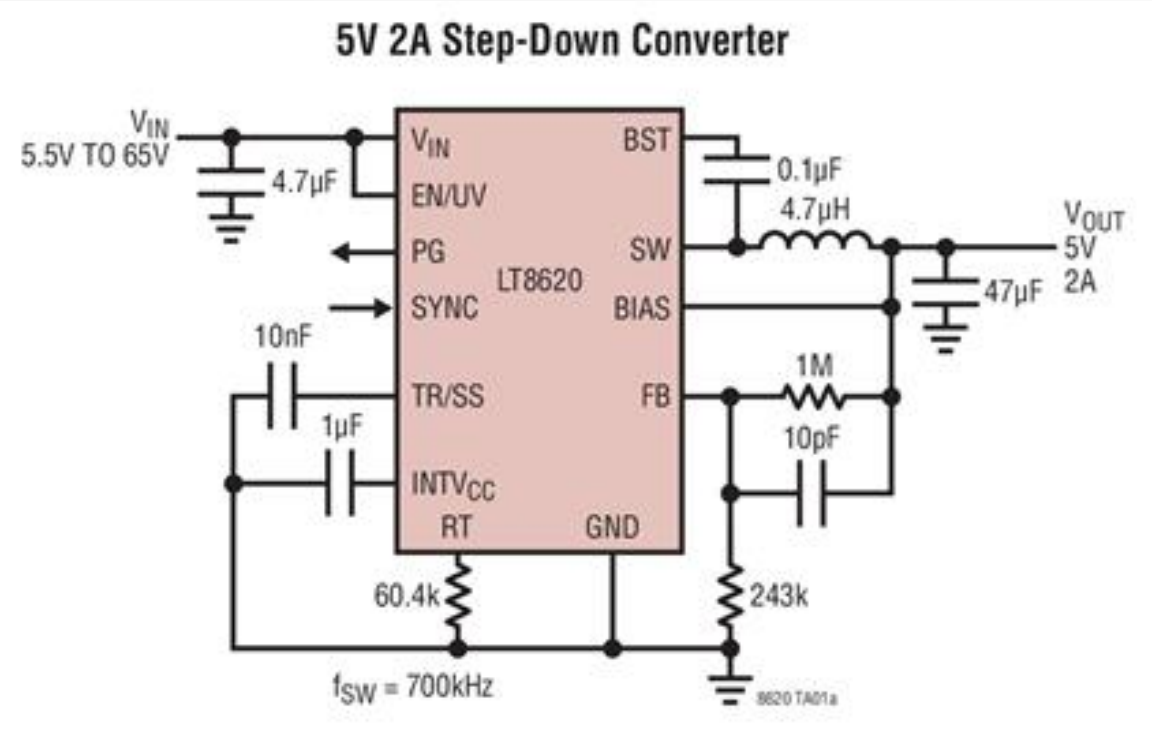
Schema bloc LM2679



© https://www.onsemi.com/pub/Collateral/LM2596-D.PDF

Variante

- http://www.ti.com/product/lm2679
- http://www.linear.com/product/LT8620



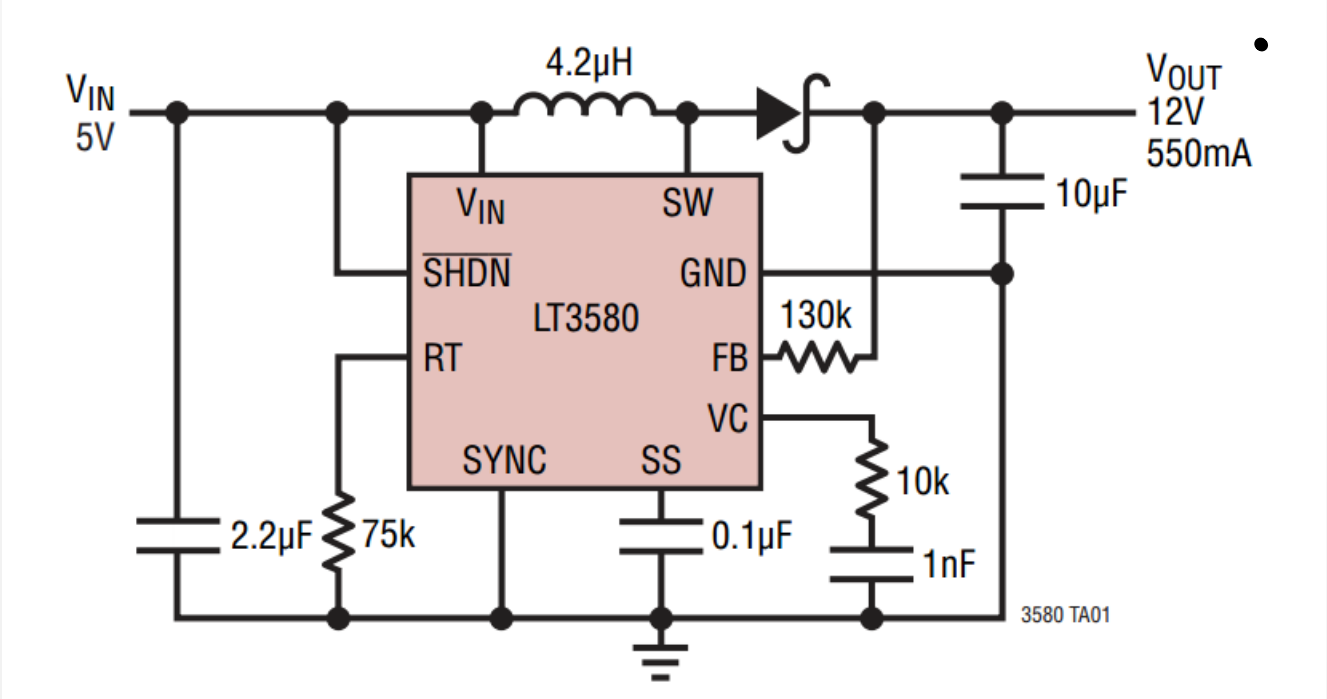
Exemple module “ready to buy”

- https://www.optimusdigital.ro/ro/surse-coboratoare-reglabile/12258-modul-dc-dc-step-down-lm2596-albastru-intrare-3-35-v-smd.html
- https://www.optimusdigital.ro/ro/surse-coboratoare-reglabile/150-modul-dc-dc-step-down-lm296.html
- https://ro.farnell.com/texas-instruments/pth08t241wad/dc-dc-converter-1-o-p-0-69-5-5v/dp/3009735?st=dc-dc
- https://ro.farnell.com/cui/vxo7803-500-m-tr/dc-dc-converter-fixed-3-3v/dp/3595427?st=dc-dc
- https://ro.farnell.com/tracopower/tsr-1-2433/converter-dc-to-dc-3-3v-1a-sip/dp/1696319



Boost – LT3580

- http://cds.linear.com/docs/en/datasheet/3580fg.pdf

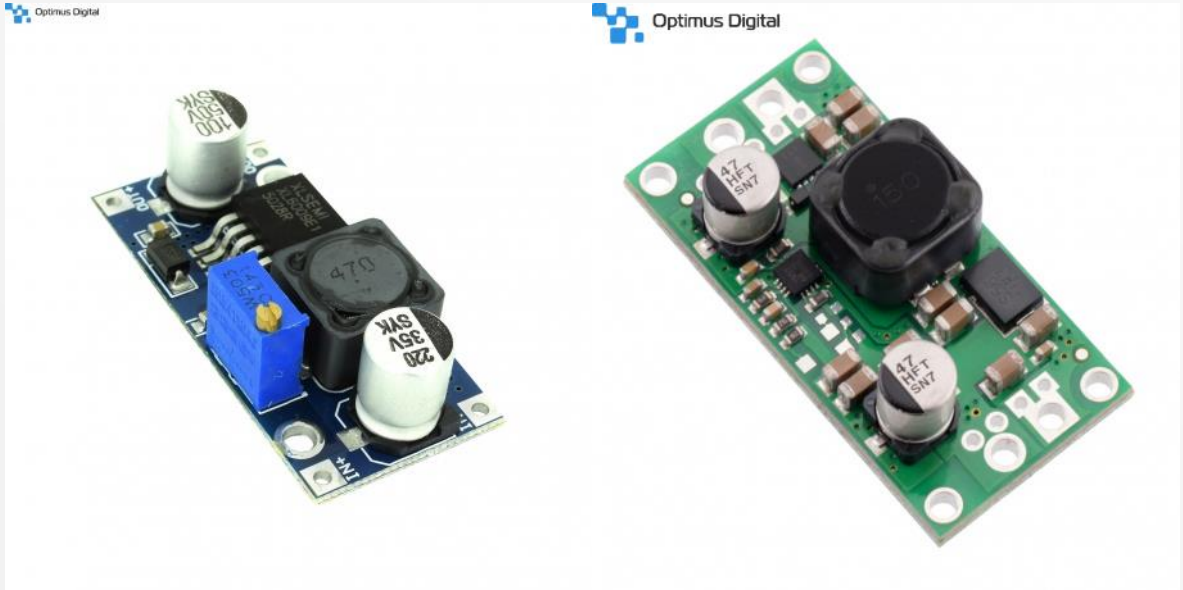


Exemple module “ready to buy”

- https://www.optimusdigital.ro/ro/surse-ridicatoare-reglabile/160-modul-dc-dc-boost-xl6009.html

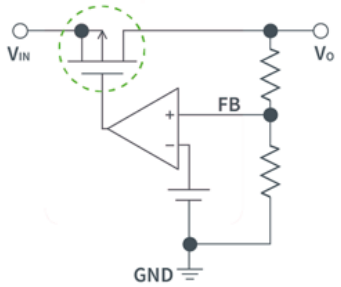
Buck-boost:

- https://www.optimusdigital.ro/ro/surse-ridicatoare-coboratoare/8275-sursa-coboratoareridicatoare-de-tensiune-de-24v-cu-regulator-s18v20f24.html



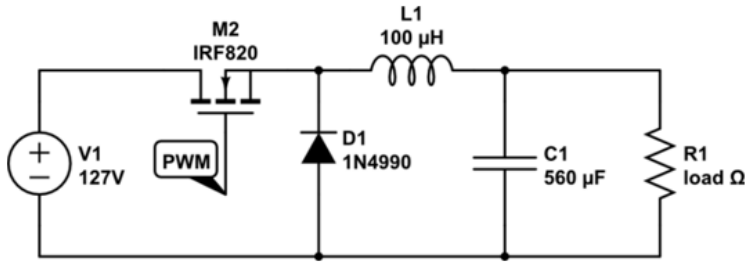
Comparație

Linear Regulators



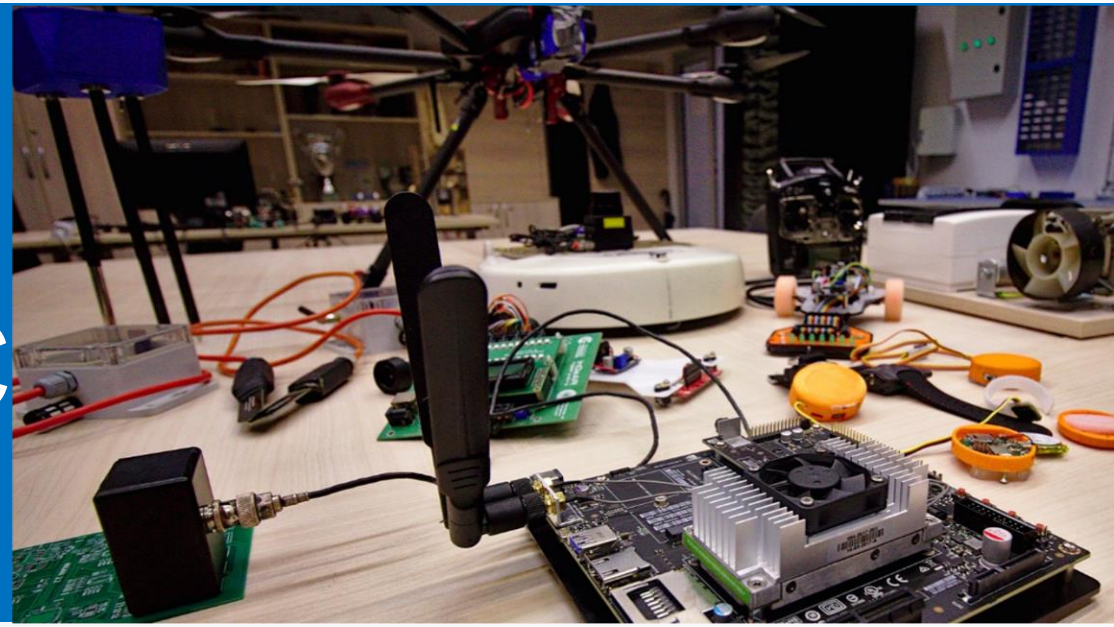
- Low Cost (low power DC applications)
- Simple to design
- High PSRR (low noise)
- Low efficiency if $V_O \ll V_I$
- Bigger heat losses than SMPS

Switch Mode Power Supplies



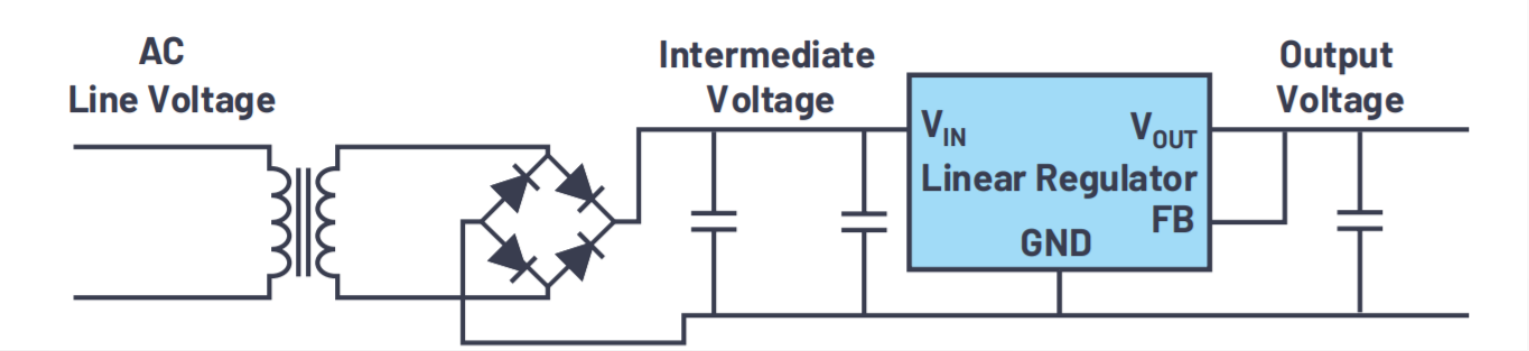
- Higher efficiency
- More W/cm^3
- Lower cost than linear supplies when used in AC-DC conversion (especially for high power)
- Source of EMI and can cause EMC problems
- Complex design (requires skilled engineer)
- Slower transient response

AC main > low DC



PM

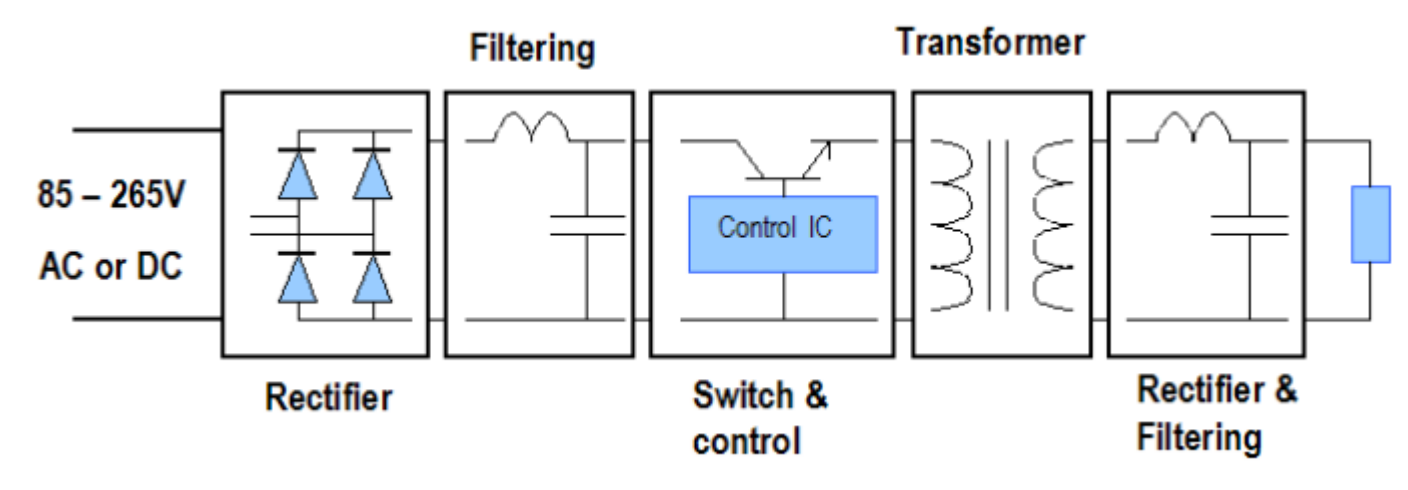
Surse AC – DC liniare



© https://www.analog.com/media/en/technical-documentation/tech-articles/how-the-smart-hardware-engineer-can-easily-design-power-supp.pdf

Surse AC – DC in comutatie

- http://ro.farnell.com/myrra/47152/power-supply-4-5w-5vdc-reg/dp/1825779

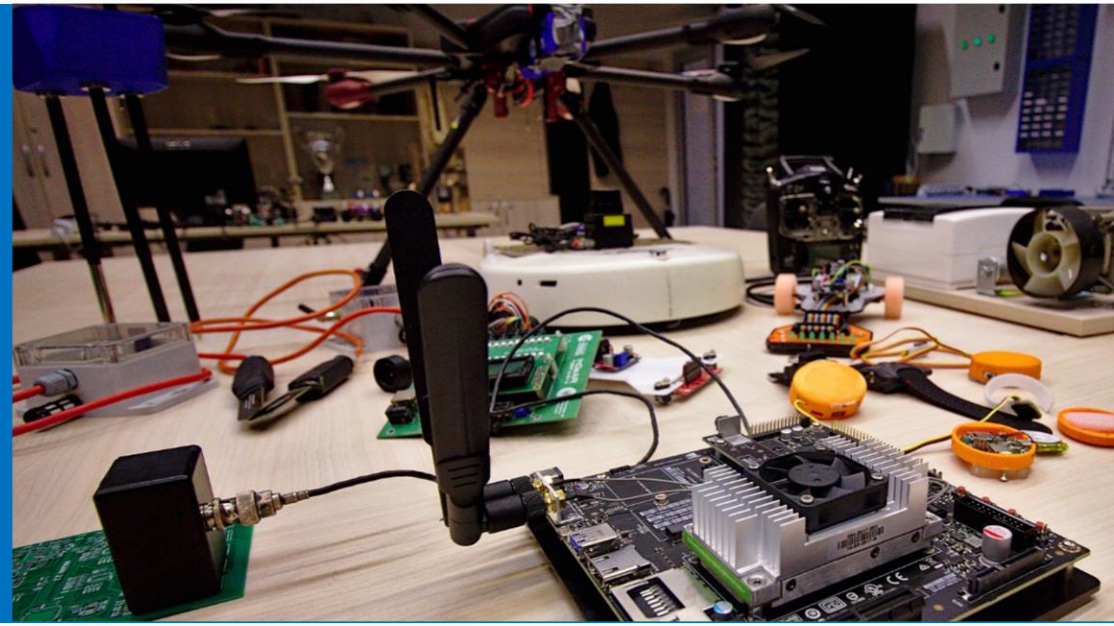


Example surse AC-DC

- https://ro.farnell.com/tdk-lambda/cus100me12-a/power-supply-medical-ac-dc-12v/dp/2913441
- https://ro.farnell.com/stontronics/pd-100-12/power-supply-enclosed-12v-100w/dp/3375267
- https://ro.farnell.com/multicomp-pro/mp003282/power-supply-ac-dc-24v-3a/dp/3408561



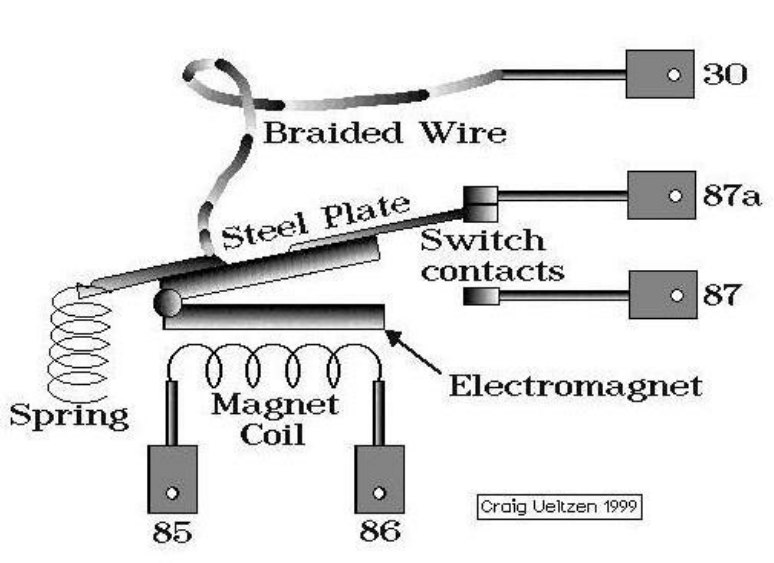
Diverse

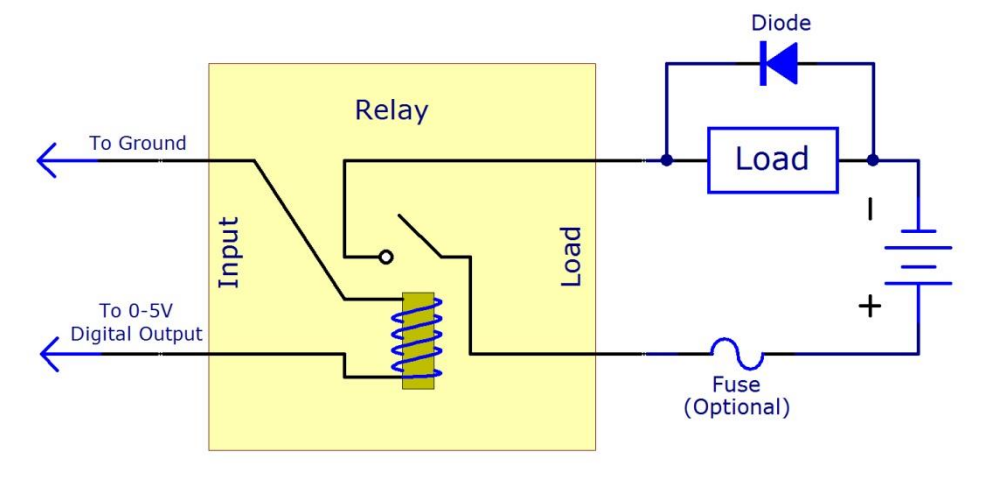


PM

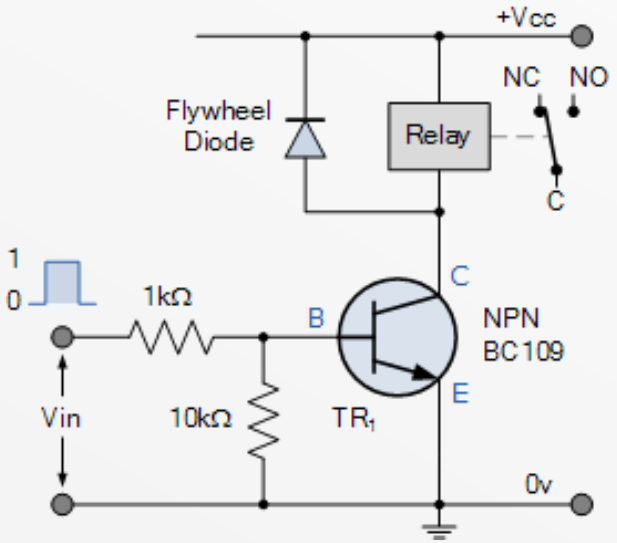
Releul

- Element electromecanic
- Când un curent electric străbate bobina, aceasta atrage o lamelă metalică, care realizează un contact electic
- Când curentul ce străbate bobina dispare, lamela metalică este readusă de către un arc în poziția inițială



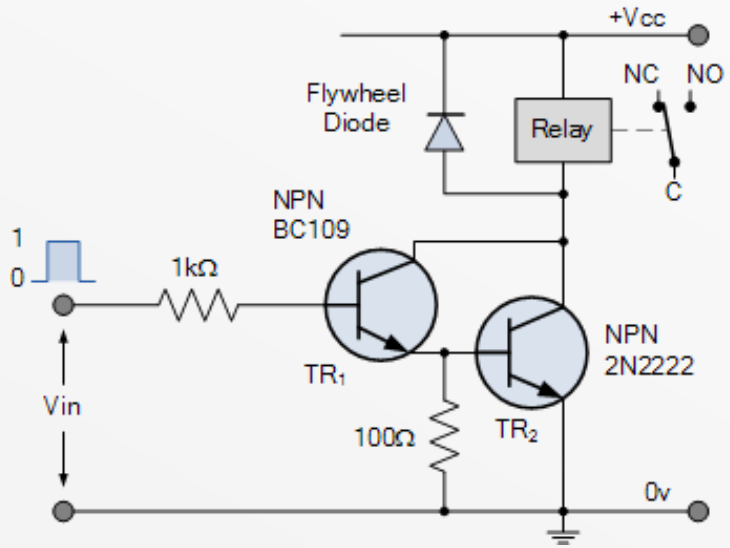


NPN



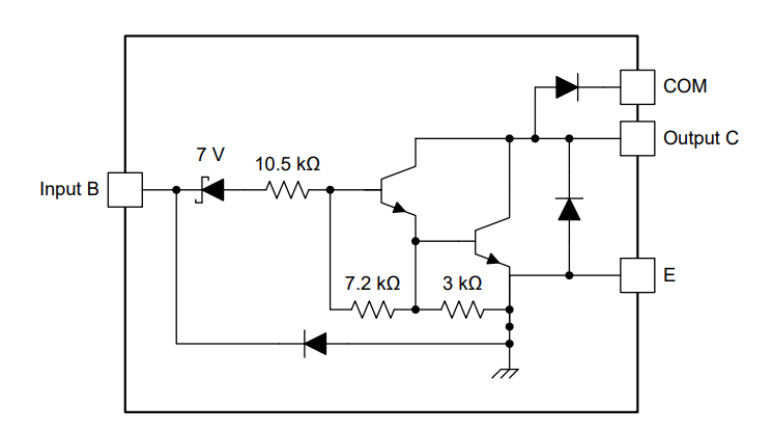
© http://www.electronics-tutorials.ws/blog/relay-switch-circuit.html

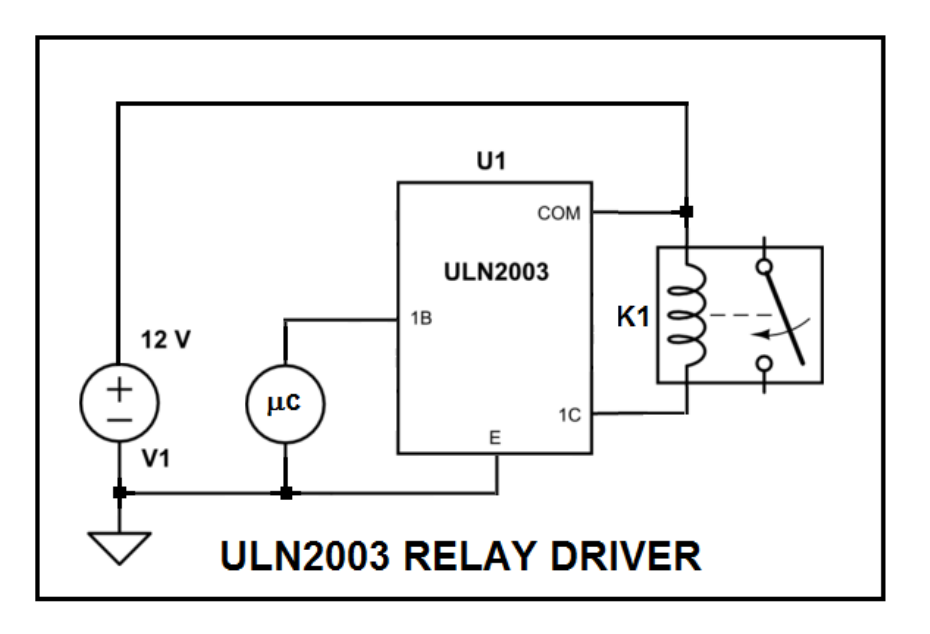
Darlinghton NPN



© http://www.electronics-tutorials.ws/blog/relay-switch-circuit.html

ULN 2003

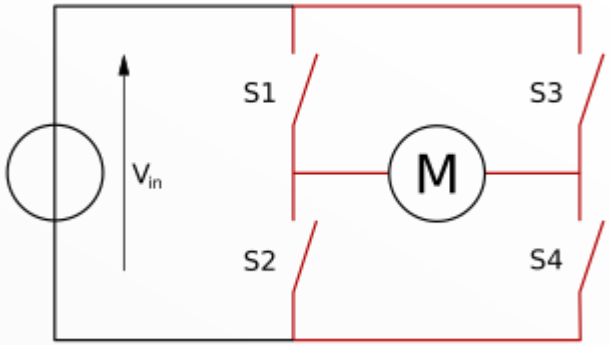




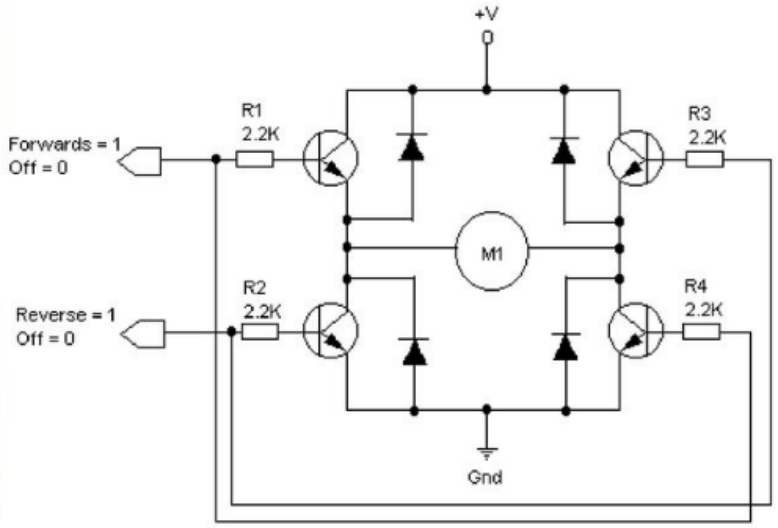
© Texas Instruments ULN2003 DS

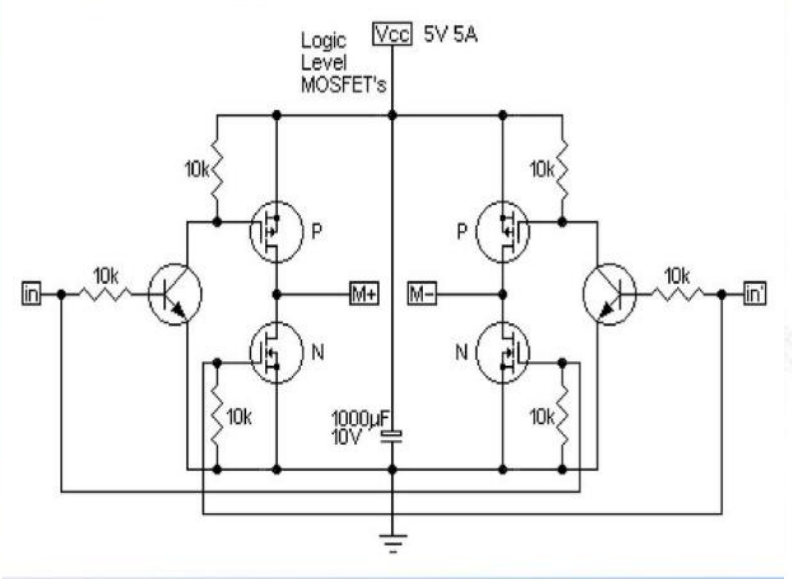
© https://electronics.stackexchange.com/

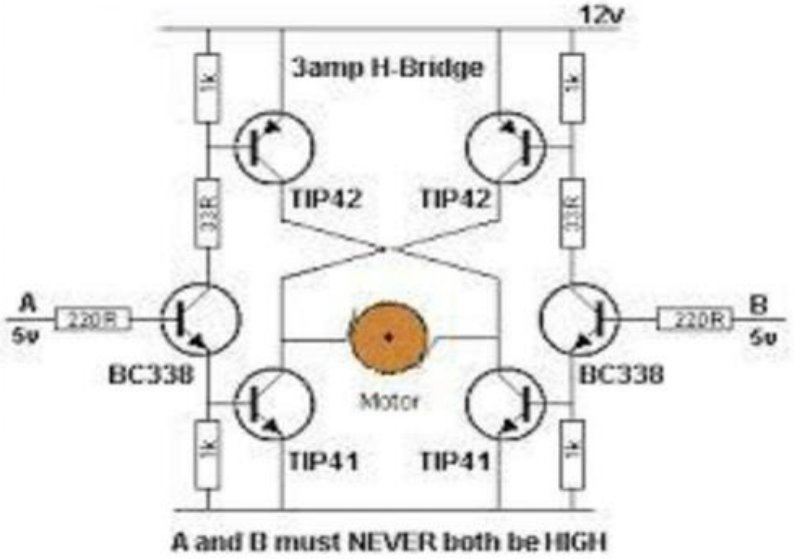
“drivere motoare”



- Puntea H
- Comuta polaritatea unei tensiuni aplicată sarcinii
- Funcționare înainte și înapoi
- Exemple de implementare

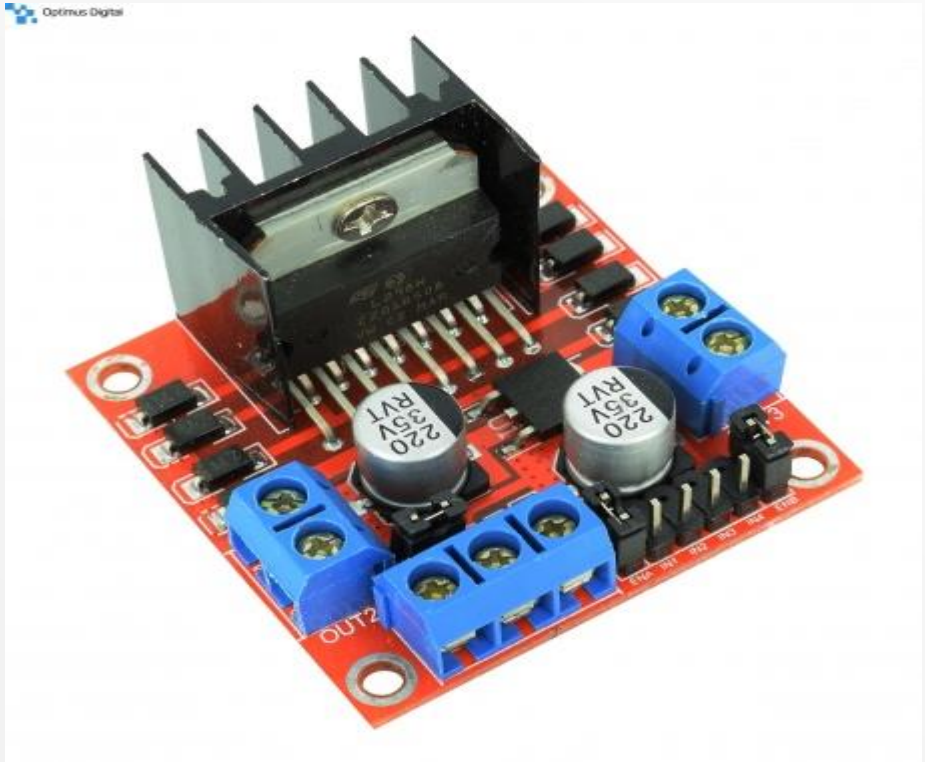






Brushed

- https://www.optimusdigital.ro/ro/drivere-de-motoare-cu-perii/145-driver-de-motoare-dual-l298n.html?search_query=l298&results=5



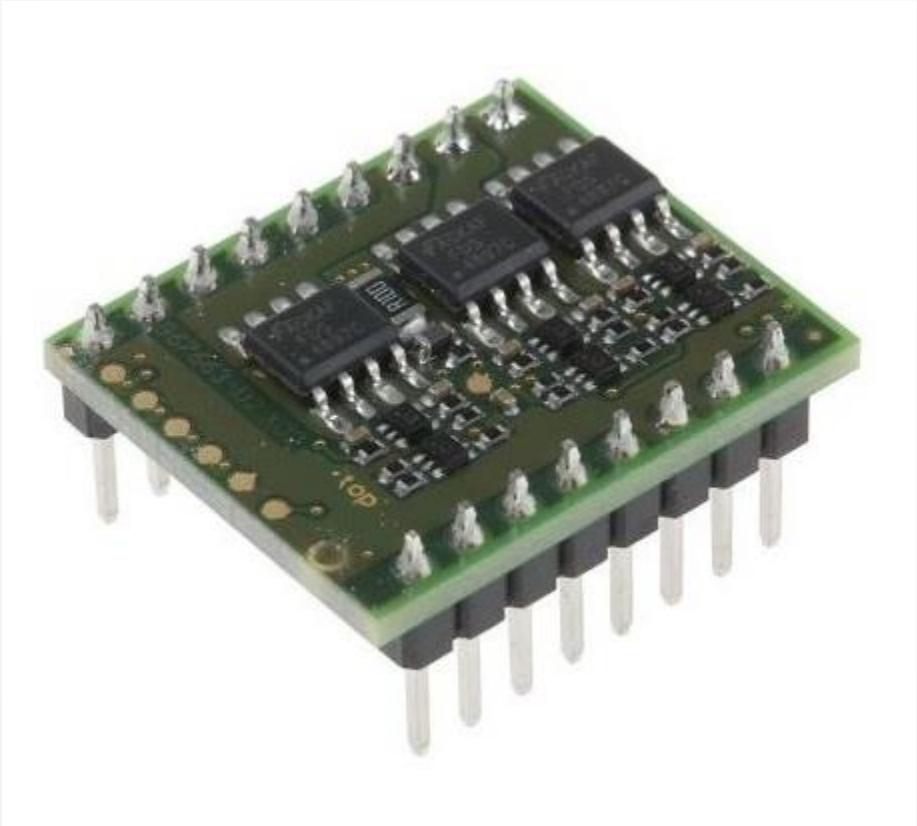
Stepper

- https://www.sigmanortec.ro/Driver-motor-pas-cu-pas-ULN2003-p126284051

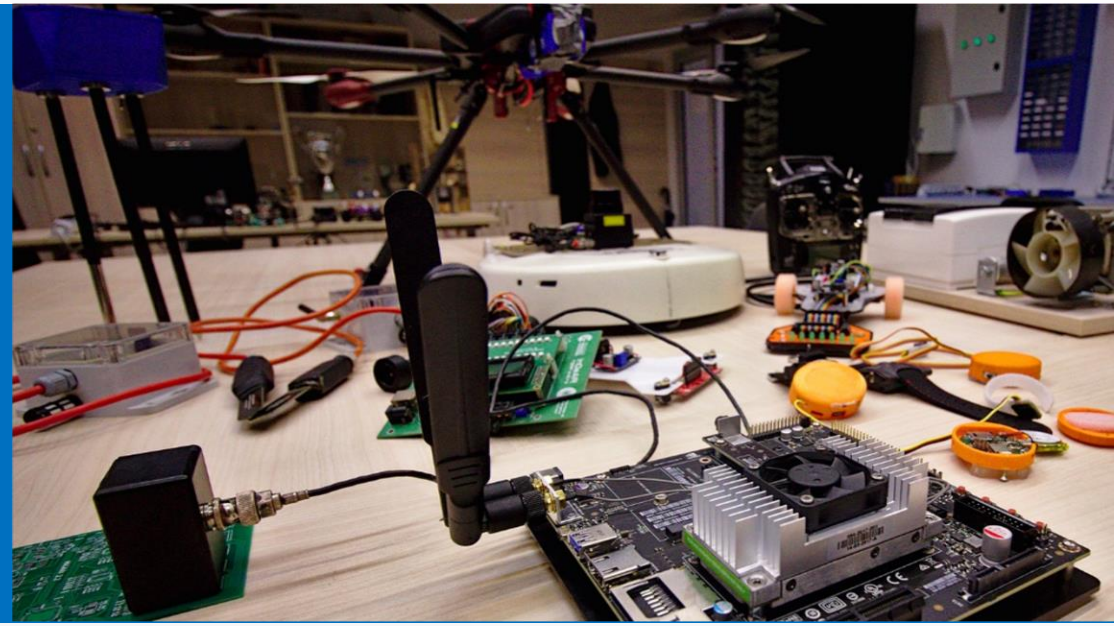


Brushless

- https://cleste.ro/driver-motor-dc-brushless-maxon-motor.html?utm_medium=GoogleAds&utm_campaign=&utm_source=



Consum de energie



PM

Consumul de energie

- În tehnologia CMOS: P ~ V_{DD}^2 · f
  - Tranzițiile costă
- Ce tranziții avem când în cod avem while(1); ?
  - Încă se execută instrucțiuni, PC se modifică etc.
  - Toate timerele active numără
  - ADC-ul poate să fie activ
  - Seriala poate transmite/recepționa
  - Circuitele de input GPIO sunt active
- Power management - procedeul prin care restricționăm ce poate face procesorul

ATMega328P Power Management

- Mai multe stări de power management
  - Mai puține periferice
  - Mai puține ceasuri active
  - Mai puține metode de revenire
- Periferice dezactivabile manual
  - PRR - Power Reduction Register
    - Un bit pt fiecare periferic: ADC, USART0, SPI, Timer/Counter1, Timer/Counter0, Timer/Counter2, TWI(I^2C)
    - One reserved bit (bit 4)

Stări de lucru

Table 9-1. Active Clock Domains and Wake-up Sources in the Different Sleep Modes.

Sleep Mode	Active Clock Domains					Oscillators		Wake-up Sources							Software BOD Disable
	clk_{CPU}	clk_{FLASH}	clk_{IO}	clk_{ADC}	clk_{ASY}	Main Clock Source Enabled	Timer Oscillator Enabled	INT1, INT0 and Pin Change	TWI Address Match	Timer2	SPM/EEPROM Ready	ADC	WDT	Other/O	
Idle			X	X	X	X	X(2)	X	X	X	X	X	X	X	
ADC noise Reduction				X	X	X	X(2)	X(3)	X	X(2)	X	X	X		
Power-down								X(3)	X				X		X
Power-save					X		X(2)	X(3)	X	X			X		X
Standby(1)						X		X(3)	X				X		X
Extended Standby					X(2)	X	X(2)	X(3)	X	X			X		X

Notes:
1. Only recommended with external crystal or resonator selected as clock source.
2. If Timer/Counter2 is running in asynchronous mode.
3. For INT1 and INT0, only level interrupt.

Stări de lucru - consum

28.3 DC Characteristics

T_A = –40°C to +125°C, V_{CC} = 2.7V to 5.5V (unless otherwise noted)

Parameter	Condition	Symbol	Min.	Typ.(2)	Max.	Units
Power supply current(1)	Active 4MHz, V_{CC} = 3V	I_{CC}		1.5	2.4	mA
	Active 8MHz, V_{CC} = 5V			5.2	10	mA
	Active 16MHz, V_{CC} = 5V			9.2	14	mA
	Idle 4MHz, V_{CC} = 3V			0.25	0.6	mA
	Idle 8MHz, V_{CC} = 5V			1.0	1.6	mA
	Idle 16MHz, V_{CC} = 5V			1.9	2.8	mA
Power-down mode(3)	WDT enabled, V_{CC} = 3V				44	µA
	WDT enabled, V_{CC} = 5V				66	µA
	WDT disabled, V_{CC} = 3V				40	µA
	WDT disabled, V_{CC} = 5V				60	µA

Notes:
1. Values with Section 9.10 “Minimizing Power Consumption” on page 36 enabled (0xFF).
2. Typical values at 25°C.
3. The current consumption values include input leakage current.

Registre

- SMCR - registru control sleep
  - biți SM - modul în care să intre
  - SE - sleep enable (se poate intra in sleep)
- *sleep* – instrucțiunea care intră în sleep efectiv
- Cu ajutor / Fără ajutor

```
void main ()
{
  while(1)
  {
    do_stuff();
    // ...

    set_sleep_mode(SLEEP_MODE_IDLE);
    sleep_mode();
  }
}
```

```
void main ()
{
  while(1)
  {
    do_stuff();
    // ...

    SMCR = (1 << SE); // pentru bitii SM2..0 -- 0 este IDLE
    __asm__ __volatile__ ( "sleep" "\n\t" :: );
    SMCR = 0;
  }
}
```

Exemplu aplicații

- Un termometru cu afișaj digital
- O dată la 1s măsoară și schimbă afișajul
- Afișaj conectat prin SPI foarte eficient
- Cum minimizăm consumul microcontroller-ului?

Datele problemei

- Presupunem un consum mediu de 100μA pentru senzor și display
- Avem la dispoziție o baterie de ceas CR2032 de 200mAh și 3.3V
- Presupunem că putem folosi toată capacitatea
- Avem un cristal de cuart, de 8MHz
- ADC configurat inițial cu prescaler maxim 128

$$t_{conversie} = 13 \text{ cicliADC} \cdot 128 \cdot t_{cicluCPU}$$
$$t_{conversie} = 208\mu s \simeq 200\mu s$$

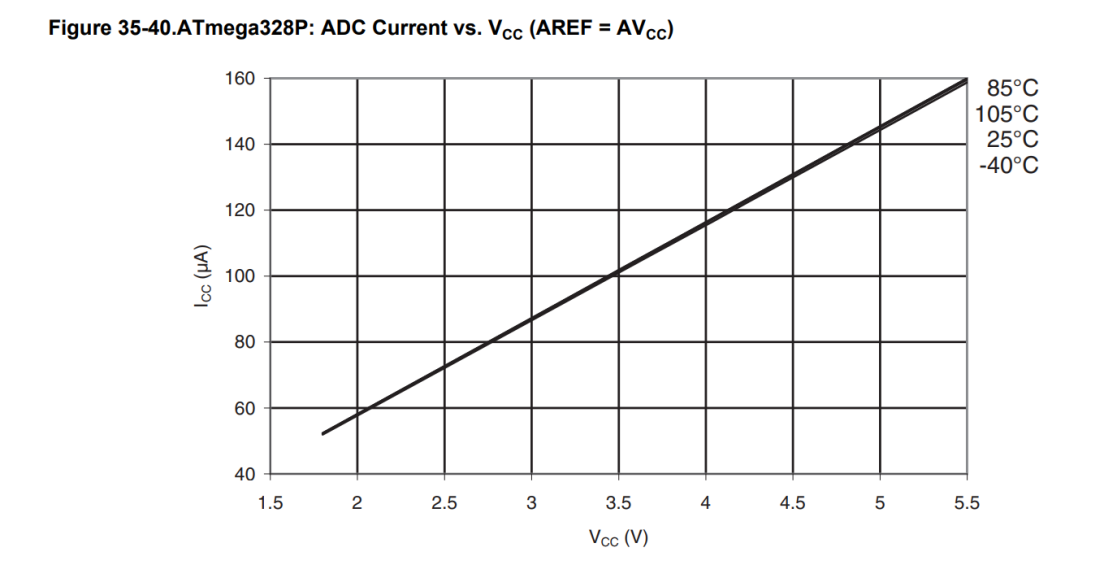
- Consumul pe SPI presupune schimbul a 100 bytes la 4MHz.

$$t_{\text{comunicare}} = \frac{100 * 8}{4000000} s$$
$$t_{\text{comunicare}} = 200\mu s$$

- Neglijăm restul operațiilor

Calcul consum activ

- $I_{CC_{\text{total}}} = I_{CC_{\text{Activ,8MHz,3.3V}}}(1 + \text{I}_{\text{ADC}} + \text{I}_{\text{SPI}} + \text{I}_{\text{TIM1}})$
- $I_{CC_{\text{total}}} = 3\text{mA} \cdot (1 + 0.042 + 0.027 + 0.016)$
- $I_{CC_{\text{total}}} = 3\text{mA} \cdot (1 + 0.042 + 0.027 + 0.016) = 3.255\text{mA}$

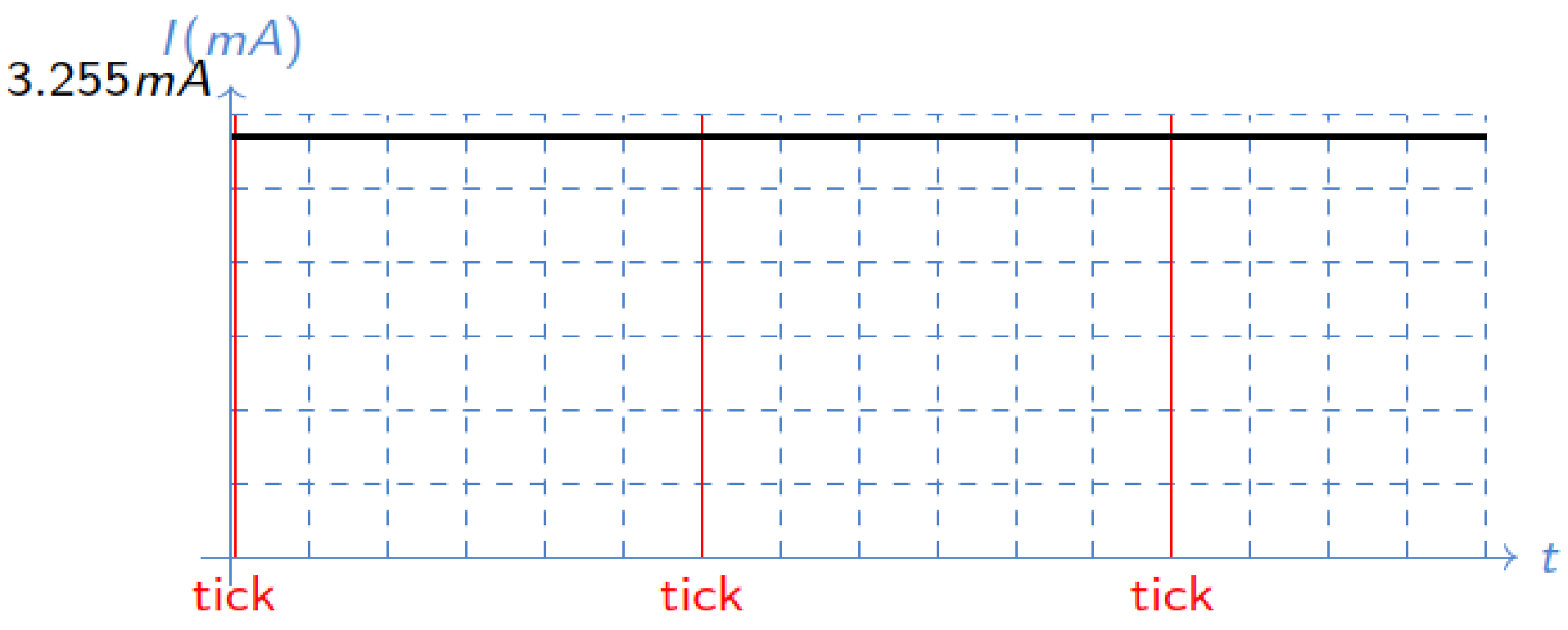


PRR bit	Additional Current consumption compared to Active with external clock (See Figure 30-1 and Figure 30-2)
PRTIM2	3.8%
PRTIM1	2.6%
PRTIM0	1.6%
PRADC	4.8%
PRSPI	2.8%

https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061B.pdf

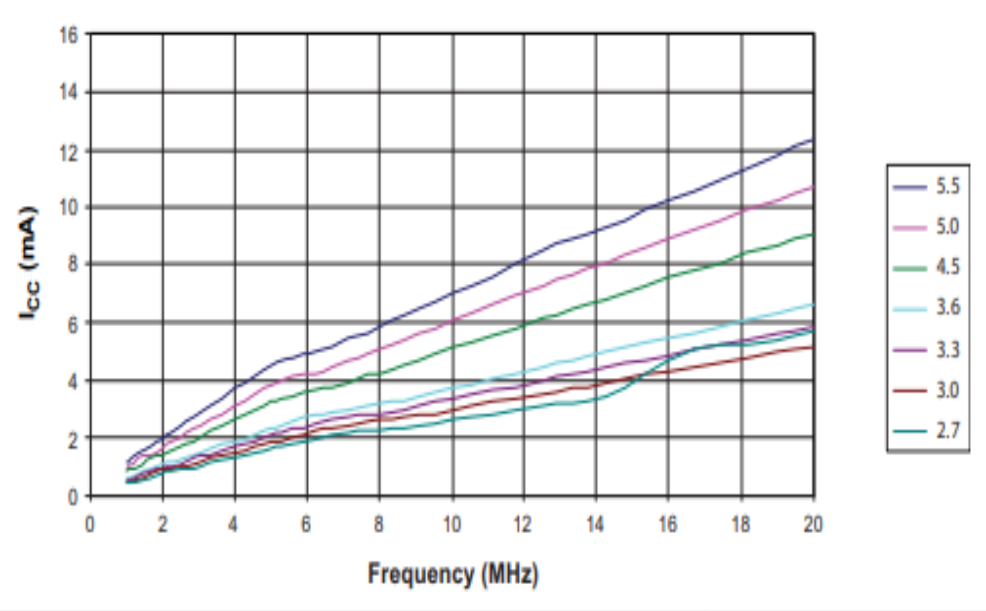
Grafic de consum

- $t_{\text{baterie}} = \frac{Capacitate}{Consum} = \frac{200}{3.26 + 0.01} = 61.1\text{h}$



Calcul consum activ optimizat

- *Dacă folosim PRR să închidem ADC și SPI când nu avem nevoie?*
- $I_{CC_{\text{adc}}} = I_{CC_{\text{Activ,8M}Hz,3.3V}}(1 + \text{I}_{\text{ADC}} + \text{I}_{\text{TIM1}}) = 3.174\text{mA}$
- $I_{CC_{\text{comunicatie}}} = I_{CC_{\text{Activ,8M}Hz,3.3V}}(1 + \text{I}_{\text{SPI}} + \text{I}_{\text{TIM1}}) = 3.129\text{mA}$
- $I_{CC_{\text{rest}}} = I_{CC_{\text{Activ,8M}Hz,3.3V}}(1 + \text{I}_{\text{TIM1}}) = 3.048\text{mA}$



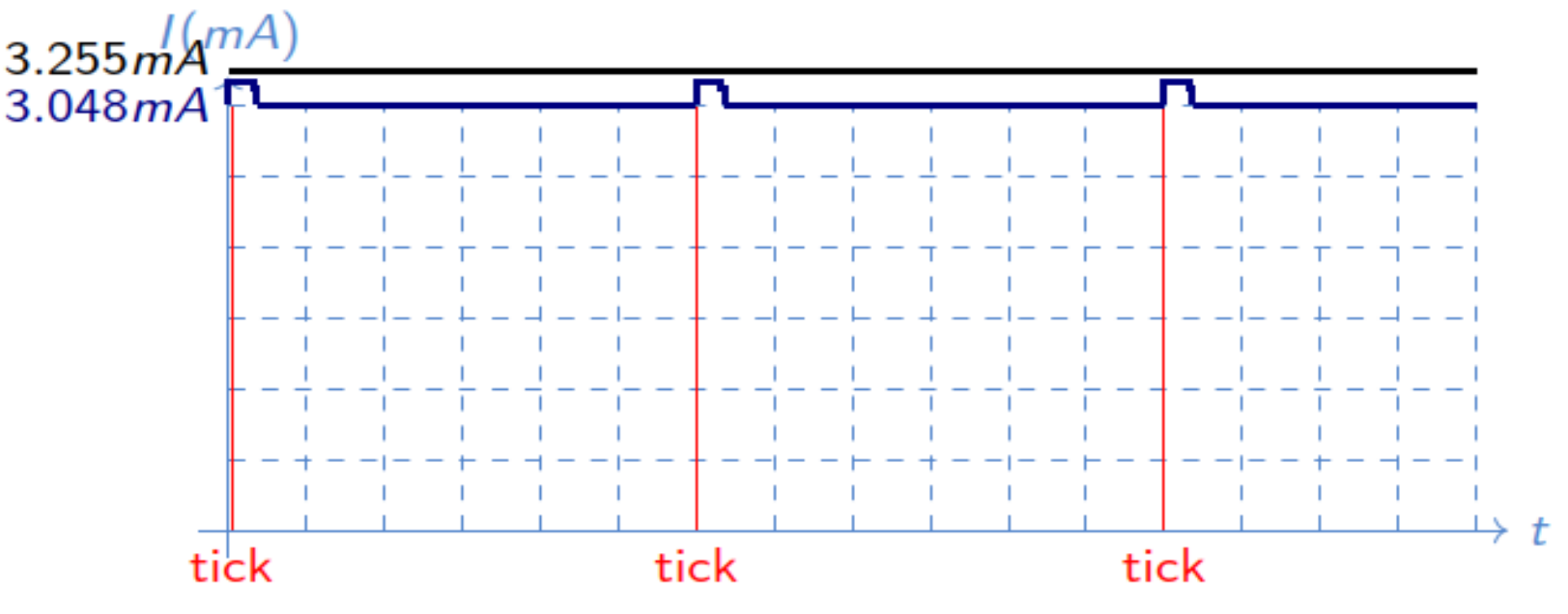
PRR bit	Additional Current consumption compared to Active with external clock (See Figure 30-1 and Figure 30-2)
PRTIM2	3.8%
PRTIM1	2.6%
PRTIM0	1.6%
PRADC	4.8%
PRSPI	2.8%

Curent mediu

- Curentul mediu este media ponderată pe timpi
- $I_{CC_{\text{mediu}}} = \frac{\sum I_{CC_x} * t_x}{\sum t_x}$
- $I_{CC_{\text{mediu}}} = \frac{I_{CC_{conversie}} * t_{conversie} + I_{CC_{comunicatie}} * t_{comunicatie} + I_{CC_{rest}} * t_{rest}}{t_{total}}$
- $I_{CC_{\text{mediu}}} = \frac{3.174 * 200 + 3.129 * 200 + 3.048 * 999600}{1000000}$
- $I_{CC_{\text{mediu}}} = 3.048\ \text{mA}$

Grafic de consum optimizat

- $t_{\text{baterie}} = \frac{Capacitate}{Consum} = \frac{200\text{mAh}}{3.05\text{mAa} + 0.01\text{mA}} = 65.3\text{h}$



Idle Mode

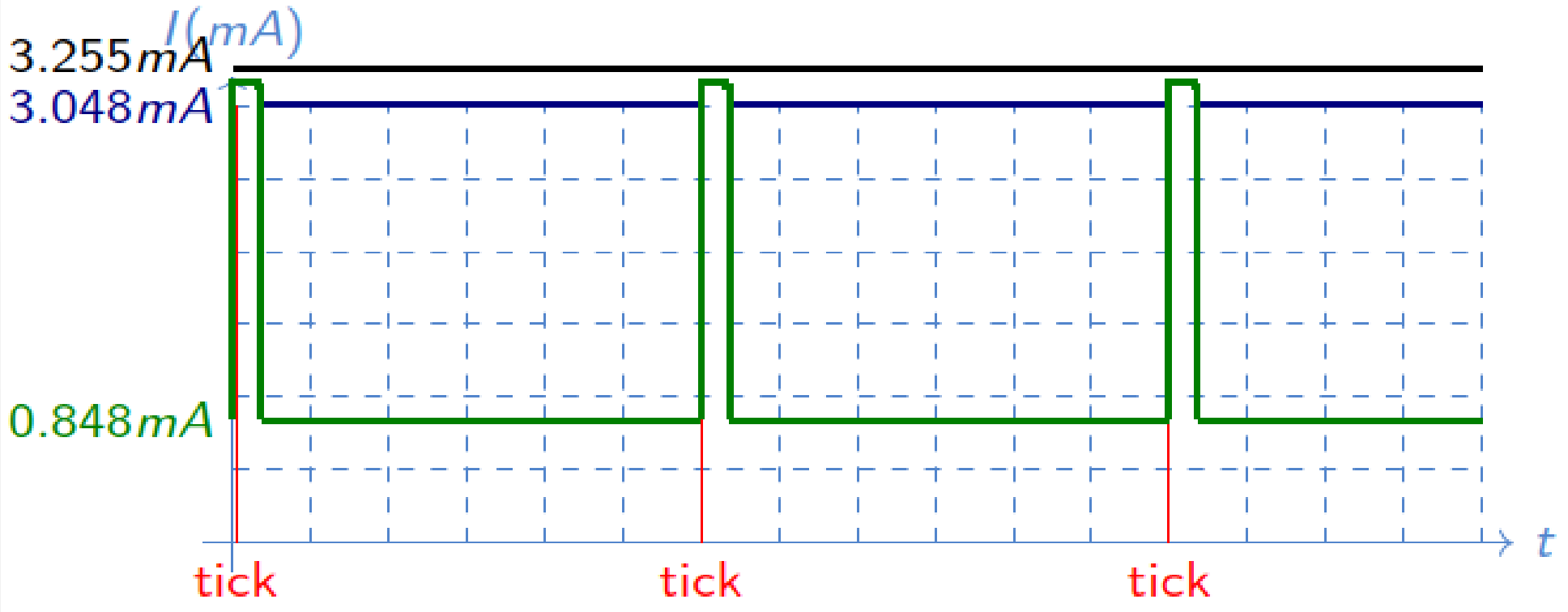
- Putem folosi Idle Mode, cu wake-up pe întreruperea de Timer1
- Putem număra maxim 4s
- Alegem întrerupere de Timer1 la 1s
- Neglijăm timpii de intrare/ieșire din Idle
- $I_{CC_{\mathrm{Idle}}} = I_{CC_{\mathrm{Idle,8M}Hz,3.3V}}(1 + \mathrm{I_{TIM1,Idle}}) = 0.8 \cdot 1.06 = 0.848\mathrm{mA}$

Curent mediu Idle Mode

- $I_{CC_{\text{mediu}}} = \frac{\sum I_{CC_x} * t_x}{\sum t_x}$
- $I_{CC_{\text{mediu}}} = \frac{I_{CC_{conversie}} * t_{conversie} + I_{CC_{comunicatie}} * t_{comunicatie} + I_{CC_{\text{Idle}}} * t_{rest}}{t_{total}}$
- $I_{CC_{\text{mediu}}} = \frac{3.174 * 200 + 3.129 * 200 + 0.848 * 999600}{1000000}$
- $I_{CC_{\text{mediu}}} = 0.848\ \text{mA}$

Grafic de consum Idle Mode

- $t_{\text{baterie}} = \frac{Capacitate}{Consum} = \frac{200mAh}{0.85\text{mA} + 0.01\text{mA}} = 232.56\text{h}$



Alte stări de power saving

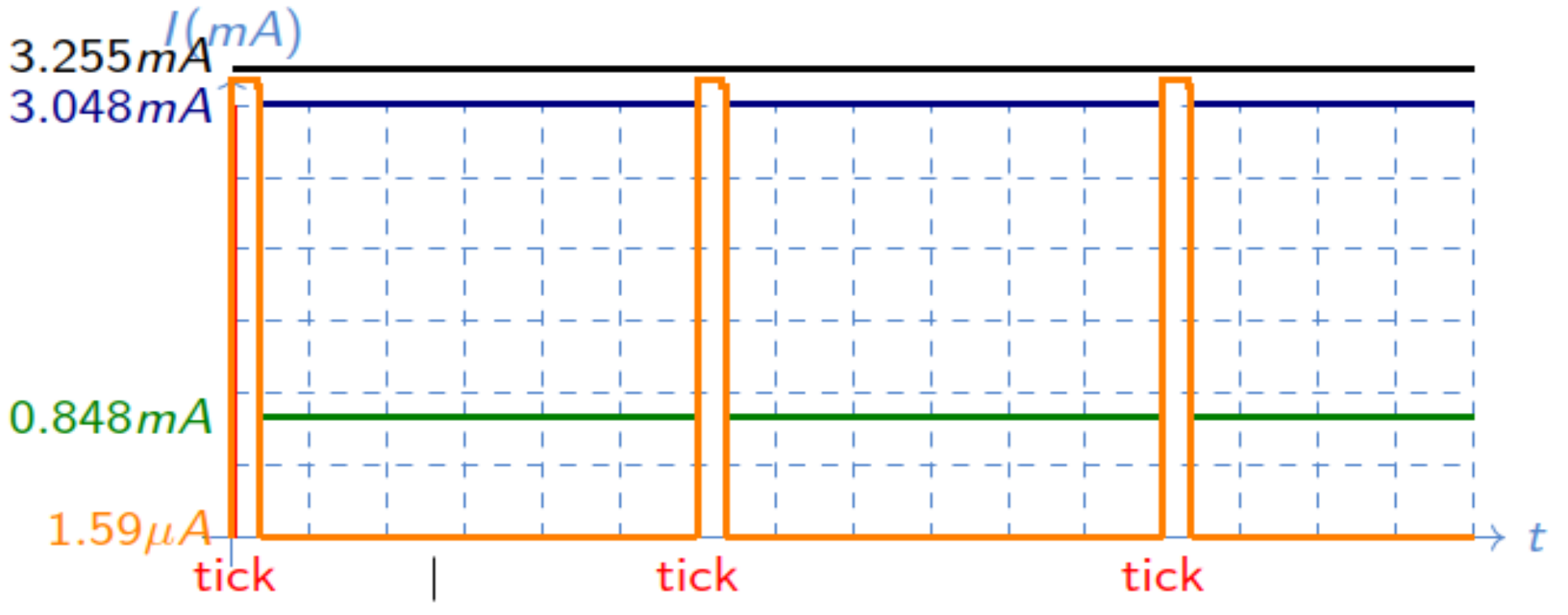
- Power-down - cel mai eficient mod de (ne)lucru
  - Poate fi trezit cu: întrerupere externă, I^2C, watchdog
  - Consum la 25◦C de 0.2µA fără watchdog
  - Noi am avea nevoie de watchdog să-l trezim → 4µA
- Power-save - Adaugă timer-ul 2 dacă funcționează cu cristal propriu
  - Consum la 25◦C, 3.3V de 0.75µA fără watchdog

Curent mediu Power-save Mode

- Curentul mediu este media ponderată pe timpi
- $I_{CC_{\text{mediu}}} = \frac{\sum I_{CC_x} * t_x}{\sum t_x}$
- $I_{CC_{\text{mediu}}} = \frac{I_{CC_{conversie}} * t_{conversie} + I_{CC_{comunicatie}} * t_{comunicatie} + I_{CC_{\text{Power-save}}} * t_{rest}}{t_{total}}$
- $I_{CC_{\text{mediu}}} = \frac{3.174 * 200 + 3.129 * 200 + 0.00075 * 999600}{1000000}$
- $I_{CC_{\text{mediu}}} = 1.59\ \mu A$

Grafic de consum Power-save Mode

- $t_{\text{baterie}} = \frac{Capacitate}{Consum} = \frac{200mAh}{0.00159\text{mA} + 0.01\text{mA}}$ = 17256h ≃ 719zile
- Consumul MCU-ului este de 2047 ori mai mic



Optimizări ADC

- Senzorul oferă o scară de 10mV/1℃ , cu o precizie de 0.5 ℃
- Rezoluția ADC-ului este de 10 biți, la 3.3V → 3.22mV
  - Nu putem reduce rezoluția ADC-ului
  - Frecvența ADC-ului trebuie să fie între 50 și 200kHz
  - Cu PS 128 la 8Mhz este la 62.5kHz → putem reduce prescaler-ul de 2 ori
  - Pentru rezoluție mai mică, puteam reduce la PS 16 sau 32

Curent mediu cu ADC rapid

- Am redus prescalar – ul de la 128 la 64

$$t_{conversie} = 13 \text{ cicliADC} \cdot 64 \cdot t_{ciclu CPU}$$

$$t_{conversie} = 104\mu s \simeq 100\mu s$$

- Curentul mediu este media ponderată pe timpi
- $I_{CC_{\text{mediu}}} = \frac{\sum I_{CC_x} * t_x}{\sum t_x}$
- $I_{CC_{\text{mediu}}} = \frac{I_{CC_{conversie}} * t_{conversie} + I_{CC_{comunicatie}} * t_{comunicatie} + I_{CC_{\text{Power-save}}} * t_{rest}}{t_{total}}$
- $I_{CC_{\text{mediu}}} = \frac{3.174 * 100 + 3.129 * 200 + 0.00075 * 999700}{1000000}$
- $I_{CC_{\text{mediu}}} = 1.27\mu A$

Optimizări frecvență

- Dar dacă am face totul mai încet?
- 1MHz MCU, SPI 500kHz (minim /2 prescaler)
- ADC prescaler 8 → 125000

$$t_{conversie} = 13 \text{ cicliADC} \cdot 8 \cdot t_{cicluCPU}$$

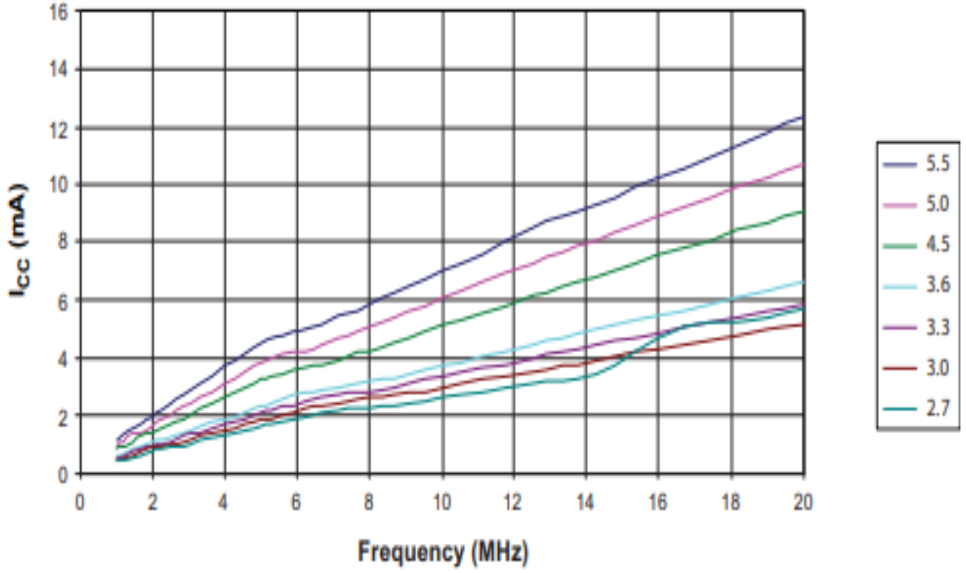
$$t_{conversie} = 104\mu s \simeq 100\mu s$$

- SPI durează mai mult

$$t_{\text{comunicare}} = \frac{100 * 8}{500000} s = 1.6ms$$

Calcul consum activ 1MHz

- $I_{CC_{\text{total}}} = I_{CC_{\text{Activ,1M}Hz,3.3V}}(1 + \text{I}_{\text{ADC}} + I_{\text{SPI}} + \text{I}_{\text{TIM1}})$
- $I_{CC_{\text{total}}} = 0.55\text{mA} \cdot (1 + 0.042 + 0.027 + 0.016) = 0.597\text{mA}$
- $I_{CC_{\text{conversie}}} = I_{CC_{\text{Activ,1M}Hz,3.3V}}(1 + I_{\text{ADC}} + \text{I}_{\text{TIM1}}) = 0.588\text{mA}$
- $I_{CC_{\text{comunicare}}} = I_{CC_{\text{Activ,1M}Hz,3.3V}}(1 + \text{I}_{\text{SPI}} + \text{I}_{\text{TIM1}}) = 0.573\text{mA}$



PRR bit	Additional Current consumption compared to Active with external clock (See Figure 30-1 and Figure 30-2)
PRTIM2	2.5%
PRTIM1	1.6%
PRTIM0	0.7%
PRADC	4.2%
PRSPI	2.7%

Curent mediu cu 1MHz

- Curentul mediu este media ponderată pe timpi
- $I_{CC_{\text{mediu}}} = \frac{\sum I_{CC_x} * t_x}{\sum t_x}$
- $I_{CC_{\text{mediu}}} = \frac{I_{CC_{conversie}} * t_{conversie} + I_{CC_{comunicatie}} * t_{comunicatie} + I_{CC_{rest}} * t_{rest}}{t_{total}}$
- $I_{CC_{\text{mediu}}} = \frac{0.588 * 100 + 0.573 * 1600 + 0.00075 * 998300}{1000000}$
- $I_{CC_{\text{mediu}}} = 0.98\ \mu A$
- Consumul mediu al MCU-ului este de 3300 ori mai mic
- *Q: cat consuma celelalte componente?*

Alte optimizări

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)
- Modelarea consumului pentru nivele de baterie diferite
- Schimbarea cerințelor pentru optimizări în plus
  - Trebuie neapărat la 1s pentru temperatură?
  - Trebuie neapărat făcut update la ecran dacă nu s-a schimbat nimic?
- Optimizări pe părțile ignorate (senzor, LCD)

Concluzii

- Cele mai mari economii - Idle sau Power-save
- Premisa - avem deja închise toate perifericele nefolosite
- Premisa2 - Avem un timp de repaus mare între procesări
- ATMega328P în general e mai eficient per MIPS la frecvențe mai mari pentru aceeași tensiune
  - Depinde și de ce alte operații mai facem
  - Degeaba reducem frecvența dacă nu reducem și tensiunea

  OBS – limitari practice!

La cursul următor

- Familia 8086
- Structură
- Caracteristici
- Particularități